Learn Being Forward

COURSES · CAMPUS PARTNERSHIPS · PLACEMENTS

COURSE NOTES · BASIC TO ADVANCED

C Programming

Complete notes for the Learn Being Forward C Programming syllabus: every module, topic and subtopic explained step by step, with diagrams, verified examples and a practice checkpoint for each module.

23	3	74	390
modules	levels	minimum days	C programs

How to use these notes

These notes follow the **Learn Being Forward C Programming syllabus** exactly: the same 23 modules, in the same order, with every topic and subtopic explained step by step. Each subtopic number (for example **3.7.2**) matches the syllabus, so you can always find where you are.

Level	Modules	Minimum	Outcome
1. Beginner	1 to 9	20 days	Write procedural programs with variables, operators, loops, arrays, strings and functions
2. Intermediate	10 to 16	22 days	Master pointers, dynamic memory, structures, the preprocessor and file handling
3. Advanced	17 to 23	32 days	Bit manipulation, modern C features, data structures, build tools, debugging and Linux systems programming
Total	1 to 23	74 days	About 13 weeks at 6 study days per week

Read
Read the concept for each subtopic and note the key points in your own words.

Type & run
Type out and run every example. Do not copy and paste code. Compare with the OUTPUT box.

Checkpoint
Finish the practice checkpoint at the end of each module before moving on.

Revise
At the end of each level, revise its modules and build one small project.

Every C program in these notes was compiled with **GCC 13** (`gcc -std=c17 -Wall -Wextra`) on Linux and run to produce the output shown. Values that depend on the machine, such as memory addresses and type sizes, are what a typical 64-bit Linux system prints; yours may differ. A few examples need C23, several source files, or a Linux system call, and their boxes say so.

HOW TO READ THE BOXES
Dark boxes with a gold top line are **C code**. Dashed cream boxes labelled **OUTPUT** show what the program prints. Gold-bordered boxes are notes and tips, red boxes warn about common mistakes, and navy boxes hold the key idea to remember.

Course plan at a glance

Module	Topic	Level	Min days
01	Introduction to C	Beginner	1
02	Setup, first program and the compilation process	Beginner	1
03	Variables, data types and constants	Beginner	2
04	Operators and expressions	Beginner	2
05	Input and output	Beginner	1
06	Control flow	Beginner	4
07	Arrays	Beginner	3

Module	Topic	Level	Min days
08	Strings	Beginner	3
09	Functions and recursion	Beginner	3
10	Storage classes, scope and lifetime	Intermediate	2
11	Pointers: fundamentals	Intermediate	4
12	Pointers: advanced	Intermediate	4
13	Dynamic memory allocation	Intermediate	3
14	Structures, unions, enums and typedef	Intermediate	4
15	The preprocessor	Intermediate	2
16	File handling	Intermediate	3
17	Bit manipulation	Advanced	2
18	Advanced C language features	Advanced	3
19	Data structures in C	Advanced	10
20	Multi-file programs, libraries and build tools	Advanced	3
21	Debugging, testing and memory safety	Advanced	3
22	Systems programming in C (Linux)	Advanced	6
23	Projects, interview preparation and next steps	Advanced	5

Table of contents

LEVEL 1 · BEGINNER LEVEL · MODULES 1 TO 9

Module 01 · Introduction to C 8

1.1 What is C? 8
1.2 History of C 9
1.3 Why do we need C? 10
1.4 Features of C 11
1.5 Where is C used? 13
1.6 Limitations of C 14
1.7 C compared with other languages 14

Module 02 · Setup, first program and the compilation process 17

2.1 Setting up the environment 17
2.2 Structure of a C program 18
2.3 Hello World, line by line 20
2.4 Compiling and running 22
2.5 The compilation process (four stages) 22
2.6 Basic syntax rules 24
2.7 Comments 26
2.8 Errors 27

Module 03 · Variables, data types and constants 31

3.1 What is a variable? 31
3.2 Creating variables 32
3.3 Rules for naming variables 34
3.4 Types of variables (by scope) 34
3.5 Data types overview 35
3.6 Integer types 36
3.7 Floating-point types 39
3.8 Characters 42
3.9 Boolean values 44
3.10 Constants and literals 45
3.11 Type conversion 48

Module 04 · Operators and expressions 53

4.1 Basics 53
4.2 Arithmetic operators 54
4.3 Increment and decrement 55
4.4 Relational operators 57
4.5 Logical operators 58
4.6 Bitwise operators 60
4.7 Assignment operators 62
4.8 Conditional (ternary) operator 64
4.9 Special operators 64
4.10 Precedence and associativity 66
4.11 math.h library 69

Module 05 · Input and output 72

5.1 Formatted output with printf 72
5.2 Formatted input with scanf 76

5.3 Character I/O 80
5.4 String I/O 81
5.5 Other I/O 82

Module 06 · Control flow 86

6.1 Decision-making statements 86
6.2 switch statement 89
6.3 Loops 91
6.4 Nested loops 96
6.5 Jump statements 100
6.6 Classic loop problems 102

Module 07 · Arrays 109

7.1 Introduction to arrays 109
7.2 One-dimensional arrays 110
7.3 Multi-dimensional arrays 113
7.4 Variable-length arrays (VLAs) 116
7.5 Array algorithms 116

Module 08 · Strings 125

8.1 Introduction to strings 125
8.2 Reading and printing strings 127
8.3 string.h functions 128
8.4 ctype.h functions 133
8.5 Conversions 134
8.6 Arrays of strings 136
8.7 Writing your own string functions 137
8.8 Classic string problems 139

Module 09 · Functions and recursion 144

9.1 Introduction to functions 144
9.2 Parts of a function 145
9.3 Types of functions by arguments and return 146
9.4 Parameter passing 148
9.5 Functions and arrays 149
9.6 How function calls work 151
9.7 Recursion 152

LEVEL 2 · INTERMEDIATE LEVEL · MODULES 10 TO 16

Module 10 · Storage classes, scope and lifetime 164

10.1 Scope 164
10.2 Lifetime (storage duration) 165
10.3 Linkage 166
10.4 Storage classes 167
10.5 Memory layout of a C program 170

Module 11 · Pointers: fundamentals 173

11.1 Memory and addresses 173
11.2 Pointer basics 174
11.3 Special pointers 176
11.4 Pointer arithmetic 178
11.5 Pointers and arrays 180
11.6 Pointers and functions: call by reference 182

Module 12 · Pointers: advanced 187
12.1 Pointers and strings 187
12.2 Pointer to pointer 191
12.3 Arrays of pointers 193
12.4 Pointer to an array 195
12.5 Function pointers 197
12.6 const and pointers 201
12.7 Reading complex declarations 203
12.8 Command-line arguments in depth 204

Module 13 · Dynamic memory allocation 208
13.1 Static vs dynamic allocation 208
13.2 stdlib.h allocation functions 209
13.3 Dynamic arrays 215
13.4 Memory errors 219
13.5 Good practices 223

Module 14 · Structures, unions, enums and typedef 227
14.1 Structures 227
14.2 Structures in depth 230
14.3 Memory layout of structures 236
14.4 Bit fields 239
14.5 Unions 240
14.6 Enumerations 242
14.7 typedef 244
14.8 Anonymous structures and unions (C11) 245

Module 15 · The preprocessor 248
15.1 Introduction 248
15.2 File inclusion 249
15.3 Macros 251
15.4 Conditional compilation 256
15.5 Include guards 259
15.6 Other directives 261
15.7 Predefined macros 263
15.8 assert.h 264

Module 16 · File handling 268
16.1 Introduction 268
16.2 Opening and closing files 270
16.3 Reading and writing text files 271
16.4 Binary files 275
16.5 Random access 277
16.6 Error handling 280
16.7 File operations 282
16.8 File programs 284

LEVEL 3 · ADVANCED LEVEL · MODULES 17 TO 23

Module 17 · Bit manipulation 290
17.1 Number systems 290
17.2 Bitwise operators in depth 293
17.3 Bit masks 296

17.4 Bit tricks 298
17.5 Endianness 301

Module 18 · Advanced C language features 304

18.1 Type qualifiers 304
18.2 Variadic functions 307
18.3 Advanced function topics 309
18.4 Generic selection 312
18.5 Undefined, unspecified and implementation-defined behaviour 313
18.6 Error handling patterns in C 315
18.7 Time and utilities 318
18.8 Modern C standards 321

Module 19 · Data structures in C 326

19.1 Fundamentals 326
19.2 Linked lists 328
19.3 Stack 337
19.4 Queue 341
19.5 Trees 345
19.6 Heaps 350
19.7 Hashing 351
19.8 Graphs 354
19.9 Sorting and searching algorithms 357

Module 20 · Multi-file programs, libraries and build tools 366

20.1 Splitting a program into files 366
20.2 Libraries 370
20.3 Make and Makefiles 372
20.4 CMake basics 375
20.5 Code organisation and style 377
20.6 Version control 378

Module 21 · Debugging, testing and memory safety 381

21.1 Debugging with GDB 381
21.2 Memory error detection 386
21.3 Static analysis 389
21.4 Testing C code 391
21.5 Secure coding 393
21.6 Performance 396

Module 22 · Systems programming in C (Linux) 401

22.1 Introduction 401
22.2 Low-level file I/O 402
22.3 Processes 406
22.4 Inter-process communication 410
22.5 Signals 414
22.6 Threads (POSIX threads) 416
22.7 Network programming 423

Module 23 · Projects, interview preparation and next steps 429

23.1 Capstone projects 429
23.2 Interview preparation 441
23.3 Next steps after C 447

LEVEL 1

Beginner level

Outcome: write procedural programs with variables, operators, loops, arrays, strings and functions.

9 MODULES · 20 DAYS MINIMUM

Module 01 · Introduction to C	1 day
Module 02 · Setup, first program and the compilation process	1 day
Module 03 · Variables, data types and constants	2 days
Module 04 · Operators and expressions	2 days
Module 05 · Input and output	1 day
Module 06 · Control flow	4 days
Module 07 · Arrays	3 days
Module 08 · Strings	3 days
Module 09 · Functions and recursion	3 days

MODULE 01 · BEGINNER

Introduction to C

Minimum duration **1 day** | Level **Beginner**

This module explains what C is, why it still matters decades after its creation, and where it is used, before any code is written.

IN THIS MODULE

1.1 What is C?
1.2 History of C
1.3 Why do we need C?
1.4 Features of C
1.5 Where is C used?
1.6 Limitations of C
1.7 C compared with other languages

1.1 What is C?

C is one of the most influential programming languages ever created. The operating system on your phone, the database behind your bank and the firmware inside your washing machine are very likely written, at least partly, in C. Learning C means learning how a computer really works.

1.1.1 Definition: a general-purpose, procedural, compiled programming language

General-purpose

Not tied to one kind of task: C is used for operating systems, games, databases, embedded devices and scientific code alike.

Procedural

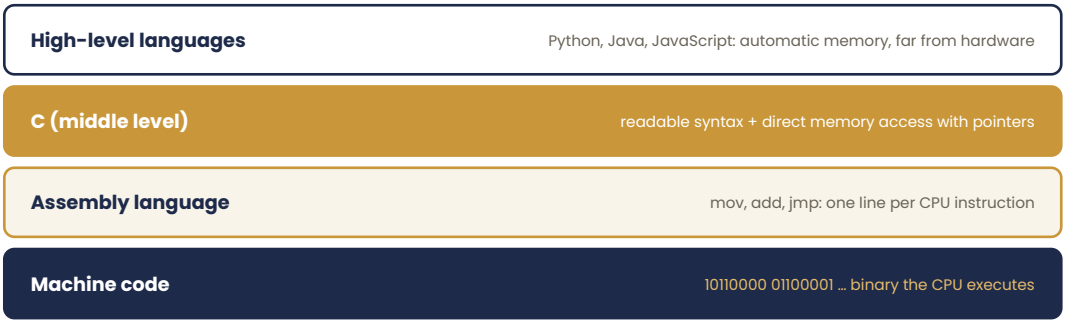
A program is a set of **procedures** (functions) that run step by step and pass data to each other.

Compiled

Source code is translated **once** by a compiler into machine code that the CPU executes directly.

1.1.2 Middle-level language: high-level syntax with low-level memory access

C sits between **high-level** languages (Python, Java), which hide the hardware, and **low-level** assembly language, which talks to the CPU directly. It gives you readable constructs such as loops, functions and structures, yet also lets you work with raw memory addresses, individual bits and hardware registers.

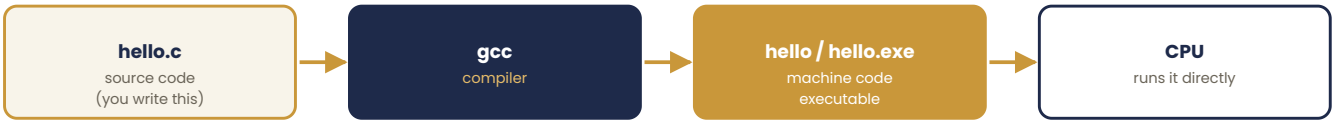


C SITS BETWEEN HUMAN-FRIENDLY AND MACHINE-FRIENDLY LANGUAGES

1.1.3 Structured programming language (functions, blocks, control structures)

C programs are built from **functions**, each doing one job; code is grouped into **blocks** with `{ }`; and the flow is controlled by **if**, **switch**, **for**, **while**. This structured style replaced the tangled "spaghetti code" of jump-based early languages and makes programs easier to read, test and maintain.

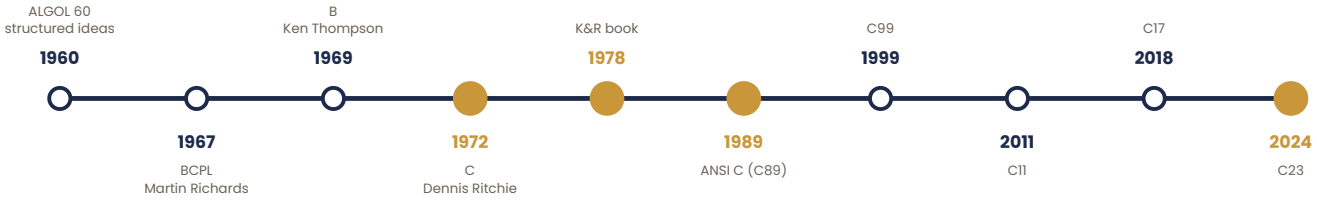
1.1.4 Compiled language: source code (.c) → machine code (executable)



C IS COMPILED AHEAD OF TIME INTO A NATIVE EXECUTABLE

	Compiled (C, C++, Go)	Interpreted (Python, JavaScript)
Translation	Once, before running	Line by line while running
Speed	Very fast	Slower
Errors found	Many at compile time	Only when that line runs
Distribution	Executable for one OS/CPU	Source + interpreter needed

1.2 History of C



FROM ALGOL TO C23

1.2.1 Predecessors: ALGOL, BCPL and B

ALGOL 60 introduced block structure and structured control flow. **BCPL** (1967, Martin Richards) was a simple language for writing compilers. **B** (1969, Ken Thompson at Bell Labs) simplified BCPL to fit on the tiny PDP-7 computer, but B had no data types: everything was a machine word. That limitation led directly to C.

1.2.2 Dennis Ritchie at Bell Labs (1972)

Dennis Ritchie extended B with **data types** (char, int, float), **structures** and a real compiler that produced efficient machine code. He named it C, the letter after B. C was designed to be close enough to the hardware to write an operating system, yet portable across machines.

1.2.3 C and the UNIX operating system

In 1973 Ritchie and Ken Thompson rewrote the UNIX kernel in C. This was revolutionary: operating systems were then written in assembly language for one specific machine. Because UNIX was now in C, it could be moved to new computers by rewriting only the compiler. UNIX and C spread together through universities, and today Linux, macOS, iOS and Android all descend from that idea.

1.2.4 "The C Programming Language" book by Kernighan and Ritchie (K&R C, 1978)

Brian Kernighan and Dennis Ritchie published *The C Programming Language* in 1978. For a decade this short book *was* the definition of the language, known as **K&R C**. It also gave the world the tradition of the "hello, world" first program.

1.2.5 Standards timeline: ANSI C / C89 → C90 → C99 → C11 → C17 → C23

Standard	Year	Key additions
ANSI C / C89	1989	First official standard: function prototypes, `void`, `const`, standard library
C90 (ISO)	1990	Same language adopted by ISO
C99	1999	`//` comments, `bool`, `long long`, `stdint.h`, variable-length arrays, declarations anywhere, `inline`, designated initializers
C11	2011	Threads, atomics, `_Generic`, `_Static_assert`, anonymous structs, removal of `gets()`
C17 (C18)	2018	Bug-fix release, no new features
C23	2024	`bool` / `true` / `false` keywords, `nullptr`, `typeof`, binary literals `0b1010`, `constexpr`, `#embed`, `auto` type inference

> **NOTE**
> In these notes we compile with `-std=c17`, which every modern compiler supports. C23 features are marked when they appear.

1.3 Why do we need C?

1.3.1 Speed and efficiency close to assembly language

C compiles directly to machine instructions with almost no hidden work: no virtual machine, no garbage collector, no interpreter. Well-written C runs within a few percent of hand-written assembly, which is why performance-critical code is still written in C.



RELATIVE RUN TIME FOR TYPICAL CPU-HEAVY BENCHMARKS (LOWER IS FASTER; ILLUSTRATIVE VALUES)

1.3.2 Direct memory access through pointers

A **pointer** holds a memory address. With pointers, C programs can manage memory precisely, build data structures, and read and write hardware registers at fixed addresses, something most high-level languages forbid.

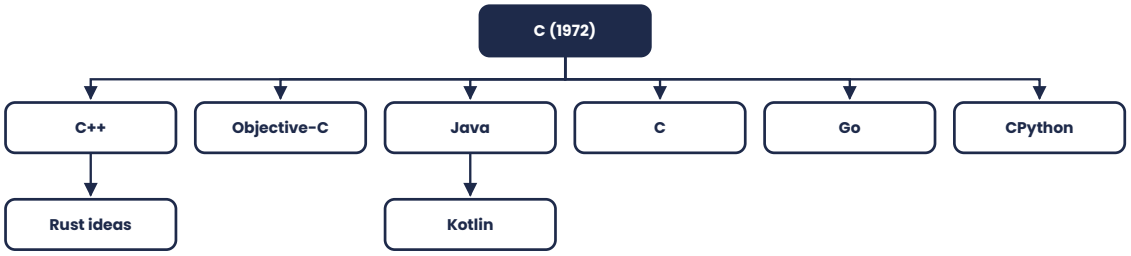
1.3.3 Portability across hardware and operating systems

A C compiler exists for almost every processor ever made, from 8-bit microcontrollers to supercomputers. Standard C source code can be recompiled for a new platform with little or no change: "write once, compile anywhere".

1.3.4 Small language with a small runtime

C has only 32 keywords in C89 (about 60 today) and a compact standard library. A C program needs almost no runtime support, so it can run on a device with a few kilobytes of memory or even with no operating system at all ("bare metal").

1.3.5 Foundation for C++, Java, C#, Python (CPython is written in C) and more



MANY LANGUAGES BORROWED C'S SYNTAX, AND MANY ARE IMPLEMENTED IN C

The curly braces, `if/for/while`, `++`, and `printf`-style formatting you will see in Java, C#, JavaScript and C++ all come from C. The standard Python interpreter (CPython), PHP, Ruby, Perl and the Lua engine are written in C. Knowing C makes every one of these easier to understand.

1.3.6 Teaches how computers really work (memory, addresses, the stack)

Because C hides so little, you learn what other languages do behind your back: how variables occupy bytes, what an address is, how function calls use the stack, how strings are just arrays of bytes, and what "allocating memory" really means. This knowledge makes you a better programmer in any language.

1.4 Features of C

1.4.1 Simple and compact

C has a small set of keywords and rules. A beginner can learn the whole core language, and an experienced programmer can keep it all in their head. The compiler itself is small enough to exist for almost any processor.

1.4.2 Procedural and structured

A C program is a collection of functions called from `main()`. Each function is a structured block with sequence, selection (if/switch) and iteration (loops), which keeps even large programs organised.

1.4.3 Fast execution

Because it compiles to native machine code and adds nothing hidden (no garbage collector, no runtime checks), C programs start instantly and run at full hardware speed.

1.4.4 Portable

Standard C code can be recompiled unchanged for Windows, Linux, macOS or a microcontroller. Only code that uses operating-system-specific functions needs adapting.

1.4.5 Rich set of operators and library functions

C offers arithmetic, relational, logical, **bitwise**, assignment, conditional and pointer operators, plus a standard library for input/output (`stdio.h`), strings (`string.h`), maths (`math.h`), memory (`stdlib.h`), time and more.

1.4.6 Pointers and dynamic memory

Pointers give direct access to memory addresses; `malloc` / `free` let a program request exactly as much memory as it needs at run time. Together they make linked lists, trees and efficient buffers possible.

1.4.7 Extensible (user-defined functions and libraries)

You can write your own functions, group them into header and source files, and build them into static or shared libraries that other programs reuse, exactly how the standard library itself is made.

1.4.8 Recursion support

A C function can call itself. Recursion gives elegant solutions to problems such as factorial, tree traversal, Tower of Hanoi and quicksort (Module 9).

Summary of the features:

Feature	What it means	Where you meet it
Simple and compact	Few keywords and rules; the whole language fits in a short book	Modules 2–3
Procedural and structured	Programs are functions and blocks with clear control flow	Modules 6, 9
Fast execution	Compiled to native code with no runtime overhead	Everywhere
Portable	Same source compiles on Windows, Linux, macOS, microcontrollers	Module 2
Rich operators and library	40+ operators incl. bitwise; stdio, string, math, stdlib…	Modules 4, 8, 17
Pointers and dynamic memory	Work with addresses; request memory at run time	Modules 11–13
Extensible	Write your own functions and build them into libraries	Modules 9, 20
Recursion	A function may call itself	Module 9

1.5 Where is C used?



C RUNS ALMOST EVERYWHERE

1.5.1 Operating systems (Linux kernel, parts of Windows and macOS)

The Linux kernel is about 30 million lines, almost all C. The cores of Windows (NT kernel) and macOS (XNU kernel) are largely C, and so are Android's low-level layers and most UNIX utilities.

1.5.2 Embedded systems and microcontrollers (Arduino, automotive, IoT)

Microwaves, washing machines, car engine controllers, pacemakers, drones and smart watches run C on small microcontrollers. Arduino sketches are C/C++. A modern car contains dozens of processors running C code.

1.5.3 Compilers and interpreters

GCC and the CPython, PHP, Ruby and Perl interpreters are written in C (LLVM/Clang are C++). When you run a Python program, C code is doing the work underneath.

1.5.4 Databases (MySQL, SQLite, PostgreSQL)

SQLite, the most widely deployed database in the world (inside every phone and browser), is about 150,000 lines of C. PostgreSQL is written in C; MySQL in C and C++.

1.5.5 Device drivers and firmware

Drivers let the operating system talk to hardware: graphics cards, keyboards, network cards, storage. Firmware is the software built into devices such as routers, printers and SSDs. Both need direct hardware access and small size, so they are written in C.

1.5.6 Networking stacks and system utilities

The TCP/IP code that moves every packet on the internet, tools like `curl`, `git` and `ssh`, and web servers like nginx and Apache are written in C.

1.5.7 Game engines and graphics libraries

Performance-critical parts of game engines, the OpenGL and Vulkan graphics APIs, and audio libraries are written in C or C++.

1.5.8 High-performance and scientific computing

Numerical libraries such as BLAS and LAPACK, the core of NumPy, weather and physics simulations, and cryptography libraries (OpenSSL) rely on C for speed.

1.6 Limitations of C

C's power comes with responsibility. It trusts the programmer completely and checks very little.

1.6.1 No object-oriented features (classes, inheritance)

C has no classes, objects, inheritance or polymorphism. Large programs are organised with structures, functions and files instead (C++ adds object orientation).

1.6.2 No built-in bounds checking or garbage collection

Writing past the end of an array is not stopped: it silently overwrites other memory, causing crashes or security holes (buffer overflows). Memory you request is never freed automatically.

1.6.3 No exception handling

There is no `try/catch`. Functions report errors through return values (often -1 or NULL) and the global `errno`, and the programmer must remember to check them.

1.6.4 Manual memory management and its risks

You allocate memory with `malloc` and must release it with `free`. Forgetting causes **memory leaks**; freeing twice or using memory after freeing it causes crashes and vulnerabilities. Module 13 and Module 21 show how to do this safely.

> **WATCH OUT**
> Around 70% of serious security bugs in large C/C++ code bases (reported by Microsoft and Google) are memory-safety errors. Writing correct C requires discipline, compiler warnings and tools such as sanitizers and Valgrind (Module 21).

1.7 C compared with other languages

1.7.1 C vs C++

	C	C++
Paradigm	Procedural	Multi-paradigm: procedural, object-oriented, generic
Classes/objects	No (structs only)	Yes
Memory	malloc/free	new/delete, RAII, smart pointers
Error handling	Return codes	Exceptions
Standard library	Small	Large (STL: vector, map, algorithms)
Compatibility	-	Mostly a superset of C

1.7.2 C vs Java (compiled to machine code vs bytecode, manual vs automatic memory)

	C	Java
Compiles to	Native machine code for one platform	Bytecode run by the JVM on any platform
Memory	Manual (malloc/free)	Automatic garbage collection

	C	Java
Pointers	Yes, with arithmetic	References only, no arithmetic
Bounds checking	No	Yes (ArrayIndexOutOfBoundsException)
OOP	No	Yes
Typical use	Systems, embedded	Enterprise apps, Android, servers

1.7.3 C vs Python (speed, typing, abstraction level)

	C	Python
Execution	Compiled, very fast	Interpreted, much slower
Typing	Static: `int x = 5;`	Dynamic: `x = 5`
Abstraction	Low: you manage memory and strings	High: lists, dicts, automatic memory
Code length	Longer	Shorter
Typical use	Performance-critical and system code	Scripting, data science, web, automation

The same task in three languages:

C: SUM OF 1 TO 100 main.c

```c
#include <stdio.h>

int main(void) {
    int sum = 0;
    for (int i = 1; i <= 100; i++)
        sum += i;
    printf("Sum of 1 to 100 = %d\n", sum);
    return 0;
}
```

OUTPUT

```
Sum of 1 to 100 = 5050
```

JAVA Java

```java
public class Sum {
    public static void main(String[] args) {
        int sum = 0;
        for (int i = 1; i <= 100; i++) sum += i;
        System.out.println("Sum of 1 to 100 = " + sum);
    }
}
```

PYTHON Python

```python
print("Sum of 1 to 100 =", sum(range(1, 101)))
```

C is the language of systems: small, fast, portable and close to the hardware. It asks more of the programmer than modern languages, and in return teaches exactly how a computer works.

PRACTICE CHECKPOINT

Explain in your own words why C is called a **middle-level language** and name **five real systems written in C**.

Sample answer: C is middle-level because it combines high-level features (readable syntax, functions, loops, structures, portability) with low-level abilities (pointers to raw memory addresses, bitwise operations, direct hardware access), so it is used both for applications and for operating systems. Five systems written in C: the **Linux kernel**, **SQLite**, **CPython** (the Python interpreter), **Git**, and the **nginx** web server (others: PostgreSQL, curl, Redis, the Windows NT kernel core).

MODULE 02 · BEGINNER

Setup, first program and the compilation process

Minimum duration **1 day** | Level **Beginner**

By the end of this module the learner has a working compiler and understands every line of Hello World and every stage of turning source code into an executable.

IN THIS MODULE

2.1 Setting up the environment
2.2 Structure of a C program
2.3 Hello World, line by line
2.4 Compiling and running
2.5 The compilation process (four stages)
2.6 Basic syntax rules
2.7 Comments
2.8 Errors

2.1 Setting up the environment

To write C you need two things: a **text editor** (or IDE) to type the source code, and a **compiler** to turn it into a program.

2.1.1 Compilers: GCC, Clang, MSVC

Compiler	From	Platforms	Notes
GCC (GNU Compiler Collection)	GNU project, 1987	Linux, Windows (MinGW), macOS	The standard on Linux; used in these notes
Clang	LLVM project, Apple	macOS (default), Linux, Windows	Very clear error messages; on macOS `gcc` is actually Clang
MSVC (cl.exe)	Microsoft	Windows (Visual Studio)	Good C++ support; C support improved with C11/C17

2.1.2 Installing GCC on Windows (MinGW-w64 / MSYS2), macOS (Xcode tools) and Linux

Windows (MSYS2, recommended): download the installer from `msys2.org` and install to `C:\msys64`. Open the "MSYS2 UCRT64" terminal and run the command below. Then add `C:\msys64\ucrt64\bin` to the Windows **PATH** (Settings → System → About → Advanced system settings → Environment Variables → Path → New) so that `gcc` works in any terminal.

Windows: installing GCC in MSYS2

```
$ pacman -Syu                                  # update MSYS2 (terminal may close; reopen it)
$ pacman -S mingw-w64-ucrt-x86_64-gcc mingw-w64-ucrt-x86_64-gdb
```

macOS: open Terminal and install Apple's Command Line Tools. This provides Clang (invoked as `gcc` or `clang`), make and git. Alternatively install real GCC with Homebrew: `brew install gcc`.

macOS: command line tools

```
$ xcode-select --install
```

Linux: GCC is usually installed already; if not, install the build tools package.

Linux

```
$ sudo apt update && sudo apt install build-essential gdb      # Ubuntu / Debian
$ sudo dnf groupinstall "Development Tools"                     # Fedora
```

2.1.3 IDEs and editors: VS Code, Code::Blocks, CLion, Dev-C++

Tool	Type	Good for
VS Code + C/C++ extension	Free editor	Lightweight, all platforms; use the integrated terminal to compile
Code::Blocks	Free IDE	Beginners on Windows (download the version bundled with MinGW)
CLion	Commercial IDE (free for students)	Professional projects, CMake, excellent debugger
Dev-C++ (Embarcadero)	Free IDE	Simple Windows IDE used in many colleges
Vim / Emacs / nano	Terminal editors	Linux servers and systems programming

2.1.4 Online compilers for quick practice

Websites such as **OnlineGDB**, **Programiz**, **Replit** and **Compiler Explorer (godbolt.org)** compile and run C in the browser with nothing to install. They are perfect for quick experiments; Compiler Explorer even shows the assembly your code produces. For real projects, install a local compiler.

2.1.5 Verifying: gcc --version

Checking the installation

```
$ gcc --version
gcc (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0
Copyright (C) 2023 Free Software Foundation, Inc.
This is free software; see the source for copying conditions.
```

> **WATCH OUT**
> "gcc is not recognized as an internal or external command" on Windows means the compiler's `bin` folder is not on PATH. Add it and open a **new** terminal.

2.2 Structure of a C program

Every C program follows the same general layout, from top to bottom:

THE SECTIONS OF A C PROGRAM

main.c

```c
/*  Documentation section
 *  Program : Area of a circle
 *  Author  : Learn Being Forward
 */
#include <stdio.h>              /* Preprocessor directives: link section */
#define PI 3.14159              /* Definition section */

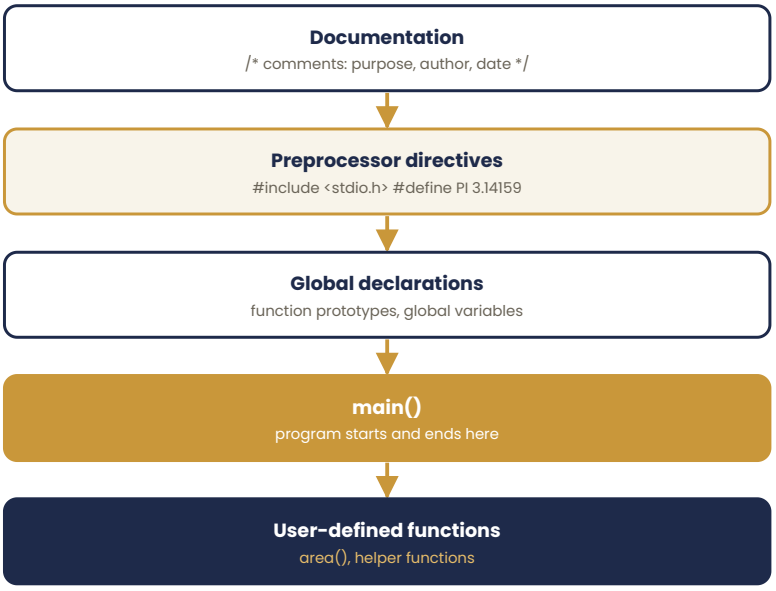
double area(double r);           /* Global declarations: prototypes, global variables */
int programs_run = 1;

int main(void) {                 /* main() function: execution starts here */
    double r = 2.5;
    printf("Area of circle with radius %.1f = %.2f\n", r, area(r));
    printf("Programs run: %d\n", programs_run);
    return 0;
}

double area(double r) {          /* User-defined functions */
    return PI * r * r;
}
```

OUTPUT

```
Area of circle with radius 2.5 = 19.63
Programs run: 1
```



THE STANDARD LAYOUT OF A C SOURCE FILE

2.2.1 Documentation section (comments)

Comments at the top describe what the program does, who wrote it and when. The compiler ignores them; humans read them.

2.2.2 Preprocessor directives (#include, #define)

Lines starting with `#` are processed **before** compilation. `#include` pastes in a header file (here, declarations of input/output functions); `#define` creates a symbolic constant or macro.

2.2.3 Global declarations

Function **prototypes** (declarations that tell the compiler a function's name, parameters and return type) and **global variables** visible to the whole file.

2.2.4 The main() function

Every C program must have exactly one `main` function. The operating system calls it to start the program, and the program ends when `main` returns.

2.2.5 User-defined functions

Your own functions, defined after (or before) `main`. They break the program into small, reusable pieces (Module 9).

2.3 Hello World, line by line

HELLO.C

hello.c

```c
#include <stdio.h>

int main(void) {
    printf("Hello, World!\n");
    return 0;
}
```

OUTPUT

```
Hello, World!
```

2.3.1 #include <stdio.h> and what a header file is

`stdio.h` ("standard input/output header") is a **header file**: a text file containing **declarations** of functions such as `printf`, `scanf` and `puts`. Including it tells the compiler what `printf` looks like (its parameters and return type). The actual compiled code of printf lives in the C standard library, which the linker connects later. Angle brackets `< >` mean "search the system include directories".

2.3.2 int main(void) and int main(int argc, char *argv[])

`int` is the return type: main returns an integer status to the operating system. `(void)` says main takes no arguments. The second standard form receives **command-line arguments**: `argc` is the count and `argv` an array of strings (Module 5.5.3 and 12.8).

MAIN WITH ARGUMENTS: RUN AS ./PROG ASHA 21

main.c

```c
#include <stdio.h>

int main(int argc, char *argv[]) {
    printf("Program name: %s\n", argv[0]);
    printf("Number of extra arguments: %d\n", argc - 1);
    for (int i = 1; i < argc; i++)
        printf("argv[%d] = %s\n", i, argv[i]);
    return 0;
}
```

OUTPUT

```
Program name: ./prog
Number of extra arguments: 2
argv[1] = Asha
argv[2] = 21
```

2.3.3 printf() and the newline \n

`printf` ("print formatted") writes text to the screen (standard output). The text between double quotes is a **string literal**. `\n` is an **escape sequence** meaning "newline": without it, the cursor stays at the end of the line and the next output continues on the same line.

EFFECT OF \N

main.c

```c
#include <stdio.h>

int main(void) {
    printf("Learn ");
    printf("Being ");
    printf("Forward\n");          /* only this one ends the line */
    printf("Line two\n\n");        /* two newlines leave an empty line */
    printf("Tab:\tafter a tab\n");
    return 0;
}
```

OUTPUT

```
Learn Being Forward
Line two

Tab:    after a tab
```

2.3.4 return 0; and exit status

`return 0;` ends `main` and hands the value **0** back to the operating system, meaning "success". Any non-zero value means an error. Shell scripts and build tools check this **exit status**. (Since C99, reaching the end of main without return also returns 0, but writing it is good practice.) `EXIT_SUCCESS` and `EXIT_FAILURE` from `stdlib.h` are portable names.

Checking the exit status

```
$ ./hello
Hello, World!
$ echo $?                # Linux/macOS: show the last exit status (Windows cmd: echo %ERRORLEVEL%)
0
```

2.4 Compiling and running

2.4.1 gcc hello.c -o hello

`gcc` is the compiler, `hello.c` the source file, and `-o hello` names the output executable. Without `-o`, GCC names it `a.out` (Linux/macOS) or `a.exe` (Windows).

2.4.2 Running ./hello (Linux/macOS) or hello.exe (Windows)

Compile and run

```
$ gcc hello.c -o hello
$ ./hello                       # Linux / macOS: ./ means "in the current folder"
Hello, World!

C:\code> gcc hello.c -o hello.exe
C:\code> hello.exe               # Windows Command Prompt (PowerShell: .\hello.exe)
Hello, World!
```

2.4.3 Useful flags: -Wall, -Wextra, -std=c17, -g, -O2

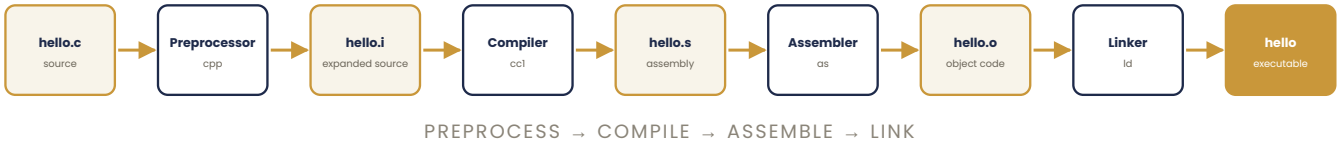
Flag	Meaning	When
`-Wall`	Enable the most useful warnings	Always
`-Wextra`	Even more warnings (unused parameters, sign comparisons…)	Always
`-std=c17`	Compile as standard C17 (or c99, c11, c2x/c23)	Always, to be explicit
`-pedantic`	Warn about non-standard extensions	Portable code
`-g`	Include debugging information for gdb	While developing
`-O0` / `-O2` / `-O3`	Optimisation level: none / standard / aggressive	-O2 for release builds
`-Werror`	Treat warnings as errors	Strict projects, CI
`-lm`	Link the math library	When using math.h on Linux

The command to remember

```
$ gcc -std=c17 -Wall -Wextra -g hello.c -o hello       # recommended everyday command
```

2.5 The compilation process (four stages)

"Compiling" is really four separate steps. GCC runs them all for you, but each can be stopped and inspected.



PREPROCESS → COMPILE → ASSEMBLE → LINK

HELLO.C USED FOR THE STAGE-BY-STAGE DEMO

hello.c

```c
#include <stdio.h>
#define GREETING "Hello, World!"

int main(void) {
    printf("%s\n", GREETING);   /* print the greeting */
    return 0;
}
```

OUTPUT

```
Hello, World!
```

2.5.1 Preprocessing (.c → .i): macros expanded, headers included

The preprocessor handles every `#` line: it pastes the whole of `stdio.h` (hundreds of lines) into the file, replaces `GREETING` with its text, and removes comments. The result is still C, just much longer.

2.5.2 Compilation (.i → .s): C to assembly

The compiler proper checks syntax and types, optimises, and translates the C code into **assembly language** for the target CPU (here x86-64).

2.5.3 Assembly (.s → .o): assembly to object code

The assembler converts assembly text into binary **machine code** in an **object file**. It is not runnable yet: calls to functions defined elsewhere (like `printf`, which GCC optimised to `puts`) are left as unresolved symbols.

2.5.4 Linking (.o + libraries → executable)

The linker combines your object file(s) with the C standard library and startup code, resolves every symbol (finds the real `puts`), and writes the final executable.

2.5.5 Seeing each stage with gcc -E, -S, -c

Each stage on a real Linux system (assembly trimmed)

```
$ gcc -E hello.c -o hello.i        # stop after preprocessing
$ wc -l hello.i
820 hello.i                          # 7 lines became 820: stdio.h was pasted in
$ tail -5 hello.i
# 4 "hello.c"
int main(void) {
    printf("%s\n", "Hello, World!");  # macro replaced, comment removed
    return 0;
}

$ gcc -S hello.c -o hello.s        # stop after compiling: assembly
$ cat hello.s
.LC0:
        .string "Hello, World!"
main:
        pushq   %rbp
        movq    %rsp, %rbp
        leaq    .LC0(%rip), %rax
        movq    %rax, %rdi
        call    puts@PLT             # printf("%s\n", s) optimised into puts(s)
        movl    $0, %eax             # return value 0
        popq    %rbp
        ret

$ gcc -c hello.c -o hello.o        # stop after assembling: object file
$ nm hello.o                       # list its symbols
0000000000000000 T main              # T = defined here
                 U puts              # U = undefined: the linker must find it

$ gcc hello.o -o hello             # link
$ ./hello
Hello, World!

$ ls -l   (sizes in bytes)
143    hello.c      21323  hello.i      668  hello.s      1504  hello.o      15960  hello
```

Stage	Command	Input → output	Tool
Preprocess	`gcc -E`	.c → .i	cpp
Compile	`gcc -S`	.i → .s	cc1
Assemble	`gcc -c`	.s → .o	as
Link	`gcc file.o -o prog`	.o + libs → executable	ld
All at once	`gcc file.c -o prog`	.c → executable	all

2.6 Basic syntax rules

2.6.1 Case sensitivity

C is **case-sensitive**: `total`, `Total` and `TOTAL` are three different names. Keywords are all lowercase: `Int` or `Printf` are errors.

2.6.2 Semicolons and statements

Every statement ends with a semicolon `;`. It is a statement **terminator**, not a separator. Forgetting it is the most common beginner error. Lines that are not statements, such as `#include` directives and the line opening a function, do not take one.

2.6.3 Blocks { } and whitespace

Curly braces group statements into a **block** (function bodies, loop bodies). Spaces, tabs and newlines are ignored by the compiler (except inside strings), so indentation is purely for humans, but it is essential for readability.

LEGAL BUT UNREADABLE: WHITESPACE DOES NOT MATTER TO THE COMPILER main.c

```c
#include <stdio.h>
int main(void){int a=5,b=7;if(a<b){printf("%d is smaller\n",a);}return 0;}
```

OUTPUT

```
5 is smaller
```

2.6.4 Tokens: keywords, identifiers, constants, strings, operators, special symbols

The compiler first splits source code into **tokens**, the smallest meaningful units.



INT TOTAL = PRICE * 3; SPLIT INTO SEVEN TOKENS

Token type	Examples
Keywords	`int`, `return`, `if`, `while`
Identifiers	`main`, `total`, `student_count`
Constants (literals)	`42`, `3.14`, `'A'`, `0x1F`
String literals	`"Hello"`
Operators	`+ - * / = == && ++`
Special symbols / punctuators	`; , { } ( ) [ ] #`

2.6.5 The 32 original keywords (plus C99/C11/C23 additions)

Keywords are reserved words with a fixed meaning; they cannot be used as names. C89 defined 32:

auto	break	case	char
const	continue	default	do
double	else	enum	extern
float	for	goto	if
int	long	register	return

short	signed	sizeof	static
struct	switch	typedef	union
unsigned	void	volatile	while

Standard	Added keywords
C99	`inline`, `restrict`, `_Bool`, `_Complex`, `_Imaginary`
C11	`_Alignas`, `_Alignof`, `_Atomic`, `_Generic`, `_Noreturn`, `_Static_assert`, `_Thread_local`
C23	`bool`, `true`, `false`, `nullptr`, `constexpr`, `typeof`, `typeof_unqual`, `static_assert`, `thread_local`, `alignas`, `alignof`, `_BitInt`, `_Decimal32/64/128`

2.6.6 Identifier naming rules and conventions (snake_case)

- Allowed characters: letters `a–z A–Z`, digits `0–9` and underscore `_`.
- Must not start with a digit: `2nd_place` is invalid.
- Must not be a keyword.
- Names starting with an underscore followed by a capital letter, or two underscores, are reserved for the implementation: avoid them.
- Convention in C: **snake_case** for variables and functions (`total_marks`, `read_input()`), **UPPER_CASE** for macros and constants (`MAX_SIZE`).

Valid	Invalid	Why invalid
`age`	`2age`	starts with a digit
`student_name`	`student-name`	`-` is not allowed
`_count`	`int`	keyword
`marks2025`	`total marks`	space not allowed

2.7 Comments

2.7.1 Single-line // (C99)

Everything from `//` to the end of the line is ignored. Added in C99 (borrowed from C++).

2.7.2 Multi-line /* */

Everything between `/*` and `*/` is ignored, across any number of lines. This form cannot be nested: a `*/` inside ends the comment early.

BOTH COMMENT STYLES

main.c

```c
#include <stdio.h>

/*
 * Multi-line comment:
 * explains the purpose of the program.
 */
int main(void) {
    int marks = 87;          // single-line comment after code
    // printf("this line is disabled\n");
    printf("Marks: %d\n", marks /* comments can even sit inside a line */);
    return 0;
}
```

OUTPUT

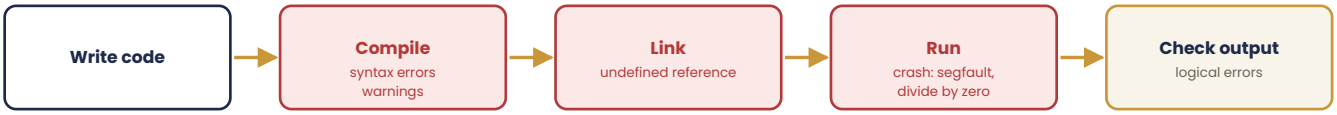
```
Marks: 87
```

TIP

Good comments explain **why**, not what the code obviously does. `i++; // increment i` adds nothing; `// skip the header row` does.

2.8 Errors

Errors are part of programming. Learning to read the compiler's messages is a key skill. Errors come in four kinds, found at different times.



WHERE EACH KIND OF ERROR IS CAUGHT

2.8.1 Syntax (compile-time) errors

Code that breaks the grammar of C: a missing semicolon, an unmatched brace, a misspelt keyword, an undeclared variable. The compiler reports the file, **line and column**, and stops. Fix the **first** error first; later errors are often consequences of it.

MISSING SEMICOLON

main.c

```
#include <stdio.h>

int main(void) {
    int count = 10
    printf("%d\n", count);
    return 0;
}
```

COMPILER OUTPUT

```
main.c: In function ‘main’:
main.c:5:5: error: expected ‘,’ or ‘;’ before ‘printf’
    5 |     printf("%d\n", count);
      |     ^~~~~~
main.c:4:9: warning: unused variable ‘count’ [-Wunused-variable]
    4 |     int count = 10
      |         ^~~~~
```

2.8.2 Linker errors (undefined reference)

The code compiles, but the linker cannot find a function or variable that was declared and used: a function prototype exists but no definition, a misspelt function name, or a missing library (`-lm`).

greet() was declared but never defined

```
$ gcc link.c -o link
/usr/bin/ld: /tmp/ccNplwaL.o: in function `main':
link.c:(.text+0x9): undefined reference to `greet'
collect2: error: ld returned 1 exit status
```

2.8.3 Runtime errors (segmentation fault, division by zero)

The program compiles and starts, then crashes while running. A **segmentation fault** means it accessed memory it does not own (a NULL or invalid pointer, an array far out of bounds). Integer division by zero kills the program with a floating point exception signal.

A SEGMENTATION FAULT

main.c

```
#include <stdio.h>

int main(void) {
    int *p = NULL;          /* points nowhere */
    printf("before\n");
    *p = 42;                /* writing through NULL: crash */
    printf("after\n");
    return 0;
}
```

OUTPUT

```
before
Segmentation fault (core dumped)
```

2.8.4 Logical errors

The program runs without crashing but gives the **wrong answer**. The compiler cannot help; you find them by testing with known inputs, printing intermediate values, or using a debugger.

A LOGICAL ERROR: INTEGER DIVISION

main.c

```c
#include <stdio.h>

int main(void) {
    int a = 7, b = 8;
    double avg_wrong = (a + b) / 2;      /* integer division happens first: 15 / 2 = 7 */
    double avg_right = (a + b) / 2.0;    /* one operand is double: 7.5 */
    printf("wrong average: %.1f\n", avg_wrong);
    printf("right average: %.1f\n", avg_right);
    return 0;
}
```

OUTPUT

```
wrong average: 7.0
right average: 7.5
```

2.8.5 Warnings and why to treat them seriously

Warnings mean "this is legal C, but probably a mistake". The program still compiles, which is exactly why they are dangerous. Always compile with `-Wall -Wextra` and fix every warning.

COMPILES, BUT GCC WARNS TWICE

main.c

```c
#include <stdio.h>

int main(void) {
    int x = 5;
    if (x = 10)                      /* assignment, not comparison! */
        printf("x is ten?\n");
    printf("x is now %d\n", x);
    int unused;
    return 0;
}
```

COMPILER WARNINGS, THEN OUTPUT

```
main.c: In function ‘main’:
main.c:5:9: warning: suggest parentheses around assignment used as truth value [-Wparentheses]
    5 |     if (x = 10)                      /* assignment, not comparison! */
      |         ^
main.c:8:9: warning: unused variable ‘unused’ [-Wunused-variable]
    8 |     int unused;
      |         ^~~~~~
$ ./prog
x is ten?
x is now 10
```

Make it a habit: `gcc -std=c17 -Wall -Wextra -g file.c -o file`. Read the first error, fix it, recompile. Zero warnings is the goal.

PRACTICE CHECKPOINT

Compile Hello World stage by stage with **-E**, **-S** and **-c**, open each output file, and explain what changed.

Sample answer (see the real output in 2.5.5):

1. `gcc -E hello.c -o hello.i`: the 7-line file grows to about 800 lines because the whole of `stdio.h` is pasted in; `GREETING` is replaced by `"Hello, World!"`; the comment is gone. It is still C.
2. `gcc -S hello.c -o hello.s`: the C code is now x86-64 assembly: the string is placed in a read-only data section (`.LC0`), and `main` sets up a stack frame, loads the string's address, calls `puts` (GCC optimised `printf("%s\n", …)`) and returns 0 in `eax`.
3. `gcc -c hello.c -o hello.o`: a binary object file. `nm hello.o` shows `main` as defined (T) and `puts` as undefined (U): it cannot run yet.
4. `gcc hello.o -o hello`: the linker joins the object file with the C library, resolving `puts`, and produces a runnable executable roughly ten times larger.

MODULE 03 · BEGINNER

Variables, data types and constants

Minimum duration **2 days** | Level **Beginner**

A variable is a named memory location that holds a value of a declared type. This module covers creating variables, every data type and its size, and constants.

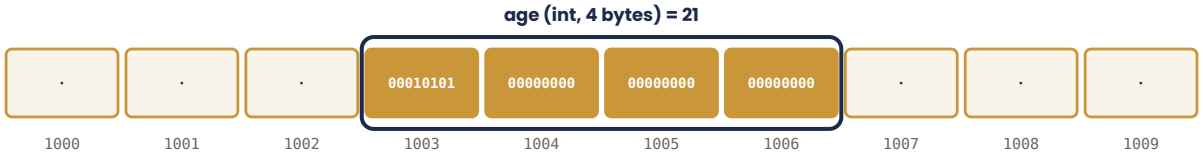
IN THIS MODULE

3.1 What is a variable?
3.2 Creating variables
3.3 Rules for naming variables
3.4 Types of variables (by scope)
3.5 Data types overview
3.6 Integer types
3.7 Floating-point types
3.8 Characters
3.9 Boolean values
3.10 Constants and literals
3.11 Type conversion

3.1 What is a variable?

3.1.1 Variable as a named memory location

Memory (RAM) is a long row of numbered **bytes**. A **variable** is a name you give to a group of those bytes so you can store a value there and read it back, without remembering the numeric address. When you write `int age = 21;`, the compiler reserves 4 bytes, remembers that `age` means those bytes, and stores 21 in them.



THE VARIABLE AGE OCCUPIES 4 BYTES STARTING AT ADDRESS 1003 (BYTE ORDER SHOWN LITTLE-ENDIAN)

3.1.2 Name, type, value and address

Property	Example for `int age = 21;`	How to see it
Name	`age`	In the source code
Type	`int` : how many bytes and how to interpret them	`sizeof(age)` → 4
Value	`21`	`printf("%d", age)`
Address	e.g. `0x7ffd5c3a1b2c`	`printf("%p", (void *)&age)`

THE FOUR PROPERTIES OF A VARIABLE

main.c

```c
#include <stdio.h>

int main(void) {
    int age = 21;
    printf("name   : age\n");
    printf("value  : %d\n", age);
    printf("size   : %zu bytes\n", sizeof(age));
    printf("address: %p\n", (void *)&age);    /* changes every run (address randomisation) */
    return 0;
}
```

OUTPUT (address differs on every run)

```
name   : age
value  : 21
size   : 4 bytes
address: 0x7fff251c71e4
```

3.1.3 Why C needs a declared type

Memory only stores bits. The **type** tells the compiler (1) how many bytes to reserve, (2) how to interpret the bits (the same 4 bytes mean something different as an `int` and as a `float`), and (3) which operations are allowed. Because types are fixed at compile time (**static typing**), the compiler can generate fast machine code and catch many mistakes before the program runs.

THE SAME BYTES, TWO INTERPRETATIONS

main.c

```c
#include <stdio.h>
#include <string.h>

int main(void) {
    float f = 1.0f;
    unsigned int bits;
    memcpy(&bits, &f, sizeof bits);          /* copy the same 4 bytes into an int */
    printf("as float       : %f\n", f);
    printf("same bits as int: %u (0x%08X)\n", bits, bits);
    return 0;
}
```

OUTPUT

```
as float       : 1.000000
same bits as int: 1065353216 (0x3F800000)
```

3.2 Creating variables

3.2.1 Declaration: int age;

A **declaration** introduces a name and its type: `type name;`. Since C99, declarations can appear anywhere in a block (C89 required them at the top).

3.2.2 Initialization: int age = 21;

Initialization gives a variable its first value at the moment it is created. **Assignment** (`age = 22;`) changes the value later.

3.2.3 Multiple declarations in one line

DECLARING AND INITIALIZING

main.c

```c
#include <stdio.h>

int main(void) {
    int age;                    /* declaration */
    age = 21;                   /* assignment */
    int marks = 87;             /* declaration + initialization */
    int x = 1, y = 2, z;        /* several in one line; z is not initialized */
    double price = 99.5, tax = 0.18;
    z = x + y;
    printf("age=%d marks=%d x=%d y=%d z=%d\n", age, marks, x, y, z);
    printf("price with tax = %.2f\n", price * (1 + tax));
    return 0;
}
```

OUTPUT

```
age=21 marks=87 x=1 y=2 z=3
price with tax = 117.41
```

WATCH OUT

`int a, b = 5;` initializes only **b**. `a` is left uninitialized.

3.2.4 Garbage values in uninitialized local variables

A local variable that is declared but not initialized contains whatever bits were left in that memory by earlier code: a **garbage value**. Reading it is undefined behaviour. Global and `static` variables, in contrast, are automatically set to zero.

GARBAGE IN AN UNINITIALIZED LOCAL

main.c

```c
#include <stdio.h>

int global_count;                   /* globals are zero-initialized */

void dirty_the_stack(void) { volatile int junk = 12345; (void)junk; }
void show(void) {
    int local;                      /* NOT initialized: garbage */
    printf("uninitialized local : %d (garbage: could be anything)\n", local);
}
int main(void) {
    printf("global              : %d\n", global_count);
    dirty_the_stack();
    show();
    return 0;
}
```

OUTPUT

```
global              : 0
uninitialized local : 12345 (garbage: could be anything)
```

TIP

Always initialize variables when you declare them. `-Wall` warns about many uninitialized uses.

3.2.5 Declaration vs definition

A **declaration** tells the compiler that a name exists and what its type is. A **definition** also creates it: reserves memory for a variable or provides the body of a function. A variable may be declared many times (e.g. with `extern` in several files) but defined only **once**.

	Declaration only	Definition (also a declaration)
Variable	`extern int count;`	`int count;` / `int count = 0;`
Function	`int square(int n);` (prototype)	`int square(int n) { return n * n; }`
Memory reserved?	No	Yes
How many times?	Any number	Exactly once in the program

3.3 Rules for naming variables

3.3.1 Allowed characters (letters, digits, underscore)

3.3.2 Cannot start with a digit or be a keyword

Identifiers may contain letters, digits and underscores, must start with a letter or underscore, cannot be a keyword, and are case-sensitive (2.6.6). Very long names are allowed; at least the first 63 characters are significant.

Name	Valid?	Reason
`total_marks`	✓	
`_temp`	✓	(but leading underscores are best avoided)
`roll2`	✓	
`2roll`	✗	starts with a digit
`roll no`	✗	space
`float`	✗	keyword
`price$`	✗	$ is not standard (GCC allows it as an extension)

3.3.3 Meaningful names and conventions

- Use names that say what the value means: `total_marks`, not `tm` or `x`.
- Single letters are fine for loop counters (`i`, `j`) and coordinates (`x`, `y`).
- C style: `snake_case` for variables and functions, `UPPER_CASE` for macros/constants, often `PascalCase` or `snake_case_t` for typedef names.
- Booleans read as questions: `is_valid`, `has_paid`.

3.4 Types of variables (by scope)

Scope is the part of the program where a name can be used. Module 10 covers scope, lifetime and linkage in depth; here are the four kinds you meet first.

3.4.1 Local variables

Declared inside a function or block; usable only inside that block; created when the block starts and destroyed when it ends; not initialized automatically.

3.4.2 Global variables

Declared outside all functions; usable by every function in the file (and other files via `extern`); exist for the whole program run; zero-initialized. Use them sparingly: any function can change them, which makes bugs hard to trace.

3.4.3 Static variables (introduced here, detailed in Module 10)

A `static` local variable keeps its value between function calls: it is created once and lives for the whole program, but is visible only inside its function.

3.4.4 Formal parameters

The parameters in a function definition behave like local variables initialized with the values passed by the caller.

LOCAL, GLOBAL, STATIC AND PARAMETER VARIABLES main.c

```c
#include <stdio.h>

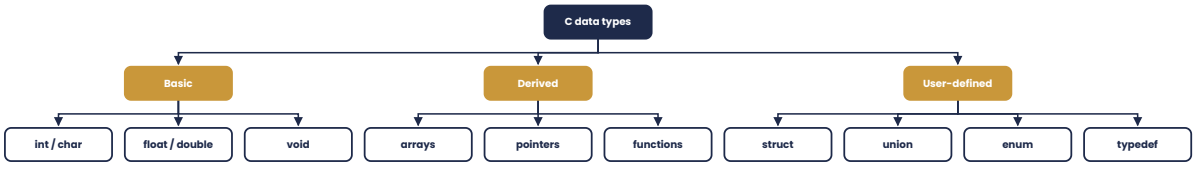
int calls_total = 0;                /* global */

int add(int a, int b) {             /* a, b: formal parameters */
    static int my_calls = 0;        /* static local: remembers between calls */
    int result = a + b;             /* local */
    my_calls++;
    calls_total++;
    printf("  add(%d, %d) call #%d\n", a, b, my_calls);
    return result;
}
int main(void) {
    int x = add(2, 3);              /* x: local to main */
    int y = add(10, 20);
    printf("x=%d y=%d, total calls=%d\n", x, y, calls_total);
    return 0;
}
```

OUTPUT

```
  add(2, 3) call #1
  add(10, 20) call #2
x=5 y=30, total calls=2
```

3.5 Data types overview



CLASSIFICATION OF C DATA TYPES

3.5.1 Basic (primary) types: int, char, float, double, void

Type	Typical size	Holds	Example
`char`	1 byte	A character or small integer	`char grade = 'A';`
`int`	4 bytes	Whole numbers	`int age = 21;`
`float`	4 bytes	Decimal numbers, ~7 digits precision	`float pi = 3.14f;`
`double`	8 bytes	Decimal numbers, ~15 digits precision	`double g = 9.80665;`
`void`	-	"No type": functions returning nothing, generic pointers	`void print(void);`

3.5.2 Derived types: arrays, pointers, functions

Built from basic types: an **array** is a sequence of elements of one type (`int marks[5]`, Module 7), a **pointer** holds an address (`int *p`, Module 11), and a **function** type describes parameters and return type (Module 9).

3.5.3 User-defined types: struct, union, enum, typedef

Types you create: `struct` groups different fields into one record, `union` overlaps fields in the same memory, `enum` names a set of integer constants, and `typedef` gives a type a new name (all in Module 14).

3.6 Integer types

3.6.1 char (1 byte), short, int, long, long long

3.6.2 Type modifiers: signed, unsigned, short, long

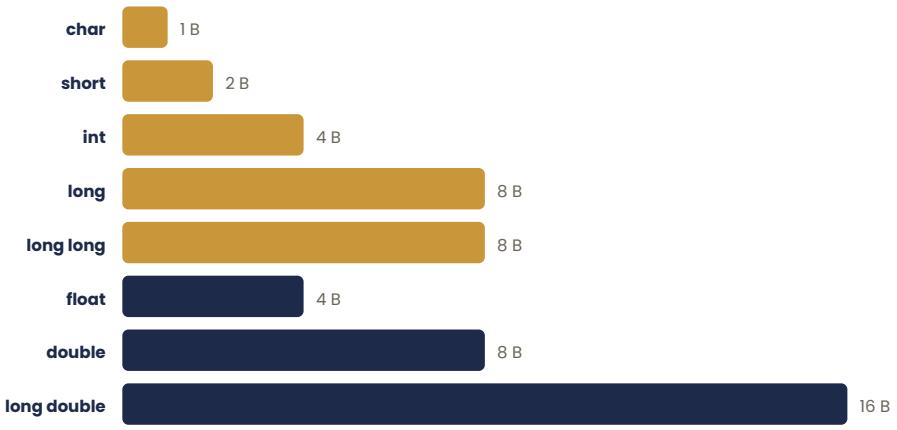
Modifiers adjust the basic types. `short` and `long` change the size; `signed` (the default for int) allows negatives; `unsigned` allows only zero and positives, doubling the maximum value. `short` = `short int`, `long` = `long int`, `unsigned` alone = `unsigned int`.

3.6.3 Ranges and why sizes depend on the platform

The C standard fixes only **minimum** sizes and an order (`char ≤ short ≤ int ≤ long ≤ long long`), so that C can run efficiently on any CPU. On today's 64-bit Linux and macOS, `long` is 8 bytes; on 64-bit Windows it is only 4 bytes. That is why portable code uses fixed-width types (3.6.6) when the exact size matters.

Type	Bytes (64-bit Linux)	Range	printf
`char` / `signed char`	1	-128 … 127	`%c` / `%hhd`
`unsigned char`	1	0 … 255	`%hhu`
`short`	2	-32,768 … 32,767	`%hd`
`unsigned short`	2	0 … 65,535	`%hu`
`int`	4	-2,147,483,648 … 2,147,483,647	`%d`
`unsigned int`	4	0 … 4,294,967,295	`%u`

Type	Bytes (64-bit Linux)	Range	printf
`long`	8 (4 on Windows)	$\pm 9.22 \times 10^{18}$	`%ld`
`unsigned long`	8 (4 on Windows)	$0 \ldots 1.84 \times 10^{19}$	`%lu`
`long long`	8	−9,223,372,036,854,775,808 … 9,223,372,036,854,775,807	`%lld`
`unsigned long long`	8	0 … 18,446,744,073,709,551,615	`%llu`



SIZES ON A 64-BIT LINUX SYSTEM (INTEGER TYPES GOLD, FLOATING TYPES NAVY)

3.6.4 sizeof operator

`sizeof` gives the size in bytes of a type or an expression, computed at compile time. Its result has type `size_t`, printed with `%zu`. Parentheses are required for a type name (`sizeof(int)`) but optional for a variable (`sizeof x`).

3.6.5 limits.h constants (INT_MAX, INT_MIN, CHAR_BIT)

SIZES AND RANGES FROM LIMITS.H

main.c

```c
#include <stdio.h>
#include <limits.h>

int main(void) {
    printf("CHAR_BIT  = %d bits per byte\n", CHAR_BIT);
    printf("char      : %zu byte,  %d to %d\n", sizeof(char), CHAR_MIN, CHAR_MAX);
    printf("short     : %zu bytes, %d to %d\n", sizeof(short), SHRT_MIN, SHRT_MAX);
    printf("int       : %zu bytes, %d to %d\n", sizeof(int), INT_MIN, INT_MAX);
    printf("unsigned  : %zu bytes, 0 to %u\n", sizeof(unsigned), UINT_MAX);
    printf("long      : %zu bytes, %ld to %ld\n", sizeof(long), LONG_MIN, LONG_MAX);
    printf("long long : %zu bytes, %lld to %lld\n", sizeof(long long), LLONG_MIN, LLONG_MAX);
    printf("ull max   : %llu\n", ULLONG_MAX);
    return 0;
}
```

OUTPUT

```
CHAR_BIT  = 8 bits per byte
char      : 1 byte,  -128 to 127
short     : 2 bytes, -32768 to 32767
int       : 4 bytes, -2147483648 to 2147483647
unsigned  : 4 bytes, 0 to 4294967295
long      : 8 bytes, -9223372036854775808 to 9223372036854775807
long long : 8 bytes, -9223372036854775808 to 9223372036854775807
ull max   : 18446744073709551615
```

3.6.6 Fixed-width types in stdint.h (int8_t, int32_t, uint64_t)

C99's `<stdint.h>` provides integer types with an **exact** width on every platform: `int8_t`, `int16_t`, `int32_t`, `int64_t` and the unsigned `uint8_t … uint64_t`. Use them for file formats, network protocols, hardware registers and any data whose size must not change. `<inttypes.h>` provides matching printf macros such as `PRId64`.

EXACT-WIDTH INTEGERS

main.c

```c
#include <stdio.h>
#include <stdint.h>
#include <inttypes.h>

int main(void) {
    int8_t   tiny   = -100;
    uint8_t  pixel  = 255;                      /* one colour channel */
    int32_t  salary = 2000000000;
    uint64_t bytes  = 18000000000000000000u;    /* bigger than any int64_t */
    printf("int8_t   %" PRId8  " (%zu byte)\n", tiny, sizeof tiny);
    printf("uint8_t  %" PRIu8  " (%zu byte)\n", pixel, sizeof pixel);
    printf("int32_t  %" PRId32 " (%zu bytes)\n", salary, sizeof salary);
    printf("uint64_t %" PRIu64 " (%zu bytes)\n", bytes, sizeof bytes);
    printf("INT32_MAX = %" PRId32 ", UINT8_MAX = %d\n", INT32_MAX, UINT8_MAX);
    return 0;
}
```

OUTPUT

```
int8_t   -100 (1 byte)
uint8_t  255 (1 byte)
int32_t  2000000000 (4 bytes)
uint64_t 18000000000000000000 (8 bytes)
INT32_MAX = 2147483647, UINT8_MAX = 255
```

3.6.7 size_t

`size_t` is an **unsigned** integer type big enough to hold the size of any object in memory (8 bytes on 64-bit systems). It is the type of `sizeof`, of `strlen()`'s result and of array indexes in the standard library. Print it with `%zu`. Because it is unsigned, beware of loops counting down past zero (3.11.5).

3.7 Floating-point types

3.7.1 float, double, long double

Type	Size	Significant digits	Range (approx.)	printf / scanf	Literal
`float`	4 bytes	6–7	$\pm 3.4 \times 10^{38}$	`%f` / `%f`	`3.14f`
`double`	8 bytes	15–16	$\pm 1.8 \times 10^{308}$	`%f` / `%lf`	`3.14`
`long double`	16 bytes (x86-64 Linux; 8 on MSVC)	18–19	$\pm 1.2 \times 10^{4932}$	`%Lf`	`3.14L`

`double` is the default for decimal literals and maths functions; use it unless memory is tight (large arrays, embedded devices), where `float` saves half the space.

3.7.2 Precision and float.h

HOW MANY DIGITS ARE REALLY STORED?

main.c

```c
#include <stdio.h>
#include <float.h>

int main(void) {
    float  f = 1.0f / 3.0f;
    double d = 1.0 / 3.0;
    long double ld = 1.0L / 3.0L;
    printf("float       : %.20f  (FLT_DIG = %d)\n", f, FLT_DIG);
    printf("double      : %.20f  (DBL_DIG = %d)\n", d, DBL_DIG);
    printf("long double : %.20Lf  (LDBL_DIG = %d)\n", ld, LDBL_DIG);
    printf("FLT_MAX = %e, DBL_MAX = %e\n", FLT_MAX, DBL_MAX);
    printf("DBL_EPSILON = %e (smallest x with 1 + x != 1)\n", DBL_EPSILON);
    return 0;
}
```

OUTPUT

```
float       : 0.33333334326744079590  (FLT_DIG = 6)
double      : 0.33333333333333331483  (DBL_DIG = 15)
long double : 0.33333333333333333334  (LDBL_DIG = 18)
FLT_MAX = 3.402823e+38, DBL_MAX = 1.797693e+308
DBL_EPSILON = 2.220446e-16 (smallest x with 1 + x != 1)
```

3.7.3 IEEE 754 representation (overview)

Almost every computer stores floating-point numbers in the **IEEE 754** format, a binary version of scientific notation: value = $(-1)^{sign} \times 1.mantissa \times 2^{exponent\ -\ bias}$. A `float` uses 1 sign bit, 8 exponent bits and 23 mantissa (fraction) bits; a `double` uses 1, 11 and 52.



float 5.75 = 101.11_2 = 1.0111×2^2 → 0 10000001 01110000000000000000000 = 0x40B80000

IEEE 754 SINGLE PRECISION (FLOAT) LAYOUT

LOOKING INSIDE A FLOAT

main.c

```c
#include <stdio.h>
#include <string.h>
#include <stdint.h>

int main(void) {
    float f = 5.75f;
    uint32_t bits;
    memcpy(&bits, &f, 4);
    printf("5.75 as hex : 0x%08X\n", bits);
    printf("sign        : %u\n", bits >> 31);
    printf("exponent    : %u (2^%d)\n", (bits >> 23) & 0xFF, (int)((bits >> 23) & 0xFF) - 127);
    printf("mantissa    : 0x%06X\n", bits & 0x7FFFFF);
    return 0;
}
```

OUTPUT

```
5.75 as hex : 0x40B80000
sign        : 0
exponent    : 129 (2^2)
mantissa    : 0x380000
```

3.7.4 Precision problems when comparing floats

Most decimal fractions (0.1, 0.2) cannot be represented exactly in binary, just as 1/3 cannot be written exactly in decimal. Tiny rounding errors appear, so **never compare floating-point values with ==**. Compare the difference against a small tolerance (epsilon) instead.

WHY == FAILS FOR FLOATING POINT

main.c

```c
#include <stdio.h>
#include <math.h>

int main(void) {
    double a = 0.1 + 0.2;
    printf("0.1 + 0.2 = %.17f\n", a);
    printf("a == 0.3 ? %s\n", a == 0.3 ? "yes" : "no");
    printf("|a - 0.3| < 1e-9 ? %s\n", fabs(a - 0.3) < 1e-9 ? "yes" : "no");

    float sum = 0.0f;
    for (int i = 0; i < 10; i++) sum += 0.1f;
    printf("0.1f added 10 times = %.7f\n", sum);
    return 0;
}
```

OUTPUT

```
0.1 + 0.2 = 0.30000000000000004
a == 0.3 ? no
|a - 0.3| < 1e-9 ? yes
0.1f added 10 times = 1.0000001
```

WATCH OUT

Never use `float` / `double` for money. Store amounts as integers in the smallest unit (paise, cents): `long long paise = 199999;`.

3.8 Characters

3.8.1 char as a small integer

A `char` is a 1-byte integer. A character literal such as `'A'` is simply the number 65. Printing with `%c` shows the character; with `%d` the number. You can do arithmetic on characters.

CHARACTERS ARE NUMBERS main.c

```c
#include <stdio.h>

int main(void) {
    char c = 'A';
    printf("%c has code %d\n", c, c);
    printf("next letter: %c\n", c + 1);
    printf("lowercase  : %c (difference %d)\n", c + 32, 'a' - 'A');
    char digit = '7';
    printf("'7' as a number: %d\n", digit - '0');
    for (char ch = 'a'; ch <= 'e'; ch++) printf("%c=%d ", ch, ch);
    printf("\n");
    return 0;
}
```

OUTPUT

```
A has code 65
next letter: B
lowercase  : a (difference 32)
'7' as a number: 7
a=97 b=98 c=99 d=100 e=101
```

3.8.2 ASCII table and character codes

ASCII (American Standard Code for Information Interchange) assigns codes 0–127 to characters. Codes 0–31 are control characters (newline is 10, tab is 9), 32 is space, digits `'0'–'9'` are 48–57, uppercase `'A'–'Z'` 65–90, lowercase `'a'–'z'` 97–122. Letters and digits are consecutive, which makes tricks like `ch - '0'` work.

Code	Char	Code	Char	Code	Char	Code	Char
9	\t (tab)	48	0	65	A	97	a
10	\n (newline)	49	1	66	B	98	b
13	\r (return)	57	9	90	Z	122	z
32	(space)	33	!	64	@	126	~
0	\0 (null)	35	#	95	_	127	DEL

PRINTING THE ASCII TABLE

main.c

```c
#include <stdio.h>

int main(void) {
    printf("Printable ASCII table (32-126):\n");
    for (int code = 32; code <= 126; code++) {
        printf("%3d:%c  ", code, code);
        if ((code - 31) % 10 == 0) printf("\n");
    }
    printf("\n");
    return 0;
}
```

OUTPUT

```
Printable ASCII table (32-126):
 32:    33:!   34:"   35:#   36:$   37:%   38:&   39:'   40:(   41:)
 42:*   43:+   44:,   45:-   46:.   47:/   48:0   49:1   50:2   51:3
 52:4   53:5   54:6   55:7   56:8   57:9   58::   59:;   60:<   61:=
 62:>   63:?   64:@   65:A   66:B   67:C   68:D   69:E   70:F   71:G
 72:H   73:I   74:J   75:K   76:L   77:M   78:N   79:O   80:P   81:Q
 82:R   83:S   84:T   85:U   86:V   87:W   88:X   89:Y   90:Z   91:[
 92:\   93:]   94:^   95:_   96:`   97:a   98:b   99:c  100:d  101:e
102:f  103:g  104:h  105:i  106:j  107:k  108:l  109:m  110:n  111:o
112:p  113:q  114:r  115:s  116:t  117:u  118:v  119:w  120:x  121:y
122:z  123:{  124:|  125:}  126:~
```

3.8.3 Escape sequences (\n, \t, \\, \', \", \0)

Escape	Meaning	ASCII
`\n`	Newline	10
`\t`	Horizontal tab	9
`\\`	Backslash	92
`\'`	Single quote	39
`\"`	Double quote	34
`\0`	Null character (ends strings)	0
`\r`	Carriage return	13
`\a`	Alert (beep)	7
`\b`	Backspace	8
`\x41`, `\101`	Character by hex / octal code (both 'A')	65

ESCAPE SEQUENCES IN ACTION

main.c

```c
#include <stdio.h>

int main(void) {
    printf("Name\tMarks\n");
    printf("Asha\t91\n");
    printf("Path: C:\\Users\\Lohit\n");
    printf("She said \"Hello\" and it\'s fine\n");
    printf("Hex \\x41 = \x41, octal \\101 = \101\n");
    printf("%d\n", '\0');
    return 0;
}
```

OUTPUT

```
Name    Marks
Asha    91
Path: C:\Users\Lohit
She said "Hello" and it's fine
Hex \x41 = A, octal \101 = A
0
```

3.9 Boolean values

3.9.1 Zero is false, non-zero is true

Traditional C has no boolean type. Any integer can be used as a condition: **0 means false**, **any other value means true**. Comparison and logical operators produce `1` (true) or `0` (false).

3.9.2 _Bool and stdbool.h (C99)

C99 added the type `_Bool` (holds only 0 or 1; any non-zero value assigned becomes 1) and the header `<stdbool.h>`, which defines the friendlier names `bool`, `true` and `false`.

3.9.3 bool, true, false as keywords (C23)

In C23, `bool`, `true` and `false` are real keywords, so `<stdbool.h>` is no longer needed (including it is harmless).

TRUTH VALUES IN C main.c

```c
#include <stdio.h>
#include <stdbool.h>

int main(void) {
    int x = 5;
    printf("x > 3  -> %d\n", x > 3);
    printf("x == 3 -> %d\n", x == 3);
    if (42) printf("42 counts as true\n");
    if (!0) printf("0 counts as false, so !0 is true\n");

    bool passed = true;
    _Bool b = 7;                        /* any non-zero becomes 1 */
    printf("passed = %d, b = %d, sizeof(bool) = %zu\n", passed, b, sizeof(bool));
    bool is_even = (x % 2 == 0);
    printf("%d is %s\n", x, is_even ? "even" : "odd");
    return 0;
}
```

OUTPUT

```
x > 3  -> 1
x == 3 -> 0
42 counts as true
0 counts as false, so !0 is true
passed = 1, b = 1, sizeof(bool) = 1
5 is odd
```

3.10 Constants and literals

3.10.1 Integer literals: decimal, octal 0, hexadecimal 0x, binary 0b (C23)

FOUR WAYS TO WRITE 255 main.c

```c
#include <stdio.h>

int main(void) {
    int dec = 255;
    int oct = 0377;          /* leading 0 means octal! */
    int hex = 0xFF;
    int bin = 0b11111111;    /* C23 (GCC has long supported it as an extension) */
    printf("%d %d %d %d\n", dec, oct, hex, bin);
    printf("255 in octal: %o, hex: %x / %X, with prefix: %#x\n", dec, dec, dec, dec);
    int surprise = 010;      /* not ten! */
    printf("010 = %d\n", surprise);
    return 0;
}
```

OUTPUT

```
255 255 255 255
255 in octal: 377, hex: ff / FF, with prefix: 0xff
010 = 8
```

WATCH OUT

A leading zero makes a number **octal**: `int pin = 0123;` is 83, not 123.

3.10.2 Suffixes: U, L, LL, F

Suffix	Type	Example
none (integer)	int (or larger if it does not fit)	`42`
`U`	unsigned	`42U`
`L`	long	`42L`
`LL`	long long	`9000000000LL`
`UL`, `ULL`	unsigned long / long long	`42ULL`
none (decimal)	double	`3.14`
`F`	float	`3.14F`
`L` (decimal)	long double	`3.14L`
`e`	exponent (scientific)	`6.022e23`, `1.5e-3`

3.10.3 Character and string literals

A **character literal** uses single quotes and is one character: `'A'`, `'\n'`. A **string literal** uses double quotes and is an array of characters automatically ended by `'\0'`: `"A"` occupies **2** bytes. Adjacent string literals are joined by the compiler: `"Learn " "Being"` is `"Learn Being"`.

CHARACTER VS STRING LITERALS main.c

```c
#include <stdio.h>

int main(void) {
    printf("sizeof('A')  = %zu (a char literal has type int in C)\n", sizeof('A'));
    printf("sizeof(\"A\")  = %zu ('A' plus '\\0')\n", sizeof("A"));
    printf("sizeof(\"Hi!\") = %zu\n", sizeof("Hi!"));
    printf("Learn " "Being " "Forward\n");    /* adjacent literals are concatenated */
    return 0;
}
```

```
OUTPUT
sizeof('A')  = 4 (a char literal has type int in C)
sizeof("A")  = 2 ('A' plus '\0')
sizeof("Hi!") = 4
Learn Being Forward
```

3.10.4 const qualifier

`const` makes a variable **read-only** after initialization. The compiler rejects any attempt to modify it. A const variable has a type, a scope and an address, and debuggers can show it.

MODIFYING A CONST IS A COMPILE ERROR main.c

```c
#include <stdio.h>

int main(void) {
    const double GST_RATE = 0.18;
    const int MAX_STUDENTS = 60;
    double price = 1000;
    printf("GST on %.0f = %.2f, max students = %d\n", price, price * GST_RATE, MAX_STUDENTS);
    GST_RATE = 0.28;            /* error */
    return 0;
}
```

COMPILER OUTPUT

```
main.c: In function ‘main’:
main.c:8:14: error: assignment of read-only variable ‘GST_RATE’
    8 |     GST_RATE = 0.28;            /* error */
      |              ^
```

3.10.5 #define symbolic constants

`#define NAME value` tells the preprocessor to replace every occurrence of `NAME` with `value` before compilation. No semicolon, no type, no memory: pure text substitution. By convention the names are UPPER_CASE.

SYMBOLIC CONSTANTS WITH #DEFINE main.c

```c
#include <stdio.h>
#define PI 3.14159265
#define MAX_MARKS 100
#define COLLEGE "Learn Being Forward"

int main(void) {
    double r = 3.0;
    printf("%s\n", COLLEGE);
    printf("Area = %.3f, out of %d marks\n", PI * r * r, MAX_MARKS);
    return 0;
}
```

OUTPUT

```
Learn Being Forward
Area = 28.274, out of 100 marks
```

3.10.6 const vs #define

	`const` variable	`#define` macro
Handled by	Compiler	Preprocessor (text replacement)
Type	Yes: type-checked	No type
Scope	Normal block/file scope	From the definition to the end of the file
Memory/address	Usually has an address	None
Visible in debugger	Yes	No (already replaced)
Usable as array size in a global array / case label	No in C (yes in C23 with `constexpr`)	Yes

	`const` variable	`#define` macro
Typical use	Typed constants inside functions	Array sizes, configuration, conditional compilation

3.10.7 enum constants (introduced; detailed in Module 14)

An `enum` defines a set of named integer constants, numbered 0, 1, 2… by default. They are true compile-time constants, have names that debuggers show, and group related values.

ENUM CONSTANTS main.c

```c
#include <stdio.h>

enum weekday { MON, TUE, WED, THU, FRI, SAT, SUN };
enum { BOARD_SIZE = 8 };            /* a handy way to make an integer constant */

int main(void) {
    enum weekday today = WED;
    printf("WED = %d, SUN = %d\n", today, SUN);
    int board[BOARD_SIZE][BOARD_SIZE] = {0};
    printf("board has %zu cells\n", sizeof board / sizeof board[0][0]);
    return 0;
}
```

OUTPUT

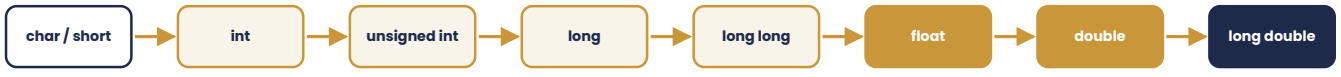
```
WED = 2, SUN = 6
board has 64 cells
```

3.11 Type conversion

When an expression mixes types, or a value is stored into a variable of another type, C converts values. Some conversions are automatic (**implicit**); others you request with a **cast**.

3.11.1 Implicit conversion and the usual arithmetic conversions

In an arithmetic expression with two different types, the "smaller" operand is converted to the "larger" type before the operation (the **usual arithmetic conversions**). In assignments, the value is converted to the type of the variable, possibly losing information.



CONVERSION RANKING: THE OPERAND OF LOWER RANK IS CONVERTED TO THE HIGHER ONE

3.11.2 Integer promotion

Before any arithmetic, values of types smaller than `int` (`char`, `short`, `_Bool`) are **promoted** to `int`. So `char + char` is computed as `int`.

PROMOTION AND IMPLICIT CONVERSION

main.c

```c
#include <stdio.h>

int main(void) {
    char a = 100, b = 100;
    int sum = a + b;                    /* promoted to int: 200, no overflow */
    char c = a + b;                     /* 200 does not fit in signed char */
    printf("int sum = %d, char sum = %d\n", sum, c);
    printf("sizeof(a + b) = %zu (int)\n", sizeof(a + b));

    int i = 7;
    double d = 2.0;
    printf("7 / 2   = %d (int / int)\n", i / 2);
    printf("7 / 2.0 = %.2f (int converted to double)\n", i / d);
    int truncated = 9.99;               /* double -> int: fraction dropped */
    printf("int x = 9.99 -> %d\n", truncated);
    return 0;
}
```

OUTPUT

```
int sum = 200, char sum = -56
sizeof(a + b) = 4 (int)
7 / 2   = 3 (int / int)
7 / 2.0 = 3.50 (int converted to double)
int x = 9.99 -> 9
```

3.11.3 Explicit casting (type) value

A **cast** `(type) expression` converts a value explicitly. Use it to force floating-point division, to make an intended narrowing conversion visible, or to convert pointers.

CASTING

main.c

```c
#include <stdio.h>

int main(void) {
    int total = 425, count = 6;
    printf("without cast: %d\n", total / count);
    printf("with cast   : %.2f\n", (double)total / count);       /* cast one operand */
    printf("wrong cast  : %.2f\n", (double)(total / count));     /* cast after integer division! */
    double price = 249.75;
    int rupees = (int)price;                                     /* truncation toward zero */
    int paise = (int)((price - rupees) * 100 + 0.5);
    printf("%.2f -> %d rupees %d paise\n", price, rupees, paise);
    printf("(int)-3.7 = %d, (char)65 = %c\n", (int)-3.7, (char)65);
    return 0;
}
```

OUTPUT

```
without cast: 70
with cast   : 70.83
wrong cast  : 70.00
249.75 -> 249 rupees 75 paise
(int)-3.7 = -3, (char)65 = A
```

3.11.4 Overflow, truncation and sign problems

Unsigned overflow is well defined: values wrap around modulo 2^n (UINT_MAX + 1 becomes 0). **Signed** overflow is **undefined behaviour**: the program may give a wrong value or the optimiser may assume it

never happens. Converting a value that does not fit into a smaller type truncates high bits; converting a negative number to unsigned produces a huge positive value.

WRAP-AROUND AND TRUNCATION

main.c

```c
#include <stdio.h>
#include <limits.h>

int main(void) {
    unsigned int u = UINT_MAX;
    u = u + 1;                              /* well-defined wrap-around */
    printf("UINT_MAX + 1 = %u\n", u);

    unsigned char small = 300;              /* 300 % 256 = 44 */
    printf("unsigned char = 300 -> %d\n", small);

    int neg = -1;
    unsigned int big = neg;                 /* -1 becomes UINT_MAX */
    printf("(unsigned)-1 = %u\n", big);

    long long safe = (long long)INT_MAX + 1;   /* widen BEFORE adding */
    printf("INT_MAX + 1 computed in long long = %lld\n", safe);
    int years = 70, secs_per_year = 31536000;
    long long life = (long long)years * secs_per_year;
    printf("seconds in 70 years = %lld (int would overflow)\n", life);
    return 0;
}
```

OUTPUT

```
UINT_MAX + 1 = 0
unsigned char = 300 -> 44
(unsigned)-1 = 4294967295
INT_MAX + 1 computed in long long = 2147483648
seconds in 70 years = 2207520000 (int would overflow)
```

3.11.5 Signed vs unsigned comparison pitfalls

When a signed and an unsigned value of the same rank are compared, the signed one is converted to unsigned. So `-1 < 1u` is **false**! This bites when comparing an `int` with `sizeof` or `strlen()` (both unsigned). `-Wextra` warns about it.

THE SIGNED/UNSIGNED TRAP

main.c

```c
#include <stdio.h>
#include <string.h>

int main(void) {
    int a = -1;
    unsigned int b = 1;
    if (a < b) printf("-1 < 1u\n");
    else       printf("surprise: -1 is NOT less than 1u (it became %u)\n", (unsigned)a);

    const char *s = "abc";
    int i = -1;
    if (i < (int)strlen(s)) printf("with a cast the comparison is correct\n");

    for (size_t k = 3; k-- > 0; )            /* safe countdown with an unsigned index */
        printf("k = %zu\n", k);
    return 0;
}
```

OUTPUT

```
surprise: -1 is NOT less than 1u (it became 4294967295)
with a cast the comparison is correct
k = 2
k = 1
k = 0
```

Choose the type that fits the data (int for counts, double for measurements, size_t for sizes, fixed-width types for exact layouts), initialize every variable, and make narrowing conversions explicit with casts.

PRACTICE CHECKPOINT

Print the size and range of every basic type on your machine using `sizeof` and `limits.h`.

SAMPLE SOLUTION

main.c

```c
#include <stdio.h>
#include <limits.h>
#include <float.h>
#include <stdbool.h>

int main(void) {
    printf("%-20s %6s  %-26s %s\n", "TYPE", "BYTES", "MINIMUM", "MAXIMUM");
    printf("%-20s %6zu  %-26d %d\n",   "char",               sizeof(char),               CHAR_MIN,
CHAR_MAX);
    printf("%-20s %6zu  %-26d %d\n",   "signed char",        sizeof(signed char),        SCHAR_MIN,
SCHAR_MAX);
    printf("%-20s %6zu  %-26d %u\n",   "unsigned char",      sizeof(unsigned char),      0,
UCHAR_MAX);
    printf("%-20s %6zu  %-26d %d\n",   "short",              sizeof(short),              SHRT_MIN,
SHRT_MAX);
    printf("%-20s %6zu  %-26d %u\n",   "unsigned short",     sizeof(unsigned short),     0,
USHRT_MAX);
    printf("%-20s %6zu  %-26d %d\n",   "int",                sizeof(int),                INT_MIN,
INT_MAX);
    printf("%-20s %6zu  %-26d %u\n",   "unsigned int",       sizeof(unsigned int),       0,
UINT_MAX);
    printf("%-20s %6zu  %-26ld %ld\n", "long",               sizeof(long),               LONG_MIN,
LONG_MAX);
    printf("%-20s %6zu  %-26d %lu\n",  "unsigned long",      sizeof(unsigned long),      0,
ULONG_MAX);
    printf("%-20s %6zu  %-26lld %lld\n", "long long",        sizeof(long long),          LLONG_MIN,
LLONG_MAX);
    printf("%-20s %6zu  %-26d %llu\n", "unsigned long long", sizeof(unsigned long long), 0,
ULLONG_MAX);
    printf("%-20s %6zu  %-26e %e\n",   "float",              sizeof(float),              FLT_MIN,
FLT_MAX);
    printf("%-20s %6zu  %-26e %e\n",   "double",             sizeof(double),             DBL_MIN,
DBL_MAX);
    printf("%-20s %6zu  %-26Le %Le\n", "long double",        sizeof(long double),        LDBL_MIN,
LDBL_MAX);
    printf("%-20s %6zu  %-26d %d\n",   "bool",               sizeof(bool),               false,
true);
    printf("%-20s %6zu\n",             "pointer (void *)",   sizeof(void *));
    return 0;
}
```

OUTPUT

```
TYPE                  BYTES  MINIMUM                    MAXIMUM
char                      1  -128                       127
signed char               1  -128                       127
unsigned char             1  0                          255
short                     2  -32768                     32767
unsigned short            2  0                          65535
int                       4  -2147483648                2147483647
unsigned int              4  0                          4294967295
long                      8  -9223372036854775808       9223372036854775807
unsigned long             8  0                          18446744073709551615
long long                 8  -9223372036854775808       9223372036854775807
unsigned long long        8  0                          18446744073709551615
float                     4  1.175494e-38                3.402823e+38
double                    8  2.225074e-308               1.797693e+308
long double              16  3.362103e-4932              1.189731e+4932
bool                      1  0                          1
pointer (void *)          8
```

MODULE 04 · BEGINNER

Operators and expressions

Minimum duration **2 days** | Level **Beginner**

Operators act on values to produce new values; this module covers every C operator group and the order of evaluation.

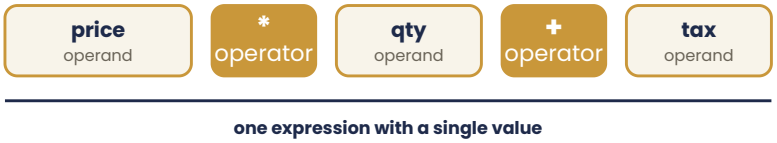
IN THIS MODULE

4.1 Basics
4.2 Arithmetic operators
4.3 Increment and decrement
4.4 Relational operators
4.5 Logical operators
4.6 Bitwise operators
4.7 Assignment operators
4.8 Conditional (ternary) operator
4.9 Special operators
4.10 Precedence and associativity
4.11 math.h library

4.1 Basics

4.1.1 Operators, operands and expressions

An **operator** is a symbol that performs an operation (`+`, `*`, `==`). The values it works on are **operands**. An **expression** is any combination of operands and operators that produces a value: `price * qty + tax`. Every expression in C has a value and a type, even an assignment (`x = 5` has the value 5).



OPERANDS AND OPERATORS FORM AN EXPRESSION

4.1.2 Unary, binary and ternary operators

Kind	Operands	Examples
Unary	1	`-x`, `!flag`, `++i`, `~bits`, `sizeof x`, `&x`, `*p`
Binary	2	`a + b`, `a < b`, `a && b`, `a = b`
Ternary	3	`cond ? a : b` (the only one)

4.2 Arithmetic operators

4.2.1 + - * / %

Operator	Meaning	Example (a = 17, b = 5)	Result
`+`	Addition	`a + b`	22
`-`	Subtraction	`a - b`	12
`*`	Multiplication	`a * b`	85
`/`	Division	`a / b`	3 (integer division)
`%`	Remainder (modulus), integers only	`a % b`	2
unary `-`, `+`	Negation / plus	`-a`	-17

ARITHMETIC OPERATORS

main.c

```c
#include <stdio.h>

int main(void) {
    int a = 17, b = 5;
    printf("a + b = %d\n", a + b);
    printf("a - b = %d\n", a - b);
    printf("a * b = %d\n", a * b);
    printf("a / b = %d\n", a / b);
    printf("a %% b = %d\n", a % b);           /* %% prints a percent sign */
    double x = 17, y = 5;
    printf("x / y = %.2f\n", x / y);
    int seconds = 7384;
    printf("%d s = %d h %d min %d s\n", seconds, seconds / 3600, seconds % 3600 / 60, seconds % 60);
    return 0;
}
```

OUTPUT

```
a + b = 22
a - b = 12
a * b = 85
a / b = 3
a % b = 2
x / y = 3.40
7384 s = 2 h 3 min 4 s
```

4.2.2 Integer division and truncation

When both operands are integers, `/` performs **integer division**: the result is an integer and any fraction is discarded, truncating **toward zero** (C99 and later). `7 / 2` is 3 and `-7 / 2` is -3. If either operand is floating point, real division is performed. Dividing an integer by zero is undefined behaviour (usually a crash).

4.2.3 Modulus with negative numbers

`%` gives the remainder, with the rule `(a / b) * b + a % b == a`. Because division truncates toward zero, the result of `%` has the **sign of the left operand**: `-7 % 3` is -1, `7 % -3` is 1. To get a mathematically positive remainder use `((a % n) + n) % n`. For floating-point remainders use `fmod()` (4.11).

DIVISION AND REMAINDER WITH NEGATIVES

main.c

```c
#include <stdio.h>

int main(void) {
    printf(" 7 /  2 = %2d    7 %%  2 = %2d\n",  7 / 2,  7 % 2);
    printf("-7 /  2 = %2d   -7 %%  2 = %2d\n", -7 / 2, -7 % 2);
    printf(" 7 / -2 = %2d    7 %% -2 = %2d\n",  7 / -2,  7 % -2);
    printf("-7 / -2 = %2d   -7 %% -2 = %2d\n", -7 / -2, -7 % -2);
    int day = -3, n = 7;
    printf("positive remainder of -3 mod 7 = %d\n", ((day % n) + n) % n);
    printf("is 2026 even? %s\n", 2026 % 2 == 0 ? "yes" : "no");
    printf("last digit of 9875 = %d\n", 9875 % 10);
    return 0;
}
```

OUTPUT

```
 7 /  2 =  3    7 %  2 =  1
-7 /  2 = -3   -7 %  2 = -1
 7 / -2 = -3    7 % -2 =  1
-7 / -2 =  3   -7 % -2 = -1
positive remainder of -3 mod 7 = 4
is 2026 even? yes
last digit of 9875 = 5
```

4.3 Increment and decrement

4.3.1 Pre-increment ++x and post-increment x++

4.3.2 Pre and post decrement

`++` adds 1 and `--` subtracts 1 from a variable. The **prefix** form (`++x`) changes the variable first and the expression's value is the **new** value. The **postfix** form (`x++`) uses the **old** value as the expression's value and then changes the variable. Used alone as a statement (`i++;`), both forms do the same thing.

PREFIX VS POSTFIX main.c

```c
#include <stdio.h>

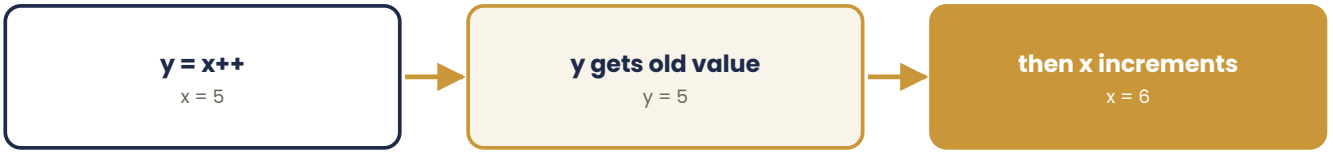
int main(void) {
    int x = 5, y;
    y = ++x;                    /* x becomes 6, then y = 6 */
    printf("y = ++x : x=%d y=%d\n", x, y);
    x = 5;
    y = x++;                    /* y = 5, then x becomes 6 */
    printf("y = x++ : x=%d y=%d\n", x, y);
    x = 5;
    y = --x;
    printf("y = --x : x=%d y=%d\n", x, y);
    x = 5;
    y = x--;
    printf("y = x-- : x=%d y=%d\n", x, y);
    int arr[] = {10, 20, 30}, i = 0;
    printf("arr[i++] = %d, then i = %d\n", arr[i++], i);
    return 0;
}
```

OUTPUT

```
y = ++x : x=6 y=6
y = x++ : x=6 y=5
y = --x : x=4 y=4
y = x-- : x=4 y=5
arr[i++] = 10, then i = 1
```

NOTE

The last line reads `i` in the same call where it is modified; the order of evaluating printf's arguments is unspecified, so another compiler might print a different `i`. Avoid this style (4.3.3).



POSTFIX: USE, THEN CHANGE. PREFIX: CHANGE, THEN USE

4.3.3 Undefined behaviour in expressions like i = i++ + ++i

If a variable is modified twice, or modified and read for another purpose, between two **sequence points** (4.10.3), the behaviour is **undefined**: the C standard places no requirements on the result, and different compilers (or optimisation levels) give different answers. Never write such code. GCC warns with `-Wall`.

GCC WARNS ABOUT UNDEFINED BEHAVIOUR

main.c

```c
#include <stdio.h>

int main(void) {
    int i = 5;
    i = i++ + ++i;              /* UNDEFINED: i modified twice without a sequence point */
    printf("i = %d (meaningless)\n", i);
    return 0;
}
```

COMPILER WARNING, THEN OUTPUT

```
main.c: In function ‘main’:
main.c:5:7: warning: operation on ‘i’ may be undefined [-Wsequence-point]
    5 |     i = i++ + ++i;              /* UNDEFINED: i modified twice without a sequence point */
      |     ~~^~~~~~~~~~
main.c:5:7: warning: operation on ‘i’ may be undefined [-Wsequence-point]
$ ./prog
i = 12 (meaningless)
```

WATCH OUT

Rule: in one expression, modify a variable at most once, and do not read it elsewhere in that expression.

4.4 Relational operators

4.4.1 == != > < >= <=

4.4.2 Result is 1 or 0

Relational operators compare two values and produce an `int`: **1** if the relation is true, **0** if false.

Operator	Meaning	7 op 3	3 op 3
`==`	equal to	0	1
`!=`	not equal to	1	0
`>`	greater than	1	0
`<`	less than	0	0
`>=`	greater than or equal	1	1
`<=`	less than or equal	0	1

RELATIONAL OPERATORS PRODUCE 0 OR 1

main.c

```c
#include <stdio.h>

int main(void) {
    int a = 7, b = 3;
    printf("a == b : %d\n", a == b);
    printf("a != b : %d\n", a != b);
    printf("a >  b : %d\n", a > b);
    printf("a <  b : %d\n", a < b);
    printf("a >= 7 : %d\n", a >= 7);
    int count = (a > 1) + (b > 1) + (0 > 1);     /* results are ordinary ints */
    printf("how many are > 1: %d\n", count);
    int x = 5;
    printf("1 < x < 3 is %d (!) because (1 < 5) is 1 and 1 < 3\n", 1 < x < 3);
    printf("correct: %d\n", 1 < x && x < 3);
    return 0;
}
```

OUTPUT

```
a == b : 0
a != b : 1
a >  b : 1
a <  b : 0
a >= 7 : 1
how many are > 1: 2
1 < x < 3 is 1 (!) because (1 < 5) is 1 and 1 < 3
correct: 0
```

4.5 Logical operators

4.5.1 &&, ||, !

Logical operators combine conditions: `&&` (AND) is true when both operands are true; `||` (OR) when at least one is; `!` (NOT) inverts. Any non-zero operand counts as true; the result is always 0 or 1.

4.5.2 Short-circuit evaluation

`&&` and `||` evaluate their left operand first and stop as soon as the result is known: for `A && B`, if A is false, B is **never evaluated**; for `A || B`, if A is true, B is skipped. This is guaranteed, and it lets you write safe guards such as `if (p != NULL && p->value > 0)` or `if (n != 0 && total / n > 50)`.

SHORT-CIRCUIT EVALUATION

main.c

```c
#include <stdio.h>

int check(const char *name, int value) {
    printf("  evaluating %s\n", name);
    return value;
}
int main(void) {
    printf("0 && B:\n");
    if (check("A", 0) && check("B", 1)) printf("  true\n"); else printf("  false (B skipped)\n");
    printf("1 || B:\n");
    if (check("A", 1) || check("B", 0)) printf("  true (B skipped)\n");
    int n = 0, total = 500;
    if (n != 0 && total / n > 50) printf("high average\n");
    else printf("safe: division never happened\n");
    printf("!0 = %d, !5 = %d, !!5 = %d\n", !0, !5, !!5);
    return 0;
}
```

OUTPUT

```
0 && B:
  evaluating A
  false (B skipped)
1 || B:
  evaluating A
  true (B skipped)
safe: division never happened
!0 = 1, !5 = 0, !!5 = 1
```

4.5.3 Truth tables

A	B	A && B	A || B	!A
0	0	0	0	1
0	1	0	1	1
1	0	0	1	0
1	1	1	1	0

COMBINING CONDITIONS

main.c

```c
#include <stdio.h>

int main(void) {
    int age = 22, marks = 78, has_id = 1;
    int eligible = (age >= 18 && age <= 25) && (marks >= 60 || has_id);
    printf("eligible for scholarship: %d\n", eligible);
    char c = 'g';
    printf("'%c' is a lowercase letter: %d\n", c, c >= 'a' && c <= 'z');
    int year = 2024;
    int leap = (year % 4 == 0 && year % 100 != 0) || year % 400 == 0;
    printf("%d leap year: %d\n", year, leap);
    return 0;
}
```

OUTPUT

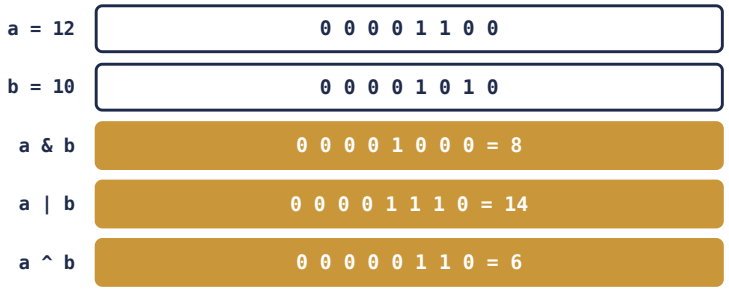
```
eligible for scholarship: 1
'g' is a lowercase letter: 1
2024 leap year: 1
```

4.6 Bitwise operators

Bitwise operators work on the individual **bits** of integers. They are essential for hardware registers, flags, compression, cryptography and fast arithmetic tricks. Module 17 goes much deeper.

4.6.1 &, |, ^, ~

Operator	Name	Rule per bit	12 op 10 (1100, 1010)
`&`	AND	1 if both bits are 1	8 (1000)
`|`	OR	1 if either bit is 1	14 (1110)
`^`	XOR	1 if the bits differ	6 (0110)
`~`	NOT (one's complement)	flips every bit	`~12` = -13



BITWISE AND, OR AND XOR, BIT BY BIT (8-BIT VIEW)

4.6.2 Left shift << and right shift >>

`x << n` shifts bits left by n places, filling with zeros: equivalent to multiplying by 2^n (if no bits fall off). `x >> n` shifts right: for unsigned values it divides by 2^n; for negative signed values the result is implementation-defined (usually an arithmetic shift that keeps the sign). Shifting by a negative amount or by the type's width or more is undefined.

BITWISE OPERATORS WITH A BIT PRINTER

main.c

```c
#include <stdio.h>

void print_bits(const char *label, unsigned int x) {
    printf("%-8s", label);
    for (int i = 7; i >= 0; i--) printf("%u", (x >> i) & 1u);
    printf(" = %u\n", x);
}
int main(void) {
    unsigned int a = 12, b = 10;
    print_bits("a", a);
    print_bits("b", b);
    print_bits("a & b", a & b);
    print_bits("a | b", a | b);
    print_bits("a ^ b", a ^ b);
    print_bits("a << 2", a << 2);         /* 12 * 4 = 48 */
    print_bits("a >> 2", a >> 2);         /* 12 / 4 = 3 */
    printf("~12 (signed int) = %d\n", ~12);
    printf("1 << 10 = %d\n", 1 << 10);
    return 0;
}
```

OUTPUT

```
a       00001100 = 12
b       00001010 = 10
a & b   00001000 = 8
a | b   00001110 = 14
a ^ b   00000110 = 6
a << 2  00110000 = 48
a >> 2  00000011 = 3
~12 (signed int) = -13
1 << 10 = 1024
```

4.6.3 Binary representation and two's complement

Unsigned integers are plain binary: bit k is worth 2^k. Signed integers on every modern machine use **two's complement** (made mandatory in C23): the top bit has a **negative** weight. To negate a number, invert all bits and add 1. That is why `~x == -x - 1`, and why an 8-bit signed value ranges from -128 to 127.

Value	8-bit two's complement	How
5	0000 0101	
-5	1111 1011	invert 0000 0101 → 1111 1010, add 1
127	0111 1111	largest
-128	1000 0000	smallest
-1	1111 1111	all ones

TWO'S COMPLEMENT IN PRACTICE

main.c

```c
#include <stdio.h>

void bits8(signed char v) {
    printf("%5d : ", v);
    for (int i = 7; i >= 0; i--) printf("%d", (v >> i) & 1);
    printf("\n");
}
int main(void) {
    bits8(5); bits8(-5); bits8(127); bits8(-128); bits8(-1);
    int x = 5;
    printf("~x + 1 = %d (equals -x)\n", ~x + 1);
    return 0;
}
```

OUTPUT

```
    5 : 00000101
   -5 : 11111011
  127 : 01111111
 -128 : 10000000
   -1 : 11111111
~x + 1 = -5 (equals -x)
```

4.7 Assignment operators

4.7.1 = and compound assignment += -= *= /= %= &= |= ^= <<= >>=

`=` stores the right-hand value in the left-hand variable (an **lvalue**: something with a storage location). An assignment is itself an expression whose value is the assigned value, so assignments can be chained: `a = b = c = 0;` (right to left). Compound operators combine an operation and assignment: `x += 5` means `x = x + 5`, but x is evaluated only once.

Operator	Equivalent	Operator	Equivalent
`x += y`	`x = x + y`	`x &= y`	`x = x & y`
`x -= y`	`x = x - y`	`x |= y`	`x = x | y`
`x *= y`	`x = x * y`	`x ^= y`	`x = x ^ y`
`x /= y`	`x = x / y`	`x <<= y`	`x = x << y`
`x %= y`	`x = x % y`	`x >>= y`	`x = x >> y`

ASSIGNMENT AND COMPOUND ASSIGNMENT main.c

```c
#include <stdio.h>

int main(void) {
    int a, b, c;
    a = b = c = 10;                 /* chained, evaluated right to left */
    printf("a=%d b=%d c=%d\n", a, b, c);
    int x = 20;
    x += 5;  printf("x += 5  -> %d\n", x);
    x -= 3;  printf("x -= 3  -> %d\n", x);
    x *= 2;  printf("x *= 2  -> %d\n", x);
    x /= 4;  printf("x /= 4  -> %d\n", x);
    x %= 7;  printf("x %%= 7  -> %d\n", x);
    x <<= 3; printf("x <<= 3 -> %d\n", x);
    x |= 1;  printf("x |= 1  -> %d\n", x);
    double total = 1000;
    total *= 1.18;                  /* add 18% */
    printf("total with GST = %.2f\n", total);
    return 0;
}
```

OUTPUT

```
a=10 b=10 c=10
x += 5  -> 25
x -= 3  -> 22
x *= 2  -> 44
x /= 4  -> 11
x %= 7  -> 4
x <<= 3 -> 32
x |= 1  -> 33
total with GST = 1180.00
```

4.7.2 = vs == mistake

`=` assigns; `==` compares. Writing `if (x = 5)` assigns 5 to x and the condition is always true (5 is non-zero): a silent, classic bug. Compile with `-Wall` (it suggests parentheses), and some programmers write the constant first (`if (5 == x)`) so that a typo becomes a compile error.

ASSIGNMENT INSIDE A CONDITION main.c

```c
#include <stdio.h>

int main(void) {
    int marks = 35;
    if ((marks = 100))                    /* bug: assignment (extra parentheses silence the
warning) */
        printf("Full marks! (marks is now %d)\n", marks);
    marks = 35;
    if (marks == 100)                     /* correct comparison */
        printf("Full marks!\n");
    else
        printf("marks = %d\n", marks);
    return 0;
}
```

OUTPUT

```
Full marks! (marks is now 100)
marks = 35
```

4.8 Conditional (ternary) operator

4.8.1 condition ? a : b

`condition ? expr1 : expr2` evaluates the condition; if true, the whole expression has the value of `expr1`, otherwise `expr2`. Only one of the two is evaluated. It is a compact `if-else` that produces a **value**, ideal inside assignments and printf arguments. Avoid nesting it deeply.

THE TERNARY OPERATOR

main.c

```c
#include <stdio.h>

int main(void) {
    int a = 14, b = 9;
    int max = (a > b) ? a : b;
    printf("max = %d\n", max);
    int n = 7;
    printf("%d is %s\n", n, n % 2 == 0 ? "even" : "odd");
    int marks = 82;
    char grade = marks >= 90 ? 'A' : marks >= 75 ? 'B' : marks >= 50 ? 'C' : 'F';   /* chained */
    printf("grade %c\n", grade);
    int items = 1;
    printf("%d item%s in cart\n", items, items == 1 ? "" : "s");
    return 0;
}
```

OUTPUT

```
max = 14
7 is odd
grade B
1 item in cart
```

4.9 Special operators

4.9.1 sizeof

`sizeof` yields the size in bytes of a type or object as a `size_t`, computed at **compile time**; its operand is not evaluated (so `sizeof(i++)` does not increment i). For an array it gives the size of the whole array, which is used to count elements.

SIZEOF IN ACTION main.c

```c
#include <stdio.h>

int main(void) {
    int arr[10];
    double d;
    int i = 0;
    printf("sizeof(int) = %zu, sizeof d = %zu\n", sizeof(int), sizeof d);
    printf("sizeof arr = %zu bytes, elements = %zu\n", sizeof arr, sizeof arr / sizeof arr[0]);
    printf("sizeof(i++) = %zu, i is still %d\n", sizeof(i++), i);
    printf("sizeof(3.0f + 1) = %zu (float)\n", sizeof(3.0f + 1));
    return 0;
}
```

OUTPUT

```
sizeof(int) = 4, sizeof d = 8
sizeof arr = 40 bytes, elements = 10
sizeof(i++) = 4, i is still 0
sizeof(3.0f + 1) = 4 (float)
```

4.9.2 Comma operator

`expr1, expr2` evaluates expr1, discards its value, then evaluates expr2, which becomes the value of the whole expression. There is a sequence point between them. It is mainly used in `for` loops to update two variables. (The commas separating function arguments or declarations are *not* the comma operator.)

THE COMMA OPERATOR main.c

```c
#include <stdio.h>

int main(void) {
    int x = (printf("first "), 10 + 5);          /* value of the right operand */
    printf("x = %d\n", x);
    for (int i = 0, j = 10; i < j; i += 3, j -= 3) /* two loop variables */
        printf("i=%d j=%d\n", i, j);
    return 0;
}
```

OUTPUT

```
first x = 15
i=0 j=10
i=3 j=7
```

4.9.3 Address-of & and dereference * (introduced; detailed in Module 11)

`&x` gives the memory address of x. If `p` holds an address, `*p` accesses the value stored there. These two operators are the foundation of pointers.

A FIRST LOOK AT & AND *

main.c

```c
#include <stdio.h>

int main(void) {
    int x = 42;
    int *p = &x;                    /* p stores the address of x */
    printf("x = %d, *p = %d\n", x, *p);
    *p = 99;                        /* change x through p */
    printf("after *p = 99, x = %d\n", x);
    printf("p == &x ? %d\n", p == &x);
    return 0;
}
```

OUTPUT

```
x = 42, *p = 42
after *p = 99, x = 99
p == &x ? 1
```

4.9.4 Member access . and -> (introduced; detailed in Module 14)

For a structure variable `s`, `s.field` accesses a member. For a pointer to a structure `ps`, `ps->field` is shorthand for `(*ps).field`.

MEMBER ACCESS

main.c

```c
#include <stdio.h>

struct point { int x, y; };

int main(void) {
    struct point pt = {3, 4};
    struct point *pp = &pt;
    printf("pt.x = %d, pt.y = %d\n", pt.x, pt.y);
    pp->x = 10;                                  /* same as (*pp).x = 10 */
    printf("after pp->x = 10: pt.x = %d\n", pt.x);
    return 0;
}
```

OUTPUT

```
pt.x = 3, pt.y = 4
after pp->x = 10: pt.x = 10
```

4.10 Precedence and associativity

4.10.1 Precedence table

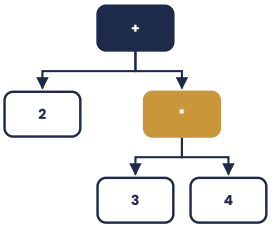
Precedence decides which operator binds first when several appear without parentheses: `2 + 3 * 4` is 14 because `*` has higher precedence than `+`. From highest to lowest:

Level	Operators	Associativity
1	`()` call, `[]`, `.`, `->`, postfix `++ --`	left to right
2	prefix `++ --`, unary `+ -`, `! ~`, `*` (dereference), `&` (address), `sizeof`, `(type)` cast	**right to left**
3	`* / %`	left to right

Level	Operators	Associativity
4	`+ -`	left to right
5	`<< >>`	left to right
6	`< <= > >=`	left to right
7	`== !=`	left to right
8	`&` (bitwise AND)	left to right
9	`^`	left to right
10	`|`	left to right
11	`&&`	left to right
12	`||`	left to right
13	`?:`	**right to left**
14	`= += -= *= /= %= &= ^= |= <<= >>=`	**right to left**
15	`,`	left to right

4.10.2 Left-to-right vs right-to-left associativity

Associativity decides the grouping when operators of the **same** precedence meet: `100 / 10 / 2` is `(100 / 10) / 2` = 5 (left to right), while `a = b = 5` is `a = (b = 5)` (right to left), and `*p++` is `*(p++)`.



EXPRESSION TREE OF 2 + 3 * 4: THE HIGHER-PRECEDENCE * IS EVALUATED FIRST

PRECEDENCE AND ASSOCIATIVITY EXAMPLES main.c

```c
#include <stdio.h>

int main(void) {
    printf("2 + 3 * 4       = %d\n", 2 + 3 * 4);
    printf("(2 + 3) * 4     = %d\n", (2 + 3) * 4);
    printf("100 / 10 / 2    = %d (left to right)\n", 100 / 10 / 2);
    printf("10 - 4 - 3      = %d\n", 10 - 4 - 3);
    int a, b;
    a = b = 7;
    printf("a = b = 7       -> a=%d b=%d (right to left)\n", a, b);
    printf("5 & 3 == 3      = %d (== binds tighter than &!)\n", 5 & 3 == 3);
    printf("(5 & 3) == 1    = %d\n", (5 & 3) == 1);
    printf("!0 + 1          = %d\n", !0 + 1);
    printf("-2 * -3 + 1     = %d\n", -2 * -3 + 1);
    return 0;
}
```

OUTPUT

```
2 + 3 * 4       = 14
(2 + 3) * 4     = 20
100 / 10 / 2    = 5 (left to right)
10 - 4 - 3      = 3
a = b = 7       -> a=7 b=7 (right to left)
5 & 3 == 3      = 1 (== binds tighter than &!)
(5 & 3) == 1    = 1
!0 + 1          = 2
-2 * -3 + 1     = 7
```

4.10.3 Sequence points and order of evaluation

Precedence tells you how an expression is **grouped**, not the **order** in which operands are evaluated. In `f() + g()`, the compiler may call g before f. Guaranteed ordering exists only at **sequence points**: the end of a full expression (`;`), after the first operand of `&&`, `||`, `?:` and the comma operator, and before a function is called (after its arguments are evaluated). Between sequence points, never depend on order (4.3.3).

GROUPING IS FIXED; EVALUATION ORDER MAY NOT BE main.c

```c
#include <stdio.h>

int f(void) { printf("f "); return 1; }
int g(void) { printf("g "); return 2; }

int main(void) {
    int r = f() + g();          /* order of the two calls is unspecified */
    printf("-> %d\n", r);
    int s = f() && g();         /* && guarantees f first */
    printf("-> %d\n", s);
    return 0;
}
```

OUTPUT

```
f g -> 3
f g -> 1
```

4.10.4 Using parentheses for clarity

Nobody memorises the whole table. When in doubt, and always when mixing bitwise, logical and comparison operators, add parentheses. They cost nothing and make the intention obvious: `if ((flags & MASK) == MASK)`, `avg = (a + b + c) / 3.0`.

4.11 math.h library

4.11.1 pow, sqrt, abs, fabs, ceil, floor, round, fmod

Function	Header	Does	Example → result
`pow(x, y)`	math.h	x^y	`pow(2, 10)` → 1024.0
`sqrt(x)`	math.h	square root	`sqrt(144)` → 12.0
`cbrt(x)`	math.h	cube root	`cbrt(27)` → 3.0
`abs(n)`	**stdlib.h**	absolute value of an int	`abs(-7)` → 7
`fabs(x)`	math.h	absolute value of a double	`fabs(-2.5)` → 2.5
`ceil(x)`	math.h	round up	`ceil(4.1)` → 5.0
`floor(x)`	math.h	round down	`floor(4.9)` → 4.0
`round(x)`	math.h	nearest, halves away from zero	`round(2.5)` → 3.0
`trunc(x)`	math.h	drop the fraction	`trunc(-3.7)` → -3.0
`fmod(x, y)`	math.h	floating-point remainder	`fmod(7.5, 2)` → 1.5
`sin cos tan`	math.h	trigonometry (radians)	`sin(M_PI/2)` → 1.0
`log log10 exp`	math.h	natural log, base-10 log, e^x	`log10(1000)` → 3.0
`hypot(x, y)`	math.h	$\sqrt{(x^2 + y^2)}$	`hypot(3, 4)` → 5.0

COMMON MATH.H FUNCTIONS

main.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <math.h>

int main(void) {
    printf("pow(2, 10)   = %.1f\n", pow(2, 10));
    printf("sqrt(144)    = %.1f\n", sqrt(144));
    printf("cbrt(27)     = %.1f\n", cbrt(27));
    printf("abs(-7)      = %d\n", abs(-7));
    printf("fabs(-2.5)   = %.1f\n", fabs(-2.5));
    printf("ceil(4.1)    = %.1f, floor(4.9) = %.1f\n", ceil(4.1), floor(4.9));
    printf("round(2.5)   = %.1f, round(-2.5) = %.1f\n", round(2.5), round(-2.5));
    printf("trunc(-3.7)  = %.1f\n", trunc(-3.7));
    printf("fmod(7.5, 2) = %.1f\n", fmod(7.5, 2));
    printf("hypot(3, 4)  = %.1f\n", hypot(3, 4));
    double pi = acos(-1.0);
    printf("pi = %.6f, sin(pi/6) = %.2f\n", pi, sin(pi / 6));
    printf("log10(1000)  = %.1f, exp(1) = %.5f\n", log10(1000), exp(1));
    double principal = 100000, rate = 0.08;
    printf("amount after 5 years at 8%% = %.2f\n", principal * pow(1 + rate, 5));
    return 0;
}
```

OUTPUT

```
pow(2, 10)   = 1024.0
sqrt(144)    = 12.0
cbrt(27)     = 3.0
abs(-7)      = 7
fabs(-2.5)   = 2.5
ceil(4.1)    = 5.0, floor(4.9) = 4.0
round(2.5)   = 3.0, round(-2.5) = -3.0
trunc(-3.7)  = -3.0
fmod(7.5, 2) = 1.5
hypot(3, 4)  = 5.0
pi = 3.141593, sin(pi/6) = 0.50
log10(1000)  = 3.0, exp(1) = 2.71828
amount after 5 years at 8% = 146932.81
```

4.11.2 Linking the math library with -lm

On Linux, the math functions live in a separate library, `libm`. If you call a function that GCC cannot compute at compile time (for example with a variable argument), you must link it with `-lm` **after** the source file, or the linker reports *undefined reference to `sqrt'*. Windows (MinGW) and macOS link it automatically.

Forgetting -lm on Linux

```
$ gcc area.c -o area
/usr/bin/ld: /tmp/ccx2Lk3a.o: in function `main':
area.c:(.text+0x2e): undefined reference to `sqrt'
collect2: error: ld returned 1 exit status

$ gcc area.c -o area -lm        # link libm: works
$ ./area
```

PRACTICE CHECKPOINT

A **simple calculator**, an **even/odd check with bitwise AND**, and a **swap of two numbers using XOR**.

SAMPLE SOLUTION main.c

```c
#include <stdio.h>

int main(void) {
    double a, b;
    char op;
    printf("Enter expression (e.g. 12 * 4): ");
    scanf("%lf %c %lf", &a, &op, &b);
    switch (op) {
        case '+': printf("%.2f + %.2f = %.2f\n", a, b, a + b); break;
        case '-': printf("%.2f - %.2f = %.2f\n", a, b, a - b); break;
        case '*': printf("%.2f * %.2f = %.2f\n", a, b, a * b); break;
        case '/':
            if (b != 0) printf("%.2f / %.2f = %.2f\n", a, b, a / b);
            else printf("Error: division by zero\n");
            break;
        default: printf("Unknown operator %c\n", op);
    }

    int n;
    printf("Enter an integer: ");
    scanf("%d", &n);
    printf("%d is %s (last bit = %d)\n", n, (n & 1) ? "odd" : "even", n & 1);

    int x, y;
    printf("Enter two integers to swap: ");
    scanf("%d %d", &x, &y);
    printf("before: x = %d, y = %d\n", x, y);
    x = x ^ y;      /* x holds x^y */
    y = x ^ y;      /* (x^y)^y = x */
    x = x ^ y;      /* (x^y)^x = y */
    printf("after : x = %d, y = %d\n", x, y);
    return 0;
}
```

CONSOLE (user input in gold)

```
Enter expression (e.g. 12 * 4): 12 * 4
12.00 * 4.00 = 48.00
Enter an integer: 37
37 is odd (last bit = 1)
Enter two integers to swap: 5 9
before: x = 5, y = 9
after : x = 9, y = 5
```

NOTE

The XOR swap is a classic puzzle, but it fails if both operands are the same variable (x ^ x is 0). In real code a temporary variable is clearer and just as fast.

MODULE 05 · BEGINNER

Input and output

Minimum duration **1 day** | Level **Beginner**

C performs input and output through library functions in stdio.h; this module covers formatted and character I/O.

IN THIS MODULE

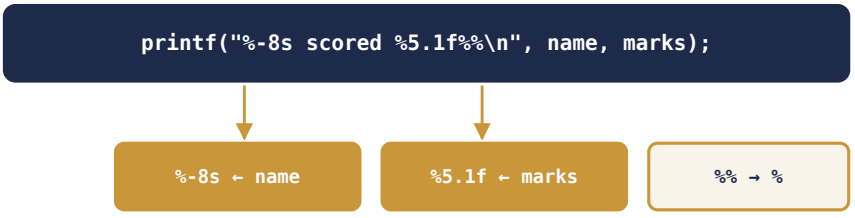
5.1 Formatted output with printf
5.2 Formatted input with scanf
5.3 Character I/O
5.4 String I/O
5.5 Other I/O

NOTE

In the boxes labelled **CONSOLE**, text shown in **gold** is what the user typed at the keyboard; everything else is printed by the program.

5.1 Formatted output with printf

`printf(format, values...)` prints the **format string**, replacing each **conversion specification** (starting with `%`) with the next value, converted as the specifier says. The number and types of values must match the specifiers; a mismatch is undefined behaviour (GCC warns with `-Wall`).



EACH % SPECIFIER TAKES THE NEXT ARGUMENT IN ORDER

5.1.1 Format specifiers: %d, %i, %u, %ld, %lld, %f, %lf, %e, %c, %s, %p, %x, %o, %zu, %%

Specifier	Type	Example	Prints
`%d`, `%i`	int (decimal)	`printf("%d", 42)`	42
`%u`	unsigned int	`printf("%u", 42u)`	42
`%ld` / `%lld`	long / long long	`printf("%lld", 9000000000LL)`	9000000000
`%hd` / `%hhd`	short / signed char		
`%f`	double (and float, which is promoted)	`printf("%f", 3.5)`	3.500000

Specifier	Type	Example	Prints
`%lf`	double (same as %f in printf; required in scanf)		
`%Lf`	long double		
`%e` / `%E`	scientific notation	`printf("%e", 123456.0)`	1.234560e+05
`%g`	shorter of %f / %e	`printf("%g", 0.0001)`	0.0001
`%c`	character	`printf("%c", 65)`	A
`%s`	string (char *)	`printf("%s", "hi")`	hi
`%p`	pointer (void *)	`printf("%p", (void*)&x)`	0x7ffd…
`%x` / `%X`	unsigned in hex	`printf("%x", 255)`	ff
`%o`	unsigned in octal	`printf("%o", 8)`	10
`%zu`	size_t	`printf("%zu", sizeof(int))`	4
`%%`	a literal percent sign	`printf("50%%")`	50%

EVERY COMMON SPECIFIER

main.c

```c
#include <stdio.h>

int main(void) {
    int n = 255;
    unsigned u = 4000000000u;
    long long big = 9000000000LL;
    double d = 1234.5678;
    char c = 'Z';
    const char *s = "Learn Being Forward";
    printf("%%d   : %d\n", n);
    printf("%%u   : %u\n", u);
    printf("%%lld: %lld\n", big);
    printf("%%f   : %f\n", d);
    printf("%%e   : %e\n", d);
    printf("%%g   : %g\n", d);
    printf("%%c   : %c (code %d)\n", c, c);
    printf("%%s   : %s\n", s);
    printf("%%x %%X %%o : %x %X %o\n", n, n, n);
    printf("%%zu : %zu\n", sizeof(double));
    printf("%%p   : %p\n", (void *)&n);
    printf("50%% done\n");
    return 0;
}
```

OUTPUT (the address differs on every run)

```
%d   : 255
%u   : 4000000000
%lld: 9000000000
%f   : 1234.567800
%e   : 1.234568e+03
%g   : 1234.57
%c   : Z (code 90)
%s   : Learn Being Forward
%x %X %o : ff FF 377
%zu : 8
%p   : 0x7ffe48ce1d78
50% done
```

5.1.2 Width, precision and alignment (%5d, %-10s, %.2f)

Between `%` and the letter you can add a **width** (minimum characters; padded with spaces), a **precision** after a dot (digits after the decimal point for floats; maximum characters for strings), and a `-` to **left-align**. `*` takes the width or precision from an argument.

FORMATTING A TABLE

main.c

```c
#include <stdio.h>

int main(void) {
    printf("[%d]\n", 42);
    printf("[%5d]   width 5, right-aligned\n", 42);
    printf("[%-5d]   width 5, left-aligned\n", 42);
    printf("[%.2f]\n", 3.14159);
    printf("[%8.2f] width 8, 2 decimals\n", 3.14159);
    printf("[%.3s]    first 3 chars of a string\n", "Forward");
    printf("[%*d]   width from an argument\n", 6, 42);
    printf("\n%-10s %5s %8s\n", "Name", "Age", "Marks");
    printf("%-10s %5d %8.1f\n", "Asha", 21, 91.5);
    printf("%-10s %5d %8.1f\n", "Ravi", 22, 78.25);
    printf("%-10s %5d %8.1f\n", "Meenakshi", 20, 85.0);
    return 0;
}
```

OUTPUT

```
[42]
[   42]   width 5, right-aligned
[42   ]   width 5, left-aligned
[3.14]
[    3.14] width 8, 2 decimals
[For]    first 3 chars of a string
[    42]   width from an argument

Name         Age    Marks
Asha          21     91.5
Ravi          22     78.2
Meenakshi     20     85.0
```

5.1.3 Flags: +, 0,

Flag	Effect	Example	Prints
`-`	left-align in the field	`%-6d|`	`42 |`
`+`	always show the sign	`%+d`	+42
space	space before positive numbers	`% d`	42
`0`	pad with zeros instead of spaces	`%05d`	00042
`#`	alternate form: 0x / 0 prefix, always a decimal point	`%#x` , `%#o`	0x2a, 052

FLAGS

main.c

```c
#include <stdio.h>

int main(void) {
    printf("%+d %+d\n", 42, -42);
    printf("%05d | %08.3f\n", 42, 3.14159);
    printf("%#x %#o %#X\n", 42, 42, 42);
    printf("Invoice no: INV-%06d\n", 731);
    printf("Time %02d:%02d:%02d\n", 9, 5, 7);
    return 0;
}
```

OUTPUT

```
+42 -42
00042 | 0003.142
0x2a 052 0X2A
Invoice no: INV-000731
Time 09:05:07
```

5.1.4 Return value of printf

`printf` returns the **number of characters printed** (not counting the terminating null), or a negative value on error. It is rarely used, but handy for aligning output.

PRINTF RETURNS A COUNT

main.c

```c
#include <stdio.h>

int main(void) {
    int n = printf("Learn Being Forward\n");
    printf("the previous printf wrote %d characters (including \\n)\n", n);
    int w = printf("Total");
    printf("%*s%d\n", 15 - w, "", 499);      /* pad so the number starts at column 15 */
    return 0;
}
```

OUTPUT

```
Learn Being Forward
the previous printf wrote 20 characters (including \n)
Total          499
```

5.2 Formatted input with scanf

`scanf(format, addresses...)` reads input from the keyboard (standard input), converts it according to the format specifiers, and stores the results **at the addresses** you pass.

5.2.1 Reading int, float, char and strings

Type	scanf specifier	Call
int	`%d`	`scanf("%d", &n)`
float	`%f`	`scanf("%f", &f)`
double	`%lf` (not %f!)	`scanf("%lf", &d)`
char	`%c` (or `" %c"`)	`scanf(" %c", &c)`

Type	scanf specifier	Call
string (one word)	`%s` with a width	`scanf("%19s", name)` (no &)
long long	`%lld`	`scanf("%lld", &big)`

READING DIFFERENT TYPES

main.c

```c
#include <stdio.h>

int main(void) {
    int age;
    double height;
    char grade;
    char name[20];
    printf("Enter your first name: ");
    scanf("%19s", name);                /* array name is already an address */
    printf("Enter your age: ");
    scanf("%d", &age);
    printf("Enter height in metres: ");
    scanf("%lf", &height);
    printf("Enter grade: ");
    scanf(" %c", &grade);               /* space skips the leftover newline */
    printf("\n%s, %d years, %.2f m, grade %c\n", name, age, height, grade);
    return 0;
}
```

CONSOLE (user input in gold)

```
Enter your first name: Asha
Enter your age: 21
Enter height in metres: 1.62
Enter grade: A

Asha, 21 years, 1.62 m, grade A
```

5.2.2 Why & is needed in scanf

C passes arguments **by value**: a function receives copies. If you passed `age`, scanf would get a copy of its current (garbage) value and could not change the original. Passing `&age` gives scanf the **address** of the variable, so it can store the converted value there. Arrays (strings) are the exception: an array name already converts to the address of its first element, so no `&`.



SCANF WRITES THROUGH THE ADDRESS YOU GIVE IT

WATCH OUT

Forgetting `&` (`scanf("%d", age)`) passes a garbage value as an address: the program usually crashes. `-Wall` warns about it.

5.2.3 Reading multiple values

One scanf can read several values. Whitespace in the input (spaces, tabs, newlines) separates numbers, and most specifiers skip leading whitespace automatically. Literal characters in the format must match

the input exactly, which is how you parse formats like dates.

SEVERAL VALUES IN ONE CALL main.c

```c
#include <stdio.h>

int main(void) {
    int a, b, c;
    printf("Enter three marks: ");
    scanf("%d %d %d", &a, &b, &c);
    printf("total = %d, average = %.2f\n", a + b + c, (a + b + c) / 3.0);

    int d, m, y;
    printf("Enter date (dd/mm/yyyy): ");
    scanf("%d/%d/%d", &d, &m, &y);      /* the slashes must appear in the input */
    printf("day %d, month %d, year %d\n", d, m, y);
    return 0;
}
```

CONSOLE (user input in gold)

```
Enter three marks: 78 85 91
total = 254, average = 84.67
Enter date (dd/mm/yyyy): 15/08/2026
day 15, month 8, year 2026
```

5.2.4 The leftover newline problem and " %c"

When you type `21` and press Enter, the input buffer contains `"21\n"`. `%d` consumes "21" but leaves the `'\n'`. A following `scanf("%c", &ch)` reads that newline immediately instead of waiting for you. Unlike `%d`, `%c` does not skip whitespace. Fix: put a **space before %c** (`" %c"`), which tells scanf to skip any whitespace first.



THE NEWLINE LEFT IN THE INPUT BUFFER

THE BUG AND THE FIX

main.c

```c
#include <stdio.h>

int main(void) {
    int qty;
    char answer, confirm;
    printf("Quantity: ");
    scanf("%d", &qty);
    printf("Gift wrap (y/n)? ");
    scanf("%c", &answer);                /* BUG: reads the leftover '\n' */
    printf("[answer code = %d, i.e. newline]\n", answer);
    printf("Gift wrap (y/n)? ");
    scanf(" %c", &confirm);              /* FIX: space skips whitespace */
    printf("qty = %d, gift wrap = %c\n", qty, confirm);
    return 0;
}
```

CONSOLE (user input in gold)

```
Quantity: 3
Gift wrap (y/n)? [answer code = 10, i.e. newline]
Gift wrap (y/n)? y
qty = 3, gift wrap = y
```

5.2.5 Return value of scanf and input validation

`scanf` returns the **number of items successfully converted and stored**, or `EOF` if input ended before the first conversion. Always check it: if the user types letters where a number is expected, scanf stops, returns 0, and leaves the bad characters in the buffer, which must be discarded before retrying.

VALIDATING NUMERIC INPUT

main.c

```c
#include <stdio.h>

int main(void) {
    int age;
    while (1) {
        printf("Enter age: ");
        int ok = scanf("%d", &age);
        if (ok == 1 && age > 0 && age < 130) break;
        if (ok == EOF) { printf("no input\n"); return 1; }
        printf("Invalid input, try again.\n");
        int ch;
        while ((ch = getchar()) != '\n' && ch != EOF) { }   /* discard the rest of the line */
    }
    printf("Age accepted: %d\n", age);
    return 0;
}
```

CONSOLE (user input in gold)

```
Enter age: twenty
Invalid input, try again.
Enter age: -5
Invalid input, try again.
Enter age: 21
Age accepted: 21
```

5.2.6 Dangers of %s without a width limit

`%s` reads characters until whitespace, with **no idea how big your array is**. A long word overflows the array and overwrites neighbouring memory: a buffer overflow, a classic security hole. Always give a width one less than the array size (`%19s` for `char name[20]`, leaving room for `'\0'`). Note also that `%s` stops at the first space, so it cannot read full names; use `fgets` (5.4.2).

A WIDTH LIMIT STOPS AT 7 CHARACTERS main.c

```c
#include <stdio.h>

int main(void) {
    char city[8];
    printf("City: ");
    scanf("%7s", city);                /* reads at most 7 chars + '\0' */
    printf("stored: \"%s\"\n", city);
    return 0;
}
```

```
CONSOLE (user input in gold)
City: Thiruvananthapuram
stored: "Thiruva"
```

5.3 Character I/O

5.3.1 getchar() and putchar()

`getchar()` reads **one character** from standard input and returns it as an `int`; `putchar(c)` writes one character. They are the simplest and fastest I/O functions, ideal for processing text character by character.

5.3.2 EOF

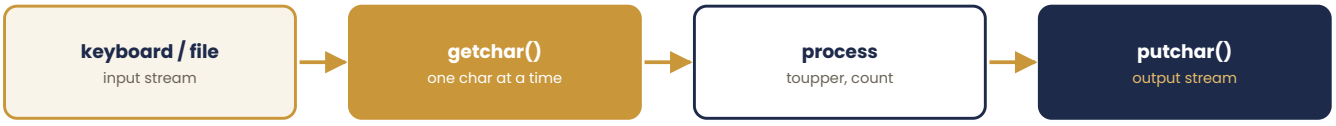
`EOF` ("end of file", usually -1) is returned by getchar when there is no more input: the end of a redirected file, or when the user presses **Ctrl+D** (Linux/macOS) or **Ctrl+Z then Enter** (Windows). Because EOF is not a valid `char` value, the variable receiving getchar's result must be an **int**.

COPYING INPUT TO OUTPUT IN UPPERCASE main.c

```c
#include <stdio.h>
#include <ctype.h>

int main(void) {
    int ch;                                 /* int, so it can hold EOF */
    int chars = 0, lines = 0;
    printf("Type text (Ctrl+D to finish):\n");
    while ((ch = getchar()) != EOF) {
        putchar(toupper(ch));               /* echo in uppercase */
        chars++;
        if (ch == '\n') lines++;
    }
    printf("[%d characters, %d lines]\n", chars, lines);
    return 0;
}
```

```
CONSOLE (user input in gold)
Type text (Ctrl+D to finish):
learn being forward
LEARN BEING FORWARD
c is fun
C IS FUN
[29 characters, 2 lines]
```



THE CLASSIC CHARACTER-PROCESSING LOOP

5.4 String I/O

5.4.1 puts()

`puts(s)` prints a string followed by a newline. It is simpler and slightly faster than `printf("%s\n", s)`, and it never interprets `%` characters. `fputs(s, stdout)` prints without adding a newline.

5.4.2 fgets() for safe line input

`fgets(buf, size, stdin)` reads a **whole line** including spaces, but never more than `size - 1` characters, and always adds `'\0'`. It keeps the newline character if there was room, so it is usually removed afterwards. It returns NULL at end of input. This is the safe, standard way to read text.

READING A FULL LINE SAFELY

main.c

```c
#include <stdio.h>
#include <string.h>

int main(void) {
    char name[40];
    printf("Full name: ");
    if (fgets(name, sizeof name, stdin) == NULL) return 1;
    printf("raw length %zu (includes the newline)\n", strlen(name));
    name[strcspn(name, "\n")] = '\0';          /* remove the trailing newline */
    printf("Hello, %s! (%zu characters)\n", name, strlen(name));
    puts("puts adds the newline for you");
    fputs("fputs does not", stdout);
    putchar('\n');
    return 0;
}
```

CONSOLE (user input in gold)

```
Full name: Lohit Kumar
raw length 12 (includes the newline)
Hello, Lohit Kumar! (11 characters)
puts adds the newline for you
fputs does not
```

5.4.3 Why gets() was removed (C11)

`gets(buf)` read a line into a buffer with **no size limit at all**, so any long input overflowed the buffer. It was exploited by the famous 1988 Morris worm. It was deprecated in C99 and **removed** from the language in C11; modern compilers refuse it or warn loudly. Always use `fgets`.

WATCH OUT

Never use `gets()`. If old code or a textbook uses it, replace it with `fgets(buf, sizeof buf, stdin)` and strip the newline.

5.5 Other I/O

5.5.1 sprintf() and snprintf()

`sprintf(buf, format, ...)` works like printf but writes into a **character array** instead of the screen. It does not know the buffer size, so it can overflow. `snprintf(buf, size, format, ...)` writes at most `size - 1` characters plus `'\0'` and returns the length the full output **would** have had, which lets you detect truncation. Prefer snprintf.

FORMATTING INTO A STRING

main.c

```c
#include <stdio.h>

int main(void) {
    char line[64];
    int roll = 7;
    double marks = 88.456;
    sprintf(line, "Roll %03d scored %.1f", roll, marks);
    puts(line);

    char small[10];
    int needed = snprintf(small, sizeof small, "Invoice-%d", 123456);
    printf("stored \"%s\", needed %d chars -> %s\n", small, needed,
           needed >= (int)sizeof small ? "truncated" : "fits");

    char filename[32];
    for (int i = 1; i <= 3; i++) {
        snprintf(filename, sizeof filename, "report_%02d.txt", i);
        printf("%s\n", filename);
    }
    return 0;
}
```

OUTPUT

```
Roll 007 scored 88.5
stored "Invoice-1", needed 14 chars -> truncated
report_01.txt
report_02.txt
report_03.txt
```

5.5.2 sscanf()

`sscanf(str, format, ...)` works like scanf but **reads from a string**. A robust input pattern is: read a whole line with `fgets`, then parse it with `sscanf` and check the return value. Bad input then never gets stuck in the input buffer.

PARSING STRINGS WITH SSCANF

main.c

```c
#include <stdio.h>

int main(void) {
    const char *record = "101,Asha,91.5";
    int id; char name[20]; double marks;
    int n = sscanf(record, "%d,%19[^,],%lf", &id, name, &marks);   /* %[^,] reads up to a comma */
    printf("fields read: %d -> id=%d name=%s marks=%.1f\n", n, id, name, marks);

    char line[64];
    int h, m;
    printf("Enter time as HH:MM: ");
    fgets(line, sizeof line, stdin);
    if (sscanf(line, "%d:%d", &h, &m) == 2 && h < 24 && m < 60)
        printf("%d minutes after midnight\n", h * 60 + m);
    else
        printf("bad time\n");
    return 0;
}
```

CONSOLE (user input in gold)

```
fields read: 3 -> id=101 name=Asha marks=91.5
Enter time as HH:MM: 09:45
585 minutes after midnight
```

5.5.3 Command-line arguments argc and argv

Programs can receive input when they are started: `./grades Asha 91`. With `int main(int argc, char *argv[])`, `argc` is the number of words (including the program name) and `argv[0]`…`argv[argc-1]` are the words as strings. Numbers arrive as text and must be converted (`atoi`, `strtol`, Module 8). Module 12.8 covers option parsing.

RUN AS: ./PROG 25 17

main.c

```c
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char *argv[]) {
    if (argc < 3) {
        fprintf(stderr, "usage: %s <a> <b>\n", argv[0]);
        return 1;
    }
    int a = atoi(argv[1]);
    int b = atoi(argv[2]);
    printf("argc = %d\n", argc);
    for (int i = 0; i < argc; i++) printf("argv[%d] = \"%s\"\n", i, argv[i]);
    printf("%d + %d = %d\n", a, b, a + b);
    return 0;
}
```

OUTPUT

```
argc = 3
argv[0] = "./prog"
argv[1] = "25"
argv[2] = "17"
25 + 17 = 42
```



argc = 3 (argv[argc] is always NULL)

HOW THE COMMAND LINE ./PROG 25 17 ARRIVES IN MAIN

PRACTICE CHECKPOINT

Read a student's name with `fgets` and three marks with `scanf`, then print the total and average to two decimal places in a formatted table.

SAMPLE SOLUTION main.c

```c
#include <stdio.h>
#include <string.h>

int main(void) {
    char name[50];
    int m1, m2, m3;

    printf("Student name : ");
    if (fgets(name, sizeof name, stdin) == NULL) return 1;
    name[strcspn(name, "\n")] = '\0';

    printf("Three marks  : ");
    if (scanf("%d %d %d", &m1, &m2, &m3) != 3) {
        printf("Please enter three whole numbers.\n");
        return 1;
    }
    int total = m1 + m2 + m3;
    double average = total / 3.0;

    printf("\n+----------------------+-------+-------+-------+-------+---------+\n");
    printf("| %-20s | %5s | %5s | %5s | %5s | %7s |\n", "Name", "M1", "M2", "M3", "Total", "Average");
    printf("+----------------------+-------+-------+-------+-------+---------+\n");
    printf("| %-20s | %5d | %5d | %5d | %5d | %7.2f |\n", name, m1, m2, m3, total, average);
    printf("+----------------------+-------+-------+-------+-------+---------+\n");
    return 0;
}
```

CONSOLE (user input in gold)

```
Student name : Meera Krishnan
Three marks  : 88 76 93

+----------------------+-------+-------+-------+-------+---------+
| Name                 |    M1 |    M2 |    M3 | Total | Average |
+----------------------+-------+-------+-------+-------+---------+
| Meera Krishnan       |    88 |    76 |    93 |   257 |   85.67 |
+----------------------+-------+-------+-------+-------+---------+
```

MODULE 06 · BEGINNER

Control flow

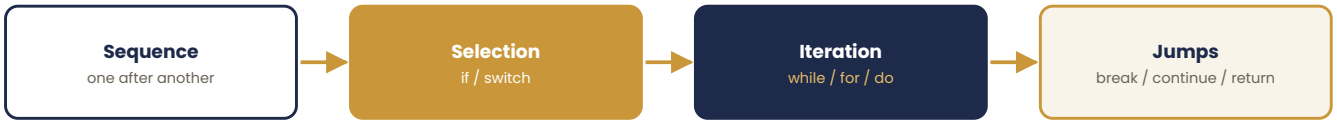
Minimum duration **4 days** | Level **Beginner**

Control flow decides which statements run and how often: decisions, loops and jumps.

IN THIS MODULE

6.1	Decision-making statements	6.4	Nested loops
6.2	switch statement	6.5	Jump statements
6.3	Loops	6.6	Classic loop problems

By default statements run one after another from top to bottom (**sequence**). Control-flow statements change that order: **selection** chooses between paths (if, switch), **iteration** repeats a block (while, do-while, for), and **jumps** transfer control (break, continue, goto, return). Every algorithm can be built from sequence, selection and iteration.



THE BUILDING BLOCKS OF CONTROL FLOW

6.1 Decision-making statements

6.1.1 if

`if (condition) statement;` runs the statement only when the condition is **true** (non-zero). To run several statements, put them in a block `{ }`.

A SIMPLE IF — main.c

```c
#include <stdio.h>

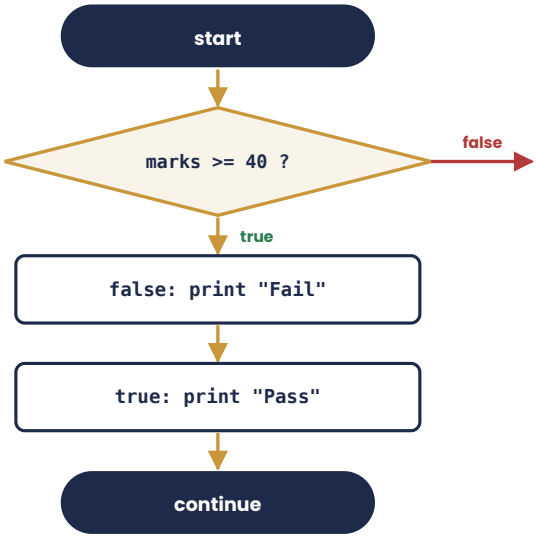
int main(void) {
    int marks = 72;
    if (marks >= 40)
        printf("You passed.\n");
    if (marks >= 90) {
        printf("Excellent!\n");
        printf("You get a certificate of merit.\n");
    }
    printf("Done.\n");
    return 0;
}
```

OUTPUT

```
You passed.
Done.
```

6.1.2 if-else

`if (condition) A else B` chooses exactly one of two paths.



IF-ELSE: ONE OF TWO BRANCHES RUNS

EVEN OR ODD main.c

```c
#include <stdio.h>

int main(void) {
    int n;
    printf("Enter a number: ");
    scanf("%d", &n);
    if (n % 2 == 0)
        printf("%d is even\n", n);
    else
        printf("%d is odd\n", n);
    return 0;
}
```

```
CONSOLE (user input in gold)
Enter a number: 17
17 is odd
```

6.1.3 else-if ladder

To choose among more than two possibilities, chain `else if`. Conditions are tested top to bottom and only the **first** true branch runs; the final `else` catches everything else. Order the tests carefully (most specific or highest first).

GRADE CALCULATOR

main.c

```c
#include <stdio.h>

int main(void) {
    int marks;
    printf("Enter marks (0-100): ");
    scanf("%d", &marks);
    if (marks < 0 || marks > 100)
        printf("Invalid marks\n");
    else if (marks >= 90)
        printf("Grade A+\n");
    else if (marks >= 75)
        printf("Grade A\n");
    else if (marks >= 60)
        printf("Grade B\n");
    else if (marks >= 40)
        printf("Grade C\n");
    else
        printf("Grade F (fail)\n");
    return 0;
}
```

CONSOLE (user input in gold)

```
Enter marks (0-100): 78
Grade A
```

6.1.4 Nested if

An if can contain another if. Nesting expresses conditions that only make sense when an outer condition holds. Deep nesting is hard to read; often it can be flattened with `&&` or an else-if ladder.

LARGEST OF THREE, WITH NESTING

main.c

```c
#include <stdio.h>

int main(void) {
    int a = 14, b = 27, c = 9;
    if (a >= b) {
        if (a >= c) printf("largest = %d\n", a);
        else        printf("largest = %d\n", c);
    } else {
        if (b >= c) printf("largest = %d\n", b);
        else        printf("largest = %d\n", c);
    }
    int age = 20, has_licence = 1;
    if (age >= 18) {
        if (has_licence) printf("You may drive.\n");
        else             printf("Get a licence first.\n");
    } else {
        printf("Too young to drive.\n");
    }
    return 0;
}
```

OUTPUT

```
largest = 27
You may drive.
```

6.1.5 Common mistakes (= in conditions, missing braces, dangling else)

= in conditions: `if (x = 0)` assigns and is always false (4.7.2). **Missing braces**: without `{ }` only the **first** statement belongs to the if; indentation does not matter to the compiler. **Stray semicolon**: `if (x > 0);` has an empty body. **Dangling else**: an else always pairs with the **nearest** unmatched if, whatever the indentation suggests.

MISSING BRACES AND THE DANGLING ELSE

main.c

```c
#include <stdio.h>

int main(void) {
    int balance = 50;
    if (balance > 100)
        printf("Balance is high\n");
        printf("This line is NOT part of the if!\n");     /* missing braces */

    int x = 5, y = -1;
    if (x > 0)
        if (y > 0)
            printf("both positive\n");
    else                                  /* looks like it pairs with the first if ... */
        printf("x is not positive?\n");     /* ... but it belongs to if (y > 0) */
    return 0;
}
```

OUTPUT

```
This line is NOT part of the if!
x is not positive?
```

TIP

Always use braces, even for single statements. It prevents these bugs and makes adding lines later safe.

6.2 switch statement

6.2.1 switch, case, break, default

`switch` compares one integer expression against several constant `case` labels and jumps to the matching one. `break` leaves the switch; `default` runs when no case matches (it is optional and may appear anywhere, conventionally last).

DAY OF THE WEEK

main.c

```c
#include <stdio.h>

int main(void) {
    int day;
    printf("Enter day number (1-7): ");
    scanf("%d", &day);
    switch (day) {
        case 1: printf("Monday\n");    break;
        case 2: printf("Tuesday\n");   break;
        case 3: printf("Wednesday\n"); break;
        case 4: printf("Thursday\n");  break;
        case 5: printf("Friday\n");    break;
        case 6: printf("Saturday\n");  break;
        case 7: printf("Sunday\n");    break;
        default: printf("Invalid day\n");
    }
    return 0;
}
```

CONSOLE (user input in gold)

```
Enter day number (1-7): 5
Friday
```

6.2.2 Fall-through behaviour

Without `break`, execution **falls through** into the next case's statements. Usually this is a bug, but it is also used deliberately to let several cases share code. Mark intentional fall-through with a comment (C23: `[[fallthrough]];`).

ACCIDENTAL AND INTENTIONAL FALL-THROUGH

main.c

```c
#include <stdio.h>

int main(void) {
    char grade = 'B';
    printf("Missing breaks:\n");
    switch (grade) {
        case 'A': printf("  Excellent\n");
        case 'B': printf("  Good\n");          /* matches here ... */
        case 'C': printf("  Average\n");       /* ... and falls through */
        default:  printf("  Keep working\n");
    }
    char ch = 'e';
    switch (ch) {                             /* deliberate fall-through: shared code */
        case 'a': case 'e': case 'i': case 'o': case 'u':
            printf("'%c' is a vowel\n", ch);
            break;
        default:
            printf("'%c' is a consonant\n", ch);
    }
    return 0;
}
```

OUTPUT

```
Missing breaks:
  Good
  Average
  Keep working
'e' is a vowel
```

6.2.3 Allowed types (integer and character constants only)

The switch expression must have an **integer** type (int, char, enum, long…). Case labels must be **integer constant expressions**, all different. You cannot switch on a `double` or a string, and you cannot write ranges or variables in case labels (GCC offers `case 1 ... 5:` as a non-standard extension). For strings, use `strcmp` in an if-else chain.

NOT ALLOWED IN A SWITCH

C

```c
switch (price) { case 9.99: ... }       /* error: float is not allowed */
switch (name)  { case "Asha": ... }     /* error: strings are not allowed */
int limit = 10; case limit: ...          /* error: case label must be a constant */
```

6.2.4 switch vs else-if ladder

	switch	else-if ladder
Tests	Equality of one integer against constants	Any condition: ranges, floats, strings, combinations
Readability	Very clear for many discrete values (menus, commands)	Better for ranges
Speed	Compiler can build a jump table: O(1)	Tests one by one
Fall-through	Possible (needs break)	No

6.3 Loops

A **loop** repeats a block while a condition holds. Each repetition is an **iteration**. Every loop needs (1) initialization of a control variable, (2) a condition to test, and (3) an update that eventually makes the condition false.

6.3.1 while loop

`while (condition) body` tests the condition **before** each iteration; if it is false at the start, the body never runs. Use it when the number of iterations is not known in advance.

WHILE LOOPS

main.c

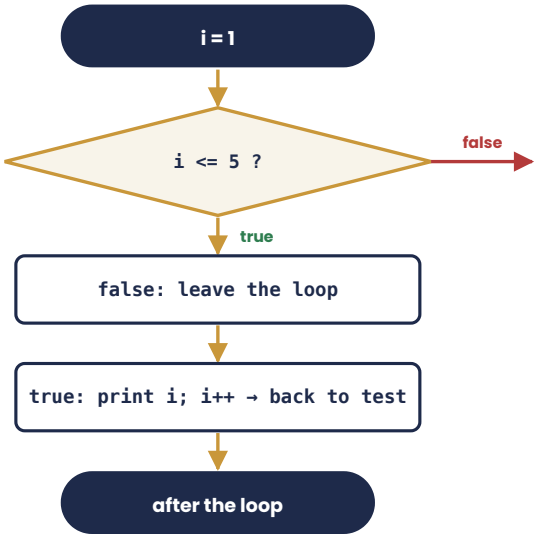
```c
#include <stdio.h>

int main(void) {
    int i = 1;                      /* 1. initialize */
    while (i <= 5) {                /* 2. test */
        printf("%d ", i);
        i++;                        /* 3. update */
    }
    printf("\n");

    int n = 9875, digits = 0;
    while (n > 0) {                 /* unknown number of iterations */
        n /= 10;
        digits++;
    }
    printf("9875 has %d digits\n", digits);
    return 0;
}
```

OUTPUT

```
1 2 3 4 5
9875 has 4 digits
```



WHILE: THE CONDITION IS CHECKED BEFORE EVERY ITERATION

6.3.2 do-while loop

`do body while (condition);` runs the body **first** and tests afterwards, so the body always executes at least once. Ideal for menus and input validation. Note the semicolon after the condition.

DO-WHILE FOR INPUT main.c

```c
#include <stdio.h>

int main(void) {
    int pin, attempts = 0;
    do {
        printf("Enter PIN: ");
        scanf("%d", &pin);
        attempts++;
    } while (pin != 4321 && attempts < 3);
    printf(pin == 4321 ? "Access granted after %d attempt(s)\n" : "Card blocked after %d attempts\n",
attempts);

    int x = 100;
    do { printf("runs once even though x > 10 (x = %d)\n", x); } while (x < 10);
    return 0;
}
```

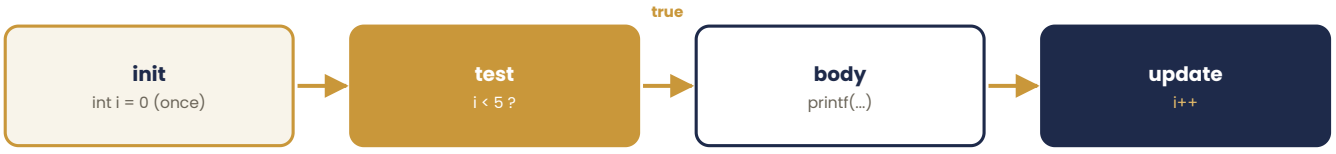
CONSOLE (user input in gold)

```
Enter PIN: 1111
Enter PIN: 4321
Access granted after 2 attempt(s)
runs once even though x > 10 (x = 100)
```

6.3.3 for loop

6.3.3.1 Initialization, condition, update

`for (init; condition; update) body` puts the three parts of a counting loop in one line. The order is: init once → test → body → update → test → body → … until the test fails. Any part may be empty; an empty condition means "true".



EXECUTION ORDER OF FOR (INIT; TEST; UPDATE): AFTER UPDATE, CONTROL RETURNS TO TEST

FOR LOOPS

main.c

```c
#include <stdio.h>

int main(void) {
    for (int i = 0; i < 5; i++)
        printf("i = %d\n", i);
    printf("countdown: ");
    for (int i = 10; i > 0; i -= 3) printf("%d ", i);
    printf("\nmultiplication table of 7:\n");
    for (int k = 1; k <= 10; k++) printf("7 x %2d = %2d\n", k, 7 * k);
    int sum = 0;
    for (int k = 2; k <= 100; k += 2) sum += k;
    printf("sum of even numbers 2..100 = %d\n", sum);
    return 0;
}
```

OUTPUT

```
i = 0
i = 1
i = 2
i = 3
i = 4
countdown: 10 7 4 1
multiplication table of 7:
7 x  1 =  7
7 x  2 = 14
7 x  3 = 21
7 x  4 = 28
7 x  5 = 35
7 x  6 = 42
7 x  7 = 49
7 x  8 = 56
7 x  9 = 63
7 x 10 = 70
sum of even numbers 2..100 = 2550
```

6.3.3.2 Multiple variables with the comma operator

TWO LOOP VARIABLES

main.c

```c
#include <stdio.h>

int main(void) {
    char word[] = "LEVEL";
    int is_pal = 1;
    for (int i = 0, j = 4; i < j; i++, j--) {        /* two indexes moving towards each other */
        printf("compare word[%d]=%c with word[%d]=%c\n", i, word[i], j, word[j]);
        if (word[i] != word[j]) is_pal = 0;
    }
    printf("%s is %sa palindrome\n", word, is_pal ? "" : "not ");
    return 0;
}
```

OUTPUT

```
compare word[0]=L with word[4]=L
compare word[1]=E with word[3]=E
LEVEL is a palindrome
```

6.3.3.3 Declaring the loop variable inside for (C99)

Since C99 the loop variable can be declared in the init part: `for (int i = 0; …)`. Its scope is limited to the loop, so it cannot be misused afterwards and the same name can be reused in the next loop. In C89 it had to be declared before the loop.

6.3.4 Infinite loops

A loop whose condition never becomes false runs forever: `while (1)`, `for (;;)`. Intentional infinite loops are common (servers, game loops, embedded main loops) and are left with `break` or `return`. Accidental ones come from forgetting the update or updating the wrong way. Stop a stuck program with **Ctrl+C**.

AN INTENTIONAL INFINITE LOOP WITH BREAK — main.c

```c
#include <stdio.h>

int main(void) {
    int total = 0, value;
    for (;;) {                              /* intentional infinite loop */
        printf("Enter amount (0 to finish): ");
        scanf("%d", &value);
        if (value == 0) break;              /* the way out */
        total += value;
    }
    printf("Total = %d\n", total);
    /* accidental: for (int i = 0; i < 10; i--)  -> i never reaches 10 */
    return 0;
}
```

CONSOLE (user input in gold)

```
Enter amount (0 to finish): 250
Enter amount (0 to finish): 125
Enter amount (0 to finish): 75
Enter amount (0 to finish): 0
Total = 450
```

6.3.5 Entry-controlled vs exit-controlled loops

	Entry-controlled (while, for)	Exit-controlled (do-while)
Condition tested	Before the body	After the body
Minimum iterations	0	1
Typical use	Counting, processing while data remains	Menus, "ask again until valid"

6.3.6 Choosing the right loop

- Known number of iterations, counting: **for**.
- Repeat while a condition holds, possibly zero times: **while** (reading until EOF, digit loops).
- Body must run at least once: **do-while** (menus, input validation).
- Any loop can be rewritten as another; choose the one that reads most naturally.

6.4 Nested loops

6.4.1 Loop inside a loop

A loop can contain another loop. For each iteration of the **outer** loop, the **inner** loop runs completely. With outer n and inner m iterations, the inner body runs n × m times. Nested loops process rows and columns: tables, matrices, patterns.

A 5 X 5 MULTIPLICATION GRID main.c

```c
#include <stdio.h>

int main(void) {
    printf("    ");
    for (int c = 1; c <= 5; c++) printf("%4d", c);
    printf("\n");
    for (int r = 1; r <= 5; r++) {          /* outer: rows */
        printf("%4d", r);
        for (int c = 1; c <= 5; c++)        /* inner: columns */
            printf("%4d", r * c);
        printf("\n");
    }
    return 0;
}
```

OUTPUT

```
       1   2   3   4   5
   1   1   2   3   4   5
   2   2   4   6   8  10
   3   3   6   9  12  15
   4   4   8  12  16  20
   5   5  10  15  20  25
```

6.4.2 Pattern printing

Pattern programs train you to think in rows and columns. The method: the outer loop counts **rows**; for each row, work out **how many** spaces and symbols to print as a function of the row number; inner loops print them; then print a newline.

6.4.2.1 Star patterns (square, triangles, pyramid, diamond)

FIVE STAR PATTERNS

main.c

```c
#include <stdio.h>

int main(void) {
    int n = 4;
    printf("Square:\n");
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= n; j++) printf("* ");
        printf("\n");
    }
    printf("Right triangle:\n");
    for (int i = 1; i <= n; i++) {          /* row i has i stars */
        for (int j = 1; j <= i; j++) printf("* ");
        printf("\n");
    }
    printf("Inverted triangle:\n");
    for (int i = n; i >= 1; i--) {
        for (int j = 1; j <= i; j++) printf("* ");
        printf("\n");
    }
    printf("Pyramid:\n");
    for (int i = 1; i <= n; i++) {          /* n-i spaces, then 2i-1 stars */
        for (int s = 1; s <= n - i; s++) printf(" ");
        for (int j = 1; j <= 2 * i - 1; j++) printf("*");
        printf("\n");
    }
    printf("Diamond:\n");
    for (int i = 1; i <= n; i++) {
        for (int s = 1; s <= n - i; s++) printf(" ");
        for (int j = 1; j <= 2 * i - 1; j++) printf("*");
        printf("\n");
    }
    for (int i = n - 1; i >= 1; i--) {
        for (int s = 1; s <= n - i; s++) printf(" ");
        for (int j = 1; j <= 2 * i - 1; j++) printf("*");
        printf("\n");
    }
    return 0;
}
```

OUTPUT

```
Square:
* * * *
* * * *
* * * *
* * * *
Right triangle:
*
* *
* * *
* * * *
Inverted triangle:
* * * *
* * *
* *
*
Pyramid:
   *
  ***
 *****
*******
Diamond:
   *
  ***
 *****
*******
 *****
  ***
   *
```

Row i (n = 4)	Spaces = n - i	Stars = 2i - 1	Printed
1	3	1	`   *`
2	2	3	`  ***`
3	1	5	` *****`
4	0	7	`*******`

6.4.2.2 Number patterns (Floyd's triangle, Pascal's triangle)

Floyd's triangle prints consecutive numbers, row i containing i numbers. In **Pascal's triangle** each number is the sum of the two above it; the k-th entry of row n is the binomial coefficient C(n, k), which can be computed from the previous entry: C(n, k) = C(n, k-1) × (n - k + 1) / k.

NUMBER PATTERNS

main.c

```c
#include <stdio.h>

int main(void) {
    int rows = 5, num = 1;
    printf("Floyd's triangle:\n");
    for (int i = 1; i <= rows; i++) {
        for (int j = 1; j <= i; j++) printf("%3d", num++);
        printf("\n");
    }
    printf("Pascal's triangle:\n");
    for (int n = 0; n < rows + 1; n++) {
        for (int s = 0; s < rows - n; s++) printf("  ");
        int c = 1;                                  /* C(n, 0) */
        for (int k = 0; k <= n; k++) {
            printf("%4d", c);
            c = c * (n - k) / (k + 1);              /* C(n, k+1) */
        }
        printf("\n");
    }
    printf("Number pyramid:\n");
    for (int i = 1; i <= 4; i++) {
        for (int s = 1; s <= 4 - i; s++) printf(" ");
        for (int j = 1; j <= i; j++) printf("%d", j);
        for (int j = i - 1; j >= 1; j--) printf("%d", j);
        printf("\n");
    }
    return 0;
}
```

OUTPUT

```
Floyd's triangle:
  1
  2  3
  4  5  6
  7  8  9 10
 11 12 13 14 15
Pascal's triangle:
             1
          1   1
        1   2   1
      1   3   3   1
    1   4   6   4   1
  1   5  10  10   5   1
Number pyramid:
   1
  121
 12321
1234321
```

6.4.2.3 Character patterns

CHARACTER PATTERNS

main.c

```c
#include <stdio.h>

int main(void) {
    for (int i = 0; i < 5; i++) {              /* A, AB, ABC ... */
        for (char c = 'A'; c <= 'A' + i; c++) printf("%c", c);
        printf("\n");
    }
    printf("\n");
    char ch = 'A';
    for (int i = 1; i <= 4; i++) {             /* A, BB, CCC ... */
        for (int j = 1; j <= i; j++) printf("%c", ch);
        ch++;
        printf("\n");
    }
    return 0;
}
```

OUTPUT

```
A
AB
ABC
ABCD
ABCDE

A
BB
CCC
DDDD
```

6.5 Jump statements

6.5.1 break

`break` immediately leaves the **innermost** loop or switch. Execution continues after it. In nested loops it exits only one level.

6.5.2 continue

`continue` skips the rest of the current iteration and jumps to the next one: in a `for` loop to the update, in `while` / `do-while` to the condition test.

BREAK AND CONTINUE

main.c

```c
#include <stdio.h>

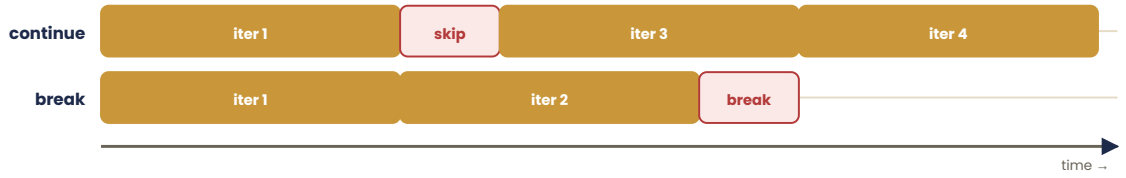
int main(void) {
    printf("first multiple of 7 above 50: ");
    for (int n = 51; ; n++)
        if (n % 7 == 0) { printf("%d\n", n); break; }

    printf("odd numbers 1-15 except multiples of 3: ");
    for (int i = 1; i <= 15; i++) {
        if (i % 2 == 0) continue;           /* skip even */
        if (i % 3 == 0) continue;           /* skip multiples of 3 */
        printf("%d ", i);
    }
    printf("\n");

    for (int i = 1; i <= 3; i++)            /* break leaves only the inner loop */
        for (int j = 1; j <= 3; j++) {
            if (j == 2) break;
            printf("(%d,%d) ", i, j);
        }
    printf("\n");
    return 0;
}
```

OUTPUT

```
first multiple of 7 above 50: 56
odd numbers 1-15 except multiples of 3: 1 5 7 11 13
(1,1) (2,1) (3,1)
```



CONTINUE SKIPS ONE ITERATION; BREAK ENDS THE LOOP

6.5.3 goto and labels (and why to avoid it)

`goto label;` jumps unconditionally to `label:` anywhere in the same function. Unrestricted jumps create tangled "spaghetti code" that is hard to follow, so structured programming avoids goto. The one widely accepted use in C is jumping to a **cleanup section** at the end of a function when an error occurs, or breaking out of deeply nested loops.

THE ACCEPTABLE GOTO: LEAVING NESTED LOOPS

main.c

```c
#include <stdio.h>

int main(void) {
    int grid[3][3] = {{1, 2, 3}, {4, 42, 6}, {7, 8, 9}};
    int target = 42;
    for (int r = 0; r < 3; r++)
        for (int c = 0; c < 3; c++)
            if (grid[r][c] == target) {
                printf("found %d at row %d, col %d\n", target, r, c);
                goto done;                    /* leaves both loops at once */
            }
    printf("not found\n");
done:
    printf("search finished\n");
    return 0;
}
```

OUTPUT

```
found 42 at row 1, col 1
search finished
```

6.5.4 return

`return` ends the current function immediately, optionally giving back a value. In `main`, it ends the program. Returning early from a function is often cleaner than deep nesting (Module 9).

6.6 Classic loop problems

6.6.1 Sum of digits, reverse a number, palindrome number

All three use the same two operations: `n % 10` extracts the last digit and `n / 10` removes it. Reversing builds a new number with `rev = rev * 10 + digit`.

DIGIT MANIPULATION

main.c

```c
#include <stdio.h>

int main(void) {
    int n = 12321, temp = n, sum = 0, rev = 0;
    while (temp > 0) {
        int digit = temp % 10;              /* last digit */
        sum += digit;
        rev = rev * 10 + digit;             /* append to the reversed number */
        temp /= 10;                         /* drop last digit */
    }
    printf("number      : %d\n", n);
    printf("sum of digits: %d\n", sum);
    printf("reversed    : %d\n", rev);
    printf("palindrome  : %s\n", rev == n ? "yes" : "no");
    return 0;
}
```

OUTPUT

```
number      : 12321
sum of digits: 9
reversed    : 12321
palindrome  : yes
```

Step	temp	digit	sum	rev
start	12321	-	0	0
1	1232	1	1	1
2	123	2	3	12
3	12	3	6	123
4	1	2	8	1232
5	0	1	9	12321

6.6.2 Factorial and Fibonacci series

FACTORIAL AND FIBONACCI

main.c

```c
#include <stdio.h>

int main(void) {
    int n = 15;
    unsigned long long fact = 1;
    for (int i = 2; i <= n; i++) fact *= i;
    printf("%d! = %llu\n", n, fact);

    int terms = 15;
    long long a = 0, b = 1;
    printf("Fibonacci: ");
    for (int i = 0; i < terms; i++) {
        printf("%lld ", a);
        long long next = a + b;
        a = b;
        b = next;
    }
    printf("\n");
    return 0;
}
```

OUTPUT

```
15! = 1307674368000
Fibonacci: 0 1 1 2 3 5 8 13 21 34 55 89 144 233 377
```

NOTE

Factorials grow very fast: 13! already overflows a 32-bit int and 21! overflows a 64-bit unsigned long long.

6.6.3 Prime check and primes in a range

A number n > 1 is **prime** if no number from 2 to √n divides it. Checking only up to √n (i.e. `i * i <= n`) is much faster than checking up to n - 1, because if n = a × b then one factor is at most √n.

PRIME NUMBERS main.c

```c
#include <stdio.h>

int main(void) {
    int n = 97, is_prime = (n > 1);
    for (int i = 2; i * i <= n; i++)
        if (n % i == 0) { is_prime = 0; break; }
    printf("%d is %s\n", n, is_prime ? "prime" : "not prime");

    printf("primes between 1 and 60: ");
    int count = 0;
    for (int x = 2; x <= 60; x++) {
        int prime = 1;
        for (int d = 2; d * d <= x; d++)
            if (x % d == 0) { prime = 0; break; }
        if (prime) { printf("%d ", x); count++; }
    }
    printf("\n(%d primes)\n", count);
    return 0;
}
```

OUTPUT

```
97 is prime
primes between 1 and 60: 2 3 5 7 11 13 17 19 23 29 31 37 41 43 47 53 59
(17 primes)
```

6.6.4 Armstrong, perfect and strong numbers

Number type	Definition	Example
Armstrong	Sum of each digit raised to the number of digits equals the number	153 = $1^3 + 5^3 + 3^3$
Perfect	Sum of proper divisors equals the number	28 = 1 + 2 + 4 + 7 + 14
Strong	Sum of factorials of digits equals the number	145 = 1! + 4! + 5!

SEARCHING FOR SPECIAL NUMBERS main.c

```c
#include <stdio.h>

int main(void) {
    printf("Armstrong numbers up to 10000: ");
    for (int n = 1; n <= 10000; n++) {
        int digits = 0, t = n;
        while (t) { digits++; t /= 10; }
        int sum = 0; t = n;
        while (t) {
            int d = t % 10, p = 1;
            for (int k = 0; k < digits; k++) p *= d;
            sum += p; t /= 10;
        }
        if (sum == n) printf("%d ", n);
    }
    printf("\nPerfect numbers up to 10000: ");
    for (int n = 2; n <= 10000; n++) {
        int sum = 1;
        for (int d = 2; d * d <= n; d++)
            if (n % d == 0) { sum += d; if (d != n / d) sum += n / d; }
        if (sum == n) printf("%d ", n);
    }
    printf("\nStrong numbers up to 100000: ");
    int fact[10] = {1, 1, 2, 6, 24, 120, 720, 5040, 40320, 362880};
    for (int n = 1; n <= 100000; n++) {
        int sum = 0, t = n;
        while (t) { sum += fact[t % 10]; t /= 10; }
        if (sum == n) printf("%d ", n);
    }
    printf("\n");
    return 0;
}
```

OUTPUT

```
Armstrong numbers up to 10000: 1 2 3 4 5 6 7 8 9 153 370 371 407 1634 8208 9474
Perfect numbers up to 10000: 6 28 496 8128
Strong numbers up to 100000: 1 2 145 40585
```

6.6.5 GCD, LCM and number base conversion

The **GCD** (greatest common divisor) is found with **Euclid's algorithm**: gcd(a, b) = gcd(b, a mod b) until b becomes 0. The LCM follows from `lcm(a, b) = a / gcd(a, b) * b`. To convert a number to base b, repeatedly divide by b; the remainders, read in reverse, are the digits.

GCD, LCM AND BASE CONVERSION

main.c

```c
#include <stdio.h>

int main(void) {
    int a = 84, b = 36, x = a, y = b;
    while (y != 0) {                        /* Euclid */
        int r = x % y;
        printf("  gcd(%d, %d) -> remainder %d\n", x, y, r);
        x = y; y = r;
    }
    int gcd = x;
    printf("GCD(%d, %d) = %d, LCM = %d\n", a, b, gcd, a / gcd * b);

    int n = 156;
    char digits[33]; int len = 0;
    int t = n;
    do { digits[len++] = '0' + t % 2; t /= 2; } while (t > 0);
    printf("%d in binary: ", n);
    while (len > 0) putchar(digits[--len]);
    printf("\n%d in octal: %o, in hex: %X\n", n, n, n);

    const char *bin = "10011100";
    int value = 0;
    for (int i = 0; bin[i] != '\0'; i++) value = value * 2 + (bin[i] - '0');
    printf("binary %s = %d\n", bin, value);
    return 0;
}
```

OUTPUT

```
  gcd(84, 36) -> remainder 12
  gcd(36, 12) -> remainder 0
GCD(84, 36) = 12, LCM = 252
156 in binary: 10011100
156 in octal: 234, in hex: 9C
binary 10011100 = 156
```

PRACTICE CHECKPOINT

A **menu-driven program** using `switch` inside a `do-while` loop, plus **five pattern programs**.

SAMPLE SOLUTION

main.c

```c
#include <stdio.h>

void square(int n)   { for (int i = 0; i < n; i++) { for (int j = 0; j < n; j++) printf("* ");
printf("\n"); } }
void triangle(int n) { for (int i = 1; i <= n; i++) { for (int j = 0; j < i; j++) printf("* ");
printf("\n"); } }
void pyramid(int n) {
    for (int i = 1; i <= n; i++) {
        for (int s = 0; s < n - i; s++) printf(" ");
        for (int j = 0; j < 2 * i - 1; j++) printf("*");
        printf("\n");
    }
}
void floyd(int n) { int k = 1; for (int i = 1; i <= n; i++) { for (int j = 0; j < i; j++)
printf("%3d", k++); printf("\n"); } }
void letters(int n) { for (int i = 0; i < n; i++) { for (char c = 'A'; c <= 'A' + i; c++) printf("%c",
c); printf("\n"); } }

int main(void) {
    int choice, n;
    do {
        printf("\n=== PATTERN MENU ===\n");
        printf("1. Square  2. Triangle  3. Pyramid  4. Floyd  5. Letters  0. Exit\n");
        printf("Choice: ");
        if (scanf("%d", &choice) != 1) break;
        if (choice >= 1 && choice <= 5) {
            printf("Size: ");
            scanf("%d", &n);
        }
        switch (choice) {
            case 1: square(n);   break;
            case 2: triangle(n); break;
            case 3: pyramid(n);  break;
            case 4: floyd(n);    break;
            case 5: letters(n);  break;
            case 0: printf("Goodbye!\n"); break;
            default: printf("Invalid choice\n");
        }
    } while (choice != 0);
    return 0;
}
```

CONSOLE (user input in gold)

```
=== PATTERN MENU ===
1. Square  2. Triangle  3. Pyramid  4. Floyd  5. Letters  0. Exit
Choice: 3
Size: 4
   *
  ***
 *****
*******

=== PATTERN MENU ===
1. Square  2. Triangle  3. Pyramid  4. Floyd  5. Letters  0. Exit
Choice: 4
Size: 4
  1
  2  3
  4  5  6
  7  8  9 10

=== PATTERN MENU ===
1. Square  2. Triangle  3. Pyramid  4. Floyd  5. Letters  0. Exit
Choice: 9
Invalid choice

=== PATTERN MENU ===
1. Square  2. Triangle  3. Pyramid  4. Floyd  5. Letters  0. Exit
Choice: 0
Goodbye!
```

MODULE 07 · BEGINNER

Arrays

Minimum duration **3 days** | Level **Beginner**

An array stores a fixed number of elements of one type in contiguous memory, accessed by index.

IN THIS MODULE

7.1 Introduction to arrays
7.2 One-dimensional arrays
7.3 Multi-dimensional arrays
7.4 Variable-length arrays (VLAs)
7.5 Array algorithms

7.1 Introduction to arrays

7.1.1 Why arrays

Suppose a class has 60 students. Storing their marks in 60 separate variables (`m1, m2, … m60`) makes it impossible to loop over them, average them or sort them. An **array** is a single variable that holds many values of the same type, each reached by a number called its **index**: `marks[0]`, `marks[1]`, … With a loop, one line of code processes them all.

FIVE VALUES, ONE LOOP — main.c

```c
#include <stdio.h>

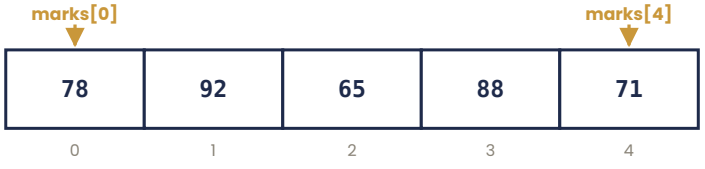
int main(void) {
    int marks[5] = {78, 92, 65, 88, 71};        /* one name, five values */
    int sum = 0;
    for (int i = 0; i < 5; i++) sum += marks[i];
    printf("total = %d, average = %.1f\n", sum, sum / 5.0);
    return 0;
}
```

OUTPUT

```
total = 394, average = 78.8
```

7.1.2 Contiguous memory and zero-based indexing

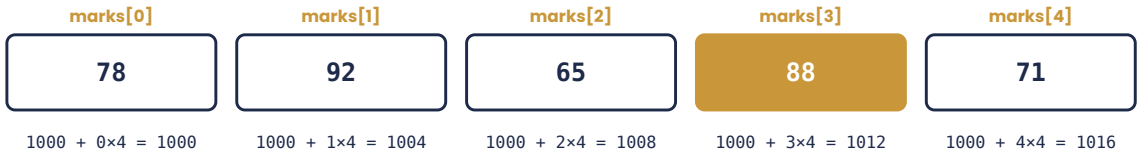
The elements of an array are stored **next to each other** in memory, with no gaps. Indexes start at **0**: an array of n elements has valid indexes 0 to n - 1. The index is really an **offset**: how many elements away from the start.



INT MARKS[5]: FIVE INTS SIDE BY SIDE, INDEXES 0 TO 4

7.1.3 Address of an element: base + index × size

Because elements are contiguous and all the same size, the address of element i is **base address + i × sizeof(element)**. The CPU computes it in one step, so accessing any element is equally fast, **O(1)**: this is called **random access**.



WITH BASE ADDRESS 1000 AND 4-BYTE INTS, MARKS[3] IS AT 1000 + 3 × 4 = 1012

ELEMENTS ARE 4 BYTES APART main.c

```c
#include <stdio.h>

int main(void) {
    int marks[5] = {78, 92, 65, 88, 71};
    for (int i = 0; i < 5; i++)
        printf("marks[%d] = %2d at %p (offset %td bytes)\n", i, marks[i], (void *)&marks[i],
               (char *)&marks[i] - (char *)&marks[0]);
    return 0;
}
```

OUTPUT (addresses differ per run; offsets do not)

```
marks[0] = 78 at 0x7ffea81bde50 (offset 0 bytes)
marks[1] = 92 at 0x7ffea81bde54 (offset 4 bytes)
marks[2] = 65 at 0x7ffea81bde58 (offset 8 bytes)
marks[3] = 88 at 0x7ffea81bde5c (offset 12 bytes)
marks[4] = 71 at 0x7ffea81bde60 (offset 16 bytes)
```

7.2 One-dimensional arrays

7.2.1 Declaration int a[5];

`type name[size];` declares an array. The size must be a positive integer constant (or, for VLAs, an expression: 7.4). The array's size is fixed for its whole lifetime. Like other locals, a local array declared without initialization contains garbage.

7.2.2 Initialization: full, partial, without size

WAYS TO INITIALIZE

main.c

```c
#include <stdio.h>

void show(const char *label, int a[], int n) {
    printf("%-22s", label);
    for (int i = 0; i < n; i++) printf("%d ", a[i]);
    printf("\n");
}
int main(void) {
    int full[5]    = {1, 2, 3, 4, 5};      /* every element given */
    int partial[5] = {7, 8};               /* rest become 0 */
    int zeros[5]   = {0};                  /* common idiom: all zeros */
    int nosize[]   = {10, 20, 30};         /* size deduced: 3 */
    show("full:", full, 5);
    show("partial {7, 8}:", partial, 5);
    show("zeros {0}:", zeros, 5);
    show("nosize (3 elements):", nosize, 3);
    printf("sizeof nosize = %zu bytes\n", sizeof nosize);
    return 0;
}
```

OUTPUT

```
full:                  1 2 3 4 5
partial {7, 8}:        7 8 0 0 0
zeros {0}:             0 0 0 0 0
nosize (3 elements):   10 20 30
sizeof nosize = 12 bytes
```

7.2.3 Designated initializers (C99)

C99 lets you initialize specific elements by index: `int a[10] = {[2] = 5, [7] = 9};`. All others become 0. Useful for lookup tables.

DESIGNATED INITIALIZERS

main.c

```c
#include <stdio.h>

int main(void) {
    int a[8] = {[1] = 10, [5] = 50, [6] = 60};
    for (int i = 0; i < 8; i++) printf("%d ", a[i]);
    printf("\n");
    const char *month_days_name[] = {[0] = "Jan", [1] = "Feb", [11] = "Dec"};
    printf("%s %s %s\n", month_days_name[0], month_days_name[1], month_days_name[11]);
    return 0;
}
```

OUTPUT

```
0 10 0 0 0 50 60 0
Jan Feb Dec
```

7.2.4 Reading and printing with loops

INPUT AND OUTPUT WITH LOOPS main.c

```c
#include <stdio.h>

int main(void) {
    int n;
    printf("How many numbers (max 10)? ");
    scanf("%d", &n);
    int a[10];
    for (int i = 0; i < n; i++) {
        printf("a[%d] = ", i);
        scanf("%d", &a[i]);
    }
    printf("You entered:");
    for (int i = 0; i < n; i++) printf(" %d", a[i]);
    printf("\nIn reverse:");
    for (int i = n - 1; i >= 0; i--) printf(" %d", a[i]);
    printf("\n");
    return 0;
}
```

CONSOLE (user input in gold)

```
How many numbers (max 10)? 4
a[0] = 12
a[1] = 7
a[2] = 30
a[3] = 5
You entered: 12 7 30 5
In reverse: 5 30 7 12
```

7.2.5 Finding the length with sizeof(a) / sizeof(a[0])

`sizeof a` gives the size of the whole array in bytes; dividing by the size of one element gives the number of elements. A common macro is `#define LEN(a) (sizeof(a) / sizeof((a)[0]))`. This works **only where the array itself is visible**; inside a function that received the array as a parameter it has decayed into a pointer and the trick fails (9.5.1).

COMPUTING THE NUMBER OF ELEMENTS main.c

```c
#include <stdio.h>
#define LEN(a) (sizeof(a) / sizeof((a)[0]))

int main(void) {
    double prices[] = {99.5, 120.0, 45.25, 300.0, 15.0, 72.5};
    printf("sizeof prices = %zu, sizeof prices[0] = %zu, length = %zu\n",
           sizeof prices, sizeof prices[0], LEN(prices));
    double total = 0;
    for (size_t i = 0; i < LEN(prices); i++) total += prices[i];
    printf("total = %.2f\n", total);
    return 0;
}
```

OUTPUT

```
sizeof prices = 48, sizeof prices[0] = 8, length = 6
total = 652.25
```

7.2.6 No bounds checking and its dangers

C does **not** check indexes. `a[5]` on a 5-element array, or `a[-1]`, silently reads or writes memory outside the array: it may change another variable, crash, or appear to work. This is **undefined behaviour** and the cause of countless security vulnerabilities (buffer overflows). You must keep every index between 0 and n - 1 yourself. Tools such as AddressSanitizer (Module 21) catch these errors.

READING PAST THE END (UNDEFINED BEHAVIOUR)

main.c

```c
#include <stdio.h>

int main(void) {
    int before = 111;
    int a[3] = {1, 2, 3};
    int after = 222;
    for (int i = 0; i <= 3; i++)                /* off-by-one: i == 3 is out of bounds */
        printf("a[%d] = %d\n", i, a[i]);
    printf("before = %d, after = %d\n", before, after);
    return 0;
}
```

OUTPUT (a[3] is whatever happens to be next in memory)

```
a[0] = 1
a[1] = 2
a[2] = 3
a[3] = -1699635456
before = 111, after = 222
```

WATCH OUT

The most common array bug is the **off-by-one** error: `for (i = 0; i <= n; i++)` instead of `i < n`.

7.3 Multi-dimensional arrays

7.3.1 2D arrays (matrices)

A 2D array is an "array of arrays": `int m[3][4];` has 3 rows, each an array of 4 ints. Access an element with two indexes, `m[row][col]`. Nested loops process it: the outer loop over rows, the inner over columns. Initializers can be nested; missing elements become 0, and the first dimension can be omitted.

A 3 X 4 MATRIX

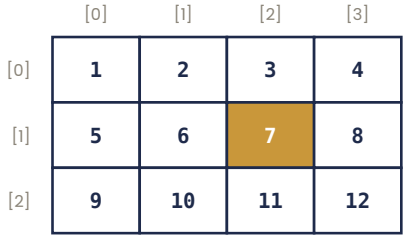
main.c

```c
#include <stdio.h>

int main(void) {
    int m[3][4] = {
        {1, 2, 3, 4},
        {5, 6, 7, 8},
        {9, 10, 11, 12}
    };
    printf("rows = %zu, cols = %zu\n", sizeof m / sizeof m[0], sizeof m[0] / sizeof m[0][0]);
    for (int r = 0; r < 3; r++) {
        for (int c = 0; c < 4; c++) printf("%4d", m[r][c]);
        printf("\n");
    }
    printf("m[1][2] = %d\n", m[1][2]);
    int marks[][3] = {{78, 82, 90}, {65, 70}};       /* first size deduced; missing -> 0 */
    printf("marks[1][2] = %d\n", marks[1][2]);
    return 0;
}
```

OUTPUT

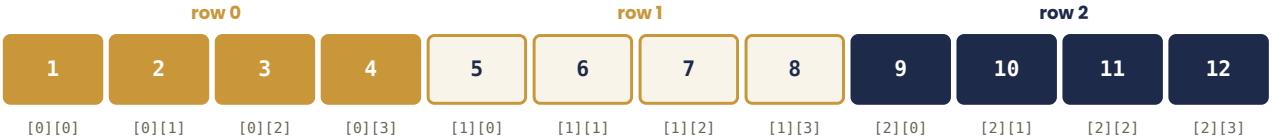
```
rows = 3, cols = 4
   1   2   3   4
   5   6   7   8
   9  10  11  12
m[1][2] = 7
marks[1][2] = 0
```



M[1][2] = 7: ROW 1, COLUMN 2

7.3.2 Row-major memory layout

Memory is one-dimensional, so C stores a 2D array **row by row** ("row-major order"): all of row 0, then all of row 1, and so on. The address of `m[r][c]` is base + (r × COLS + c) × sizeof(element). Looping row by row (inner loop over columns) visits memory sequentially, which is much faster for the CPU cache than looping column by column.



ROW-MAJOR ORDER: THE 3 X 4 MATRIX AS IT REALLY SITS IN MEMORY

THE MATRIX IS ONE BLOCK OF 12 INTS main.c

```c
#include <stdio.h>

int main(void) {
    int m[3][4] = {{1, 2, 3, 4}, {5, 6, 7, 8}, {9, 10, 11, 12}};
    int *flat = &m[0][0];                          /* view the same memory as a 1D array */
    for (int i = 0; i < 12; i++) printf("%d ", flat[i]);
    printf("\nm[2][1] is flat[2*4 + 1] = %d\n", flat[2 * 4 + 1]);
    return 0;
}
```

OUTPUT

```
1 2 3 4 5 6 7 8 9 10 11 12
m[2][1] is flat[2*4 + 1] = 10
```

7.3.3 3D arrays

Dimensions can go further: `int sales[2][3][4];` could hold 2 years × 3 regions × 4 quarters. Each extra dimension adds a nested loop. Memory is still one contiguous block in row-major order (the last index changes fastest).

A 3D ARRAY main.c

```c
#include <stdio.h>

int main(void) {
    int sales[2][3][4];                        /* [year][region][quarter] */
    for (int y = 0; y < 2; y++)
        for (int r = 0; r < 3; r++)
            for (int q = 0; q < 4; q++)
                sales[y][r][q] = (y + 1) * 100 + r * 10 + q;
    for (int y = 0; y < 2; y++) {
        printf("Year %d\n", 2025 + y);
        for (int r = 0; r < 3; r++) {
            int total = 0;
            printf("  region %d:", r);
            for (int q = 0; q < 4; q++) { printf(" %4d", sales[y][r][q]); total += sales[y][r][q]; }
            printf("  total %d\n", total);
        }
    }
    printf("sizeof sales = %zu bytes (2*3*4 ints)\n", sizeof sales);
    return 0;
}
```

OUTPUT

```
Year 2025
  region 0:  100  101  102  103  total 406
  region 1:  110  111  112  113  total 446
  region 2:  120  121  122  123  total 486
Year 2026
  region 0:  200  201  202  203  total 806
  region 1:  210  211  212  213  total 846
  region 2:  220  221  222  223  total 886
sizeof sales = 96 bytes (2*3*4 ints)
```

7.4 Variable-length arrays (VLAs)

7.4.1 VLAs in C99 and optional status in C11

C99 allows the size of a local array to be a run-time expression: `int a[n];`. The array is created on the stack when the declaration is reached and destroyed at the end of the block. VLAs **cannot be initialized** with `= {...}`, cannot be global or static, and a large n can overflow the stack with no way to detect the failure. Because of these problems C11 made VLAs an **optional** feature (MSVC does not support them). For sizes that come from the user, dynamic allocation with `malloc` (Module 13) is safer.

A VARIABLE-LENGTH ARRAY main.c

```c
#include <stdio.h>

int main(void) {
    int n;
    printf("Number of subjects: ");
    scanf("%d", &n);
    if (n <= 0 || n > 100) return 1;          /* guard the size */
    int marks[n];                             /* VLA: size known only at run time */
    for (int i = 0; i < n; i++) marks[i] = 60 + i * 7;
    printf("sizeof marks = %zu (computed at run time)\n", sizeof marks);
    for (int i = 0; i < n; i++) printf("%d ", marks[i]);
    printf("\n");
    return 0;
}
```

CONSOLE (user input in gold)

```
Number of subjects: 4
sizeof marks = 16 (computed at run time)
60 67 74 81
```

7.5 Array algorithms

7.5.1 Searching

7.5.1.1 Linear search

Check each element in turn until the target is found or the array ends. Works on **any** array (sorted or not). Time: **O(n)**.

7.5.1.2 Binary search

For a **sorted** array: look at the middle element; if it is the target, done; if the target is smaller, continue in the left half, otherwise the right half. Each step halves the range, so at most about $\log_2 n$ steps: **O(log n)**. A million elements need at most 20 comparisons.

LINEAR AND BINARY SEARCH

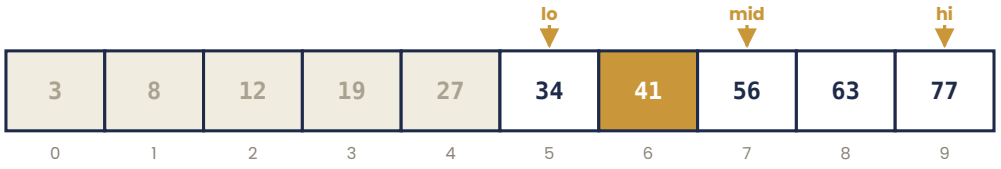
main.c

```c
#include <stdio.h>

int linear_search(const int a[], int n, int key) {
    for (int i = 0; i < n; i++)
        if (a[i] == key) return i;
    return -1;
}
int binary_search(const int a[], int n, int key) {
    int lo = 0, hi = n - 1;
    while (lo <= hi) {
        int mid = lo + (hi - lo) / 2;           /* avoids overflow of (lo + hi) */
        printf("  lo=%d hi=%d mid=%d a[mid]=%d\n", lo, hi, mid, a[mid]);
        if (a[mid] == key) return mid;
        if (key < a[mid]) hi = mid - 1;
        else lo = mid + 1;
    }
    return -1;
}
int main(void) {
    int u[] = {42, 7, 19, 88, 3, 56};
    printf("linear search 88 in unsorted array -> index %d\n", linear_search(u, 6, 88));
    int s[] = {3, 8, 12, 19, 27, 34, 41, 56, 63, 77};
    printf("binary search 41:\n");
    printf("-> index %d\n", binary_search(s, 10, 41));
    printf("binary search 20:\n");
    printf("-> index %d\n", binary_search(s, 10, 20));
    return 0;
}
```

OUTPUT

```
linear search 88 in unsorted array -> index 3
binary search 41:
  lo=0 hi=9 mid=4 a[mid]=27
  lo=5 hi=9 mid=7 a[mid]=56
  lo=5 hi=6 mid=5 a[mid]=34
  lo=6 hi=6 mid=6 a[mid]=41
-> index 6
binary search 20:
  lo=0 hi=9 mid=4 a[mid]=27
  lo=0 hi=3 mid=1 a[mid]=8
  lo=2 hi=3 mid=2 a[mid]=12
  lo=3 hi=3 mid=3 a[mid]=19
-> index -1
```



BINARY SEARCH FOR 41: AFTER THE FIRST STEP THE LEFT HALF IS DISCARDED

	Linear search	Binary search
Requires sorted data	No	Yes
Time	O(n)	O(log n)
n = 1,000,000 (worst case)	1,000,000 comparisons	20 comparisons

	Linear search	Binary search
Best for	Small or unsorted arrays, one-off searches	Large sorted arrays, repeated searches

7.5.2 Sorting

Sorting arranges elements in order. The three simple O(n²) algorithms below are the foundation for understanding faster ones (quicksort, merge sort: Module 19) and the library's `qsort` (Module 12).

7.5.2.1 Bubble sort

Repeatedly walk through the array swapping adjacent elements that are out of order. After each pass the largest remaining element has "bubbled" to the end. If a pass makes no swaps, the array is sorted and we can stop early.

BUBBLE SORT, PASS BY PASS main.c

```c
#include <stdio.h>
#include <stdbool.h>

void print(const int a[], int n) { for (int i = 0; i < n; i++) printf("%d ", a[i]); printf("\n"); }

void bubble_sort(int a[], int n) {
    for (int pass = 0; pass < n - 1; pass++) {
        bool swapped = false;
        for (int j = 0; j < n - 1 - pass; j++)
            if (a[j] > a[j + 1]) {
                int t = a[j]; a[j] = a[j + 1]; a[j + 1] = t;
                swapped = true;
            }
        printf("after pass %d: ", pass + 1); print(a, n);
        if (!swapped) break;                    /* already sorted */
    }
}
int main(void) {
    int a[] = {29, 10, 14, 37, 13};
    bubble_sort(a, 5);
    return 0;
}
```

OUTPUT

```
after pass 1: 10 14 29 13 37
after pass 2: 10 14 13 29 37
after pass 3: 10 13 14 29 37
after pass 4: 10 13 14 29 37
```

7.5.2.2 Selection sort

For each position i, find the **smallest** element in the unsorted part (i to n-1) and swap it into position i. It makes at most n - 1 swaps, which helps when swaps are expensive.

SELECTION SORT main.c

```c
#include <stdio.h>

void print(const int a[], int n) { for (int i = 0; i < n; i++) printf("%d ", a[i]); printf("\n"); }

void selection_sort(int a[], int n) {
    for (int i = 0; i < n - 1; i++) {
        int min = i;
        for (int j = i + 1; j < n; j++)
            if (a[j] < a[min]) min = j;
        int t = a[i]; a[i] = a[min]; a[min] = t;
        printf("i=%d: smallest %d moved to front -> ", i, a[i]); print(a, n);
    }
}
int main(void) {
    int a[] = {64, 25, 12, 22, 11};
    selection_sort(a, 5);
    return 0;
}
```

OUTPUT

```
i=0: smallest 11 moved to front -> 11 25 12 22 64
i=1: smallest 12 moved to front -> 11 12 25 22 64
i=2: smallest 22 moved to front -> 11 12 22 25 64
i=3: smallest 25 moved to front -> 11 12 22 25 64
```

7.5.2.3 Insertion sort

Build the sorted array one element at a time, like sorting playing cards in your hand: take the next element (the **key**) and shift larger elements of the sorted part one place right until the key's position is found. Very fast on nearly sorted data (close to O(n)), and used inside many fast sorting libraries for small subarrays.

INSERTION SORT

main.c

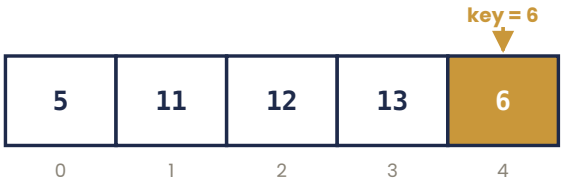
```c
#include <stdio.h>

void print(const int a[], int n) { for (int i = 0; i < n; i++) printf("%d ", a[i]); printf("\n"); }

void insertion_sort(int a[], int n) {
    for (int i = 1; i < n; i++) {
        int key = a[i], j = i - 1;
        while (j >= 0 && a[j] > key) {          /* shift bigger elements right */
            a[j + 1] = a[j];
            j--;
        }
        a[j + 1] = key;                         /* drop the key into its gap */
        printf("insert %2d -> ", key); print(a, n);
    }
}
int main(void) {
    int a[] = {12, 11, 13, 5, 6};
    insertion_sort(a, 5);
    return 0;
}
```

OUTPUT

```
insert 11 -> 11 12 13 5 6
insert 13 -> 11 12 13 5 6
insert  5 -> 5 11 12 13 6
insert  6 -> 5 6 11 12 13
```



INSERTING 6: 13, 12 AND 11 SHIFT RIGHT, THEN 6 DROPS INTO INDEX 1

Algorithm	Best	Average / worst	Swaps	Stable?	Notes
Bubble	O(n) with early exit	O(n²)	Many	Yes	Simplest to explain
Selection	O(n²)	O(n²)	≤ n - 1	No	Few writes
Insertion	O(n)	O(n²)	Many shifts	Yes	Best of the three in practice

7.5.3 Common problems: max/min, second largest, reverse, rotate, remove duplicates, frequency count, merge two arrays

SEVEN CLASSIC ARRAY PROBLEMS

main.c

```c
#include <stdio.h>

void print(const char *label, const int a[], int n) {
    printf("%-18s", label);
    for (int i = 0; i < n; i++) printf("%d ", a[i]);
    printf("\n");
}
int main(void) {
    int a[] = {4, 9, 2, 9, 7, 2, 5};
    int n = 7;
```

```
    print("array:", a, n);

    int max = a[0], min = a[0];                              /* max / min */
    for (int i = 1; i < n; i++) { if (a[i] > max) max = a[i]; if (a[i] < min) min = a[i]; }
    printf("max = %d, min = %d\n", max, min);

    int first = -1, second = -1;                             /* second largest (distinct) */
    for (int i = 0; i < n; i++) {
        if (a[i] > first) { second = first; first = a[i]; }
        else if (a[i] > second && a[i] != first) second = a[i];
    }
    printf("second largest = %d\n", second);

    int r[7]; for (int i = 0; i < n; i++) r[i] = a[i];       /* reverse in place */
    for (int i = 0, j = n - 1; i < j; i++, j--) { int t = r[i]; r[i] = r[j]; r[j] = t; }
    print("reversed:", r, n);

    int k = 2, rot[7];                                       /* rotate left by k */
    for (int i = 0; i < n; i++) rot[i] = a[(i + k) % n];
    print("rotated left 2:", rot, n);

    int u[7], m = 0;                                         /* remove duplicates */
    for (int i = 0; i < n; i++) {
        int seen = 0;
        for (int j = 0; j < m; j++) if (u[j] == a[i]) { seen = 1; break; }
        if (!seen) u[m++] = a[i];
    }
    print("unique:", u, m);

    int freq[10] = {0};                                      /* frequency count (values 0-9) */
    for (int i = 0; i < n; i++) freq[a[i]]++;
    printf("frequencies:      ");
    for (int v = 0; v < 10; v++) if (freq[v]) printf("%d:%d ", v, freq[v]);
    printf("\n");

    int x[] = {1, 4, 7, 10}, y[] = {2, 3, 8, 12, 15}, z[9];  /* merge two sorted arrays */
    int i = 0, j = 0, t = 0;
    while (i < 4 && j < 5) z[t++] = (x[i] <= y[j]) ? x[i++] : y[j++];
    while (i < 4) z[t++] = x[i++];
    while (j < 5) z[t++] = y[j++];
    print("merged sorted:", z, t);
    return 0;
}
```

```
OUTPUT
array:           4 9 2 9 7 2 5
max = 9, min = 2
second largest = 7
reversed:        5 2 7 9 2 9 4
rotated left 2:  2 9 7 2 5 4 9
unique:          4 9 2 7 5
frequencies:     2:2 4:1 5:1 7:1 9:2
merged sorted:   1 2 3 4 7 8 10 12 15
```

7.5.4 Matrix problems: addition, multiplication, transpose, diagonal sums, spiral print

Matrix multiplication: if A is r × n and B is n × c, then C = A × B is r × c with `C[i][j] = Σ A[i][k] × B[k][j]` for k = 0…n−1: row i of A combined with column j of B. The number of columns of A must equal the number of rows of B.

MATRIX OPERATIONS AND SPIRAL PRINT

main.c

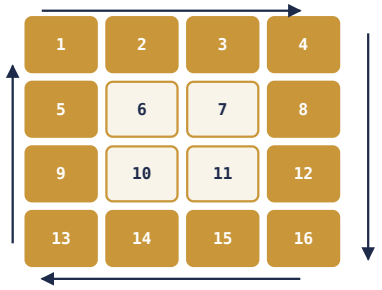
```c
#include <stdio.h>
#define N 3

void show(const char *title, int m[N][N]) {
    printf("%s\n", title);
    for (int i = 0; i < N; i++) { for (int j = 0; j < N; j++) printf("%5d", m[i][j]); printf("\n"); }
}
int main(void) {
    int a[N][N] = {{1, 2, 3}, {4, 5, 6}, {7, 8, 9}};
    int b[N][N] = {{9, 8, 7}, {6, 5, 4}, {3, 2, 1}};
    int sum[N][N], prod[N][N], tr[N][N];
    for (int i = 0; i < N; i++)
        for (int j = 0; j < N; j++) {
            sum[i][j] = a[i][j] + b[i][j];
            tr[j][i] = a[i][j];
            prod[i][j] = 0;
            for (int k = 0; k < N; k++) prod[i][j] += a[i][k] * b[k][j];
        }
    show("A + B:", sum);
    show("A x B:", prod);
    show("transpose of A:", tr);
    int d1 = 0, d2 = 0;
    for (int i = 0; i < N; i++) { d1 += a[i][i]; d2 += a[i][N - 1 - i]; }
    printf("main diagonal sum = %d, anti-diagonal sum = %d\n", d1, d2);

    int s[4][4] = {{1, 2, 3, 4}, {5, 6, 7, 8}, {9, 10, 11, 12}, {13, 14, 15, 16}};
    int top = 0, bottom = 3, left = 0, right = 3;
    printf("spiral: ");
    while (top <= bottom && left <= right) {
        for (int j = left; j <= right; j++) printf("%d ", s[top][j]);
        top++;
        for (int i = top; i <= bottom; i++) printf("%d ", s[i][right]);
        right--;
        if (top <= bottom) { for (int j = right; j >= left; j--) printf("%d ", s[bottom][j]); bottom-
-; }
        if (left <= right) { for (int i = bottom; i >= top; i--) printf("%d ", s[i][left]); left++; }
    }
    printf("\n");
    return 0;
}
```

OUTPUT

```
A + B:
   10   10   10
   10   10   10
   10   10   10
A x B:
   30   24   18
   84   69   54
  138  114   90
transpose of A:
    1    4    7
    2    5    8
    3    6    9
main diagonal sum = 15, anti-diagonal sum = 15
spiral: 1 2 3 4 8 12 16 15 14 13 9 5 6 7 11 10
```



SPIRAL ORDER: OUTER RING (GOLD) CLOCKWISE, THEN THE INNER RING

PRACTICE CHECKPOINT

Read n numbers into an array, sort them with **insertion sort**, then **binary-search** for a value; **multiply two matrices**.

SAMPLE SOLUTION main.c

```c
#include <stdio.h>

int main(void) {
    int n, a[50];
    printf("How many numbers? ");
    scanf("%d", &n);
    printf("Enter %d numbers: ", n);
    for (int i = 0; i < n; i++) scanf("%d", &a[i]);

    for (int i = 1; i < n; i++) {                    /* insertion sort */
        int key = a[i], j = i - 1;
        while (j >= 0 && a[j] > key) { a[j + 1] = a[j]; j--; }
        a[j + 1] = key;
    }
    printf("Sorted: ");
    for (int i = 0; i < n; i++) printf("%d ", a[i]);

    int key;
    printf("\nSearch for: ");
    scanf("%d", &key);
    int lo = 0, hi = n - 1, pos = -1;                /* binary search */
    while (lo <= hi) {
        int mid = lo + (hi - lo) / 2;
        if (a[mid] == key) { pos = mid; break; }
        if (key < a[mid]) hi = mid - 1; else lo = mid + 1;
    }
    if (pos >= 0) printf("%d found at index %d\n", key, pos);
    else printf("%d not found\n", key);

    int r1, c1, r2, c2, A[10][10], B[10][10], C[10][10];
    printf("Rows and columns of A: "); scanf("%d %d", &r1, &c1);
    printf("Rows and columns of B: "); scanf("%d %d", &r2, &c2);
    if (c1 != r2) { printf("Cannot multiply: columns of A must equal rows of B\n"); return 1; }
    printf("Enter A (%d values): ", r1 * c1);
    for (int i = 0; i < r1; i++) for (int j = 0; j < c1; j++) scanf("%d", &A[i][j]);
    printf("Enter B (%d values): ", r2 * c2);
    for (int i = 0; i < r2; i++) for (int j = 0; j < c2; j++) scanf("%d", &B[i][j]);
    for (int i = 0; i < r1; i++)
        for (int j = 0; j < c2; j++) {
            C[i][j] = 0;
            for (int k = 0; k < c1; k++) C[i][j] += A[i][k] * B[k][j];
        }
    printf("A x B =\n");
    for (int i = 0; i < r1; i++) { for (int j = 0; j < c2; j++) printf("%6d", C[i][j]); printf("\n");
```

```
}
    return 0;
}
```

CONSOLE (user input in gold)

```
How many numbers? 6
Enter 6 numbers: 42 7 19 88 3 56
Sorted: 3 7 19 42 56 88
Search for: 19
19 found at index 2
Rows and columns of A: 2 3
Rows and columns of B: 3 2
Enter A (6 values): 1 2 3 4 5 6
Enter B (6 values): 7 8 9 10 11 12
A x B =
    58    64
   139   154
```

MODULE 08 · BEGINNER

Strings

Minimum duration **3 days** Level **Beginner**

In C, a string is an array of characters ending with the null character \0; this module covers how strings are stored and manipulated.

IN THIS MODULE

8.1 Introduction to strings
8.2 Reading and printing strings
8.3 string.h functions
8.4 ctype.h functions
8.5 Conversions
8.6 Arrays of strings
8.7 Writing your own string functions
8.8 Classic string problems

8.1 Introduction to strings

8.1.1 Character arrays and the null terminator \0

C has no separate string type. A **string** is a sequence of characters stored in a `char` array and **terminated by the null character** `'\0'` (the byte with value 0). Every string function relies on that terminator to know where the string ends: `strlen`, `printf("%s")` and the rest simply walk forward until they meet `\0`. An array holding a string of n characters therefore needs at least **n + 1** bytes.



CHAR S[8] = "HELLO": FIVE CHARACTERS, THE TERMINATOR, AND TWO UNUSED BYTES

8.1.2 Declaring and initializing: char s[10] = "hi"; and char s[] = {'h','i','\0'};

WAYS TO CREATE A STRING

main.c

```c
#include <stdio.h>
#include <string.h>

int main(void) {
    char a[10] = "hi";                    /* 'h','i','\0', rest filled with '\0' */
    char b[] = "hello";                   /* size deduced: 6 (5 chars + '\0') */
    char c[] = {'h', 'i', '\0'};          /* character by character: must add '\0' */
    char d[] = {'n', 'o'};                /* NOT a string: no terminator! */
    printf("a = \"%s\", sizeof %zu, strlen %zu\n", a, sizeof a, strlen(a));
    printf("b = \"%s\", sizeof %zu, strlen %zu\n", b, sizeof b, strlen(b));
    printf("c = \"%s\", sizeof %zu, strlen %zu\n", c, sizeof c, strlen(c));
    printf("d has sizeof %zu but no '\\0', so printing it with %%s is undefined\n", sizeof d);
    return 0;
}
```

OUTPUT

```
a = "hi", sizeof 10, strlen 2
b = "hello", sizeof 6, strlen 5
c = "hi", sizeof 3, strlen 2
d has sizeof 2 but no '\0', so printing it with %s is undefined
```

Declaration	sizeof	strlen	Note
`char a[10] = "hi";`	10	2	extra bytes zeroed
`char b[] = "hello";`	6	5	room for `\0` added automatically
`char c[] = {'h','i','\0'};`	3	2	terminator written by hand
`char e[5] = "hello";`	5	?	legal in C, but no room for `\0`: not a string!

8.1.3 String literals and read-only memory

A **string literal** such as `"Learn"` is stored by the compiler in a static, usually **read-only** area of memory for the whole run of the program. Identical literals may share storage. Attempting to modify a literal is undefined behaviour and on Linux crashes with a segmentation fault.

8.1.4 Character array vs string literal pointer

`char arr[] = "text";` creates a **modifiable array** initialized with a copy of the literal. `char *p = "text";` creates a **pointer** to the literal itself, which must not be modified (declare it `const char *` to let the compiler enforce that).

AN ARRAY IS A COPY; A LITERAL IS READ-ONLY

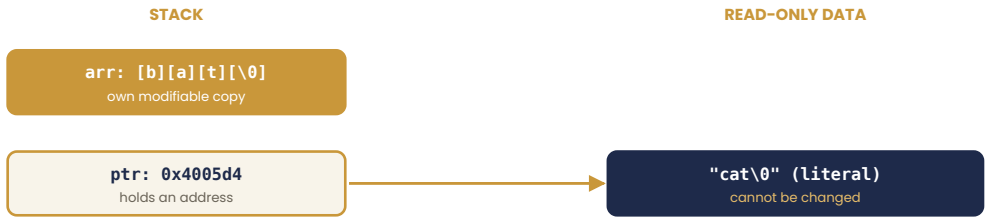
main.c

```c
#include <stdio.h>

int main(void) {
    char arr[] = "cat";             /* own copy on the stack: modifiable */
    const char *ptr = "cat";        /* points to read-only literal */
    arr[0] = 'b';
    printf("arr = %s, ptr = %s\n", arr, ptr);
    printf("sizeof arr = %zu (array), sizeof ptr = %zu (pointer)\n", sizeof arr, sizeof ptr);
    ptr = "dog";                    /* the pointer itself can point elsewhere */
    printf("ptr now = %s\n", ptr);
    char *bad = "cat";
    bad[0] = 'b';                   /* writing to a literal: crash */
    printf("never printed\n");
    return 0;
}
```

OUTPUT

```
arr = bat, ptr = cat
sizeof arr = 4 (array), sizeof ptr = 8 (pointer)
ptr now = dog
Segmentation fault (core dumped)
```



CHAR ARR[] COPIES THE TEXT; CONST CHAR *PTR ONLY POINTS AT IT

8.2 Reading and printing strings

8.2.1 scanf("%s") and its limits

`scanf("%s", s)` skips leading whitespace and reads characters up to the next whitespace: it reads **one word** only, and without a width it can overflow the array (5.2.6). Always use a width: `scanf("%29s", s)` for `char s[30]`. To read up to a newline with scanf, use a scanset: `scanf(" %29[^\n]", s)`.

8.2.2 fgets() and removing the trailing newline

`fgets(s, sizeof s, stdin)` reads a whole line including spaces, never overflows, and keeps the `'\n'` if it fits. Remove it with `s[strcspn(s, "\n")] = '\0';`. If the line was longer than the buffer, the rest stays in the input and will be read by the next call.

8.2.3 puts() and printf("%s")

READING AND PRINTING STRINGS

main.c

```c
#include <stdio.h>
#include <string.h>

int main(void) {
    char first[20], full[40];
    printf("First name: ");
    scanf("%19s", first);                    /* one word */
    int c; while ((c = getchar()) != '\n' && c != EOF) { }   /* discard rest of line */
    printf("Full name: ");
    fgets(full, sizeof full, stdin);         /* whole line, spaces included */
    full[strcspn(full, "\n")] = '\0';
    printf("first = [%s]\n", first);
    printf("full  = [%s]\n", full);
    puts(full);                              /* adds a newline */
    printf("%.5s...\n", full);               /* print only the first 5 characters */
    printf("[%10s] [%-10s]\n", first, first); /* width and alignment work for strings too */
    return 0;
}
```

CONSOLE (user input in gold)

```
First name: Asha Rao
Full name: Asha Lakshmi Rao
first = [Asha]
full  = [Asha Lakshmi Rao]
Asha Lakshmi Rao
Asha ...
[      Asha] [Asha      ]
```

8.3 string.h functions

The header `<string.h>` provides the standard string and memory functions. They all assume valid, terminated strings and destination buffers that are **big enough**: they never check sizes themselves.

8.3.1 strlen()

`strlen(s)` returns the number of characters **before** the terminator, as `size_t`. It walks the whole string each time (O(n)), so do not call it in a loop condition on a long string you are not changing; store the length in a variable.

8.3.2 strcpy() and strncpy()

`strcpy(dest, src)` copies src including its `\0` into dest; dest must be large enough. `strncpy(dest, src, n)` copies at most n characters, but if src is n characters or longer it does **not** add a terminator, and if shorter it pads with zeros. Always terminate manually after strncpy. (You cannot assign arrays with `=`: `dest = src;` is an error.)

8.3.3 strcat() and strncat()

`strcat(dest, src)` appends src to the end of dest. `strncat(dest, src, n)` appends at most n characters and **always** adds the terminator, so dest needs room for strlen(dest) + n + 1 bytes.

COPYING AND CONCATENATING main.c

```c
#include <stdio.h>
#include <string.h>

int main(void) {
    char greeting[40];
    strcpy(greeting, "Learn");              /* copy */
    strcat(greeting, " Being");             /* append */
    strcat(greeting, " Forward");
    printf("%s (length %zu)\n", greeting, strlen(greeting));

    char code[6];
    strncpy(code, "JAVA2026", sizeof code - 1);   /* copies 5 chars, no '\0' added */
    code[sizeof code - 1] = '\0';                 /* terminate it ourselves */
    printf("strncpy result: %s\n", code);

    char path[20] = "/home/";
    strncat(path, "lohit/documents/notes", sizeof path - strlen(path) - 1);   /* never overflow */
    printf("strncat result: %s (%zu chars)\n", path, strlen(path));
    return 0;
}
```

OUTPUT

```
Learn Being Forward (length 19)
strncpy result: JAVA2
strncat result: /home/lohit/documen (19 chars)
```

8.3.4 strcmp() and strncmp()

`strcmp(a, b)` compares strings character by character (by character code) and returns **0** if equal, a **negative** value if a comes first, a **positive** value if a comes after. Never compare strings with `==`: that compares the **addresses**, not the contents. `strncmp(a, b, n)` compares at most n characters (useful for prefixes).

COMPARING STRINGS main.c

```c
#include <stdio.h>
#include <string.h>

int main(void) {
    char a[] = "apple", b[] = "apple", c[] = "banana";
    printf("strcmp(apple, apple)  = %d\n", strcmp(a, b));
    printf("strcmp(apple, banana) < 0 ? %s\n", strcmp(a, c) < 0 ? "yes" : "no");
    printf("strcmp(banana, apple) > 0 ? %s\n", strcmp(c, a) > 0 ? "yes" : "no");
    printf("strcmp(Zoo, apple) < 0 ? %s (uppercase codes are smaller)\n", strcmp("Zoo", "apple") < 0 ?
"yes" : "no");
    printf("a == b (addresses)    = %d\n", a == b);
    printf("strncmp(\"program\", \"progress\", 5) = %d\n", strncmp("program", "progress", 5));
    const char *cmd = "exit";
    if (strcmp(cmd, "exit") == 0) printf("command recognised\n");
    return 0;
}
```

OUTPUT

```
strcmp(apple, apple)  = 0
strcmp(apple, banana) < 0 ? yes
strcmp(banana, apple) > 0 ? yes
strcmp(Zoo, apple) < 0 ? yes (uppercase codes are smaller)
a == b (addresses)    = 0
strncmp("program", "progress", 5) = 0
command recognised
```

8.3.5 strchr(), strrchr(), strstr()

`strchr(s, c)` returns a pointer to the **first** occurrence of character c in s, or NULL; `strrchr` finds the **last** occurrence; `strstr(s, sub)` finds the first occurrence of a substring. Subtracting the string's start from the result gives the index.

SEARCHING INSIDE STRINGS main.c

```c
#include <stdio.h>
#include <string.h>

int main(void) {
    const char *email = "lohit.kumar@learnbeingforward.in";
    const char *at = strchr(email, '@');
    const char *dot = strrchr(email, '.');
    if (at) printf("'@' at index %td, domain = %s\n", at - email, at + 1);
    if (dot) printf("last '.' at index %td, extension = %s\n", dot - email, dot + 1);
    const char *found = strstr(email, "being");
    printf("\"being\" %s", found ? "found at index " : "not found");
    if (found) printf("%td", found - email);
    printf("\nstrchr(email, '#') = %s\n", strchr(email, '#') ? "found" : "NULL");
    return 0;
}
```

OUTPUT

```
'@' at index 11, domain = learnbeingforward.in
last '.' at index 29, extension = in
"being" found at index 17
strchr(email, '#') = NULL
```

8.3.6 strtok()

`strtok(s, delims)` splits a string into **tokens** separated by any of the delimiter characters. The first call passes the string; later calls pass **NULL** to continue. It **modifies** the string (writes `\0` over delimiters), keeps hidden state (so it is not safe for two strings at once or in threads; POSIX offers `strtok_r`), and treats consecutive delimiters as one.

SPLITTING WITH STRTOK main.c

```c
#include <stdio.h>
#include <string.h>

int main(void) {
    char line[] = "Asha,21,,CSE;Pune";
    printf("tokens:\n");
    for (char *tok = strtok(line, ",;"); tok != NULL; tok = strtok(NULL, ",;"))
        printf("  [%s]\n", tok);
    char sentence[] = "  C   is a  powerful language ";
    int words = 0;
    for (char *w = strtok(sentence, " "); w; w = strtok(NULL, " ")) words++;
    printf("word count = %d\n", words);
    return 0;
}
```

```
OUTPUT
tokens:
  [Asha]
  [21]
  [CSE]
  [Pune]
word count = 5
```



STRTOK WRITES \0 OVER THE DELIMITER THAT ENDS EACH TOKEN; THE EXTRA COMMA IS SKIPPED, NOT OVERWRITTEN

8.3.7 memcpy(), memmove(), memset(), memcmp()

The `mem` functions work on raw **bytes** with an explicit count, ignoring `\0`, so they work for any data (arrays of ints, structures). `memcpy(dst, src, n)` copies n bytes (regions must not overlap); `memmove` does the same but handles overlapping regions correctly; `memset(p, value, n)` fills n bytes with a byte value (use it with 0, not with arbitrary ints); `memcmp(a, b, n)` compares n bytes.

BYTE-LEVEL MEMORY FUNCTIONS

main.c

```c
#include <stdio.h>
#include <string.h>

int main(void) {
    int src[5] = {1, 2, 3, 4, 5}, dst[5];
    memcpy(dst, src, sizeof src);                    /* copy 20 bytes */
    printf("memcpy : %d %d %d %d %d\n", dst[0], dst[1], dst[2], dst[3], dst[4]);

    char text[] = "abcdef";
    memmove(text + 2, text, 4);                      /* overlapping: shift right by 2 */
    printf("memmove: %s\n", text);

    int zeros[6];
    memset(zeros, 0, sizeof zeros);                  /* set all bytes to 0 */
    char stars[11];
    memset(stars, '*', 10); stars[10] = '\0';
    printf("memset : %d %d ... and %s\n", zeros[0], zeros[5], stars);

    printf("memcmp : %d (equal)\n", memcmp(src, dst, sizeof src));
    return 0;
}
```

OUTPUT

```
memcpy : 1 2 3 4 5
memmove: ababcd
memset : 0 0 ... and **********
memcmp : 0 (equal)
```

8.3.8 Buffer overflows and safer alternatives

A **buffer overflow** happens when a function writes past the end of an array: `strcpy` into a too-small buffer, `strcat` without room, `sprintf` producing too much text, `scanf("%s")` without a width. The extra bytes overwrite neighbouring variables, return addresses and other data, causing crashes or security exploits. Safer habits:

Risky	Safer alternative
`gets(s)`	`fgets(s, sizeof s, stdin)`
`scanf("%s", s)`	`scanf("%29s", s)` or fgets
`strcpy(d, s)`	check `strlen(s) < sizeof d` first, or `snprintf(d, sizeof d, "%s", s)`
`strcat(d, s)`	`strncat(d, s, sizeof d - strlen(d) - 1)`
`sprintf(d, ...)`	`snprintf(d, sizeof d, ...)` and check the return value
`strncpy` without termination	terminate manually, or use snprintf

AVOIDING OVERFLOW

main.c

```c
#include <stdio.h>
#include <string.h>

int main(void) {
    char small[8];
    const char *input = "a very long user name";
    int needed = snprintf(small, sizeof small, "%s", input);     /* truncates safely */
    printf("stored \"%s\" (%zu chars); full text needed %d -> %s\n",
           small, strlen(small), needed, needed >= (int)sizeof small ? "TRUNCATED" : "ok");
    if (strlen(input) < sizeof small) strcpy(small, input);
    else printf("refusing strcpy: input too long for the buffer\n");
    return 0;
}
```

OUTPUT

```
stored "a very " (7 chars); full text needed 21 -> TRUNCATED
refusing strcpy: input too long for the buffer
```

WATCH OUT

Every time you write into a char array, ask: "How do I know it fits?" If you cannot answer, the code is unsafe.

8.4 ctype.h functions

8.4.1 isalpha(), isdigit(), isspace(), isupper(), islower(), isalnum(), ispunct()

8.4.2 toupper(), tolower()

`<ctype.h>` classifies and converts single characters. The test functions return non-zero (true) or 0. Pass them an `unsigned char` value or EOF; passing a negative `char` (possible for non-ASCII bytes) is undefined, hence the cast in robust code.

Function	True for	Function	True for
`isalpha(c)`	letters A–Z a–z	`isupper(c)`	A–Z
`isdigit(c)`	0–9	`islower(c)`	a–z
`isalnum(c)`	letter or digit	`isspace(c)`	space, \t, \n, \r, \v, \f
`ispunct(c)`	printable non-alphanumeric: ! , . ? …	`isxdigit(c)`	0–9 a–f A–F
`toupper(c)`	returns the uppercase version (others unchanged)	`tolower(c)`	returns the lowercase version

CLASSIFYING AND CONVERTING CHARACTERS main.c

```c
#include <stdio.h>
#include <ctype.h>

int main(void) {
    const char *s = "Hello, LBF 2026!\tOK";
    int letters = 0, upper = 0, digits = 0, spaces = 0, punct = 0;
    for (int i = 0; s[i]; i++) {
        unsigned char c = (unsigned char)s[i];
        if (isalpha(c)) { letters++; if (isupper(c)) upper++; }
        else if (isdigit(c)) digits++;
        else if (isspace(c)) spaces++;
        else if (ispunct(c)) punct++;
    }
    printf("letters %d (upper %d), digits %d, spaces %d, punctuation %d\n", letters, upper, digits,
spaces, punct);
    char title[] = "learn being forward";
    int new_word = 1;
    for (int i = 0; title[i]; i++) {
        if (isspace((unsigned char)title[i])) new_word = 1;
        else if (new_word) { title[i] = toupper((unsigned char)title[i]); new_word = 0; }
    }
    printf("title case: %s\n", title);
    return 0;
}
```

OUTPUT

```
letters 10 (upper 6), digits 4, spaces 3, punctuation 2
title case: Learn Being Forward
```

8.5 Conversions

8.5.1 atoi(), atof()

`atoi(s)` ("ASCII to integer") and `atof(s)` (to double) from `<stdlib.h>` convert the leading part of a string to a number. They are simple but have **no error reporting**: `atoi("abc")` and `atoi("0")` both return 0, and overflow is undefined.

8.5.2 strtol(), strtod() with error checking

`strtol(s, &end, base)` converts to `long` in any base (2–36; 0 auto-detects 0x/0 prefixes) and sets `end` to the first character it did not use; `strtod` does the same for doubles. Check: `end == s` means no digits; `*end != '\0'` means trailing garbage; `errno == ERANGE` means overflow.

CONVERTING TEXT TO NUMBERS SAFELY

main.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <errno.h>

int parse_int(const char *s, long *out) {
    char *end;
    errno = 0;
    long v = strtol(s, &end, 10);
    if (end == s) return 0;                 /* no digits at all */
    if (*end != '\0') return 0;             /* junk after the number */
    if (errno == ERANGE) return 0;          /* out of range */
    *out = v;
    return 1;
}
int main(void) {
    printf("atoi(\"42\") = %d, atoi(\"42abc\") = %d, atoi(\"abc\") = %d\n", atoi("42"), atoi("42abc"),
atoi("abc"));
    printf("atof(\"3.75kg\") = %.2f\n", atof("3.75kg"));
    const char *tests[] = {"123", "-56", "12x", "", "99999999999999999999"};
    for (int i = 0; i < 5; i++) {
        long v;
        if (parse_int(tests[i], &v)) printf("\"%s\" -> %ld\n", tests[i], v);
        else printf("\"%s\" -> invalid\n", tests[i]);
    }
    printf("strtol(\"ff\", 16) = %ld, strtol(\"0x1A\", 0) = %ld, strtol(\"1011\", 2) = %ld\n",
           strtol("ff", NULL, 16), strtol("0x1A", NULL, 0), strtol("1011", NULL, 2));
    char *end;
    double d = strtod("9.81 m/s2", &end);
    printf("strtod: %.2f, rest = \"%s\"\n", d, end);
    return 0;
}
```

OUTPUT

```
atoi("42") = 42, atoi("42abc") = 42, atoi("abc") = 0
atof("3.75kg") = 3.75
"123" -> 123
"-56" -> -56
"12x" -> invalid
"" -> invalid
"99999999999999999999" -> invalid
strtol("ff", 16) = 255, strtol("0x1A", 0) = 26, strtol("1011", 2) = 11
strtod: 9.81, rest = " m/s2"
```

8.5.3 Numbers to strings with sprintf() / snprintf()

BUILDING STRINGS FROM NUMBERS

main.c

```c
#include <stdio.h>

int main(void) {
    char buf[32];
    int roll = 42; double gpa = 8.756;
    snprintf(buf, sizeof buf, "%d", roll);
    printf("int -> \"%s\"\n", buf);
    snprintf(buf, sizeof buf, "%.2f", gpa);
    printf("double -> \"%s\"\n", buf);
    snprintf(buf, sizeof buf, "STU-%05d-%s", roll, "CSE");
    printf("formatted id -> \"%s\"\n", buf);
    snprintf(buf, sizeof buf, "%x", 48879);
    printf("hex -> \"%s\"\n", buf);
    return 0;
}
```

OUTPUT

```
int -> "42"
double -> "8.76"
formatted id -> "STU-00042-CSE"
hex -> "beef"
```

8.6 Arrays of strings

8.6.1 2D character arrays

`char names[5][20];` holds 5 strings of up to 19 characters each, in one rectangular block of 100 bytes. Each row is a separate string: `names[i]`. Short names waste the rest of their row.

A 2D CHAR ARRAY

main.c

```c
#include <stdio.h>
#include <string.h>

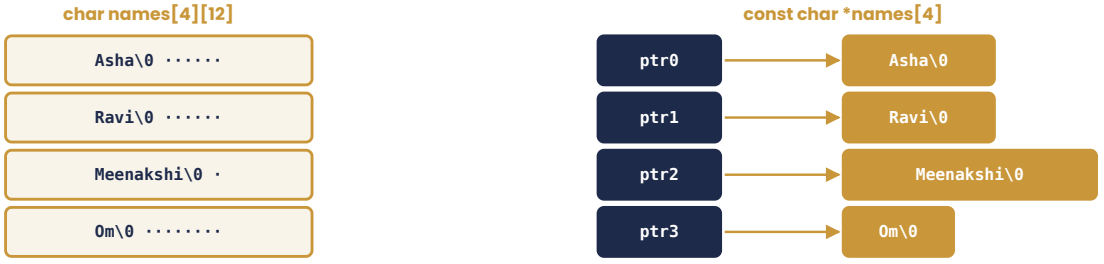
int main(void) {
    char names[4][12] = {"Asha", "Ravi", "Meenakshi", "Om"};
    printf("sizeof names = %zu bytes (4 x 12)\n", sizeof names);
    for (int i = 0; i < 4; i++)
        printf("names[%d] = %-10s (%zu chars, row of 12 bytes)\n", i, names[i], strlen(names[i]));
    strcpy(names[3], "Omkar");                /* rows are modifiable arrays */
    printf("changed: %s\n", names[3]);
    printf("first letter of each: ");
    for (int i = 0; i < 4; i++) printf("%c", names[i][0]);
    printf("\n");
    return 0;
}
```

OUTPUT

```
sizeof names = 48 bytes (4 x 12)
names[0] = Asha       (4 chars, row of 12 bytes)
names[1] = Ravi       (4 chars, row of 12 bytes)
names[2] = Meenakshi  (9 chars, row of 12 bytes)
names[3] = Om         (2 chars, row of 12 bytes)
changed: Omkar
first letter of each: ARMO
```

8.6.2 Array of character pointers (covered again in Module 12)

`const char *names[] = {"Asha", "Ravi", …};` is an array of **pointers**, each pointing to a literal of its own length: no wasted space, and sorting means swapping pointers (8 bytes) instead of copying whole strings. The strings are read-only literals.



FIXED ROWS (LEFT) VS POINTERS TO STRINGS OF EXACT LENGTH (RIGHT)

8.7 Writing your own string functions

8.7.1 my_strlen, my_strcpy, my_strcmp, my_strcat

Writing the library functions yourself shows exactly how they work: each is a loop that walks the characters until `\0`.

THE LIBRARY FUNCTIONS, HAND-WRITTEN

main.c

```c
#include <stdio.h>

size_t my_strlen(const char *s) {
    size_t n = 0;
    while (s[n] != '\0') n++;
    return n;
}
char *my_strcpy(char *dest, const char *src) {
    size_t i = 0;
    while ((dest[i] = src[i]) != '\0') i++;     /* copies the terminator too */
    return dest;
}
int my_strcmp(const char *a, const char *b) {
    size_t i = 0;
    while (a[i] != '\0' && a[i] == b[i]) i++;
    return (unsigned char)a[i] - (unsigned char)b[i];
}
char *my_strcat(char *dest, const char *src) {
    size_t d = my_strlen(dest), i = 0;
    while (src[i] != '\0') { dest[d + i] = src[i]; i++; }
    dest[d + i] = '\0';
    return dest;
}
int main(void) {
    char buf[40];
    my_strcpy(buf, "Learn");
    my_strcat(buf, " Being");
    my_strcat(buf, " Forward");
    printf("\"%s\" length %zu\n", buf, my_strlen(buf));
    printf("my_strcmp(\"abc\", \"abd\") = %d\n", my_strcmp("abc", "abd"));
    printf("my_strcmp(\"abc\", \"abc\") = %d\n", my_strcmp("abc", "abc"));
    printf("my_strcmp(\"b\", \"a\")     = %d\n", my_strcmp("b", "a"));
    return 0;
}
```

OUTPUT

```
"Learn Being Forward" length 19
my_strcmp("abc", "abd") = -1
my_strcmp("abc", "abc") = 0
my_strcmp("b", "a")     = 1
```

8.8 Classic string problems

8.8.1 Reverse a string and reverse words

REVERSING CHARACTERS AND WORDS

main.c

```c
#include <stdio.h>
#include <string.h>

void reverse_range(char *s, int i, int j) {
    while (i < j) { char t = s[i]; s[i] = s[j]; s[j] = t; i++; j--; }
}
int main(void) {
    char s[] = "FORWARD";
    reverse_range(s, 0, (int)strlen(s) - 1);
    printf("reversed: %s\n", s);

    char sentence[] = "C is fun to learn";
    int n = (int)strlen(sentence);
    reverse_range(sentence, 0, n - 1);                  /* 1. reverse everything */
    for (int start = 0, i = 0; i <= n; i++)             /* 2. reverse each word back */
        if (sentence[i] == ' ' || sentence[i] == '\0') {
            reverse_range(sentence, start, i - 1);
            start = i + 1;
        }
    printf("words reversed: %s\n", sentence);
    return 0;
}
```

OUTPUT

```
reversed: DRAWROF
words reversed: learn to fun is C
```

8.8.2 Palindrome and anagram check

A **palindrome** reads the same backwards ("madam"); compare characters from both ends moving inward, usually ignoring case and non-letters. Two words are **anagrams** if they contain the same letters with the same counts ("listen"/"silent"); count letter frequencies in an array of 26 and compare.

PALINDROMES AND ANAGRAMS

main.c

```c
#include <stdio.h>
#include <ctype.h>
#include <string.h>

int is_palindrome(const char *s) {
    int i = 0, j = (int)strlen(s) - 1;
    while (i < j) {
        if (!isalnum((unsigned char)s[i])) { i++; continue; }
        if (!isalnum((unsigned char)s[j])) { j--; continue; }
        if (tolower((unsigned char)s[i]) != tolower((unsigned char)s[j])) return 0;
        i++; j--;
    }
    return 1;
}
int is_anagram(const char *a, const char *b) {
    int count[26] = {0};
    for (; *a; a++) if (isalpha((unsigned char)*a)) count[tolower((unsigned char)*a) - 'a']++;
    for (; *b; b++) if (isalpha((unsigned char)*b)) count[tolower((unsigned char)*b) - 'a']--;
    for (int i = 0; i < 26; i++) if (count[i] != 0) return 0;
    return 1;
}
int main(void) {
    const char *p[] = {"madam", "Racecar", "A man, a plan, a canal: Panama", "Forward"};
    for (int i = 0; i < 4; i++) printf("\"%s\" palindrome? %s\n", p[i], is_palindrome(p[i]) ? "yes" :
"no");
    printf("listen/silent anagram? %s\n", is_anagram("listen", "silent") ? "yes" : "no");
    printf("Dormitory/dirty room anagram? %s\n", is_anagram("Dormitory", "dirty room") ? "yes" :
"no");
    printf("hello/world anagram? %s\n", is_anagram("hello", "world") ? "yes" : "no");
    return 0;
}
```

OUTPUT

```
"madam" palindrome? yes
"Racecar" palindrome? yes
"A man, a plan, a canal: Panama" palindrome? yes
"Forward" palindrome? no
listen/silent anagram? yes
Dormitory/dirty room anagram? yes
hello/world anagram? no
```

8.8.3 Count vowels, consonants, digits, spaces

COUNTING CHARACTER CLASSES main.c

```c
#include <stdio.h>
#include <ctype.h>
#include <string.h>

int main(void) {
    const char *s = "Learn Being Forward opened in 2024 with 3 courses";
    int vowels = 0, consonants = 0, digits = 0, spaces = 0;
    for (int i = 0; s[i]; i++) {
        unsigned char c = (unsigned char)tolower((unsigned char)s[i]);
        if (strchr("aeiou", c) && c) vowels++;
        else if (isalpha(c)) consonants++;
        else if (isdigit(c)) digits++;
        else if (c == ' ') spaces++;
    }
    printf("vowels %d, consonants %d, digits %d, spaces %d\n", vowels, consonants, digits, spaces);
    return 0;
}
```

OUTPUT

```
vowels 14, consonants 22, digits 5, spaces 8
```

8.8.4 Character frequency and first non-repeating character

An `int count[256]` array indexed by character code counts every character in one pass. A second pass over the string finds the first character whose count is 1: O(n) overall.

FREQUENCY TABLE main.c

```c
#include <stdio.h>

int main(void) {
    const char *s = "programming";
    int count[256] = {0};
    for (int i = 0; s[i]; i++) count[(unsigned char)s[i]]++;
    printf("frequencies in \"%s\": ", s);
    for (int c = 0; c < 256; c++) if (count[c]) printf("%c=%d ", c, count[c]);
    printf("\n");
    for (int i = 0; s[i]; i++)
        if (count[(unsigned char)s[i]] == 1) { printf("first non-repeating: '%c'\n", s[i]); break; }
    return 0;
}
```

OUTPUT

```
frequencies in "programming": a=1 g=2 i=1 m=2 n=1 o=1 p=1 r=2
first non-repeating: 'p'
```

8.8.5 Remove duplicates, toggle case, sort strings alphabetically

THREE MORE STRING CLASSICS

main.c

```c
#include <stdio.h>
#include <ctype.h>
#include <string.h>

int main(void) {
    char s[] = "bookkeeper";
    int seen[256] = {0}, k = 0;
    for (int i = 0; s[i]; i++)
        if (!seen[(unsigned char)s[i]]) { seen[(unsigned char)s[i]] = 1; s[k++] = s[i]; }
    s[k] = '\0';
    printf("remove duplicates: %s\n", s);

    char t[] = "LeArN c";
    for (int i = 0; t[i]; i++)
        t[i] = isupper((unsigned char)t[i]) ? tolower((unsigned char)t[i]) : toupper((unsigned
char)t[i]);
    printf("toggle case: %s\n", t);

    char names[5][12] = {"Ravi", "asha", "Meera", "Dev", "Kiran"};
    for (int i = 0; i < 4; i++)                          /* bubble sort with strcmp */
        for (int j = 0; j < 4 - i; j++)
            if (strcmp(names[j], names[j + 1]) > 0) {
                char tmp[12];
                strcpy(tmp, names[j]); strcpy(names[j], names[j + 1]); strcpy(names[j + 1], tmp);
            }
    printf("sorted:");
    for (int i = 0; i < 5; i++) printf(" %s", names[i]);
    printf("   (\"asha\" is last: lowercase letters have larger codes)\n");
    return 0;
}
```

OUTPUT

```
remove duplicates: bokepr
toggle case: lEaRn C
sorted: Dev Kiran Meera Ravi asha   ("asha" is last: lowercase letters have larger codes)
```

PRACTICE CHECKPOINT

Implement your own `strlen`, `strcpy`, `strcmp` and `strcat` (see 8.7.1), then write a **word counter** and an **anagram checker**.

SAMPLE SOLUTION

main.c

```c
#include <stdio.h>
#include <ctype.h>
#include <string.h>

int count_words(const char *s) {
    int words = 0, in_word = 0;
    for (; *s; s++) {
        if (isspace((unsigned char)*s)) in_word = 0;
        else if (!in_word) { in_word = 1; words++; }   /* start of a new word */
    }
    return words;
}
int anagram(const char *a, const char *b) {
    int freq[26] = {0};
    for (; *a; a++) if (isalpha((unsigned char)*a)) freq[tolower((unsigned char)*a) - 'a']++;
    for (; *b; b++) if (isalpha((unsigned char)*b)) freq[tolower((unsigned char)*b) - 'a']--;
    for (int i = 0; i < 26; i++) if (freq[i]) return 0;
    return 1;
}
int main(void) {
    char line[200], w1[50], w2[50];
    printf("Enter a sentence: ");
    fgets(line, sizeof line, stdin);
    printf("Word count: %d\n", count_words(line));
    printf("First word : ");
    fgets(w1, sizeof w1, stdin); w1[strcspn(w1, "\n")] = '\0';
    printf("Second word: ");
    fgets(w2, sizeof w2, stdin); w2[strcspn(w2, "\n")] = '\0';
    printf("\"%s\" and \"%s\" are %sanagrams\n", w1, w2, anagram(w1, w2) ? "" : "not ");
    return 0;
}
```

CONSOLE (user input in gold)

```
Enter a sentence:   Learning   C builds strong   foundations
Word count: 5
First word : The eyes
Second word: They see
"The eyes" and "They see" are anagrams
```

MODULE 09 · BEGINNER

Functions and recursion

Minimum duration **3 days** | Level **Beginner**

Functions divide a program into named, reusable blocks; recursion is a function solving a problem by calling itself on a smaller input.

IN THIS MODULE

9.1 Introduction to functions
9.2 Parts of a function
9.3 Types of functions by arguments and return
9.4 Parameter passing
9.5 Functions and arrays
9.6 How function calls work
9.7 Recursion

9.1 Introduction to functions

9.1.1 Why functions (modularity, reuse, readability)

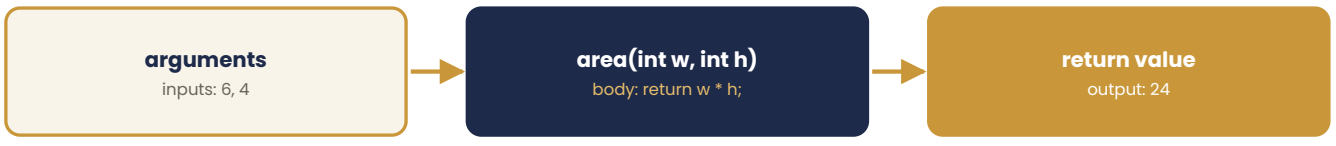
A **function** is a named block of code that performs one task. You define it once and **call** it whenever that task is needed, passing inputs (arguments) and receiving a result. Every C program already uses functions: `main` is one, and `printf` is another.

Modularity
A big problem is split into small functions (read_marks, compute_grade, print_report), each easy to write and test.

Reuse
Write once, call many times, even from other programs (libraries).

Readability
`if (is_prime(n))` explains itself; the details are hidden inside the function.

Maintenance
A bug fix in one function fixes every place that calls it.



A FUNCTION: INPUTS IN, WORK DONE, ONE RESULT OUT

9.1.2 Library functions vs user-defined functions

	Library (predefined) functions	User-defined functions
Written by	Compiler/library vendors	You
Declared in	Standard headers: stdio.h, string.h, math.h…	Your own source or header files
Examples	`printf`, `strlen`, `sqrt`, `malloc`, `qsort`	`calculate_fee`, `is_leap_year`

	Library (predefined) functions	User-defined functions
Use	#include the header and call	Declare (prototype), define and call

9.2 Parts of a function

DECLARATION, DEFINITION AND CALL

main.c

```c
#include <stdio.h>

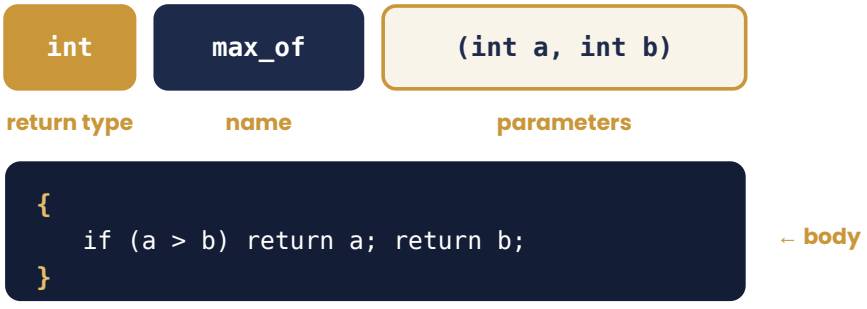
int max_of(int a, int b);               /* 1. declaration (prototype) */

int main(void) {
    int x = 17, y = 42;
    int bigger = max_of(x, y);          /* 3. call */
    printf("max of %d and %d is %d\n", x, y, bigger);
    printf("max of 5 and -3 is %d\n", max_of(5, -3));
    return 0;
}

int max_of(int a, int b) {              /* 2. definition: header + body */
    if (a > b)
        return a;                       /* 4. return statement */
    return b;
}
```

OUTPUT

```
max of 17 and 42 is 42
max of 5 and -3 is 5
```



ANATOMY OF A FUNCTION DEFINITION

9.2.1 Function declaration (prototype)

A **prototype** tells the compiler a function's name, return type and parameter types **before** it is used, ending with a semicolon: `int max_of(int a, int b);` (parameter names are optional: `int max_of(int, int);`). With it the compiler checks every call and converts arguments to the right types. Prototypes usually go at the top of the file or in a header file. Calling a function with no visible declaration is an error in C99 and later.

9.2.2 Function definition

The **definition** is the header (return type, name, parameters) followed by the **body** in braces. It may appear before or after main; if after, a prototype must come first. A function must be defined exactly once in the whole program, and functions cannot be defined inside other functions in standard C.

9.2.3 Function call

A call `name(arguments)` evaluates the arguments, copies their values into the parameters, jumps to the function, runs its body, and comes back with the return value, which replaces the call in the expression.

9.2.4 Return type and return statement

`return expression;` ends the function and sends the value back, converted to the declared return type. A function can have several return statements; the first one reached wins. A non-void function that reaches its closing brace without returning gives an undefined result (except main, which returns 0).

9.2.5 void functions and void parameter lists

A function that returns nothing has return type `void`; it ends at its closing brace or at a plain `return;`. Writing `(void)` as the parameter list says "takes no arguments". In C (unlike C++), an **empty** list `()` in older standards meant "unspecified arguments" and disabled checking, so always write `(void)` (C23 finally makes `()` mean the same as `(void)`).

VOID FUNCTIONS — main.c

```c
#include <stdio.h>

void print_line(int n) {                  /* returns nothing */
    for (int i = 0; i < n; i++) putchar('-');
    putchar('\n');
}
void greet(void) {                        /* takes nothing */
    printf("Welcome to Learn Being Forward\n");
}
void check_age(int age) {
    if (age < 0) {
        printf("invalid age\n");
        return;                           /* early exit from a void function */
    }
    printf("age %d recorded\n", age);
}
int main(void) {
    print_line(30);
    greet();
    print_line(30);
    check_age(-4);
    check_age(21);
    return 0;
}
```

OUTPUT

```
------------------------------
Welcome to Learn Being Forward
------------------------------
invalid age
age 21 recorded
```

9.3 Types of functions by arguments and return

Functions can be grouped by whether they take arguments and whether they return a value. All four are useful; functions with arguments **and** a return value are the most reusable, because they neither read input nor print output themselves.

9.3.1 No arguments, no return value

9.3.2 Arguments, no return value

9.3.3 No arguments, return value

9.3.4 Arguments and return value

THE FOUR CATEGORIES IN ONE PROGRAM

main.c

```c
#include <stdio.h>

void show_banner(void) {                        /* 9.3.1 no args, no return */
    printf("=== Rectangle tool ===\n");
}
void print_area(int w, int h) {                 /* 9.3.2 args, no return */
    printf("area of %d x %d = %d\n", w, h, w * h);
}
int read_positive(void) {                       /* 9.3.3 no args, returns a value */
    int v;
    do {
        printf("enter a positive number: ");
        scanf("%d", &v);
    } while (v <= 0);
    return v;
}
int perimeter(int w, int h) {                   /* 9.3.4 args and return */
    return 2 * (w + h);
}
int main(void) {
    show_banner();
    int w = read_positive();
    int h = read_positive();
    print_area(w, h);
    printf("perimeter = %d\n", perimeter(w, h));
    return 0;
}
```

CONSOLE (user input in gold)

```
=== Rectangle tool ===
enter a positive number: -3
enter a positive number: 8
enter a positive number: 5
area of 8 x 5 = 40
perimeter = 26
```

Category	Prototype	Typical use
No args, no return	`void show_menu(void);`	Fixed output: menus, banners
Args, no return	`void print_report(int marks);`	Display or act on data
No args, return	`int read_choice(void);`	Read and validate input
Args and return	`double average(int a, int b, int c);`	Calculations: most reusable

9.4 Parameter passing

9.4.1 Actual vs formal parameters

Formal parameters are the variables listed in the function definition (`int a, int b`). **Actual parameters** (arguments) are the values in the call (`max_of(x, 10)`). At the call, each argument is evaluated and its value is used to initialize the matching parameter.

9.4.2 Call by value

C always passes arguments **by value**: the function receives **copies**. Changing a parameter changes only the copy; the caller's variable is untouched. This protects the caller's data, but it means a function cannot change its caller's variables through ordinary parameters.

9.4.3 Call by reference using pointers (introduced; detailed in Module 12)

To let a function modify a caller's variable, pass its **address** (`&x`) and have the function work through a pointer parameter (`int *p`, using `*p`). Technically the address itself is still passed by value, but the effect is "call by reference". This is exactly how `scanf` works.

9.4.4 Swap example: why call by value fails

CALL BY VALUE VS CALL BY ADDRESS

main.c

```c
#include <stdio.h>

void swap_wrong(int a, int b) {           /* receives copies */
    int t = a; a = b; b = t;
    printf("  inside swap_wrong: a=%d b=%d\n", a, b);
}
void swap(int *a, int *b) {               /* receives addresses */
    int t = *a; *a = *b; *b = t;
}
int main(void) {
    int x = 10, y = 20;
    swap_wrong(x, y);
    printf("after swap_wrong: x=%d y=%d  (unchanged)\n", x, y);
    swap(&x, &y);
    printf("after swap      : x=%d y=%d  (swapped)\n", x, y);
    return 0;
}
```

OUTPUT

```
  inside swap_wrong: a=20 b=10
after swap_wrong: x=10 y=20  (unchanged)
after swap      : x=20 y=10  (swapped)
```



COPIES ARE SWAPPED AND THROWN AWAY; THROUGH POINTERS THE ORIGINALS ARE SWAPPED

9.5 Functions and arrays

9.5.1 Passing a 1D array (array decays to a pointer)

When you pass an array to a function, C does **not** copy the array. The array name **decays** into a pointer to its first element, and that pointer is passed. So: (1) the function works on the **original** array and can modify it; (2) inside the function, `sizeof` gives the size of a pointer, not of the array. The parameter can be written `int a[]` or `int *a`: they mean the same thing. Use `const int a[]` when the function should not modify the array.

9.5.2 Passing the size along with the array

ARRAYS ARE PASSED AS POINTERS

main.c

```c
#include <stdio.h>

void print_array(const int a[], int n) {        /* const: promise not to modify */
    for (int i = 0; i < n; i++) printf("%d ", a[i]);
    printf("\n");
}
void double_all(int a[], int n) {               /* modifies the caller's array */
    for (int i = 0; i < n; i++) a[i] *= 2;
}
double average(const int *a, int n) {           /* same as const int a[] */
    long sum = 0;
    for (int i = 0; i < n; i++) sum += a[i];
    return n ? (double)sum / n : 0;
}
void sizes(int a[]) {
    printf("inside the function: sizeof a = %zu (just a pointer!)\n", sizeof a);
}
int main(void) {
    int marks[] = {70, 85, 92, 64, 78};
    int n = sizeof marks / sizeof marks[0];     /* compute size where the array is visible */
    printf("in main: sizeof marks = %zu\n", sizeof marks);
    sizes(marks);
    print_array(marks, n);
    printf("average = %.2f\n", average(marks, n));
    double_all(marks, n);
    print_array(marks, n);
    return 0;
}
```

OUTPUT

```
in main: sizeof marks = 20
inside the function: sizeof a = 8 (just a pointer!)
70 85 92 64 78
average = 77.80
140 170 184 128 156
```

9.5.3 Passing a 2D array

For a 2D array parameter, every dimension except the first must be specified, because the compiler needs the row length to compute `a[i][j]`: `void print(int m[][4], int rows)`. With C99 VLA parameters the column count can be passed first: `void print(int rows, int cols, int m[rows][cols])`.

PASSING 2D ARRAYS

main.c

```c
#include <stdio.h>

void print_fixed(int m[][3], int rows) {                    /* column count fixed at 3 */
    for (int i = 0; i < rows; i++) { for (int j = 0; j < 3; j++) printf("%4d", m[i][j]); printf("\n");
}
}
int sum_vla(int rows, int cols, int m[rows][cols]) {          /* C99: sizes as parameters */
    int s = 0;
    for (int i = 0; i < rows; i++) for (int j = 0; j < cols; j++) s += m[i][j];
    return s;
}
int main(void) {
    int a[2][3] = {{1, 2, 3}, {4, 5, 6}};
    int b[3][4] = {{1, 1, 1, 1}, {2, 2, 2, 2}, {3, 3, 3, 3}};
    print_fixed(a, 2);
    printf("sum of a = %d, sum of b = %d\n", sum_vla(2, 3, a), sum_vla(3, 4, b));
    return 0;
}
```

OUTPUT

```
   1   2   3
   4   5   6
sum of a = 21, sum of b = 24
```

9.5.4 Passing strings

A string is a char array, so it is passed as a `char *` (or `const char *` when the function only reads it). No length is needed because the terminator marks the end.

STRINGS AS PARAMETERS

main.c

```c
#include <stdio.h>
#include <ctype.h>

int count_char(const char *s, char c) {
    int n = 0;
    for (int i = 0; s[i] != '\0'; i++) if (s[i] == c) n++;
    return n;
}
void to_upper(char s[]) {                       /* modifies the caller's string */
    for (int i = 0; s[i]; i++) s[i] = toupper((unsigned char)s[i]);
}
int main(void) {
    char name[] = "learn being forward";
    printf("'e' appears %d times\n", count_char(name, 'e'));
    to_upper(name);
    printf("%s\n", name);
    return 0;
}
```

OUTPUT

```
'e' appears 2 times
LEARN BEING FORWARD
```

9.6 How function calls work

9.6.1 The call stack and stack frames

Each running function needs memory for its parameters, local variables and the address to return to. The program keeps this in a region called the **stack**. Every call **pushes** a new **stack frame** on top; when the function returns, its frame is **popped**. The function on top is the one currently executing. Frames are last-in, first-out, matching the way calls nest.

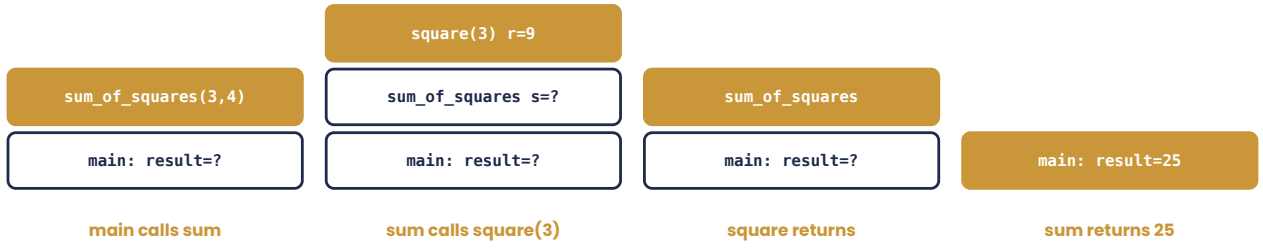
EACH CALL GETS ITS OWN FRAME (ADDRESSES GO DOWN AS THE STACK GROWS) main.c

```c
#include <stdio.h>

int square(int x) {
    int r = x * x;
    printf("      square(%d): frame at %p\n", x, (void *)&r);
    return r;
}
int sum_of_squares(int a, int b) {
    int s;
    printf("   sum_of_squares(%d, %d): frame at %p\n", a, b, (void *)&s);
    s = square(a) + square(b);
    return s;
}
int main(void) {
    int result;
    printf("main: frame at %p\n", (void *)&result);
    result = sum_of_squares(3, 4);
    printf("result = %d\n", result);
    return 0;
}
```

OUTPUT (addresses differ per run)

```
main: frame at 0x7ffe68b70274
   sum_of_squares(3, 4): frame at 0x7ffe68b70244
      square(3): frame at 0x7ffe68b70214
      square(4): frame at 0x7ffe68b70214
result = 25
```



THE CALL STACK AT FOUR MOMENTS

9.6.2 Local variables on the stack

Local variables live in their function's frame. They are created when the function is called, have garbage values unless initialized, are **separate for every call** (two calls of the same function have different copies), and **disappear** when the function returns. That is why a function must never return a pointer to one of its local variables (11.6.4).

9.6.3 inline functions (C99)

Calling a function has a small cost (pushing the frame, jumping, returning). For tiny functions called in hot loops, `inline` **suggests** that the compiler paste the body at each call site instead. Unlike a macro (Module 15), an inline function is type-checked and evaluates its arguments once. Modern compilers inline small functions automatically at `-O2` anyway. In C, a function used only in one file is best declared `static inline`.

STATIC INLINE HELPERS main.c

```c
#include <stdio.h>

static inline int max2(int a, int b) { return a > b ? a : b; }
static inline double celsius_to_f(double c) { return c * 9.0 / 5.0 + 32.0; }

int main(void) {
    int best = 0;
    int scores[] = {67, 92, 85, 74};
    for (int i = 0; i < 4; i++) best = max2(best, scores[i]);
    printf("best = %d\n", best);
    printf("37 C = %.1f F\n", celsius_to_f(37));
    return 0;
}
```

OUTPUT

```
best = 92
37 C = 98.6 F
```

9.7 Recursion

9.7.1 What recursion is

Recursion is when a function **calls itself**. It is natural for problems that contain smaller copies of themselves: n! = n × (n-1)!; a folder contains folders; to reverse a string, reverse the rest and put the first character at the end. Each call works on a smaller input until the input is small enough to answer directly.

9.7.2 Base case and recursive case

Base case

The simplest input, answered **without** recursion. It stops the chain: `if (n <= 1) return 1;`

Recursive case

Calls the function on a **smaller** input and combines the result: `return n * fact(n - 1);` Each call must move closer to the base case.

RECURSIVE FACTORIAL main.c

```c
#include <stdio.h>

unsigned long long fact(int n) {
    if (n <= 1) return 1;                /* base case */
    return n * fact(n - 1);              /* recursive case */
}
int main(void) {
    for (int i = 0; i <= 10; i++) printf("%2d! = %llu\n", i, fact(i));
    printf("20! = %llu\n", fact(20));
    return 0;
}
```

OUTPUT

```
 0! = 1
 1! = 1
 2! = 2
 3! = 6
 4! = 24
 5! = 120
 6! = 720
 7! = 5040
 8! = 40320
 9! = 362880
10! = 3628800
20! = 2432902008176640000
```

9.7.3 How the stack grows and unwinds

Each recursive call gets its own frame with its own copy of n. The stack **grows** as calls go deeper (winding) until the base case returns; then the frames return one by one (**unwinding**), each completing its multiplication.

TRACING THE WINDING AND UNWINDING

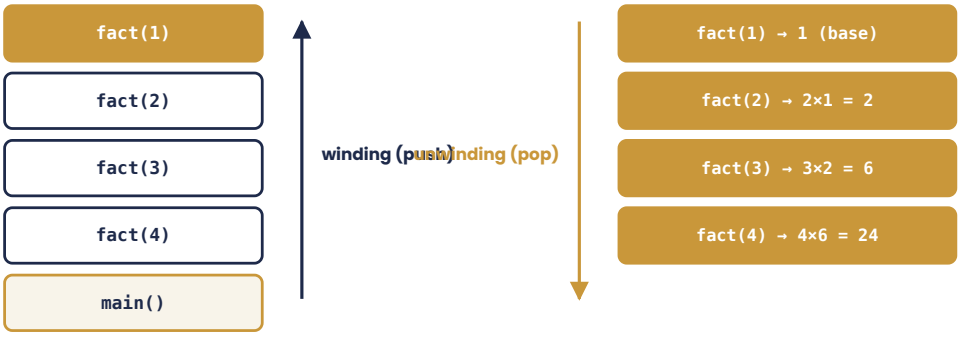
main.c

```c
#include <stdio.h>

int fact(int n, int depth) {
    printf("%*scall fact(%d)\n", depth * 2, "", n);
    if (n <= 1) {
        printf("%*sbase case -> 1\n", depth * 2, "");
        return 1;
    }
    int r = n * fact(n - 1, depth + 1);
    printf("%*sfact(%d) returns %d\n", depth * 2, "", n, r);
    return r;
}
int main(void) {
    fact(4, 0);
    return 0;
}
```

OUTPUT

```
call fact(4)
  call fact(3)
    call fact(2)
      call fact(1)
      base case -> 1
    fact(2) returns 2
  fact(3) returns 6
fact(4) returns 24
```



CALLS PILE UP UNTIL THE BASE CASE, THEN RESULTS FLOW BACK

9.7.4 Stack overflow

The stack is limited (typically 8 MB on Linux, 1 MB on Windows). Recursion without a reachable base case, or recursion that is simply too deep, keeps pushing frames until the stack is exhausted and the program crashes with a **segmentation fault** ("stack overflow"). C gives no exception to catch: prevent it by checking base cases and by using iteration for very deep problems.

RECURSION WITHOUT A BASE CASE main.c

```c
#include <stdio.h>

int depth = 0;
void forever(void) {
    char buffer[1024];                  /* each frame uses ~1 KB */
    buffer[0] = (char)depth;
    depth++;
    if (depth % 2000 == 0) { printf("depth %d\n", depth); fflush(stdout); }
    forever();                          /* no base case */
    buffer[1] = buffer[0];              /* prevents tail-call optimisation */
}
int main(void) {
    forever();
    return 0;
}
```

OUTPUT (the crash depth depends on the stack size)

```
depth 2000
depth 4000
depth 6000
Segmentation fault (core dumped)
```

9.7.5 Types of recursion

9.7.5.1 Direct and indirect

Direct recursion: f calls f. **Indirect** (mutual) recursion: f calls g and g calls f. Indirect recursion needs a prototype so that the first function can call the second before it is defined.

DIRECT AND INDIRECT RECURSION main.c

```c
#include <stdio.h>

int is_odd(int n);                      /* prototype needed for mutual recursion */
int is_even(int n) { return n == 0 ? 1 : is_odd(n - 1); }
int is_odd(int n)  { return n == 0 ? 0 : is_even(n - 1); }

void count_down(int n) {                /* direct recursion */
    if (n == 0) { printf("liftoff!\n"); return; }
    printf("%d ", n);
    count_down(n - 1);
}
int main(void) {
    count_down(5);
    printf("is_even(10) = %d, is_odd(7) = %d\n", is_even(10), is_odd(7));
    return 0;
}
```

OUTPUT

```
5 4 3 2 1 liftoff!
is_even(10) = 1, is_odd(7) = 1
```

9.7.5.2 Head and tail recursion

In **head recursion** the recursive call happens **first** and the work is done afterwards, as the stack unwinds. In **tail recursion** the recursive call is the **very last** action. Tail-recursive functions can be turned into loops by the compiler (GCC does this at `-O2`, though the C standard does not guarantee it), so they use constant stack space.

HEAD VS TAIL RECURSION main.c

```c
#include <stdio.h>

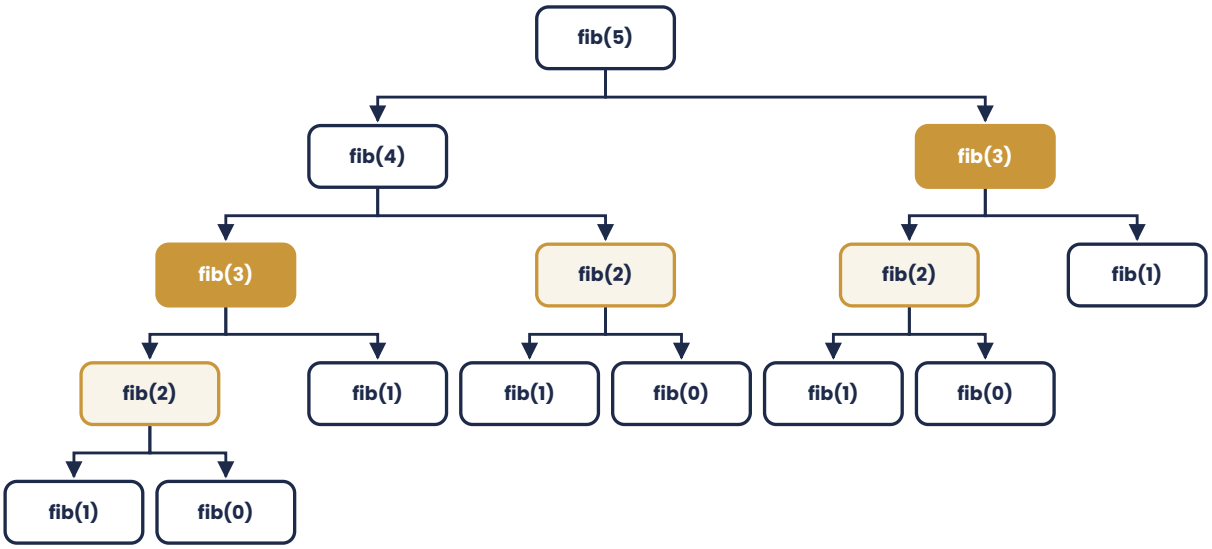
void head(int n) { if (n == 0) return; head(n - 1); printf("%d ", n); }   /* prints 1..n */
void tail(int n) { if (n == 0) return; printf("%d ", n); tail(n - 1); }   /* prints n..1 */
long sum_tail(int n, long acc) {                                          /* accumulator style */
    if (n == 0) return acc;
    return sum_tail(n - 1, acc + n);
}
int main(void) {
    printf("head: "); head(5); printf("\n");
    printf("tail: "); tail(5); printf("\n");
    printf("sum_tail(100000) = %ld\n", sum_tail(100000, 0));
    return 0;
}
```

OUTPUT

```
head: 1 2 3 4 5
tail: 5 4 3 2 1
sum_tail(100000) = 5000050000
```

9.7.5.3 Tree recursion

A function that makes **more than one** recursive call creates a tree of calls. The naive Fibonacci `fib(n) = fib(n−1) + fib(n−2)` is the classic example: the tree roughly doubles at each level, so the running time is exponential, about $O(1.6^n)$, and the same values are recomputed many times.



CALLS MADE BY FIB(5): FIB(3) IS COMPUTED TWICE (GOLD), FIB(2) THREE TIMES (CREAM)

THE COST OF TREE RECURSION main.c

```c
#include <stdio.h>

long calls = 0;
long fib(int n) { calls++; return n < 2 ? n : fib(n - 1) + fib(n - 2); }

long fib_iter(int n) {
    long a = 0, b = 1;
    for (int i = 0; i < n; i++) { long t = a + b; a = b; b = t; }
    return a;
}
int main(void) {
    for (int n = 10; n <= 30; n += 10) {
        calls = 0;
        long v = fib(n);
        printf("fib(%d) = %ld using %ld calls; iterative gives %ld in %d steps\n", n, v, calls,
fib_iter(n), n);
    }
    return 0;
}
```

OUTPUT

```
fib(10) = 55 using 177 calls; iterative gives 55 in 10 steps
fib(20) = 6765 using 21891 calls; iterative gives 6765 in 20 steps
fib(30) = 832040 using 2692537 calls; iterative gives 832040 in 30 steps
```

9.7.6 Recursion vs iteration

	Recursion	Iteration
Code	Short, mirrors the mathematical definition	Explicit loop, sometimes longer
Memory	One stack frame per active call	Constant
Speed	Call overhead; tree recursion may repeat work	Usually faster
Risk	Stack overflow if too deep	Infinite loop if the condition is wrong
Best for	Trees, divide and conquer, backtracking, nested structures	Simple counting and linear processing

9.7.7 Classic recursive problems

Recipe for any recursive function: (1) What is the smallest case I can answer directly? (2) How do I make the problem smaller? (3) How do I build the answer from the smaller answer?

9.7.7.1 Factorial, Fibonacci, sum of digits, power

FOUR RECURSIVE CLASSICS

main.c

```c
#include <stdio.h>

unsigned long long fact(int n) { return n <= 1 ? 1 : n * fact(n - 1); }
long fib(int n) { return n < 2 ? n : fib(n - 1) + fib(n - 2); }
int sum_digits(int n) { return n == 0 ? 0 : n % 10 + sum_digits(n / 10); }
long long power(long long b, int e) {            /* fast power: O(log e) */
    if (e == 0) return 1;
    long long half = power(b, e / 2);
    return e % 2 == 0 ? half * half : half * half * b;
}
int main(void) {
    printf("fact(12) = %llu\n", fact(12));
    printf("fib(0..12): ");
    for (int i = 0; i <= 12; i++) printf("%ld ", fib(i));
    printf("\nsum_digits(98765) = %d\n", sum_digits(98765));
    printf("power(2, 40) = %lld, power(3, 13) = %lld\n", power(2, 40), power(3, 13));
    return 0;
}
```

OUTPUT

```
fact(12) = 479001600
fib(0..12): 0 1 1 2 3 5 8 13 21 34 55 89 144
sum_digits(98765) = 35
power(2, 40) = 1099511627776, power(3, 13) = 1594323
```

Problem	Base case	Recursive case
n!	n ≤ 1 → 1	n × (n-1)!
fib(n)	n < 2 → n	fib(n-1) + fib(n-2)
sum of digits	n = 0 → 0	n % 10 + sum(n / 10)
b^e	e = 0 → 1	$(b^{e/2})^2$, times b if e is odd

9.7.7.2 Reverse a string, palindrome check

RECURSION ON STRINGS

main.c

```c
#include <stdio.h>
#include <string.h>

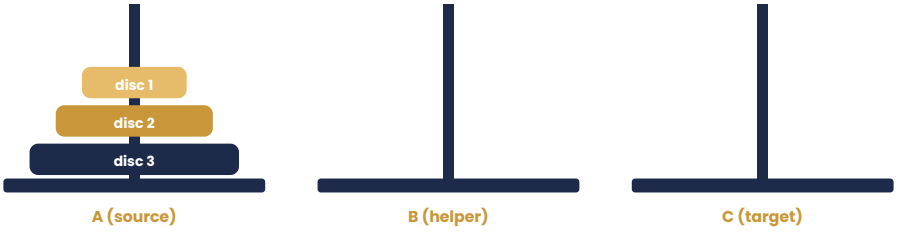
void reverse(char *s, int lo, int hi) {
    if (lo >= hi) return;                          /* base: 0 or 1 chars left */
    char t = s[lo]; s[lo] = s[hi]; s[hi] = t;
    reverse(s, lo + 1, hi - 1);                    /* the inner part */
}
int is_pal(const char *s, int lo, int hi) {
    if (lo >= hi) return 1;
    if (s[lo] != s[hi]) return 0;
    return is_pal(s, lo + 1, hi - 1);
}
void print_reverse(const char *s) {                /* head recursion prints backwards */
    if (*s == '\0') return;
    print_reverse(s + 1);
    putchar(*s);
}
int main(void) {
    char s[] = "RECURSION";
    reverse(s, 0, (int)strlen(s) - 1);
    printf("reversed: %s\n", s);
    const char *w[] = {"level", "rotor", "forward"};
    for (int i = 0; i < 3; i++) printf("%s -> %s\n", w[i], is_pal(w[i], 0, (int)strlen(w[i]) - 1) ?
"palindrome" : "not");
    printf("print_reverse: "); print_reverse("hello"); printf("\n");
    return 0;
}
```

OUTPUT

```
reversed: NOISRUCER
level -> palindrome
rotor -> palindrome
forward -> not
print_reverse: olleh
```

9.7.7.3 Tower of Hanoi

Move n discs from peg A to peg C using B, one disc at a time, never placing a larger disc on a smaller one. Insight: to move n discs, (1) move the top n-1 discs from A to B, (2) move the largest disc from A to C, (3) move the n-1 discs from B to C. That takes exactly $2^n - 1$ moves.



TOWER OF HANOI WITH 3 DISCS: THE STARTING POSITION

SOLVING TOWER OF HANOI main.c

```c
#include <stdio.h>

int moves = 0;
void hanoi(int n, char from, char via, char to) {
    if (n == 0) return;
    hanoi(n - 1, from, to, via);              /* 1. clear the way */
    printf("move disc %d from %c to %c\n", n, from, to);
    moves++;
    hanoi(n - 1, via, from, to);              /* 3. stack the rest on top */
}
int main(void) {
    hanoi(3, 'A', 'B', 'C');
    printf("total moves: %d (2^3 - 1)\n", moves);
    return 0;
}
```

OUTPUT

```
move disc 1 from A to C
move disc 2 from A to B
move disc 1 from C to B
move disc 3 from A to C
move disc 1 from B to A
move disc 2 from B to C
move disc 1 from A to C
total moves: 7 (2^3 - 1)
```

9.7.7.4 Recursive binary search

BINARY SEARCH, RECURSIVELY main.c

```c
#include <stdio.h>

int bsearch_rec(const int a[], int lo, int hi, int key) {
    if (lo > hi) return -1;                       /* empty range */
    int mid = lo + (hi - lo) / 2;
    if (a[mid] == key) return mid;
    if (key < a[mid]) return bsearch_rec(a, lo, mid - 1, key);
    return bsearch_rec(a, mid + 1, hi, key);
}
int main(void) {
    int a[] = {2, 5, 8, 12, 16, 23, 38, 56, 72, 91};
    int keys[] = {23, 2, 91, 40};
    for (int i = 0; i < 4; i++) printf("search %d -> index %d\n", keys[i], bsearch_rec(a, 0, 9,
keys[i]));
    return 0;
}
```

OUTPUT

```
search 23 -> index 5
search 2 -> index 0
search 91 -> index 9
search 40 -> index -1
```

9.7.7.5 Permutations of a string (intro to backtracking)

Backtracking builds a solution step by step: make a choice, recurse to make the remaining choices, then **undo** the choice and try the next one. To generate all permutations, swap each remaining character into the current position, recurse on the rest, then swap back. A string of n distinct characters has n! permutations.

ALL PERMUTATIONS BY BACKTRACKING

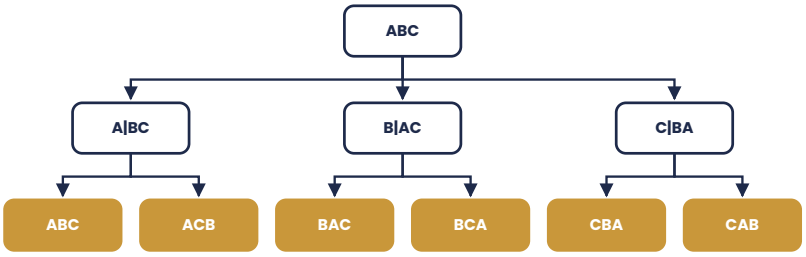
main.c

```c
#include <stdio.h>
#include <string.h>

void swap(char *a, char *b) { char t = *a; *a = *b; *b = t; }
void permute(char *s, int k, int n) {
    if (k == n - 1) { printf("%s ", s); return; }     /* all positions fixed */
    for (int i = k; i < n; i++) {
        swap(&s[k], &s[i]);                           /* choose */
        permute(s, k + 1, n);                         /* explore */
        swap(&s[k], &s[i]);                           /* un-choose (backtrack) */
    }
}
int main(void) {
    char s[] = "ABC";
    permute(s, 0, (int)strlen(s));
    printf("\n");
    char t[] = "1234";
    permute(t, 0, 4);
    printf("\n(4! = 24 permutations)\n");
    return 0;
}
```

OUTPUT

```
ABC ACB BAC BCA CBA CAB
1234 1243 1324 1342 1432 1423 2134 2143 2314 2341 2431 2413 3214 3241 3124 3142 3412 3421 4231 4213
4321 4312 4132 4123
(4! = 24 permutations)
```



THE CHOICE TREE: FIX ONE CHARACTER PER LEVEL; THE LEAVES ARE THE 6 PERMUTATIONS

PRACTICE CHECKPOINT

A library of **math functions** (prime, factorial, GCD, power) in separate functions with prototypes, plus recursive **Fibonacci** and **Tower of Hanoi**.

SAMPLE SOLUTION

main.c

```c
#include <stdio.h>

/* ---- prototypes ---- */
int is_prime(int n);
unsigned long long factorial(int n);
int gcd(int a, int b);
double power(double base, int exp);
long fibonacci(int n);
void hanoi(int n, char from, char via, char to);

int main(void) {
    printf("Primes below 30: ");
    for (int i = 0; i < 30; i++) if (is_prime(i)) printf("%d ", i);
```

```c
    printf("\n10! = %llu\n", factorial(10));
    printf("gcd(84, 120) = %d\n", gcd(84, 120));
    printf("power(1.5, 4) = %.4f, power(2, -3) = %.4f\n", power(1.5, 4), power(2, -3));
    printf("Fibonacci: ");
    for (int i = 0; i < 12; i++) printf("%ld ", fibonacci(i));
    printf("\nTower of Hanoi with 3 discs:\n");
    hanoi(3, 'A', 'B', 'C');
    return 0;
}

/* ---- definitions ---- */
int is_prime(int n) {
    if (n < 2) return 0;
    for (int d = 2; d * d <= n; d++) if (n % d == 0) return 0;
    return 1;
}
unsigned long long factorial(int n) {
    unsigned long long f = 1;
    for (int i = 2; i <= n; i++) f *= i;
    return f;
}
int gcd(int a, int b) { return b == 0 ? a : gcd(b, a % b); }       /* recursive Euclid */
double power(double base, int exp) {
    if (exp < 0) return 1.0 / power(base, -exp);
    double r = 1;
    for (int i = 0; i < exp; i++) r *= base;
    return r;
}
long fibonacci(int n) { return n < 2 ? n : fibonacci(n - 1) + fibonacci(n - 2); }
void hanoi(int n, char from, char via, char to) {
    if (n == 0) return;
    hanoi(n - 1, from, to, via);
    printf("  disc %d: %c -> %c\n", n, from, to);
    hanoi(n - 1, via, from, to);
}
```

OUTPUT

```
Primes below 30: 2 3 5 7 11 13 17 19 23 29
10! = 3628800
gcd(84, 120) = 12
power(1.5, 4) = 5.0625, power(2, -3) = 0.1250
Fibonacci: 0 1 1 2 3 5 8 13 21 34 55 89
Tower of Hanoi with 3 discs:
  disc 1: A -> C
  disc 2: A -> B
  disc 1: C -> B
  disc 3: A -> C
  disc 1: B -> A
  disc 2: B -> C
  disc 1: A -> C
```

LEVEL 2

Intermediate level

Outcome: master pointers, dynamic memory, structures, the preprocessor and file handling.

7 MODULES · 22 DAYS MINIMUM

Module 10 · Storage classes, scope and lifetime	**2 days**
Module 11 · Pointers: fundamentals	**4 days**
Module 12 · Pointers: advanced	**4 days**
Module 13 · Dynamic memory allocation	**3 days**
Module 14 · Structures, unions, enums and typedef	**4 days**
Module 15 · The preprocessor	**2 days**
Module 16 · File handling	**3 days**

MODULE 10 · INTERMEDIATE

Storage classes, scope and lifetime

Minimum duration **2 days** | Level **Intermediate**

Storage classes decide where a variable lives, how long it lives and who can see it.

IN THIS MODULE

10.1 Scope
10.2 Lifetime (storage duration)
10.3 Linkage
10.4 Storage classes
10.5 Memory layout of a C program

Every identifier in C has three independent properties. **Scope**: where in the source code the name can be used. **Lifetime** (storage duration): when, at run time, the object exists in memory. **Linkage**: whether the same name in different files or scopes refers to the same object. Storage-class keywords (`auto`, `register`, `static`, `extern`) control these properties.

Scope
Where in the code is the name visible? (compile-time property)

Lifetime
When does the memory exist? (run-time property)

Linkage
Which files share the name? (link-time property)

10.1 Scope

10.1.1 Block scope

A name declared inside a block `{ }` (including a function body and a for-loop header) is visible from its declaration to the end of that block. Local variables and function parameters have block scope.

10.1.2 Function scope (labels)

Only **labels** (targets of `goto`) have function scope: a label is visible throughout the whole function in which it appears, even before its declaration point.

10.1.3 File scope

A name declared outside all functions has **file scope**: it is visible from its declaration to the end of the source file (translation unit). Global variables and functions have file scope.

10.1.4 Function prototype scope

Parameter names in a prototype (`int area(int width, int height);`) are visible only until the end of that prototype. They are documentation; they need not match the names in the definition.

10.1.5 Shadowing of variables

An inner declaration with the same name as an outer one **hides** (shadows) the outer variable for the rest of the inner block. It is legal but confusing; compile with `-Wshadow` to get warnings.

BLOCK, FILE AND FUNCTION SCOPE; SHADOWING main.c

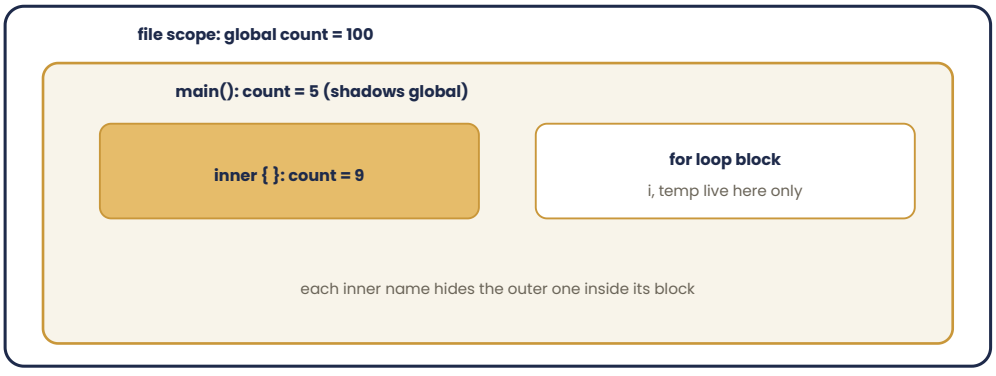
```c
#include <stdio.h>

int count = 100;                        /* file scope */

void show(int count) {                  /* parameter shadows the global */
    printf("  parameter count = %d\n", count);
}
int main(void) {
    printf("global count = %d\n", count);
    int count = 5;                      /* block scope: shadows the global */
    printf("main's count = %d\n", count);
    {
        int count = 9;                  /* inner block shadows main's count */
        printf("inner block count = %d\n", count);
    }
    printf("back in main, count = %d\n", count);
    show(42);
    for (int i = 0; i < 2; i++) { int temp = i * 10; printf("  loop temp = %d\n", temp); }
    /* printf("%d", temp);  error: temp is out of scope here */
    goto finish;                         /* label used before it appears: function scope */
    printf("skipped\n");
finish:
    printf("done\n");
    return 0;
}
```

OUTPUT

```
global count = 100
main's count = 5
inner block count = 9
back in main, count = 5
  parameter count = 42
  loop temp = 0
  loop temp = 10
done
```



NESTED SCOPES: LOOKUP GOES FROM THE INNERMOST BLOCK OUTWARD

10.2 Lifetime (storage duration)

10.2.1 Automatic

Local variables (without `static`) have **automatic** storage duration: they are created each time execution enters their block and destroyed when it leaves. They live on the stack and are not initialized automatically.

10.2.2 Static

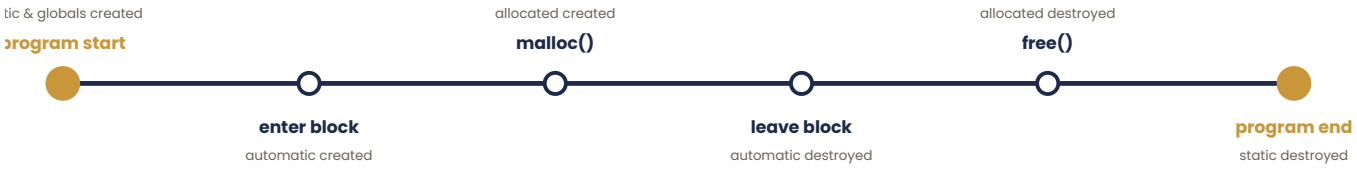
Global variables and variables declared with `static` have **static** storage duration: they exist for the **entire run** of the program, are initialized once before main starts (to zero if no initializer), and keep their values.

10.2.3 Allocated (dynamic, covered in Module 13)

Memory obtained with `malloc` / `calloc` / `realloc` exists from allocation until `free` is called, regardless of scope. It lives on the heap.

10.2.4 Thread storage duration (C11 _Thread_local)

A variable declared `_Thread_local` (C23: `thread_local`) exists once **per thread** for the lifetime of that thread: each thread gets its own copy (Module 22).



WHEN EACH KIND OF OBJECT EXISTS DURING A RUN

10.3 Linkage

10.3.1 External linkage

A name with **external linkage** refers to the same object or function in **every** file of the program. Global variables and functions have external linkage by default; other files can use them after declaring them with `extern`.

10.3.2 Internal linkage

A file-scope name declared `static` has **internal linkage**: it is private to its own source file. Another file can use the same name for something different without a clash. This is C's way of hiding implementation details.

10.3.3 No linkage

Local variables and parameters have **no linkage**: each declaration is a separate entity, even static locals.

Declared	Scope	Lifetime	Linkage
Inside a block	Block	Automatic	None
Inside a block with `static`	Block	Static	None
Outside functions	File	Static	External
Outside functions with `static`	File	Static	Internal
With `extern`	Block or file	Static (defined elsewhere)	External

10.4 Storage classes

10.4.1 auto

10.4.1.1 Default for local variables

In C17 and earlier, `auto` declares an automatic local variable, which is already the default, so it is practically never written: `auto int x = 5;` is the same as `int x = 5;`.

10.4.1.2 New meaning as type inference in C23

C23 gives `auto` a new, useful meaning borrowed from C++: in a declaration with an initializer and no type, the type is **inferred** from the initializer: `auto total = 3.5;` makes `total` a `double`.

AUTO TYPE INFERENCE (C23; COMPILE WITH -STD=C2X ON GCC 13) — main.c

```c
#include <stdio.h>

int main(void) {
    auto price = 99.5;          /* C23: inferred as double */
    auto count = 3;             /* inferred as int */
    printf("sizeof price = %zu, sizeof count = %zu\n", sizeof price, sizeof count);
    printf("total = %.2f\n", price * count);
    return 0;
}
```

OUTPUT

```
sizeof price = 8, sizeof count = 4
total = 298.50
```

10.4.2 register

`register` hints that a variable is used heavily and should be kept in a CPU register. Modern compilers ignore the hint (they allocate registers far better than people), but the rule remains that you **cannot take the address** of a register variable.

REGISTER (A HISTORICAL HINT) — main.c

```c
#include <stdio.h>

int main(void) {
    register int sum = 0;
    for (register int i = 1; i <= 100; i++) sum += i;
    printf("sum = %d\n", sum);
    /* int *p = &sum;   error: address of register variable requested */
    return 0;
}
```

OUTPUT

```
sum = 5050
```

10.4.3 static

10.4.3.1 Static local variables (keep their value between calls)

A `static` local variable is initialized **once** (with a constant expression, or zero) and keeps its value from one call to the next, although it is still visible only inside its function. Typical uses: counting calls, generating IDs, caching.

STATIC VS AUTOMATIC LOCALS main.c

```c
#include <stdio.h>

int next_id(void) {
    static int id = 1000;          /* initialized once, before main runs */
    return ++id;
}
void visit(void) {
    int automatic = 0;             /* re-created at every call */
    static int persistent = 0;     /* survives between calls */
    automatic++;
    persistent++;
    printf("automatic = %d, persistent = %d\n", automatic, persistent);
}
int main(void) {
    visit(); visit(); visit();
    int a = next_id(), b = next_id(), c = next_id();   /* separate calls: order guaranteed */
    printf("ids: %d %d %d\n", a, b, c);
    return 0;
}
```

OUTPUT

```
automatic = 1, persistent = 1
automatic = 1, persistent = 2
automatic = 1, persistent = 3
ids: 1001 1002 1003
```

10.4.3.2 Static global variables and functions (file-private)

`static` on a global variable or a function gives it **internal linkage**: invisible outside its .c file. Mark every helper function and global that other files do not need as `static`; it prevents name clashes and makes the file's public interface clear.

FILE-PRIVATE STATE AND HELPERS main.c

```c
#include <stdio.h>

static int balance = 0;                    /* private to this file */

static int is_valid(int amount) {          /* private helper */
    return amount > 0;
}
void deposit(int amount) {                 /* public: external linkage */
    if (is_valid(amount)) balance += amount;
}
int get_balance(void) { return balance; }

int main(void) {
    deposit(500);
    deposit(-20);
    deposit(250);
    printf("balance = %d\n", get_balance());
    return 0;
}
```

OUTPUT

```
balance = 750
```

10.4.4 extern

10.4.4.1 Sharing a global variable across files

To use a global variable defined in another file, **declare** it with `extern`: this says "it exists, with this type, somewhere else". Exactly one file must contain the **definition** (without extern). The usual practice is to put the extern declaration in a header file included by every file that needs it.

10.4.4.2 Declaration vs definition with extern

Line	Kind	Memory reserved?
`int counter = 0;` (file scope)	Definition	Yes
`int counter;` (file scope)	Tentative definition (becomes a definition)	Yes
`extern int counter;`	Declaration only	No
`extern int counter = 0;`	Definition (extern ignored)	Yes, but confusing: avoid

COUNTER.H AND COUNTER.C C

```c
/* counter.h : the shared declaration */
#ifndef COUNTER_H
#define COUNTER_H
extern int counter;             /* declaration: defined in counter.c */
void increment(void);
#endif

/* counter.c : the single definition */
#include "counter.h"
int counter = 0;                /* definition: memory lives here */
void increment(void) { counter++; }
```

MAIN.C USES THE SHARED COUNTER (BUILT WITH: GCC MAIN.C COUNTER.C) main.c

```c
#include <stdio.h>
#include "counter.h"

int main(void) {
    increment();
    increment();
    counter += 10;                    /* same variable as in counter.c */
    increment();
    printf("counter = %d\n", counter);
    return 0;
}
```

OUTPUT

```
counter = 13
```



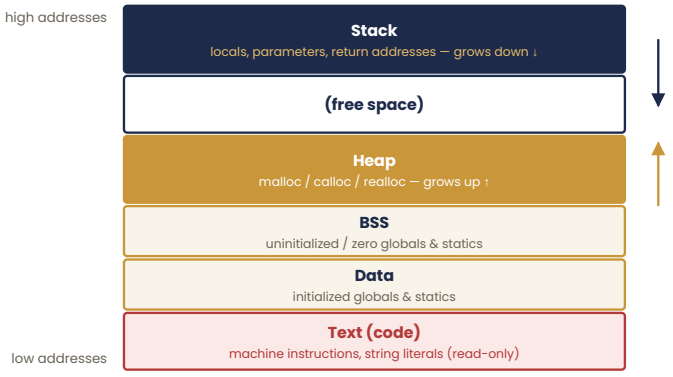
EXTERN DECLARATIONS IN MANY FILES, ONE DEFINITION IN ONE FILE

10.4.5 Comparison table: storage, default value, scope, lifetime

Storage class	Where stored	Default value	Scope	Lifetime	Linkage
`auto` (default local)	Stack	Garbage	Block	Block	None
`register`	CPU register (hint) / stack	Garbage	Block	Block	None
`static` local	Data / BSS segment	Zero	Block	Whole program	None
`static` global	Data / BSS segment	Zero	File	Whole program	Internal
global (no keyword)	Data / BSS segment	Zero	File (+ others via extern)	Whole program	External
`extern`	Wherever it is defined	(of the definition)	Where declared	Whole program	External

10.5 Memory layout of a C program

When a C program runs, the operating system gives it a **virtual address space** divided into segments, each holding a different kind of data.



TYPICAL MEMORY LAYOUT OF A RUNNING C PROGRAM

10.5.1 Text (code) segment

Holds the compiled machine instructions. It is read-only (writing to it crashes) and can be shared between several running copies of the same program. String literals and other read-only constants live in an adjacent read-only data area.

10.5.2 Initialized data segment

Holds global and static variables that have a **non-zero initializer**, e.g. `int limit = 100;`. Their initial values are stored in the executable file.

10.5.3 Uninitialized data segment (BSS)

Holds global and static variables that are uninitialized or initialized to zero. The executable only records the size; the OS fills the area with zeros at start-up. That is why globals and statics default to 0. ("BSS" is an old assembler term, Block Started by Symbol.)

10.5.4 Heap

Memory for dynamic allocation (`malloc`). It grows upward as needed and is managed by the C library's allocator. The programmer must free what is allocated.

10.5.5 Stack

Holds a frame for each active function call (parameters, local variables, return address). It grows downward. Its size is limited (commonly 8 MB on Linux), so huge local arrays or very deep recursion overflow it.

10.5.6 Which variable goes where

PRINTING ADDRESSES FROM EVERY SEGMENT main.c

```c
#include <stdio.h>
#include <stdlib.h>

int init_global = 42;            /* data */
int zero_global;                 /* BSS */
static int static_global = 7;    /* data */
const char *msg = "literal";     /* msg in data; the text "literal" in read-only data */

void where(void) {}

int main(void) {
    static int static_local;     /* BSS */
    int local = 5;               /* stack */
    int *heap = malloc(sizeof *heap);   /* the pointer is on the stack; *heap is on the heap */
    printf("text (code)     : %p\n", (void *)where);
    printf("read-only data  : %p\n", (void *)msg);
    printf("data (init)     : %p, %p\n", (void *)&init_global, (void *)&static_global);
    printf("BSS (zero)      : %p, %p\n", (void *)&zero_global, (void *)&static_local);
    printf("heap            : %p\n", (void *)heap);
    printf("stack           : %p\n", (void *)&local);
    printf("zero_global = %d, static_local = %d (BSS is zero-filled)\n", zero_global, static_local);
    free(heap);
    return 0;
}
```

OUTPUT (addresses differ per run; their order is what matters)

```
text (code)     : 0x401196
read-only data  : 0x402008
data (init)     : 0x404030, 0x404034
BSS (zero)      : 0x404044, 0x404048
heap            : 0x2fa382a0
stack           : 0x7fffd52e4eec
zero_global = 0, static_local = 0 (BSS is zero-filled)
```

The size tool reports the text, data and BSS sizes of an executable

```
$ gcc -std=c17 -no-pie segments.c -o segments
$ size segments
   text    data     bss     dec     hex filename
   1889     584      16    2489     9b9 segments
```

Rule of thumb: locals → stack; malloc → heap; globals and statics → data (if initialized to non-zero) or BSS (zero/uninitialized); code and string literals → read-only text.

PRACTICE CHECKPOINT

A function that counts how many times it has been called using a **static variable**, and a **two-file program** that shares a global counter with `extern`.

SAMPLE SOLUTION

main.c

```c
#include <stdio.h>
#include "stats.h"

int call_count(void) {
    static int calls = 0;           /* persists across calls */
    return ++calls;
}
int main(void) {
    for (int i = 0; i < 3; i++) printf("call_count() returned %d\n", call_count());
    record_visit("home");
    record_visit("courses");
    record_visit("contact");
    printf("total visits shared through extern: %d\n", total_visits);
    return 0;
}
```

OUTPUT

```
call_count() returned 1
call_count() returned 2
call_count() returned 3
  visit #1: home
  visit #2: courses
  visit #3: contact
total visits shared through extern: 3
```

THE SECOND FILE AND ITS HEADER (BUILD: GCC MAIN.C STATS.C -O VISITS)

C

```c
/* stats.h */
#ifndef STATS_H
#define STATS_H
extern int total_visits;          /* declared here, defined in stats.c */
void record_visit(const char *page);
#endif

/* stats.c */
#include <stdio.h>
#include "stats.h"

int total_visits = 0;             /* the one definition */

void record_visit(const char *page) {
    total_visits++;
    printf("  visit #%d: %s\n", total_visits, page);
}
```

MODULE 11 · INTERMEDIATE

Pointers: fundamentals

Minimum duration **4 days** | Level **Intermediate**

A pointer is a variable that stores a memory address. Pointers are the most important and most feared topic in C, so this module builds them up step by step.

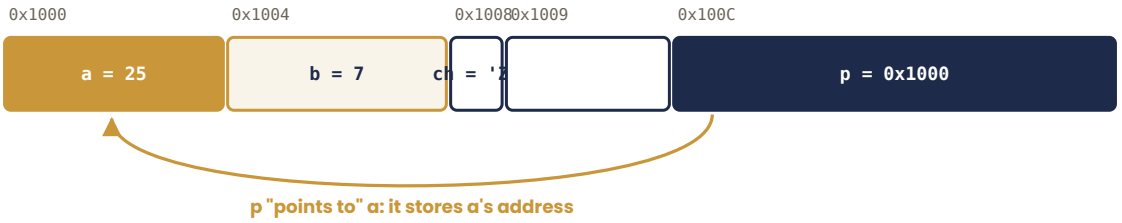
IN THIS MODULE

11.1 Memory and addresses
11.2 Pointer basics
11.3 Special pointers
11.4 Pointer arithmetic
11.5 Pointers and arrays
11.6 Pointers and functions: call by reference

11.1 Memory and addresses

11.1.1 How memory is organised (bytes and addresses)

Main memory is a huge array of **bytes**. Each byte has a unique number, its **address**, just as each house on a street has a house number. A variable occupies one or more consecutive bytes; its address is the address of its **first** byte. On a 64-bit system addresses are 64-bit numbers, usually shown in hexadecimal.



VARIABLES OCCUPY BYTE RANGES; A POINTER VARIABLE STORES ANOTHER VARIABLE'S ADDRESS

11.1.2 Address-of operator &

The unary operator `&` gives the address of a variable (or array element, or structure member): `&a`. The result is a pointer value whose type is "pointer to the variable's type" (`int *` for an int). You cannot take the address of a constant (`&5`), an expression (`&(a + 1)`) or a register variable.

11.1.3 Printing addresses with %p

Print addresses with `%p`, converting the pointer to `void *`. Addresses change between runs because modern operating systems randomize memory layout for security (ASLR).

ADDRESSES OF VARIABLES main.c

```c
#include <stdio.h>

int main(void) {
    int a = 25, b = 7;
    char ch = 'Z';
    double d = 3.5;
    printf("a  = %2d at %p (size %zu)\n", a, (void *)&a, sizeof a);
    printf("b  = %2d at %p (size %zu)\n", b, (void *)&b, sizeof b);
    printf("ch = %2c at %p (size %zu)\n", ch, (void *)&ch, sizeof ch);
    printf("d  = %.1f at %p (size %zu)\n", d, (void *)&d, sizeof d);
    return 0;
}
```

OUTPUT (addresses differ on every run)

```
a  = 25 at 0x7ffe0f395648 (size 4)
b  =  7 at 0x7ffe0f39564c (size 4)
ch =  Z at 0x7ffe0f395647 (size 1)
d  = 3.5 at 0x7ffe0f395650 (size 8)
```

11.2 Pointer basics

11.2.1 Declaring a pointer: int *p;

`int *p;` declares `p` as a **pointer to int**: a variable that can hold the address of an int. The `*` belongs to the variable, not the type: `int *p, q;` declares one pointer p and one plain int q. Write `int *p, *q;` for two pointers.

11.2.2 Initializing: p = &x;

11.2.3 Dereferencing with *

Given `p = &x;`, the expression `*p` (the **dereference** or indirection operator) means "the object p points to", i.e. x itself. `*p` can be read and assigned. Note the two different uses of `*`: in a **declaration** it says "is a pointer"; in an **expression** it means "go to the address".

11.2.4 Reading and changing a value through a pointer

USING A POINTER

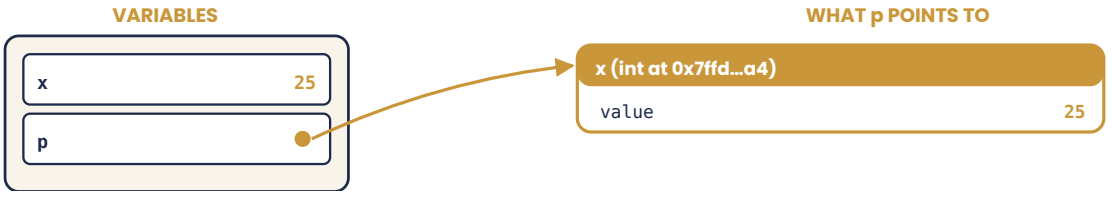
main.c

```c
#include <stdio.h>

int main(void) {
    int x = 10;
    int *p = &x;                    /* p holds the address of x */
    printf("x  = %d\n", x);
    printf("&x = %p\n", (void *)&x);
    printf("p  = %p (same address)\n", (void *)p);
    printf("*p = %d (value at that address)\n", *p);
    *p = 25;                        /* write through the pointer */
    printf("after *p = 25, x = %d\n", x);
    (*p)++;                         /* increment the pointed-to value */
    printf("after (*p)++, x = %d\n", x);
    int y = 100;
    p = &y;                         /* the pointer can be re-aimed */
    printf("p now points to y: *p = %d\n", *p);
    printf("&p = %p (the pointer variable has its own address)\n", (void *)&p);
    return 0;
}
```

OUTPUT (addresses differ per run)

```
x  = 10
&x = 0x7ffcddda0388
p  = 0x7ffcddda0388 (same address)
*p = 10 (value at that address)
after *p = 25, x = 25
after (*p)++, x = 26
p now points to y: *p = 100
&p = 0x7ffcddda0390 (the pointer variable has its own address)
```



P STORES X'S ADDRESS; *P IS X

Expression	Meaning	Value (x = 25, p = &x)
x	value of x	25
&x	address of x	0x7ffd…a4
p	value of p = an address	0x7ffd…a4
*p	object at that address = x	25
&p	address of the pointer itself	0x7ffd…98
*&x	* and & cancel out	25

11.2.5 Size of a pointer on 32-bit and 64-bit systems

A pointer stores an address, so its size depends on the platform's address width, not on what it points to: **8 bytes** on 64-bit systems and **4 bytes** on 32-bit systems, for every pointer type.

ALL POINTERS HAVE THE SAME SIZE (64-BIT LINUX)

main.c

```c
#include <stdio.h>

int main(void) {
    printf("sizeof(char *)   = %zu\n", sizeof(char *));
    printf("sizeof(int *)    = %zu\n", sizeof(int *));
    printf("sizeof(double *) = %zu\n", sizeof(double *));
    printf("sizeof(void *)   = %zu\n", sizeof(void *));
    printf("sizeof(int **)   = %zu\n", sizeof(int **));
    return 0;
}
```

OUTPUT

```
sizeof(char *)   = 8
sizeof(int *)    = 8
sizeof(double *) = 8
sizeof(void *)   = 8
sizeof(int **)   = 8
```

11.2.6 Why pointers have types

If all pointers are 8 bytes, why does `int *` differ from `char *`? The type tells the compiler (1) **how many bytes** to read or write when dereferencing, (2) **how to interpret** those bytes, and (3) how far to move in **pointer arithmetic** (11.4). The same address read through different pointer types gives different results.

SAME ADDRESS, DIFFERENT POINTER TYPES

main.c

```c
#include <stdio.h>

int main(void) {
    int n = 0x41424344;                     /* bytes 44 43 42 41 in memory (little-endian) */
    int *ip = &n;
    unsigned char *cp = (unsigned char *)&n;  /* same address, viewed as bytes */
    printf("*ip   = 0x%X (reads 4 bytes as an int)\n", *ip);
    printf("*cp   = 0x%X '%c' (reads 1 byte)\n", *cp, *cp);
    printf("bytes : %02X %02X %02X %02X\n", cp[0], cp[1], cp[2], cp[3]);
    printf("ip + 1 moves %td bytes, cp + 1 moves %td byte\n",
           (char *)(ip + 1) - (char *)ip, (char *)(cp + 1) - (char *)cp);
    return 0;
}
```

OUTPUT

```
*ip   = 0x41424344 (reads 4 bytes as an int)
*cp   = 0x44 'D' (reads 1 byte)
bytes : 44 43 42 41
ip + 1 moves 4 bytes, cp + 1 moves 1 byte
```

11.3 Special pointers

11.3.1 NULL pointer and nullptr (C23)

`NULL` (from `<stddef.h>`, `<stdio.h>`, …) is a pointer value guaranteed not to point to any object. It means "points nowhere" and is used to initialize pointers that have no target yet, to mark the end of linked lists, and as an error return (malloc, fopen, strchr). Dereferencing NULL is undefined behaviour and crashes on most systems. C23 adds the keyword `nullptr`, a null pointer constant with its own type, avoiding the ambiguity of `NULL` being just `0`.

ALWAYS CHECK FOR NULL main.c

```c
#include <stdio.h>
#include <string.h>

int main(void) {
    int *p = NULL;                          /* explicitly points nowhere */
    if (p == NULL) printf("p is NULL: do not dereference it\n");
    const char *s = "learn";
    const char *hit = strchr(s, 'z');       /* library returns NULL for "not found" */
    printf("strchr result: %s\n", hit ? hit : "(NULL, not found)");
    if (p != NULL && *p > 0) printf("never evaluated thanks to short-circuit\n");
    printf("about to dereference NULL...\n");
    printf("%d\n", *p);                     /* crash */
    return 0;
}
```

OUTPUT

```
p is NULL: do not dereference it
strchr result: (NULL, not found)
about to dereference NULL...
Segmentation fault (core dumped)
```

11.3.2 Wild (uninitialized) pointers

A pointer declared without initialization contains a garbage address and is called a **wild pointer**. Dereferencing it reads or writes random memory: it may crash, corrupt other data, or appear to work. Always initialize pointers, to a valid address or to NULL.

11.3.3 Dangling pointers

A **dangling pointer** points to memory that is no longer valid: a local variable whose function has returned, memory that has been `free`d, or a variable whose block has ended. The address is still there but the object is gone; using it is undefined behaviour.

HOW DANGLING POINTERS ARISE main.c

```c
#include <stdio.h>

int *make_dangling(void) {
    int local = 42;
    return &local;                  /* WRONG: local dies when the function returns */
}
int main(void) {
    int *wild;                      /* wild: uninitialized, never dereference */
    (void)wild;
    int *ptr;
    {
        int inner = 7;
        ptr = &inner;
        printf("inside block: *ptr = %d\n", *ptr);
    }                               /* inner's lifetime ends here: ptr now dangles */
    int *d = make_dangling();
    printf("dangling pointer value: %p (unusable)\n", (void *)d);
    return 0;
}
```

OUTPUT (GCC returns NULL for the escaped local address)

```
inside block: *ptr = 7
dangling pointer value: (nil) (unusable)
```

11.3.4 Void (generic) pointers and casting

A `void *` is a **generic pointer**: it can hold the address of any object type, and any object pointer converts to and from it **without a cast** in C. You cannot dereference a `void *` or do arithmetic on it directly (the compiler does not know the size); convert it to a typed pointer first. `malloc`, `memcpy` and `qsort` use `void *` to work with any type.

A GENERIC PRINT FUNCTION WITH VOID *

main.c

```c
#include <stdio.h>

void print_value(const void *ptr, char type) {
    switch (type) {
        case 'i': printf("int    : %d\n", *(const int *)ptr); break;
        case 'd': printf("double : %.2f\n", *(const double *)ptr); break;
        case 'c': printf("char   : %c\n", *(const char *)ptr); break;
    }
}
int main(void) {
    int i = 42; double d = 9.81; char c = 'Q';
    void *gp;
    gp = &i; print_value(gp, 'i');
    gp = &d; print_value(gp, 'd');
    gp = &c; print_value(gp, 'c');
    int *ip = gp == &c ? NULL : gp;          /* void * converts to int * without a cast */
    printf("ip is %s\n", ip ? "set" : "NULL");
    return 0;
}
```

OUTPUT

```
int    : 42
double : 9.81
char   : Q
ip is NULL
```

11.4 Pointer arithmetic

You can add integers to pointers and subtract pointers, but the arithmetic is in units of the **pointed-to type**, not bytes. This makes pointers the natural tool for walking through arrays.

11.4.1 Increment and decrement

11.4.2 Adding and subtracting integers

For `T *p`, `p + n` points n **elements** further: its address is `p + n × sizeof(T)` bytes. `p++` moves to the next element, `p--` to the previous. The result is only valid while it stays inside the same array (or one past its end).

11.4.5 Scaling by the size of the type

POINTER ARITHMETIC MOVES BY WHOLE ELEMENTS

main.c

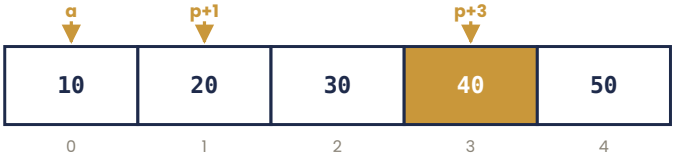
```c
#include <stdio.h>

int main(void) {
    int a[5] = {10, 20, 30, 40, 50};
    int *p = a;                                  /* &a[0] */
    printf("p     -> %p, *p = %d\n", (void *)p, *p);
    p++;
    printf("p++   -> %p, *p = %d (moved %zu bytes)\n", (void *)p, *p, sizeof(int));
    p += 2;
    printf("p += 2-> %p, *p = %d\n", (void *)p, *p);
    printf("*(p - 1) = %d, *(p + 1) = %d\n", *(p - 1), *(p + 1));

    double dv[3] = {1.5, 2.5, 3.5};
    double *dp = dv;
    char text[] = "LBF";
    char *cp = text;
    printf("double pointer step: %td bytes, char pointer step: %td byte\n",
           (char *)(dp + 1) - (char *)dp, (cp + 1) - cp);
    return 0;
}
```

OUTPUT (addresses differ per run)

```
p     -> 0x7ffdb1e4f9d0, *p = 10
p++   -> 0x7ffdb1e4f9d4, *p = 20 (moved 4 bytes)
p += 2-> 0x7ffdb1e4f9dc, *p = 40
*(p - 1) = 30, *(p + 1) = 50
double pointer step: 8 bytes, char pointer step: 1 byte
```



WITH INT *P = A, P + 3 IS 3 × 4 = 12 BYTES FURTHER AND POINTS TO A[3]

11.4.3 Subtracting two pointers (ptrdiff_t)

Subtracting two pointers into the **same array** gives the number of **elements** between them, as a signed integer of type `ptrdiff_t` (print with `%td`). It is commonly used to turn a found pointer back into an index.

11.4.4 Comparing pointers

Pointers into the same array can be compared with `< <= > >=` (which one comes first) and any pointers with `== !=` (same address?). Loops often run `while (p < end)`.

POINTER DIFFERENCE AND COMPARISON main.c

```c
#include <stdio.h>

int main(void) {
    int marks[6] = {67, 82, 91, 74, 88, 59};
    int *start = marks, *end = marks + 6;        /* one past the last element */
    int *best = start;
    for (int *p = start; p < end; p++)           /* compare pointers */
        if (*p > *best) best = p;
    printf("highest = %d at index %td\n", *best, best - start);   /* pointer difference */
    printf("elements between end and start = %td\n", end - start);
    printf("best == &marks[2] ? %s\n", best == &marks[2] ? "yes" : "no");
    return 0;
}
```

OUTPUT

```
highest = 91 at index 2
elements between end and start = 6
best == &marks[2] ? yes
```

11.4.6 Illegal operations on pointers

Operation	Allowed?	Why
`p + n`, `p - n`, `p++`	✓	move within an array
`p - q` (same array)	✓	distance in elements
`p == q`, `p < q`	✓	compare positions
`p + q`	✗	adding two addresses is meaningless
`p * 2`, `p / 2`, `p % 2`	✗	no multiplication/division of pointers
`p & mask`	✗	no bitwise ops (convert to `uintptr_t` first)
arithmetic on `void *`	✗	element size unknown (GCC allows it as an extension)
moving outside the array (except one past the end)	UB	undefined behaviour even without dereferencing

11.5 Pointers and arrays

11.5.1 Array name as a pointer to the first element

In almost every expression, an array name automatically **converts ("decays")** to a pointer to its first element: `marks` is equivalent to `&marks[0]`. That is why arrays can be passed to functions as pointers and why `int *p = marks;` works without `&`.

11.5.2 a[i] is the same as *(a + i)

The subscript operator is **defined** by pointer arithmetic: `a[i]` means `*(a + i)`. It works the same on arrays and pointers, which is why a pointer to the first element can be indexed like the array. (Because addition is commutative, even `i[a]` is legal, though no one should write it.)

FOUR WAYS TO ACCESS THE SAME ELEMENT main.c

```c
#include <stdio.h>

int main(void) {
    int a[4] = {5, 10, 15, 20};
    int *p = a;
    for (int i = 0; i < 4; i++)
        printf("a[%d]=%d  *(a+%d)=%d  p[%d]=%d  *(p+%d)=%d\n", i, a[i], i, *(a + i), i, p[i], i, *(p +
i));
    printf("a == &a[0] ? %s\n", a == &a[0] ? "yes" : "no");
    printf("2[a] = %d (legal, but never do this)\n", 2[a]);
    return 0;
}
```

OUTPUT

```
a[0]=5  *(a+0)=5  p[0]=5  *(p+0)=5
a[1]=10  *(a+1)=10  p[1]=10  *(p+1)=10
a[2]=15  *(a+2)=15  p[2]=15  *(p+2)=15
a[3]=20  *(a+3)=20  p[3]=20  *(p+3)=20
a == &a[0] ? yes
2[a] = 15 (legal, but never do this)
```

11.5.3 Traversing an array with a pointer

WALKING ARRAYS WITH POINTERS main.c

```c
#include <stdio.h>

int main(void) {
    int data[] = {3, 8, 1, 9, 4};
    int n = sizeof data / sizeof data[0];
    int sum = 0;
    for (int *p = data; p < data + n; p++)       /* p walks through the array */
        sum += *p;
    printf("sum = %d\n", sum);
    char word[] = "pointer";
    int vowels = 0;
    for (char *c = word; *c != '\0'; c++)        /* walk until the terminator */
        if (*c == 'a' || *c == 'e' || *c == 'i' || *c == 'o' || *c == 'u') vowels++;
    printf("\"%s\" has %d vowels\n", word, vowels);
    int *q = data + n - 1;                       /* walk backwards */
    printf("reverse:");
    while (q >= data) printf(" %d", *q--);
    printf("\n");
    return 0;
}
```

OUTPUT

```
sum = 25
"pointer" has 3 vowels
reverse: 4 9 1 8 3
```

11.5.4 Differences between arrays and pointers (sizeof, assignment)

Although an array name usually behaves like a pointer, an array **is not** a pointer. An array is a block of elements; a pointer is a single variable holding an address.

	Array `int a[5];`	Pointer `int *p = a;`
What it is	5 ints of storage	one variable holding an address
`sizeof`	20 (whole array)	8 (the pointer)
Assignment	`a = …` is an error: an array cannot be re-pointed	`p = other;` is fine
`&`	`&a` has type `int (*)[5]` (pointer to the whole array)	`&p` has type `int **`
Increment	`a++` is an error	`p++` is fine

AN ARRAY IS NOT A POINTER

main.c

```c
#include <stdio.h>

int main(void) {
    int a[5] = {1, 2, 3, 4, 5};
    int *p = a;
    printf("sizeof a = %zu, sizeof p = %zu\n", sizeof a, sizeof p);
    printf("a = %p, &a = %p (same address, different types)\n", (void *)a, (void *)&a);
    printf("a + 1 moves %td bytes, &a + 1 moves %td bytes\n", (char *)(a + 1) - (char *)a, (char *)(&a
+ 1) - (char *)&a);
    p++;                    /* fine */
    /* a++;                    error: lvalue required */
    printf("*p after p++ = %d\n", *p);
    return 0;
}
```

OUTPUT

```
sizeof a = 20, sizeof p = 8
a = 0x7ffea2e5b2d0, &a = 0x7ffea2e5b2d0 (same address, different types)
a + 1 moves 4 bytes, &a + 1 moves 20 bytes
*p after p++ = 2
```

11.6 Pointers and functions: call by reference

11.6.1 Passing addresses to functions

C passes arguments by value (9.4.2). To let a function modify the caller's variable, pass the variable's **address**; the function receives it in a pointer parameter and works through `*`. This simulates "call by reference".

11.6.2 Swap with pointers

MODIFYING THE CALLER'S VARIABLES

main.c

```c
#include <stdio.h>

void swap(int *a, int *b) {
    int t = *a;          /* value at a */
    *a = *b;
    *b = t;
}
void add_bonus(int *marks, int bonus) {
    if (marks == NULL) return;          /* defensive check */
    *marks += bonus;
}
int main(void) {
    int x = 3, y = 8, m = 72;
    swap(&x, &y);
    printf("after swap: x = %d, y = %d\n", x, y);
    add_bonus(&m, 5);
    printf("marks with bonus = %d\n", m);
    return 0;
}
```

OUTPUT

```
after swap: x = 8, y = 3
marks with bonus = 77
```

11.6.3 Returning multiple values through pointer parameters

A function returns at most one value, but it can deliver more results through **output parameters**: pointers to variables in which the function stores results. The return value is often used for success/failure.

OUTPUT PARAMETERS

main.c

```c
#include <stdio.h>

int divide(int a, int b, int *quotient, int *remainder) {
    if (b == 0) return 0;               /* failure */
    *quotient = a / b;
    *remainder = a % b;
    return 1;                           /* success */
}
void stats(const int *arr, int n, int *min, int *max, double *avg) {
    *min = *max = arr[0];
    long sum = 0;
    for (int i = 0; i < n; i++) {
        if (arr[i] < *min) *min = arr[i];
        if (arr[i] > *max) *max = arr[i];
        sum += arr[i];
    }
    *avg = (double)sum / n;
}
int main(void) {
    int q, r;
    if (divide(47, 5, &q, &r)) printf("47 / 5 = %d remainder %d\n", q, r);
    if (!divide(1, 0, &q, &r)) printf("division by zero rejected\n");
    int marks[] = {72, 95, 61, 88, 79};
    int lo, hi; double avg;
    stats(marks, 5, &lo, &hi, &avg);
    printf("min %d, max %d, average %.1f\n", lo, hi, avg);
    return 0;
}
```

OUTPUT

```
47 / 5 = 9 remainder 2
division by zero rejected
min 61, max 95, average 79.0
```

11.6.4 Never return the address of a local variable

A function's local variables live in its stack frame, which is destroyed when the function returns. Returning a pointer to one creates a dangling pointer (11.3.3); the next function call reuses that memory. Instead: return the value itself, let the caller provide the storage (an output parameter or buffer), use a `static` variable (shared between calls), or allocate with `malloc` (Module 13).

SAFE ALTERNATIVES TO RETURNING A LOCAL'S ADDRESS

main.c

```c
#include <stdio.h>
#include <stdlib.h>

void fill_greeting(char *buf, size_t size, const char *name) {   /* caller owns the buffer */
    snprintf(buf, size, "Hello, %s!", name);
}
int *make_array(int n) {                                          /* heap memory outlives the call */
    int *a = malloc(n * sizeof *a);
    if (a) for (int i = 0; i < n; i++) a[i] = (i + 1) * 10;
    return a;
}
int main(void) {
    char msg[32];
    fill_greeting(msg, sizeof msg, "Asha");
    printf("%s\n", msg);
    int *arr = make_array(4);
    if (arr) {
        for (int i = 0; i < 4; i++) printf("%d ", arr[i]);
        printf("\n");
        free(arr);
    }
    return 0;
}
```

OUTPUT

```
Hello, Asha!
10 20 30 40
```

PRACTICE CHECKPOINT

Swap two numbers through pointers, find the **sum and max of an array using only pointer arithmetic**, and write a function that returns **min and max through pointer parameters**.

SAMPLE SOLUTION

main.c

```c
#include <stdio.h>

void swap(int *a, int *b) { int t = *a; *a = *b; *b = t; }

int sum_ptr(const int *arr, int n) {
    int s = 0;
    for (const int *p = arr; p < arr + n; p++) s += *p;     /* no [] used */
    return s;
}
int max_ptr(const int *arr, int n) {
    const int *best = arr;
    for (const int *p = arr + 1; p < arr + n; p++)
        if (*p > *best) best = p;
    return *best;
}
void min_max(const int *arr, int n, int *min, int *max) {
    *min = *max = *arr;
    for (const int *p = arr + 1; p != arr + n; p++) {
        if (*p < *min) *min = *p;
        if (*p > *max) *max = *p;
    }
}
int main(void) {
    int a = 15, b = 40;
    printf("before swap: a = %d, b = %d\n", a, b);
    swap(&a, &b);
    printf("after swap : a = %d, b = %d\n", a, b);

    int values[] = {23, 7, 56, 14, 89, 3, 41};
    int n = sizeof values / sizeof *values;
    printf("sum = %d, max = %d\n", sum_ptr(values, n), max_ptr(values, n));
    int lo, hi;
    min_max(values, n, &lo, &hi);
    printf("min = %d, max = %d (through pointer parameters)\n", lo, hi);
    return 0;
}
```

OUTPUT

```
before swap: a = 15, b = 40
after swap : a = 40, b = 15
sum = 233, max = 89
min = 3, max = 89 (through pointer parameters)
```

MODULE 12 · INTERMEDIATE

Pointers: advanced

Minimum duration **4 days** | Level **Intermediate**

With the fundamentals in place, this module extends pointers to strings, several levels of indirection, arrays of pointers, pointers to whole arrays and pointers to functions. These are the tools that make C libraries such as qsort() possible.

IN THIS MODULE

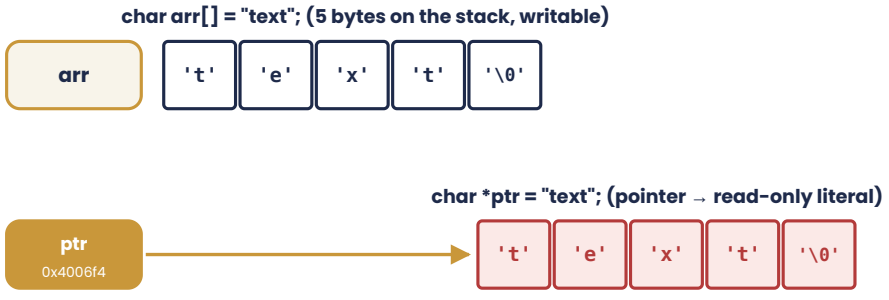
12.1 Pointers and strings
12.2 Pointer to pointer
12.3 Arrays of pointers
12.4 Pointer to an array
12.5 Function pointers
12.6 const and pointers
12.7 Reading complex declarations
12.8 Command-line arguments in depth

12.1 Pointers and strings

12.1.1 char *s = "text" vs char s[] = "text"

These two declarations look almost identical but create very different things:

- `char s[] = "text";` creates an **array of 5 chars** on the stack and copies the characters `t e x t \0` into it. The array is yours: you may modify any character. `s` itself is not a variable; it cannot be re-pointed.
- `char *s = "text";` creates a **pointer variable** (8 bytes) that points to a **string literal** stored in read-only memory. You may re-point `s` to another string, but you must **not** modify the characters; doing so is undefined behaviour and usually crashes with a segmentation fault.



AN ARRAY OWNS A WRITABLE COPY; A POINTER REFERS TO A READ-ONLY LITERAL

ARRAY VS POINTER TO A LITERAL main.c

```c
#include <stdio.h>

int main(void) {
    char arr[] = "text";
    char *ptr  = "text";

    printf("sizeof arr = %zu (whole array incl. \\0)\n", sizeof arr);
    printf("sizeof ptr = %zu (just the pointer)\n", sizeof ptr);

    arr[0] = 'n';               /* fine: the array is writable  */
    printf("arr after change: %s\n", arr);

    ptr = "other";              /* fine: re-point the pointer   */
    printf("ptr now: %s\n", ptr);
    /* arr = "other";   error: array names cannot be assigned  */
    /* ptr[0] = 'O';    compiles, but writes read-only memory: crash */
    return 0;
}
```

OUTPUT

```
sizeof arr = 5 (whole array incl. \0)
sizeof ptr = 8 (just the pointer)
arr after change: next
ptr now: other
```

TIP

Declare pointers to string literals as `const char *s = "text";`. The compiler then rejects any attempt to modify the literal at compile time instead of letting the program crash at run time.

MODIFYING A STRING LITERAL CRASHES main.c

```c
#include <stdio.h>

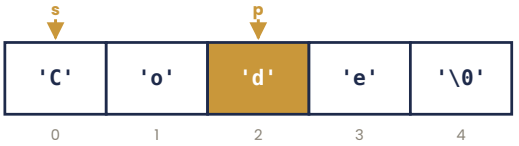
int main(void) {
    char *ptr = "text";
    printf("about to modify a string literal...\n");
    fflush(stdout);
    ptr[0] = 'n';               /* undefined behaviour */
    printf("%s\n", ptr);
    return 0;
}
```

OUTPUT (crash)

```
about to modify a string literal...
Segmentation fault (core dumped)
```

12.1.2 Traversing a string with a pointer

A string is a char array ending in `'\0'`. The usual pointer idiom walks a pointer `p` from the first character until `*p` is the null character, which is false in a condition. The loop `while (*p)` is therefore the same as `while (*p != '\0')`.



P MOVES ALONG THE STRING; THE LOOP STOPS AT \0

WALKING A STRING WITH A POINTER main.c

```c
#include <stdio.h>

int main(void) {
    const char *s = "Learn C";
    const char *p = s;
    int vowels = 0;
    while (*p) {                          /* until '\0' */
        char c = *p;
        if (c=='a'||c=='e'||c=='i'||c=='o'||c=='u'||c=='A'||c=='E'||c=='I'||c=='O'||c=='U')
            vowels++;
        putchar(c);
        putchar(p[1] ? '-' : '\n');
        p++;
    }
    printf("length = %td, vowels = %d\n", p - s, vowels);   /* pointer difference */
    return 0;
}
```

OUTPUT

```
L-e-a-r-n- -C
length = 7, vowels = 2
```

NOTE

`p - s` gives the number of elements between two pointers into the same array. Its type is `ptrdiff_t`, printed with `%td`. After the loop, p points to the terminating null character, so the difference is exactly the string length.

12.1.3 String functions written with pointers

The standard library string functions are classically written with pointers. Writing them yourself is the best practice for pointer fluency. Compare these with the index-based versions in Module 8.

STRLEN, STRCPY, STRCMP, STRCAT AND REVERSE WITH POINTERS

main.c

```c
#include <stdio.h>
#include <stddef.h>

size_t my_strlen(const char *s) {
    const char *p = s;
    while (*p) p++;
    return (size_t)(p - s);
}
char *my_strcpy(char *dst, const char *src) {
    char *d = dst;
    while ((*d++ = *src++))      /* copy, including the final '\0' */
        ;
    return dst;
}
int my_strcmp(const char *a, const char *b) {
    while (*a && *a == *b) { a++; b++; }
    return (unsigned char)*a - (unsigned char)*b;
}
char *my_strcat(char *dst, const char *src) {
    char *d = dst + my_strlen(dst);    /* jump to the end */
    while ((*d++ = *src++))
        ;
    return dst;
}
void my_strrev(char *s) {
    char *l = s, *r = s + my_strlen(s) - 1;
    while (l < r) { char t = *l; *l++ = *r; *r-- = t; }
}
int main(void) {
    char buf[40];
    my_strcpy(buf, "Pointer");
    my_strcat(buf, " power");
    printf("buf = \"%s\", length %zu\n", buf, my_strlen(buf));
    printf("cmp(apple, apply) = %d\n", my_strcmp("apple", "apply"));
    printf("cmp(same, same)   = %d\n", my_strcmp("same", "same"));
    my_strrev(buf);
    printf("reversed = %s\n", buf);
    return 0;
}
```

OUTPUT

```
buf = "Pointer power", length 13
cmp(apple, apply) = -20
cmp(same, same)   = 0
reversed = rewop retnioP
```

The heart of `my_strcpy` is the famous one-liner `while ((*d++ = *src++));`. Read it in steps: copy the character at src into d; then advance both pointers; the value of the assignment is the character copied, and when that is `'\0'` the loop ends. The extra parentheses tell the compiler the assignment inside the condition is intentional.

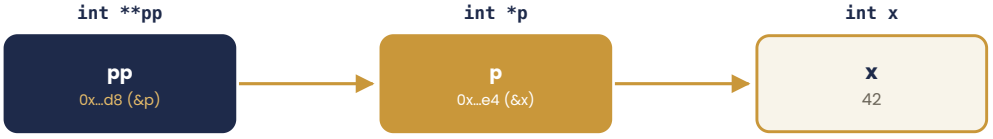
Step	*src	Copied into *d	Loop continues?
1	'P'	'P'	yes
2	'o'	'o'	yes
…	…	…	yes

Step	*src	Copied into *d	Loop continues?
7	'r'	'r'	yes
8	'\0'	'\0'	no: value is 0

12.2 Pointer to pointer

12.2.1 Declaring int **pp

A pointer is itself a variable with an address, so another pointer can point to it. `int **pp` declares a **pointer to a pointer to int**. Each `*` adds one level of indirection. Two levels are common (arrays of strings, changing a caller's pointer); three or more are rare.



*pp is p **pp is x

TWO LEVELS OF INDIRECTION

12.2.2 Double dereferencing

READING AND WRITING THROUGH INT **

main.c

```c
#include <stdio.h>

int main(void) {
    int x = 42;
    int *p = &x;
    int **pp = &p;

    printf("x    = %d\n", x);
    printf("*p   = %d\n", *p);
    printf("**pp = %d\n", **pp);
    printf("p == *pp ? %s\n", p == *pp ? "yes" : "no");
    printf("&p == pp ? %s\n", &p == pp ? "yes" : "no");

    **pp = 99;                      /* changes x through two levels */
    printf("after **pp = 99, x = %d\n", x);

    int y = 7;
    *pp = &y;                       /* changes p itself */
    printf("after *pp = &y, *p = %d\n", *p);
    return 0;
}
```

OUTPUT

```
x    = 42
*p   = 42
**pp = 42
p == *pp ? yes
&p == pp ? yes
after **pp = 99, x = 99
after *pp = &y, *p = 7
```

Expression	Type	Refers to
`pp`	`int **`	address of p
`*pp`	`int *`	p (the pointer)
`**pp`	`int`	x (the int)
`&pp`	`int ***`	address of pp

12.2.3 Changing a pointer inside a function

C passes arguments by value, and that includes pointers. If a function receives `int *p` and assigns `p = something`, only its local copy changes. To change the **caller's pointer**, pass the address of that pointer: an `int **`. This is exactly the same reasoning as passing `&x` to change an int, just one level up.

ALLOCATING MEMORY FOR THE CALLER

main.c

```c
#include <stdio.h>
#include <stdlib.h>

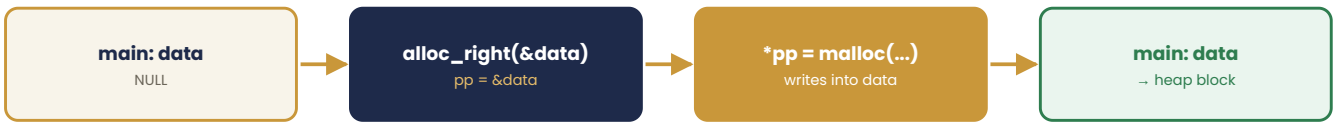
void alloc_wrong(int *p, int n) {        /* p is a copy: caller sees nothing */
    p = malloc(n * sizeof *p);
}
int alloc_right(int **pp, int n) {       /* pp points at the caller's pointer */
    *pp = malloc(n * sizeof **pp);
    if (*pp == NULL) return 0;
    for (int i = 0; i < n; i++) (*pp)[i] = i * i;
    return 1;
}
int main(void) {
    int *data = NULL;
    alloc_wrong(data, 5);
    printf("after alloc_wrong: data is %s\n", data ? "set" : "still NULL");

    if (alloc_right(&data, 5)) {
        printf("after alloc_right: ");
        for (int i = 0; i < 5; i++) printf("%d ", data[i]);
        printf("\n");
        free(data);
    }
    return 0;
}
```

OUTPUT

```
after alloc_wrong: data is still NULL
after alloc_right: 0 1 4 9 16
```

alloc_wrong also leaks the block it allocates; it is shown only to illustrate the mistake.



PASSING &DATA LETS THE FUNCTION CHANGE THE CALLER'S POINTER

Another classic use: deleting the head of a linked list or inserting at the front, where the head pointer itself must change. You will see `Node **head` in Module 19.

12.3 Arrays of pointers

12.3.1 int *arr[5]

`int *arr[5];` is an **array of 5 pointers to int**. The `[]` binds tighter than `*`, so arr is first an array, and each element is an `int *`. Each element can point to a separate int, or to the start of a separate array of a different length (a **jagged array**).

ARRAY OF INT POINTERS AND A JAGGED ARRAY — main.c

```c
#include <stdio.h>

int main(void) {
    int a = 10, b = 20, c = 30;
    int *ptrs[3] = {&a, &b, &c};
    for (int i = 0; i < 3; i++)
        printf("ptrs[%d] -> %d\n", i, *ptrs[i]);
    *ptrs[1] += 5;                      /* changes b */
    printf("b = %d\n", b);

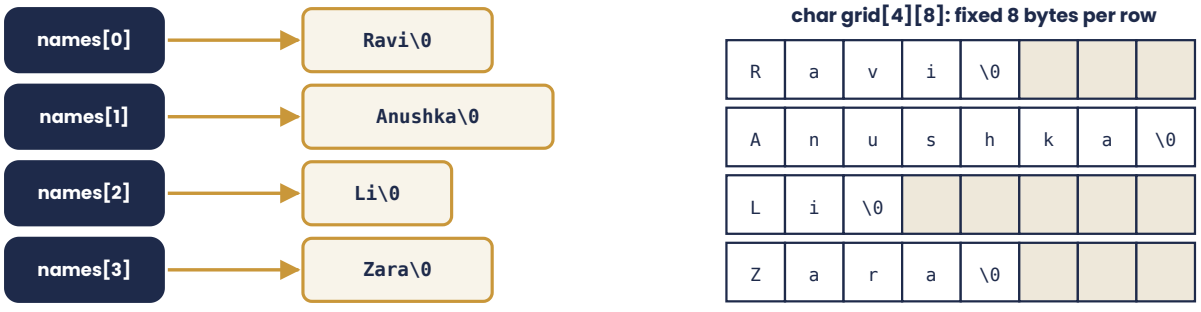
    /* jagged array: rows of different lengths */
    int r0[] = {1}, r1[] = {2, 3}, r2[] = {4, 5, 6};
    int *rows[] = {r0, r1, r2};
    int len[]   = {1, 2, 3};
    for (int i = 0; i < 3; i++) {
        for (int j = 0; j < len[i]; j++) printf("%d ", rows[i][j]);
        printf("\n");
    }
    return 0;
}
```

OUTPUT

```
ptrs[0] -> 10
ptrs[1] -> 20
ptrs[2] -> 30
b = 25
1
2 3
4 5 6
```

12.3.2 Array of strings with char *names[]

An array of `char *` is the natural way to hold a list of strings of different lengths. Each element points to its own string, so no space is wasted, unlike a 2D char array where every row has the same fixed width.



CHAR *NAMES[] (LEFT) VS A 2D CHAR ARRAY (RIGHT, GREY = WASTED BYTES)

A LIST OF NAMES main.c

```c
#include <stdio.h>

int main(void) {
    const char *names[] = {"Ravi", "Anushka", "Li", "Zara"};
    int n = sizeof names / sizeof names[0];
    for (int i = 0; i < n; i++)
        printf("%d: %-8s (first letter %c)\n", i, names[i], *names[i]);

    char grid[4][8] = {"Ravi", "Anushka", "Li", "Zara"};
    printf("sizeof names = %zu (4 pointers)\n", sizeof names);
    printf("sizeof grid  = %zu (4 x 8 chars)\n", sizeof grid);
    printf("grid[1] = %s\n", grid[1]);
    return 0;
}
```

OUTPUT

```
0: Ravi     (first letter R)
1: Anushka  (first letter A)
2: Li       (first letter L)
3: Zara     (first letter Z)
sizeof names = 32 (4 pointers)
sizeof grid  = 32 (4 x 8 chars)
grid[1] = Anushka
```

12.3.3 Sorting strings by swapping pointers

To sort an array of strings you do not need to move any characters. Simply swap the **pointers** in the array. Swapping two 8-byte pointers is far cheaper than copying whole strings, and it works no matter how long the strings are.

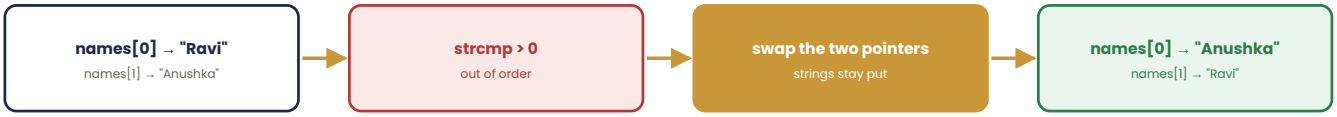
BUBBLE SORT ON AN ARRAY OF STRING POINTERS main.c

```c
#include <stdio.h>
#include <string.h>

void sort_names(const char *a[], int n) {
    for (int i = 0; i < n - 1; i++)
        for (int j = 0; j < n - 1 - i; j++)
            if (strcmp(a[j], a[j + 1]) > 0) {
                const char *t = a[j];          /* swap pointers only */
                a[j] = a[j + 1];
                a[j + 1] = t;
            }
}
int main(void) {
    const char *names[] = {"Ravi", "Anushka", "Li", "Zara", "Meera"};
    int n = sizeof names / sizeof *names;
    sort_names(names, n);
    for (int i = 0; i < n; i++) printf("%s\n", names[i]);
    return 0;
}
```

OUTPUT

```
Anushka
Li
Meera
Ravi
Zara
```



ONLY THE ARROWS MOVE, NEVER THE CHARACTERS

12.4 Pointer to an array

12.4.1 int (*p)[5] vs int *p[5]

Parentheses change everything. Precedence makes `[]` bind before `*`, so:

Declaration	Read as	sizeof on 64-bit	p + 1 moves by
`int *p[5]`	p is an array of 5 pointers to int	40	(p is an array)
`int (*p)[5]`	p is a pointer to an array of 5 ints	8	20 bytes (one whole array)

POINTER TO AN ARRAY VS POINTER TO AN ELEMENT main.c

```c
#include <stdio.h>

int main(void) {
    int row[5] = {1, 2, 3, 4, 5};
    int (*pa)[5] = &row;          /* pointer to the whole array */
    int *pe = row;                /* pointer to the first element */
    int *ap[5];                   /* array of 5 pointers */

    printf("sizeof pa = %zu, sizeof *pa = %zu\n", sizeof pa, sizeof *pa);
    printf("sizeof ap = %zu\n", sizeof ap);
    printf("(*pa)[2] = %d, pe[2] = %d\n", (*pa)[2], pe[2]);
    printf("pe + 1 moves %td bytes\n", (char *)(pe + 1) - (char *)pe);
    printf("pa + 1 moves %td bytes\n", (char *)(pa + 1) - (char *)pa);
    (void)ap;
    return 0;
}
```

OUTPUT

```
sizeof pa = 8, sizeof *pa = 20
sizeof ap = 40
(*pa)[2] = 3, pe[2] = 3
pe + 1 moves 4 bytes
pa + 1 moves 20 bytes
```

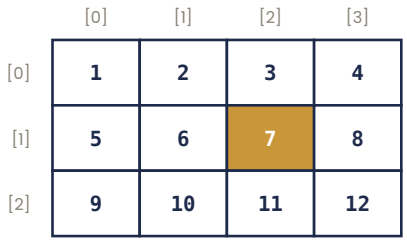
NOTE

`row`, `&row[0]` and `&row` all hold the same address, but they have different types: the first two are `int *`, the third is `int (*)[5]`. The type decides how far `+ 1` moves.

12.4.2 Pointers and 2D arrays

A 2D array `int m[3][4]` is an array of 3 rows, each an `int[4]`, stored row after row. In expressions, `m` decays to a pointer to its first row: type `int (*)[4]`. Therefore:

- `m + i` points to row i (moves 16 bytes per step).
- `*(m + i)` is row i itself, which decays to `int *` pointing at `m[i][0]`.
- `*(*(m + i) + j)` is exactly `m[i][j]`.



2D ARRAY THROUGH POINTERS

main.c

```c
#include <stdio.h>

int main(void) {
    int m[3][4] = {{1, 2, 3, 4}, {5, 6, 7, 8}, {9, 10, 11, 12}};
    printf("m[1][2]           = %d\n", m[1][2]);
    printf("*(*(m + 1) + 2)   = %d\n", *(*(m + 1) + 2));
    printf("(*(m + 1))[2]     = %d\n", (*(m + 1))[2]);
    printf("row size          = %zu bytes\n", sizeof m[0]);

    int (*row)[4] = m;                /* pointer to a row */
    for (int i = 0; i < 3; i++, row++) {
        for (int *q = *row; q < *row + 4; q++) printf("%3d", *q);
        printf("\n");
    }
    /* the whole array is contiguous: walk it with one int pointer */
    int *flat = &m[0][0];
    for (int k = 0; k < 12; k++) printf("%d ", flat[k]);
    printf("\n");
    return 0;
}
```

OUTPUT

```
m[1][2]           = 7
*(*(m + 1) + 2)   = 7
(*(m + 1))[2]     = 7
row size          = 16 bytes
  1  2  3  4
  5  6  7  8
  9 10 11 12
1 2 3 4 5 6 7 8 9 10 11 12
```

12.4.3 Passing 2D arrays with pointer syntax

A parameter declared `int a[][4]` is adjusted by the compiler to `int (*a)[4]`. The number of columns must be known so that `a + 1` knows how far to jump. Since C99 you can also pass the column count first and use a **variable-length array parameter**: `int a[][cols]`.

PASSING 2D ARRAYS

main.c

```c
#include <stdio.h>

void print_fixed(int (*a)[4], int rows) {          /* same as int a[][4] */
    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < 4; j++) printf("%3d", a[i][j]);
        printf("\n");
    }
}
int sum_any(int rows, int cols, int a[rows][cols]) {   /* C99 VLA parameter */
    int s = 0;
    for (int i = 0; i < rows; i++)
        for (int j = 0; j < cols; j++) s += a[i][j];
    return s;
}
int main(void) {
    int m[2][4] = {{1, 2, 3, 4}, {5, 6, 7, 8}};
    int t[3][2] = {{1, 1}, {2, 2}, {3, 3}};
    print_fixed(m, 2);
    printf("sum m = %d\n", sum_any(2, 4, m));
    printf("sum t = %d\n", sum_any(3, 2, t));
    return 0;
}
```

OUTPUT

```
  1  2  3  4
  5  6  7  8
sum m = 36
sum t = 12
```

WATCH OUT

A 2D array `int m[3][4]` is **not** an `int **`. Passing it to a function that takes `int **` is a type error, and casting it to `int **` crashes because there are no row pointers stored anywhere. Use `int (*)[4]` or a VLA parameter.

12.5 Function pointers

12.5.1 Declaring and assigning a function pointer

Compiled functions live in memory too, so they have addresses. A **function pointer** stores the address of a function and lets you call it indirectly. The declaration mirrors the function's prototype with `(*name)` in place of the function name:

SYNTAX

C

```c
int add(int a, int b);          /* a function                       */
int (*op)(int, int);            /* op: pointer to such a function   */
op = add;                       /* function name decays to address  */
op = &add;                      /* the same, written explicitly     */
```

WATCH OUT

Without the parentheses, `int *op(int, int);` declares a **function** named op that returns `int *`. The parentheses around `*op` are essential.

12.5.2 Calling through a function pointer

CALLING A FUNCTION THROUGH A POINTER main.c

```c
#include <stdio.h>

int add(int a, int b) { return a + b; }
int mul(int a, int b) { return a * b; }

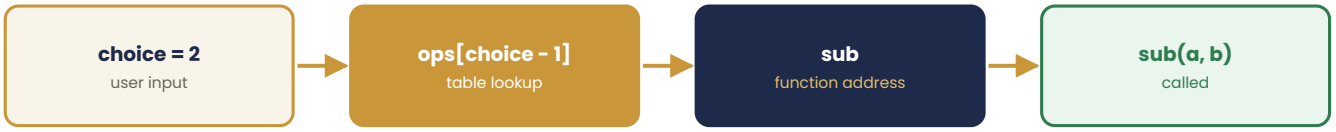
int main(void) {
    int (*op)(int, int) = add;
    printf("op(6, 7) = %d\n", op(6, 7));        /* modern style      */
    printf("(*op)(6, 7) = %d\n", (*op)(6, 7));  /* classic style     */
    op = mul;                                    /* re-point          */
    printf("after op = mul: op(6, 7) = %d\n", op(6, 7));
    printf("op == mul ? %s\n", op == mul ? "yes" : "no");
    return 0;
}
```

OUTPUT

```
op(6, 7) = 13
(*op)(6, 7) = 13
after op = mul: op(6, 7) = 42
op == mul ? yes
```

12.5.3 Arrays of function pointers (menu dispatch)

Put function pointers in an array and you can choose a function by **index** instead of writing a long switch. This is called a **dispatch table**. It is how interpreters, state machines and command menus are often built.



A DISPATCH TABLE REPLACES A SWITCH

CALCULATOR MENU WITH A TABLE OF FUNCTION POINTERS main.c

```c
#include <stdio.h>

double add(double a, double b) { return a + b; }
double sub(double a, double b) { return a - b; }
double mul(double a, double b) { return a * b; }
double dvd(double a, double b) { return a / b; }

int main(void) {
    double (*ops[])(double, double) = {add, sub, mul, dvd};
    const char *names[] = {"add", "subtract", "multiply", "divide"};
    int choice;
    double a, b;
    for (int i = 0; i < 4; i++) printf("%d. %s\n", i + 1, names[i]);
    printf("Choice: ");
    if (scanf("%d", &choice) != 1 || choice < 1 || choice > 4) {
        printf("invalid choice\n");
        return 1;
    }
    printf("Two numbers: ");
    scanf("%lf %lf", &a, &b);
    printf("%s(%.2f, %.2f) = %.2f\n", names[choice - 1], a, b, ops[choice - 1](a, b));
    return 0;
}
```

CONSOLE (user input in gold)

```
1. add
2. subtract
3. multiply
4. divide
Choice: 3
Two numbers: 4.5 2
multiply(4.50, 2.00) = 9.00
```

12.5.4 Callbacks and qsort() with a comparator

A **callback** is a function you pass to another function so that it can call you back. The standard library function `qsort()` sorts any array of anything, because **you** supply the comparison as a callback:

PROTOTYPE IN C

```c
void qsort(void *base, size_t count, size_t size,
           int (*compar)(const void *, const void *));
```

Comparator returns	Meaning
negative	first element goes before the second
0	they are equal
positive	first element goes after the second

qsort passes pointers to the two elements as `const void *`. Inside the comparator you convert them back to the real element type and compare.

QSORT WITH THREE DIFFERENT COMPARATORS

main.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int asc(const void *a, const void *b) {
    int x = *(const int *)a, y = *(const int *)b;
    return (x > y) - (x < y);          /* safe: no overflow */
}
int desc(const void *a, const void *b) { return asc(b, a); }

int by_name(const void *a, const void *b) {
    /* elements are char *, so we receive pointers to char * */
    const char *s = *(const char *const *)a;
    const char *t = *(const char *const *)b;
    return strcmp(s, t);
}
void print_ints(const int *v, int n) {
    for (int i = 0; i < n; i++) printf("%d ", v[i]);
    printf("\n");
}
int main(void) {
    int v[] = {42, 7, 19, 88, 3, 56};
    int n = sizeof v / sizeof *v;
    qsort(v, n, sizeof *v, asc);
    printf("ascending : "); print_ints(v, n);
    qsort(v, n, sizeof *v, desc);
    printf("descending: "); print_ints(v, n);

    const char *names[] = {"Ravi", "Anushka", "Li", "Zara", "Meera"};
    int m = sizeof names / sizeof *names;
    qsort(names, m, sizeof *names, by_name);
    for (int i = 0; i < m; i++) printf("%s ", names[i]);
    printf("\n");
    return 0;
}
```

OUTPUT

```
ascending : 3 7 19 42 56 88
descending: 88 56 42 19 7 3
Anushka Li Meera Ravi Zara
```

WATCH OUT

The tempting comparator `return x - y;` overflows for values such as `INT_MIN` and `INT_MAX`, producing a wrong sign. The idiom `(x > y) - (x < y)` always returns -1, 0 or 1.

You can write your own functions that take callbacks too. Here a generic `apply` runs any transformation over an array:

WRITING A FUNCTION THAT ACCEPTS A CALLBACK

main.c

```c
#include <stdio.h>

void apply(int *a, int n, int (*f)(int)) {
    for (int i = 0; i < n; i++) a[i] = f(a[i]);
}
int square(int x) { return x * x; }
int negate(int x) { return -x; }

int main(void) {
    int a[] = {1, 2, 3, 4};
    apply(a, 4, square);
    for (int i = 0; i < 4; i++) printf("%d ", a[i]);
    printf("\n");
    apply(a, 4, negate);
    for (int i = 0; i < 4; i++) printf("%d ", a[i]);
    printf("\n");
    return 0;
}
```

OUTPUT

```
1 4 9 16
-1 -4 -9 -16
```

12.6 const and pointers

With pointers there are **two** things that can be constant: the data being pointed to, and the pointer itself. Where you write `const` decides which. Rule of thumb: `const` applies to whatever is immediately to its **left**; if nothing is on its left, it applies to what is on its right.

12.6.1 Pointer to const: const int *p

`const int *p` (also written `int const *p`) means "p points to an int that must not be changed **through p**". p itself can be re-pointed. This is the type you use for read-only function parameters such as `size_t strlen(const char *s)`.

12.6.2 Const pointer: int *const p

`int *const p = &x;` means p must always point to x (it must be initialized), but x may be changed through it.

12.6.3 Const pointer to const: const int *const p

Both are locked: p cannot be re-pointed and the data cannot be changed through p.

Declaration	*p = 5	p = &y	Typical use
`int *p`	✔	✔	ordinary pointer
`const int *p`	✘	✔	read-only parameters, iterating
`int *const p`	✔	✘	fixed handle to a buffer or device
`const int *const p`	✘	✘	fixed read-only lookup table

THE THREE CONST COMBINATIONS

main.c

```c
#include <stdio.h>

int main(void) {
    int x = 10, y = 20;

    const int *p1 = &x;      /* pointer to const */
    /* *p1 = 11;    error: assignment of read-only location */
    p1 = &y;                 /* ok */

    int *const p2 = &x;      /* const pointer */
    *p2 = 11;                /* ok */
    /* p2 = &y;     error: assignment of read-only variable p2 */

    const int *const p3 = &y;
    /* *p3 = 1; p3 = &x;   both errors */

    printf("*p1 = %d, x = %d, *p3 = %d\n", *p1, x, *p3);
    return 0;
}
```

OUTPUT

```
*p1 = 20, x = 11, *p3 = 20
```

WHAT THE COMPILER SAYS

main.c

```c
#include <stdio.h>

int main(void) {
    int x = 10, y = 20;
    const int *p1 = &x;
    int *const p2 = &x;
    *p1 = 11;
    p2 = &y;
    return 0;
}
```

OUTPUT

```
main.c: In function ‘main’:
main.c:7:9: error: assignment of read-only location ‘*p1’
    7 |     *p1 = 11;
      |         ^
main.c:8:8: error: assignment of read-only variable ‘p2’
    8 |     p2 = &y;
      |        ^
main.c:6:16: warning: variable ‘p2’ set but not used [-Wunused-but-set-variable]
    6 |     int *const p2 = &x;
      |                ^~
```

NOTE

`const` only promises that **this pointer** will not be used to modify the data. The object itself may still change through another, non-const path: `x = 99;` is still legal above.

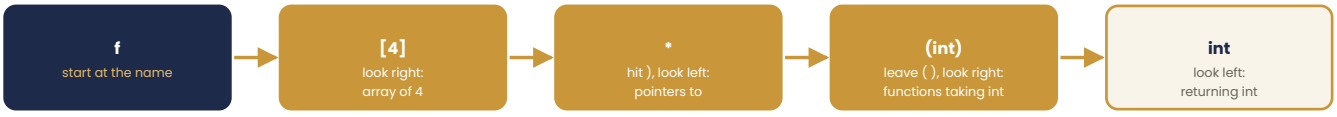
12.7 Reading complex declarations

12.7.1 The clockwise/spiral rule

C declarations are read from the **identifier outwards**, following precedence: postfix `[]` and `()` bind before prefix `*`, and parentheses group. The **clockwise/spiral rule** puts this into a simple procedure:

1. Start at the variable name.
2. Move right: read any `[]` ("array of") or `()` ("function returning").
3. When you hit a closing parenthesis or the end, move left: read any `*` ("pointer to").
4. Jump out of the parentheses and repeat, spiralling outwards, until you reach the base type.

Declaration	In English
`int *p[5]`	p is an array of 5 pointers to int
`int (*p)[5]`	p is a pointer to an array of 5 ints
`int *f(void)`	f is a function returning a pointer to int
`int (*f)(void)`	f is a pointer to a function returning int
`int (*f[4])(int)`	f is an array of 4 pointers to functions taking int and returning int
`char **argv`	argv is a pointer to a pointer to char
`int (*(*g)(int))[3]`	g is a pointer to a function taking int, returning a pointer to an array of 3 ints
`void (*signal(int, void (*)(int)))(int)`	signal is a function taking (int, pointer to function(int) returning void) and returning a pointer to function(int) returning void



READING INT (*F[4])(INT): RIGHT, LEFT, RIGHT, LEFT, SPIRALLING OUT FROM THE NAME

12.7.2 Using typedef to simplify declarations

`typedef` gives a name to a type. Naming the function-pointer type once makes every later declaration readable, and is the standard way professional code handles callbacks.

TYPEDEF TAMES FUNCTION POINTERS AND ARRAY POINTERS

main.c

```c
#include <stdio.h>

typedef int (*BinOp)(int, int);          /* BinOp: pointer to int f(int, int) */
typedef int Row[3];                      /* Row: array of 3 ints              */

int add(int a, int b) { return a + b; }
int sub(int a, int b) { return a - b; }

/* without typedef: int (*pick(char c))(int, int) */
BinOp pick(char c) { return c == '+' ? add : sub; }

int main(void) {
    BinOp table[2] = {add, sub};         /* int (*table[2])(int, int) */
    printf("table[0](8, 3) = %d\n", table[0](8, 3));
    printf("pick('-')(8, 3) = %d\n", pick('-')(8, 3));

    Row grid[2] = {{1, 2, 3}, {4, 5, 6}};
    Row *rp = grid;                      /* int (*rp)[3] */
    printf("rp[1][2] = %d\n", rp[1][2]);
    return 0;
}
```

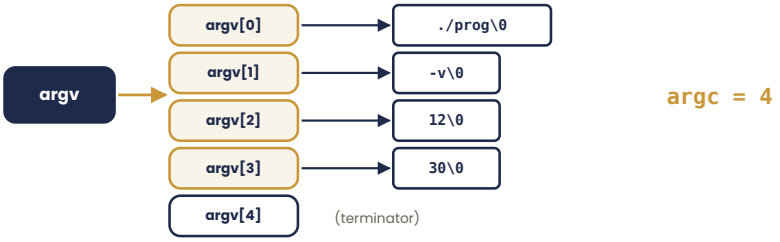
OUTPUT

```
table[0](8, 3) = 11
pick('-')(8, 3) = 5
rp[1][2] = 6
```

12.8 Command-line arguments in depth

12.8.1 char *argv[] as an array of pointers

Now the signature `int main(int argc, char *argv[])` makes complete sense: `argv` is an **array of pointers to char**, one per word on the command line, terminated by a NULL pointer. As a parameter it is adjusted to `char **argv`, so both spellings are identical.



./PROG -V 12 30 IN MEMORY

WALKING ARGV WITH A POINTER: ./PROG -V 12 30

main.c

```c
#include <stdio.h>

int main(int argc, char **argv) {
    printf("argc = %d\n", argc);
    for (char **p = argv; *p != NULL; p++)       /* walk until NULL */
        printf("argv[%td] = \"%s\" (first char '%c')\n", p - argv, *p, **p);
    return 0;
}
```

OUTPUT

```
argc = 4
argv[0] = "./prog" (first char '.')
argv[1] = "-v" (first char '-')
argv[2] = "12" (first char '1')
argv[3] = "30" (first char '3')
```

12.8.2 Parsing options

Programs usually accept **options** (starting with `-`) followed by ordinary arguments. A simple hand-written parser loops over argv, recognizes options, and converts numbers with `strtol` (which, unlike `atoi`, reports bad input).

A TINY OPTION PARSER: ./PROG -V -O MUL 3 4 5

main.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

static void usage(const char *prog) {
    fprintf(stderr, "usage: %s [-v] [-o add|mul] numbers...\n", prog);
}
int main(int argc, char *argv[]) {
    int verbose = 0;
    const char *op = "add";
    int i = 1;
    for (; i < argc && argv[i][0] == '-'; i++) {
        if (strcmp(argv[i], "-v") == 0) verbose = 1;
        else if (strcmp(argv[i], "-o") == 0 && i + 1 < argc) op = argv[++i];
        else { usage(argv[0]); return 1; }
    }
    if (i == argc) { usage(argv[0]); return 1; }
    long result = strcmp(op, "mul") == 0 ? 1 : 0;
    for (; i < argc; i++) {
        char *end;
        long v = strtol(argv[i], &end, 10);
        if (*end != '\0') { fprintf(stderr, "not a number: %s\n", argv[i]); return 1; }
        if (verbose) printf("  %s %ld\n", op, v);
        result = strcmp(op, "mul") == 0 ? result * v : result + v;
    }
    printf("result = %ld\n", result);
    return 0;
}
```

OUTPUT

```
  mul 3
  mul 4
  mul 5
result = 60
```

TIP

On Linux and macOS, the POSIX function `getopt()` in `<unistd.h>` does this parsing for you, including combined flags like `-vx`. It is covered with the other system interfaces in Module 22.

Arrays of pointers, pointers to pointers and function pointers are what make C flexible: argv, qsort and dispatch tables are all built from them. When a declaration looks confusing, start at the name, spiral outwards, and reach for typedef.

PRACTICE CHECKPOINT

Sort an array of names with **qsort and a custom comparator** (case-insensitive, then by length), and build a **calculator that dispatches operations through an array of function pointers**.

SAMPLE SOLUTION

main.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>

/* ---------- part 1: sorting names ---------- */
int ci_cmp(const void *a, const void *b) {
    const char *s = *(const char *const *)a;
    const char *t = *(const char *const *)b;
    for (; *s && *t; s++, t++) {
        int d = tolower((unsigned char)*s) - tolower((unsigned char)*t);
        if (d) return d;
    }
    return (unsigned char)*s - (unsigned char)*t;
}
int len_cmp(const void *a, const void *b) {
    size_t x = strlen(*(const char *const *)a), y = strlen(*(const char *const *)b);
    if (x != y) return (x > y) - (x < y);
    return ci_cmp(a, b);                    /* tie-break alphabetically */
}
void show(const char *title, const char **v, int n) {
    printf("%-14s:", title);
    for (int i = 0; i < n; i++) printf(" %s", v[i]);
    printf("\n");
}
/* ---------- part 2: calculator ---------- */
typedef double (*Op)(double, double);
double add(double a, double b) { return a + b; }
double sub(double a, double b) { return a - b; }
double mul(double a, double b) { return a * b; }
double dvd(double a, double b) { return a / b; }

int main(void) {
    const char *names[] = {"zara", "Anushka", "li", "Ravi", "meera", "Om"};
    int n = sizeof names / sizeof *names;
    qsort(names, n, sizeof *names, ci_cmp);
    show("alphabetical", names, n);
    qsort(names, n, sizeof *names, len_cmp);
    show("by length", names, n);

    const char symbols[] = "+-*/";
    Op ops[] = {add, sub, mul, dvd};
    double a, b;
    char sym;
```

```c
    while (scanf(" %lf %c %lf", &a, &sym, &b) == 3) {
        const char *pos = strchr(symbols, sym);
        if (pos == NULL || sym == '\0') { printf("unknown operator %c\n", sym); continue; }
        if (sym == '/' && b == 0) { printf("cannot divide by zero\n"); continue; }
        printf("%g %c %g = %g\n", a, sym, b, ops[pos - symbols](a, b));
    }
    return 0;
}
```

CONSOLE (user input in gold)

```
alphabetical  : Anushka li meera Om Ravi zara
by length     : li Om Ravi zara meera Anushka
12 + 30
12 + 30 = 42
7 * 6
7 * 6 = 42
9 / 0
cannot divide by zero
5 % 2
unknown operator %
100 - 58
100 - 58 = 42
```

MODULE 13 · INTERMEDIATE

Dynamic memory allocation

Minimum duration **3 days** | Level **Intermediate**

Until now every variable had a size fixed when the program was compiled. Dynamic memory lets a program request memory from the heap at run time, in exactly the amount it needs, and give it back when it is done. It is what makes growing lists, user-sized matrices and linked data structures possible.

IN THIS MODULE

13.1 Static vs dynamic allocation
13.2 stdlib.h allocation functions
13.3 Dynamic arrays
13.4 Memory errors
13.5 Good practices

13.1 Static vs dynamic allocation

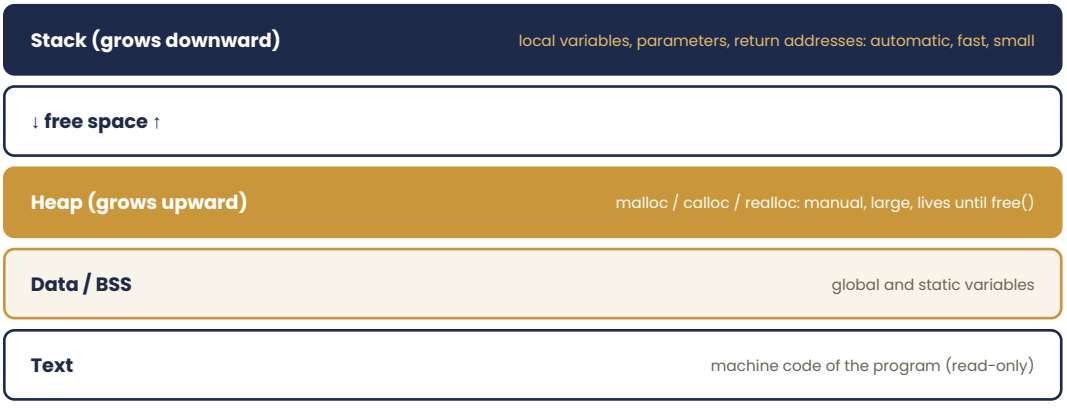
13.1.1 Limits of fixed-size arrays

A declaration such as `int marks[100];` has three limits:

- **The size is guessed in advance.** If the class has 30 students, 70 slots are wasted; if it has 150, the program cannot cope.
- **The lifetime is tied to the block.** A local array disappears when its function returns, so a function cannot build an array and hand it back.
- **The stack is small.** Typical stack limits are 1 to 8 MB. A local `double big[2000000];` (16 MB) crashes the program with a stack overflow.

Variable-length arrays (C99) fix the first problem but not the other two. **Dynamic allocation** fixes all three: the size is decided at run time, the memory lives until you free it, and it comes from the much larger heap.

13.1.2 Stack vs heap



MEMORY LAYOUT OF A RUNNING C PROGRAM

	Stack	Heap
Who manages it	the compiler, automatically	you, with malloc and free
Lifetime	until the block/function ends	until free() is called
Size	small (often 8 MB on Linux)	large (limited by RAM and OS)
Speed	very fast (move a register)	slower (allocator searches for space)
Size known at	compile time (or VLA at block entry)	run time, any moment
Typical errors	stack overflow, returning a local's address	leaks, double free, use after free

WHERE VARIABLES LIVE main.c

```c
#include <stdio.h>
#include <stdlib.h>

int global_counter = 0;                     /* data segment */

int main(void) {
    int local = 5;                          /* stack */
    int *heap = malloc(sizeof *heap);       /* heap  */
    if (heap == NULL) return 1;
    *heap = 7;
    printf("global : %p\n", (void *)&global_counter);
    printf("heap   : %p\n", (void *)heap);
    printf("stack  : %p\n", (void *)&local);
    printf("values : %d %d %d\n", global_counter, *heap, local);
    free(heap);
    return 0;
}
```

OUTPUT (addresses differ per run)

```
global : 0x55b98c8a8014
heap   : 0x55b9cc4ed2a0
stack  : 0x7ffec6ca907c
values : 0 7 5
```

The stack address is much higher than the heap and global addresses, matching the picture above: the stack sits near the top of the address space and grows down; the heap sits above the program's data and grows up.

13.2 stdlib.h allocation functions

Four functions in `<stdlib.h>` manage the heap. All allocation functions return a `void *` (a generic pointer) which C converts automatically to any object pointer type; no cast is needed in C.

Function	Prototype	What it does
malloc	`void *malloc(size_t size)`	allocates size bytes, contents **uninitialized**
calloc	`void *calloc(size_t n, size_t size)`	allocates n × size bytes, all **zero**
realloc	`void *realloc(void *p, size_t size)`	resizes a block, keeping its contents
free	`void free(void *p)`	returns a block to the heap

13.2.1 malloc()

`malloc(n)` finds n free bytes on the heap and returns the address of the first one. The bytes contain whatever was there before (garbage). Always compute the size with `sizeof`:

SNIPPET C

```c
int *a = malloc(10 * sizeof *a);     /* room for 10 ints = 40 bytes  */
double *d = malloc(sizeof *d);       /* room for one double          */
```

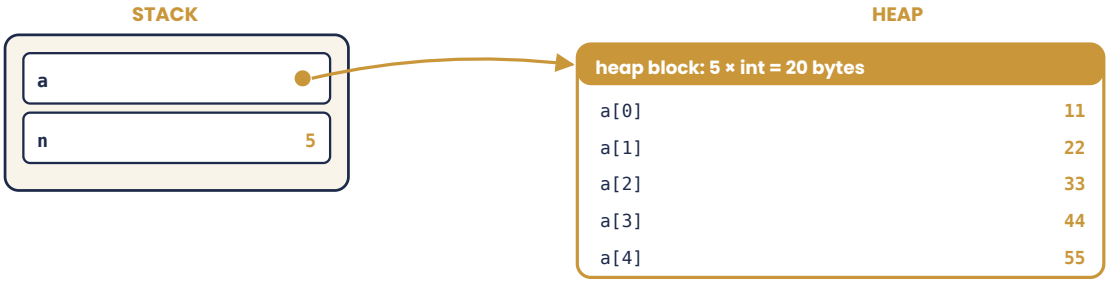
ALLOCATING AN INT ARRAY WITH MALLOC main.c

```c
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    int n = 5;
    int *a = malloc(n * sizeof *a);
    if (a == NULL) {
        fprintf(stderr, "out of memory\n");
        return 1;
    }
    for (int i = 0; i < n; i++) a[i] = (i + 1) * 11;   /* use like an array */
    for (int i = 0; i < n; i++) printf("%d ", a[i]);
    printf("\n%zu bytes requested\n", n * sizeof *a);
    free(a);
    return 0;
}
```

OUTPUT

```
11 22 33 44 55
20 bytes requested
```



THE POINTER LIVES ON THE STACK; THE BLOCK LIVES ON THE HEAP

13.2.2 calloc()

`calloc(n, size)` allocates an array of n elements of the given size and sets **every byte to zero**. It also checks that `n * size` does not overflow, which malloc cannot do for you.

CALLOC GIVES A ZEROED COUNTER ARRAY main.c

```c
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    int *counts = calloc(6, sizeof *counts);     /* 6 zeroed ints */
    if (!counts) return 1;
    int rolls[] = {3, 5, 3, 1, 6, 3, 5, 2};
    for (int i = 0; i < 8; i++) counts[rolls[i] - 1]++;
    for (int f = 0; f < 6; f++) printf("face %d: %d\n", f + 1, counts[f]);
    free(counts);
    return 0;
}
```

OUTPUT

```
face 1: 1
face 2: 1
face 3: 3
face 4: 0
face 5: 2
face 6: 1
```

13.2.3 realloc() and safe usage with a temporary pointer

`realloc(p, new_size)` changes the size of the block p. It may grow it in place, or allocate a new block, copy the old contents and free the old block. Either way it returns the (possibly new) address. If it fails, it returns NULL and **leaves the old block untouched**.

> **WATCH OUT**
> Never write `p = realloc(p, size);`. If realloc fails, p is overwritten with NULL and the only pointer to the old block is lost: a memory leak, and your data is gone. Always assign to a temporary first.

SAFE REALLOC PATTERN C

```c
int *tmp = realloc(a, new_n * sizeof *a);
if (tmp == NULL) {
    /* a is still valid: report the error, keep or free the old data */
} else {
    a = tmp;          /* only now replace the old pointer */
}
```

GROWING AND SHRINKING WITH REALLOC

main.c

```c
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    int *a = malloc(3 * sizeof *a);
    if (!a) return 1;
    for (int i = 0; i < 3; i++) a[i] = i + 1;
    printf("before: ");
    for (int i = 0; i < 3; i++) printf("%d ", a[i]);
    printf("\n");

    int *tmp = realloc(a, 6 * sizeof *a);        /* grow to 6 */
    if (tmp == NULL) { free(a); return 1; }
    a = tmp;
    for (int i = 3; i < 6; i++) a[i] = (i + 1) * 100;
    printf("after : ");
    for (int i = 0; i < 6; i++) printf("%d ", a[i]);  /* old values kept */
    printf("\n");

    tmp = realloc(a, 2 * sizeof *a);             /* shrink to 2 */
    if (tmp) a = tmp;
    printf("shrunk: %d %d\n", a[0], a[1]);
    free(a);
    return 0;
}
```

OUTPUT

```
before: 1 2 3
after : 1 2 3 400 500 600
shrunk: 1 2
```

Call	Equivalent to
`realloc(NULL, n)`	`malloc(n)`
`realloc(p, 0)`	implementation-defined; do not rely on it (C23 makes it undefined). Use free(p).

13.2.4 free()

`free(p)` gives the block back to the allocator so it can be reused. Rules:

- Only free pointers that came from malloc, calloc or realloc.
- Free each block **exactly once**.
- After free, the pointer still holds the old address but the memory is no longer yours: do not use it.
- `free(NULL)` is allowed and does nothing, so no check is needed before freeing.

13.2.5 malloc vs calloc

	malloc(n * size)	calloc(n, size)
Arguments	one: total bytes	two: count and element size
Initial contents	indeterminate (garbage)	all bits zero
Overflow check of n × size	no	yes, returns NULL

	malloc(n * size)	calloc(n, size)
Speed	slightly faster	may be slower (zeroing), but large blocks come pre-zeroed from the OS
Use when	you will overwrite every element anyway	you need zeros: counters, flags, sums

CALLOC ZEROES, MALLOC DOES NOT

main.c

```c
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    /* reuse a freed block so malloc's "garbage" is visible */
    int *old = malloc(4 * sizeof *old);
    for (int i = 0; i < 4; i++) old[i] = 1111 * (i + 1);
    free(old);
    int *m = malloc(4 * sizeof *m);            /* likely the same block */
    int *c = calloc(4, sizeof *c);
    printf("calloc: %d %d %d %d\n", c[0], c[1], c[2], c[3]);
    printf("malloc: contents are indeterminate; never read before writing\n");
    free(m);
    free(c);
    return 0;
}
```

OUTPUT

```
calloc: 0 0 0 0
malloc: contents are indeterminate; never read before writing
```

13.2.6 Checking for NULL on failure

When the heap cannot satisfy a request the allocation functions return **NULL**. Dereferencing NULL crashes the program, so every allocation must be checked. A clean approach is a small wrapper that prints an error and exits:

HANDLING ALLOCATION FAILURE

main.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <stdint.h>

void *xmalloc(size_t size) {
    void *p = malloc(size);
    if (p == NULL && size != 0) {
        fprintf(stderr, "fatal: cannot allocate %zu bytes\n", size);
        exit(EXIT_FAILURE);
    }
    return p;
}
int main(void) {
    int *ok = xmalloc(100 * sizeof *ok);
    printf("100 ints allocated\n");
    free(ok);
    void *huge = malloc(SIZE_MAX / 2);          /* far too large */
    printf("huge request returned %s\n", huge ? "memory" : "NULL");
    free(huge);
    int *fail = xmalloc(SIZE_MAX / 2);
    fail[0] = 1;                                 /* never reached */
    return 0;
}
```

OUTPUT

```
100 ints allocated
huge request returned NULL
fatal: cannot allocate 9223372036854775807 bytes
```

13.3 Dynamic arrays

13.3.1 Dynamic 1D array of user-given size

AN ARRAY WHOSE SIZE THE USER CHOOSES

main.c

```c
#include <stdio.h>
#include <stdlib.h>

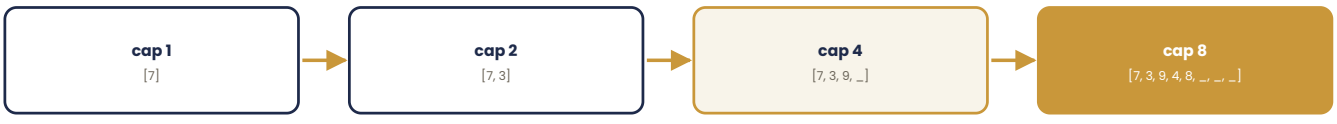
int main(void) {
    int n;
    printf("How many marks? ");
    if (scanf("%d", &n) != 1 || n <= 0) return 1;
    int *marks = malloc(n * sizeof *marks);
    if (!marks) return 1;
    int sum = 0;
    for (int i = 0; i < n; i++) {
        printf("Mark %d: ", i + 1);
        scanf("%d", &marks[i]);
        sum += marks[i];
    }
    printf("Average = %.2f\n", (double)sum / n);
    printf("Above average:");
    for (int i = 0; i < n; i++)
        if (marks[i] > (double)sum / n) printf(" %d", marks[i]);
    printf("\n");
    free(marks);
    return 0;
}
```

CONSOLE (user input in gold)

```
How many marks? 4
Mark 1: 72
Mark 2: 85
Mark 3: 64
Mark 4: 91
Average = 78.00
Above average: 85 91
```

13.3.2 Growing array (vector) with realloc

When the number of items is not known even at run time (for example, reading until end of input), keep a **size** (items used) and a **capacity** (slots allocated). When size reaches capacity, double the capacity with realloc. Doubling means that across n insertions the total copying is proportional to n, so each push costs constant time on average (amortized O(1)).



CAPACITY DOUBLES EACH TIME THE ARRAY FILLS UP

A GROWABLE VECTOR main.c

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct {
    int *data;
    size_t size, cap;
} IntVec;

int vec_push(IntVec *v, int x) {
    if (v->size == v->cap) {
        size_t ncap = v->cap ? v->cap * 2 : 1;
        int *tmp = realloc(v->data, ncap * sizeof *tmp);
        if (!tmp) return 0;
        v->data = tmp;
        v->cap = ncap;
        printf("  (grew to capacity %zu)\n", ncap);
    }
    v->data[v->size++] = x;
    return 1;
}
void vec_free(IntVec *v) { free(v->data); v->data = NULL; v->size = v->cap = 0; }

int main(void) {
    IntVec v = {NULL, 0, 0};
    int x;
    while (scanf("%d", &x) == 1)
        if (!vec_push(&v, x)) break;
    printf("%zu numbers:", v.size);
    for (size_t i = 0; i < v.size; i++) printf(" %d", v.data[i]);
    printf("\n");
    vec_free(&v);
    return 0;
}
```

CONSOLE (user input in gold)

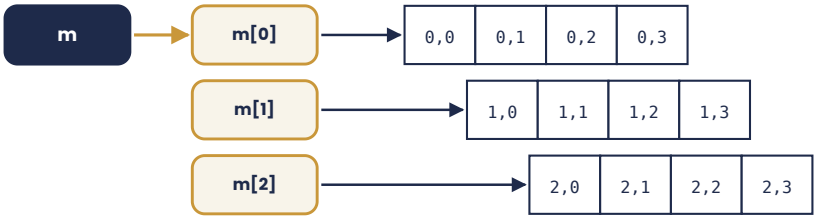
```
7 3 9 4 8 1
  (grew to capacity 1)
  (grew to capacity 2)
  (grew to capacity 4)
  (grew to capacity 8)
6 numbers: 7 3 9 4 8 1
```

13.3.3 Dynamic 2D arrays

A matrix whose rows and columns are chosen at run time can be built in two ways.

13.3.3.1 Array of pointers method

Allocate an array of `rows` row pointers, then allocate each row separately. The syntax `m[i][j]` works, rows may even have different lengths, but the memory is scattered and you need rows + 1 calls to malloc and to free.



ROW POINTERS, EACH ROW A SEPARATE HEAP BLOCK

MATRIX AS AN ARRAY OF ROW POINTERS main.c

```c
#include <stdio.h>
#include <stdlib.h>

int **alloc_matrix(int rows, int cols) {
    int **m = malloc(rows * sizeof *m);
    if (!m) return NULL;
    for (int i = 0; i < rows; i++) {
        m[i] = malloc(cols * sizeof *m[i]);
        if (!m[i]) {                        /* undo what succeeded */
            while (i--) free(m[i]);
            free(m);
            return NULL;
        }
    }
    return m;
}
void free_matrix(int **m, int rows) {
    for (int i = 0; i < rows; i++) free(m[i]);   /* rows first */
    free(m);                                     /* then the pointer array */
}
int main(void) {
    int rows = 3, cols = 4;
    int **m = alloc_matrix(rows, cols);
    if (!m) return 1;
    for (int i = 0; i < rows; i++)
        for (int j = 0; j < cols; j++) m[i][j] = i * cols + j;
    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < cols; j++) printf("%3d", m[i][j]);
        printf("\n");
    }
    free_matrix(m, rows);
    return 0;
}
```

OUTPUT

```
  0  1  2  3
  4  5  6  7
  8  9 10 11
```

13.3.3.2 Single contiguous block method

Allocate all rows × cols elements in one block and compute the position of element (i, j) as `i * cols + j`. One malloc, one free, cache-friendly. With a C99 pointer-to-VLA you can even keep the `m[i][j]` syntax.

MATRIX IN ONE CONTIGUOUS BLOCK main.c

```c
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    int rows = 3, cols = 4;

    /* (a) flat block with manual indexing */
    int *flat = malloc(rows * cols * sizeof *flat);
    if (!flat) return 1;
    for (int i = 0; i < rows; i++)
        for (int j = 0; j < cols; j++) flat[i * cols + j] = (i + 1) * (j + 1);
    printf("flat[2][3] = %d\n", flat[2 * cols + 3]);
    free(flat);

    /* (b) pointer to a VLA row: one block, real 2D syntax */
    int (*m)[cols] = malloc(rows * sizeof *m);
    if (!m) return 1;
    for (int i = 0; i < rows; i++)
        for (int j = 0; j < cols; j++) m[i][j] = (i + 1) * (j + 1);
    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < cols; j++) printf("%3d", m[i][j]);
        printf("\n");
    }
    printf("one block of %zu bytes\n", rows * sizeof *m);
    free(m);
    return 0;
}
```

OUTPUT

```
flat[2][3] = 12
  1  2  3  4
  2  4  6  8
  3  6  9 12
one block of 48 bytes
```

	Array of pointers	Single block
malloc / free calls	rows + 1	1
Jagged rows possible	yes	no
Memory layout	scattered	contiguous, cache friendly
Swap two rows	swap two pointers	copy the rows
Pass to int (*)[cols] functions	no	yes

13.3.4 Dynamic strings

Strings whose length is not known in advance (user input, joined text) are allocated with exactly `strlen + 1` bytes. POSIX and C23 provide `strdup()`, which allocates and copies in one step.

ALLOCATING STRINGS OF THE RIGHT SIZE main.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

char *my_strdup(const char *s) {
    size_t n = strlen(s) + 1;           /* +1 for '\0' */
```

```
    char *copy = malloc(n);
    if (copy) memcpy(copy, s, n);
    return copy;
}
char *join(const char *a, const char *sep, const char *b) {
    size_t n = strlen(a) + strlen(sep) + strlen(b) + 1;
    char *r = malloc(n);
    if (r) snprintf(r, n, "%s%s%s", a, sep, b);
    return r;
}
char *read_line(FILE *in) {             /* any length, grows as needed */
    size_t cap = 8, len = 0;
    char *buf = malloc(cap);
    int c;
    if (!buf) return NULL;
    while ((c = fgetc(in)) != EOF && c != '\n') {
        if (len + 1 == cap) {
            char *tmp = realloc(buf, cap *= 2);
            if (!tmp) { free(buf); return NULL; }
            buf = tmp;
        }
        buf[len++] = (char)c;
    }
    if (len == 0 && c == EOF) { free(buf); return NULL; }
    buf[len] = '\0';
    return buf;
}
int main(void) {
    char *a = my_strdup("Learn");
    char *full = join(a, " Being ", "Forward");
    printf("%s (%zu chars)\n", full, strlen(full));
    char *line = read_line(stdin);
    if (line) printf("You typed %zu characters: %s\n", strlen(line), line);
    free(a); free(full); free(line);
    return 0;
}
```

CONSOLE (user input in gold)

```
Learn Being Forward (19 chars)
a line of input that is much longer than eight bytes
You typed 52 characters: a line of input that is much longer than eight bytes
```

13.4 Memory errors

With manual memory management come a family of bugs that the compiler cannot catch. They often do not crash immediately, which makes them dangerous. The examples below are deliberately broken; Module 21 shows how Valgrind and AddressSanitizer find them automatically.

Leak
memory never freed

Dangling pointer
used after free

Double free
freed twice

Uninitialized read
garbage values

Heap overflow
writing past the end

NULL after free
the simple defence

13.4.1 Memory leaks

A **leak** happens when the last pointer to a block is lost before the block is freed. The memory stays reserved until the program ends. One leak in a short program is harmless; a leak inside a loop in a server that runs for weeks eventually exhausts memory.

A LEAK ON AN EARLY-RETURN PATH

main.c

```c
#include <stdio.h>
#include <stdlib.h>

void process(int n) {
    int *tmp = malloc(n * sizeof *tmp);
    if (!tmp) return;
    tmp[0] = n;
    if (n > 100) return;          /* BUG: early return skips free */
    free(tmp);
}
int main(void) {
    for (int i = 0; i < 1000; i++) process(1000);   /* leaks 4000 bytes each time */
    printf("finished; about %d KB were leaked\n", 1000 * 1000 * 4 / 1024);
    return 0;
}
```

OUTPUT

```
finished; about 3906 KB were leaked
```

Valgrind confirms it (output trimmed):

Terminal

```
$ gcc -g main.c -o prog
$ valgrind --leak-check=full ./prog
==5235== Memcheck, a memory error detector
==5235== Command: ./prog
finished; about 3906 KB were leaked
==5235== HEAP SUMMARY:
==5235==     in use at exit: 4,000,000 bytes in 1,000 blocks
==5235==   total heap usage: 1,001 allocs, 1 frees, 4,004,096 bytes allocated
==5235==
==5235== 4,000,000 bytes in 1,000 blocks are definitely lost in loss record 1 of 1
==5235==    at 0x4846828: malloc (in /usr/libexec/valgrind/vgpreload_memcheck-amd64-linux.so)
==5235==    by 0x1091A8: process (main.c:5)
==5235==    by 0x1091F5: main (main.c:12)
==5235==
==5235== LEAK SUMMARY:
==5235==    definitely lost: 4,000,000 bytes in 1,000 blocks
==5235== ERROR SUMMARY: 1 errors from 1 contexts (suppressed: 0 from 0)
```

13.4.2 Dangling pointers after free

After `free(p)`, p still holds the old address. Reading or writing through it (**use after free**) is undefined behaviour: the block may already belong to another allocation, so you silently corrupt someone else's data.

USE AFTER FREE READS SOMEONE ELSE'S DATA

main.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main(void) {
    char *name = malloc(16);
    strcpy(name, "Asha");
    free(name);                         /* name is now dangling */
    char *other = malloc(16);           /* allocator reuses the block */
    strcpy(other, "Balance: 5000");
    printf("name  -> %s   (use after free!)\n", name);
    printf("same block? %s\n", (void *)name == (void *)other ? "yes" : "no");
    free(other);
    return 0;
}
```

OUTPUT (undefined behaviour; typical glibc result)

```
name  -> Balance: 5000   (use after free!)
same block? yes
```

13.4.3 Double free

Freeing the same block twice corrupts the allocator's internal lists. Modern glibc detects many cases and aborts the program:

DOUBLE FREE

main.c

```c
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    int *p = malloc(sizeof *p);
    free(p);
    printf("freed once\n");
    fflush(stdout);
    free(p);                            /* BUG: second free */
    printf("never printed\n");
    return 0;
}
```

OUTPUT (crash)

```
freed once
free(): double free detected in tcache 2
Aborted (core dumped)
```

13.4.4 Using uninitialized memory

malloc does not clear memory. Reading a value before writing it gives whatever bytes happened to be there. Results change between runs and between machines, so such bugs are notoriously hard to reproduce. Use calloc when you need zeros, or initialize every element before reading it.

SNIPPET

C

```c
int *sum = malloc(sizeof *sum);
for (int i = 0; i < 10; i++) *sum += i;      /* BUG: *sum starts as garbage */

int *sum2 = calloc(1, sizeof *sum2);          /* fixed: starts at 0 */
```

13.4.5 Buffer overflow on the heap

Writing past the end of a heap block overwrites the allocator's bookkeeping data or a neighbouring block. The classic cause is forgetting the `+ 1` for the string terminator.

OFF-BY-ONE HEAP OVERFLOW

main.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main(void) {
    const char *src = "overflow";           /* 8 chars + '\0' = 9 bytes */
    char *dst = malloc(strlen(src));        /* BUG: only 8 bytes */
    strcpy(dst, src);                       /* writes 9 bytes */
    printf("%s\n", dst);                   /* may "work"... */
    free(dst);
    return 0;
}
```

Compiled with AddressSanitizer, the bug is reported at the exact line (library frames trimmed):

Terminal

```
$ gcc -g -fsanitize=address -fno-omit-frame-pointer main.c -o prog
$ ./prog
=================================================================
==5227==ERROR: AddressSanitizer: heap-buffer-overflow on address 0x502000000018
WRITE of size 9 at 0x502000000018 thread T0
    #0 0x7f35ba0a7922 in strcpy ../../../../src/libsanitizer/asan/asan_interceptors.cpp:563
    #1 0x56361b27c28a in main main.c:8

0x502000000018 is located 0 bytes after 8-byte region [0x502000000010,0x502000000018)
allocated by thread T0 here:
    #0 0x7f35ba0fd9c7 in malloc ../../../../src/libsanitizer/asan/asan_malloc_linux.cpp:69
    #1 0x56361b27c273 in main main.c:7

SUMMARY: AddressSanitizer: heap-buffer-overflow in strcpy
```

NOTE

Without a sanitizer, this program usually prints "overflow" and exits normally, which is exactly why heap overflows are dangerous: the bug hides until it corrupts something important. Compile with `-fsanitize=address` during development.

13.4.6 Setting pointers to NULL after free

A simple defence against use after free and double free: assign NULL right after freeing. A later `free(p)` becomes a harmless no-op, and an accidental `*p` crashes immediately and predictably instead of corrupting data silently.

FREE AND NULL TOGETHER

main.c

```c
#include <stdio.h>
#include <stdlib.h>

#define FREE(p) do { free(p); (p) = NULL; } while (0)

int main(void) {
    int *p = malloc(sizeof *p);
    *p = 10;
    FREE(p);
    FREE(p);                          /* safe: free(NULL) does nothing */
    if (p == NULL) printf("p is NULL, so we know it is not usable\n");
    return 0;
}
```

OUTPUT

```
p is NULL, so we know it is not usable
```

NOTE

Setting a pointer to NULL protects only **that** pointer variable. Other copies of the same address elsewhere in the program still dangle, which is why clear ownership (next section) matters.

13.5 Good practices

13.5.1 Every malloc has one matching free

For every successful allocation there must be exactly one free on every path through the program, including error paths. A common structure for functions with several allocations is the single cleanup label:

ONE EXIT POINT THAT FREES EVERYTHING

main.c

```c
#include <stdio.h>
#include <stdlib.h>

int work(int n) {
    int result = -1;
    int *a = NULL, *b = NULL;
    a = malloc(n * sizeof *a);
    if (!a) goto cleanup;
    b = malloc(n * sizeof *b);
    if (!b) goto cleanup;
    for (int i = 0; i < n; i++) { a[i] = i; b[i] = i * i; }
    result = 0;
    for (int i = 0; i < n; i++) result += a[i] * b[i];
cleanup:
    free(b);                  /* free(NULL) is fine */
    free(a);
    return result;
}
int main(void) {
    printf("sum of i^3 for i < 5 = %d\n", work(5));
    return 0;
}
```

OUTPUT

```
sum of i^3 for i < 5 = 100
```

13.5.2 Ownership: who frees the memory

Ownership means deciding which part of the program is responsible for freeing a block. Document it in the function comment and keep it consistent:

Pattern	Example	Who frees
Function returns new memory	`char *strdup(const char *)`	the caller
Caller provides the buffer	`snprintf(buf, size, ...)`	whoever allocated buf
Create / destroy pair	`Vec *vec_new(); void vec_free(Vec *)`	call the matching destroy function
Borrowed pointer	`size_t strlen(const char *)`	nobody here: the function only looks

Pairing a constructor with a destructor makes ownership obvious and keeps all freeing logic in one place, as in this small string-builder:

CONSTRUCTOR/DESTRUCTOR PAIR main.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef struct { char *buf; size_t len, cap; } StrBuf;

StrBuf *sb_new(void) {                      /* caller owns the result */
    StrBuf *s = malloc(sizeof *s);
    if (!s) return NULL;
    s->cap = 16; s->len = 0;
    s->buf = malloc(s->cap);
    if (!s->buf) { free(s); return NULL; }
    s->buf[0] = '\0';
    return s;
}
int sb_append(StrBuf *s, const char *t) {   /* borrows t */
    size_t n = strlen(t);
    if (s->len + n + 1 > s->cap) {
        size_t ncap = s->cap;
        while (s->len + n + 1 > ncap) ncap *= 2;
        char *tmp = realloc(s->buf, ncap);
        if (!tmp) return 0;
        s->buf = tmp; s->cap = ncap;
    }
    memcpy(s->buf + s->len, t, n + 1);
    s->len += n;
    return 1;
}
void sb_free(StrBuf *s) {                   /* releases everything sb_new made */
    if (!s) return;
    free(s->buf);
    free(s);
}
int main(void) {
    StrBuf *s = sb_new();
    if (!s) return 1;
    const char *words[] = {"Every ", "malloc ", "has ", "one ", "matching ", "free."};
    for (int i = 0; i < 6; i++) sb_append(s, words[i]);
    printf("%s\n(len %zu, capacity %zu)\n", s->buf, s->len, s->cap);
    sb_free(s);
```

```
    return 0;
}
```

OUTPUT

```
Every malloc has one matching free.
(len 35, capacity 64)
```

13.5.3 sizeof *p idiom

Write allocation sizes in terms of the **pointer being assigned**, not a type name:

SNIPPET C

```c
double *v = malloc(n * sizeof(double));   /* works, but repeats the type      */
double *v = malloc(n * sizeof *v);        /* preferred: follows v's type      */
```

If the type of v later changes from `double *` to `long double *`, the second form stays correct automatically, while the first silently allocates too little. `sizeof *v` does not dereference anything at run time; it only inspects the type, so it is safe even when v is uninitialized.

Also notice there is no cast on malloc's result. In C, `void *` converts implicitly, and a cast only hides mistakes (and is a C++ habit).

The heap gives you memory of any size for as long as you need it, but you are the garbage collector: check every allocation for NULL, realloc through a temporary, free exactly once, set the pointer to NULL, and make it clear who owns each block.

PRACTICE CHECKPOINT

Build a **dynamic array that grows with realloc** as the user enters numbers (stop at 0), and a **dynamically allocated matrix** of a user-given size that is filled, printed and **freed correctly**.

SAMPLE SOLUTION main.c

```c
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    /* ---- part 1: growing array ---- */
    size_t size = 0, cap = 2;
    int *nums = malloc(cap * sizeof *nums);
    if (!nums) return 1;
    int x;
    printf("Enter numbers (0 to stop): ");
    while (scanf("%d", &x) == 1 && x != 0) {
        if (size == cap) {
            int *tmp = realloc(nums, (cap * 2) * sizeof *tmp);
            if (!tmp) { free(nums); return 1; }
            nums = tmp;
            cap *= 2;
        }
        nums[size++] = x;
    }
    long sum = 0;
    for (size_t i = 0; i < size; i++) sum += nums[i];
    printf("%zu numbers stored (capacity %zu), sum = %ld\n", size, cap, sum);
    free(nums);
    nums = NULL;
```

```c
    /* ---- part 2: dynamic matrix ---- */
    int r, c;
    printf("Rows and columns: ");
    if (scanf("%d %d", &r, &c) != 2 || r <= 0 || c <= 0) return 1;
    int **m = malloc(r * sizeof *m);
    if (!m) return 1;
    for (int i = 0; i < r; i++) {
        m[i] = calloc(c, sizeof *m[i]);
        if (!m[i]) { while (i--) free(m[i]); free(m); return 1; }
    }
    for (int i = 0; i < r; i++)
        for (int j = 0; j < c; j++) m[i][j] = (i + 1) * (j + 1);
    for (int i = 0; i < r; i++) {
        for (int j = 0; j < c; j++) printf("%4d", m[i][j]);
        printf("\n");
    }
    for (int i = 0; i < r; i++) free(m[i]);    /* rows first */
    free(m);                                   /* then the row table */
    printf("matrix freed\n");
    return 0;
}
```

CONSOLE (user input in gold)

```
Enter numbers (0 to stop): 5 12 8 3 20 0
5 numbers stored (capacity 8), sum = 48
Rows and columns: 3 4
   1   2   3   4
   2   4   6   8
   3   6   9  12
matrix freed
```

MODULE 14 · INTERMEDIATE

Structures, unions, enums and typedef

Minimum duration **4 days** | Level **Intermediate**

Real-world things have several properties: a student has a roll number, a name and marks; a point has x and y. C's user-defined types let you group related data into one unit, which is how C programs model records, packets, shapes and every data structure in Module 19.

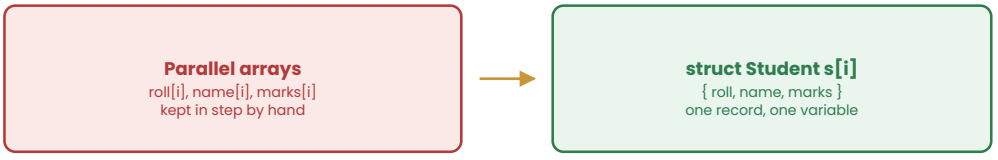
IN THIS MODULE

14.1 Structures
14.2 Structures in depth
14.3 Memory layout of structures
14.4 Bit fields
14.5 Unions
14.6 Enumerations
14.7 typedef
14.8 Anonymous structures and unions (C11)

14.1 Structures

14.1.1 Why structures

Suppose you store 50 students using separate arrays: `int roll[50]; char name[50][30]; float marks[50];`. Nothing ties `roll[7]` to `name[7]` except your discipline. Sorting by marks means swapping in three arrays at once, and passing one student to a function needs three parameters. A **structure** bundles the related fields into one variable so that a student is a single thing.



STRUCTURES REPLACE LOOSELY RELATED ARRAYS WITH RECORDS

14.1.2 Defining a struct and declaring variables

A structure definition creates a new **type**; it does not reserve memory. Memory is reserved when you declare variables of that type. The members can be of any type, including arrays and other structures.

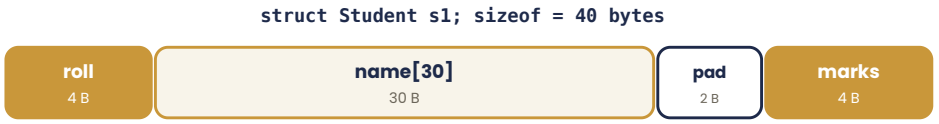
DEFINING AND DECLARING

C

```c
struct Student {            /* "struct Student" is the type name   */
    int   roll;             /* members (fields)                    */
    char  name[30];
    float marks;
};                          /* <- the semicolon is required        */

struct Student s1, s2;      /* two variables of the type           */

struct Point { int x, y; } p1, p2;   /* define and declare together */
```



MEMBERS ARE STORED IN DECLARATION ORDER (WITH PADDING, SEE 14.3)

14.1.3 Initializing structures (including designated initializers)

A structure can be initialized with a brace list in member order. C99 **designated initializers** name each member with `.member =`, so the order does not matter and any members not mentioned are set to zero.

FOUR WAYS TO INITIALIZE A STRUCTURE main.c

```c
#include <stdio.h>

struct Student {
    int roll;
    char name[30];
    float marks;
};

int main(void) {
    struct Student a = {101, "Asha", 88.5f};                  /* positional */
    struct Student b = {.name = "Ravi", .marks = 72.0f, .roll = 102};  /* designated */
    struct Student c = {.roll = 103};                          /* rest zeroed */
    struct Student d = {0};                                    /* all zero */

    printf("%d %-5s %.1f\n", a.roll, a.name, a.marks);
    printf("%d %-5s %.1f\n", b.roll, b.name, b.marks);
    printf("%d '%s' %.1f\n", c.roll, c.name, c.marks);
    printf("%d '%s' %.1f\n", d.roll, d.name, d.marks);
    return 0;
}
```

OUTPUT

```
101 Asha  88.5
102 Ravi  72.0
103 '' 0.0
0 '' 0.0
```

14.1.4 Accessing members with .

The **dot operator** selects a member of a structure variable: `s1.marks`. A member is an ordinary variable of its own type, so you can read it, assign it, take its address and pass it to scanf.

READING AND CHANGING MEMBERS

main.c

```c
#include <stdio.h>
#include <string.h>

struct Student { int roll; char name[30]; float marks; };

int main(void) {
    struct Student s;
    printf("Roll, name and marks: ");
    scanf("%d %29s %f", &s.roll, s.name, &s.marks);   /* name is an array: no & */
    s.marks += 5;                                      /* grace marks */
    if (strlen(s.name) > 0) s.name[0] = (char)(s.name[0] & ~32);  /* capitalise */
    printf("Roll %d | %s | %.2f\n", s.roll, s.name, s.marks);
    return 0;
}
```

CONSOLE (user input in gold)

```
Roll, name and marks: 17 meera 81.5
Roll 17 | Meera | 86.50
```

14.1.5 Copying and assigning structures

Unlike arrays, structures can be assigned with `=`. The whole structure is copied member by member, **including arrays inside it**. The copy is independent: changing one does not affect the other. Be careful with pointer members though: only the pointer is copied (a **shallow copy**), so both structures then point to the same data.

ASSIGNMENT COPIES THE WHOLE STRUCTURE

main.c

```c
#include <stdio.h>
#include <string.h>

struct Student { int roll; char name[30]; float marks; };
struct Tagged  { int id; char *label; };            /* pointer member */

int main(void) {
    struct Student a = {1, "Asha", 90};
    struct Student b = a;                    /* full copy, array included */
    strcpy(b.name, "Bina");
    b.marks = 75;
    printf("a: %s %.0f\n", a.name, a.marks);
    printf("b: %s %.0f\n", b.name, b.marks);

    char text[] = "shared";
    struct Tagged x = {1, text};
    struct Tagged y = x;                     /* shallow: same label pointer */
    y.label[0] = 'S';
    printf("x.label = %s, y.label = %s (same memory)\n", x.label, y.label);
    return 0;
}
```

OUTPUT

```
a: Asha 90
b: Bina 75
x.label = Shared, y.label = Shared (same memory)
```

14.1.6 Comparing structures (member by member)

The operators `==` and `!=` do **not** work on structures. Compare the members you care about. Do not use `memcmp` on whole structures: the padding bytes may hold different garbage even when all members are equal.

COMPARING TWO STRUCTURES main.c

```c
#include <stdio.h>
#include <string.h>
#include <stdbool.h>

struct Student { int roll; char name[30]; float marks; };

bool same_student(const struct Student *a, const struct Student *b) {
    return a->roll == b->roll && strcmp(a->name, b->name) == 0 && a->marks == b->marks;
}
int main(void) {
    struct Student a = {5, "Kiran", 66}, b = {5, "Kiran", 66}, c = {6, "Kiran", 66};
    /* if (a == b) ...   error: invalid operands to binary == */
    printf("a vs b: %s\n", same_student(&a, &b) ? "equal" : "different");
    printf("a vs c: %s\n", same_student(&a, &c) ? "equal" : "different");
    return 0;
}
```

OUTPUT

```
a vs b: equal
a vs c: different
```

14.2 Structures in depth

14.2.1 Arrays of structures

An array of structures is the natural "table" in C: each element is one record. Access a member of element i with `arr[i].member`.

A SHOPPING CART AS AN ARRAY OF STRUCTURES main.c

```c
#include <stdio.h>

struct Item { char name[20]; int qty; double price; };

int main(void) {
    struct Item cart[] = {
        {"Notebook", 3, 45.00},
        {"Pen",      10, 12.50},
        {"Bag",      1, 899.00},
    };
    int n = sizeof cart / sizeof cart[0];
    double total = 0;
    printf("%-10s %4s %8s %9s\n", "Item", "Qty", "Price", "Amount");
    for (int i = 0; i < n; i++) {
        double amt = cart[i].qty * cart[i].price;
        total += amt;
        printf("%-10s %4d %8.2f %9.2f\n", cart[i].name, cart[i].qty, cart[i].price, amt);
    }
    printf("%-10s %23.2f\n", "TOTAL", total);
    return 0;
}
```

OUTPUT

```
Item        Qty    Price    Amount
Notebook      3    45.00    135.00
Pen          10    12.50    125.00
Bag           1   899.00    899.00
TOTAL                       1159.00
```

	name	qty	price
cart[0]	Notebook	3	45.00
cart[1]	Pen	10	12.50
cart[2]	Bag	1	899.00

14.2.2 Nested structures

A member can itself be a structure. Access goes through each level with dots: `s.addr.city`.

STRUCTURES INSIDE STRUCTURES main.c

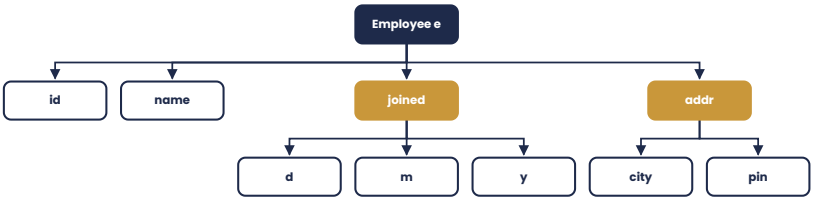
```c
#include <stdio.h>

struct Date    { int d, m, y; };
struct Address { char city[20]; int pin; };
struct Employee {
    int id;
    char name[20];
    struct Date joined;          /* nested */
    struct Address addr;         /* nested */
};

int main(void) {
    struct Employee e = {
        .id = 7, .name = "Farah",
        .joined = {12, 6, 2023},
        .addr = {.city = "Bengaluru", .pin = 560001},
    };
    printf("%s (#%d) joined %02d/%02d/%d, lives in %s - %d\n",
           e.name, e.id, e.joined.d, e.joined.m, e.joined.y, e.addr.city, e.addr.pin);
    e.addr.pin = 560034;
    printf("new pin: %d\n", e.addr.pin);
    return 0;
}
```

OUTPUT

```
Farah (#7) joined 12/06/2023, lives in Bengaluru - 560001
new pin: 560034
```



E.JOINED.Y AND E.ADDR.CITY FOLLOW THE TREE

14.2.3 Pointers to structures and the -> operator

With a pointer `struct Student *p = &s;` you can reach a member as `(*p).marks`. The parentheses are required because `.` binds tighter than `*`. This is so common that C has a shorthand: the **arrow operator** `p->marks`, which means exactly `(*p).marks`.

THE ARROW OPERATOR main.c

```c
#include <stdio.h>

struct Student { int roll; char name[30]; float marks; };

int main(void) {
    struct Student s = {21, "Neha", 79.5f};
    struct Student *p = &s;
    printf("(*p).name = %s\n", (*p).name);
    printf("p->name   = %s\n", p->name);
    p->marks = 84.0f;                       /* changes s */
    printf("s.marks   = %.1f\n", s.marks);

    struct Student class_[3] = {{1, "A", 50}, {2, "B", 60}, {3, "C", 70}};
    for (struct Student *q = class_; q < class_ + 3; q++)   /* pointer walk */
        printf("%d:%s:%.0f  ", q->roll, q->name, q->marks);
    printf("\n");
    return 0;
}
```

OUTPUT

```
(*p).name = Neha
p->name   = Neha
s.marks   = 84.0
1:A:50  2:B:60  3:C:70
```

Have	Member access	Example
a structure variable	`.`	`s.marks`
a pointer to a structure	`->`	`p->marks`
an array element	`.` after `[i]`	`arr[i].marks` or `(arr + i)->marks`

14.2.4 Structures and functions

14.2.4.1 Passing by value

Passing a structure by value copies **every byte** of it into the parameter. The function works on a copy, so changes do not reach the caller. Fine for small structures such as a Point; wasteful for large records.

14.2.4.2 Passing by pointer

Passing a pointer copies only 8 bytes and lets the function modify the original. Use `const struct X *` when the function only reads.

14.2.4.3 Returning a structure

A function can return a structure by value. This is a clean way to return several values at once, and compilers optimize it well.

VALUE, POINTER AND RETURN main.c

```c
#include <stdio.h>

struct Point { double x, y; };
struct Rect  { struct Point tl; double w, h; char title[64]; };

void move_copy(struct Point p, double dx) { p.x += dx; }          /* by value  */
void move_ref(struct Point *p, double dx) { p->x += dx; }        /* by pointer */
struct Point midpoint(struct Point a, struct Point b) {           /* returns one */
    struct Point m = {(a.x + b.x) / 2, (a.y + b.y) / 2};
    return m;
}
double area(const struct Rect *r) { return r->w * r->h; }        /* read-only */

int main(void) {
    struct Point p = {1, 1};
    move_copy(p, 10);
    printf("after move_copy: (%.0f, %.0f)\n", p.x, p.y);
    move_ref(&p, 10);
    printf("after move_ref : (%.0f, %.0f)\n", p.x, p.y);
    struct Point m = midpoint((struct Point){0, 0}, (struct Point){4, 6});   /* compound literals */
    printf("midpoint       : (%.0f, %.0f)\n", m.x, m.y);
    struct Rect r = {{0, 0}, 3, 4, "box"};
    printf("sizeof(struct Rect) = %zu, area = %.0f\n", sizeof r, area(&r));
    return 0;
}
```

OUTPUT

```
after move_copy: (1, 1)
after move_ref : (11, 1)
midpoint       : (2, 3)
sizeof(struct Rect) = 96, area = 12
```

NOTE

`(struct Point){0, 0}` is a C99 **compound literal**: an unnamed structure value created on the spot, handy for passing or assigning temporary structures.

14.2.5 Dynamically allocated structures

Structures are commonly created on the heap, especially when their number is not known in advance or they must outlive the function that creates them. Allocate with `malloc(sizeof *p)` and access with `->`.

CREATING AND DESTROYING STRUCTURES ON THE HEAP main.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

struct Book { char title[40]; char *author; int year; };

struct Book *book_new(const char *title, const char *author, int year) {
    struct Book *b = malloc(sizeof *b);
    if (!b) return NULL;
    snprintf(b->title, sizeof b->title, "%s", title);
    b->author = malloc(strlen(author) + 1);          /* member on the heap too */
    if (!b->author) { free(b); return NULL; }
    strcpy(b->author, author);
    b->year = year;
    return b;
}
void book_free(struct Book *b) {
    if (!b) return;
    free(b->author);              /* free inner allocations first */
    free(b);
}
int main(void) {
    int n = 2;
    struct Book **shelf = malloc(n * sizeof *shelf);   /* array of pointers to Book */
    shelf[0] = book_new("The C Programming Language", "Kernighan & Ritchie", 1978);
    shelf[1] = book_new("Expert C Programming", "Peter van der Linden", 1994);
    for (int i = 0; i < n; i++)
        printf("%d  %-28s %s\n", shelf[i]->year, shelf[i]->title, shelf[i]->author);
    for (int i = 0; i < n; i++) book_free(shelf[i]);
    free(shelf);
    return 0;
}
```

OUTPUT

```
1978  The C Programming Language   Kernighan & Ritchie
1994  Expert C Programming         Peter van der Linden
```

14.2.6 Self-referential structures (basis of linked lists)

A structure cannot contain a member of its own type (it would be infinitely large), but it **can** contain a **pointer** to its own type. Such a self-referential structure is the building block of linked lists, trees and graphs.

SNIPPET C

```c
struct Node {
    int data;
    struct Node *next;      /* pointer to another Node: allowed       */
 /* struct Node child;        error: field has incomplete type        */
};
```

THREE NODES LINKED BY POINTERS

main.c

```c
#include <stdio.h>

struct Node { int data; struct Node *next; };

int main(void) {
    struct Node c = {30, NULL};
    struct Node b = {20, &c};
    struct Node a = {10, &b};
    for (struct Node *p = &a; p != NULL; p = p->next)
        printf("%d -> ", p->data);
    printf("NULL\n");
    return 0;
}
```

OUTPUT

```
10 -> 20 -> 30 -> NULL
```



EACH NODE POINTS TO THE NEXT; MODULE 19 BUILDS FULL LINKED LISTS FROM THIS IDEA

14.3 Memory layout of structures

14.3.1 Padding and alignment

CPUs read memory fastest when a value of size n starts at an address that is a multiple of n (its **alignment**): an int at a multiple of 4, a double at a multiple of 8. To keep every member aligned, the compiler inserts unused **padding bytes** between members and at the end, so the structure size is also a multiple of its largest alignment (so arrays of it stay aligned).

THE SAME MEMBERS IN TWO ORDERS main.c

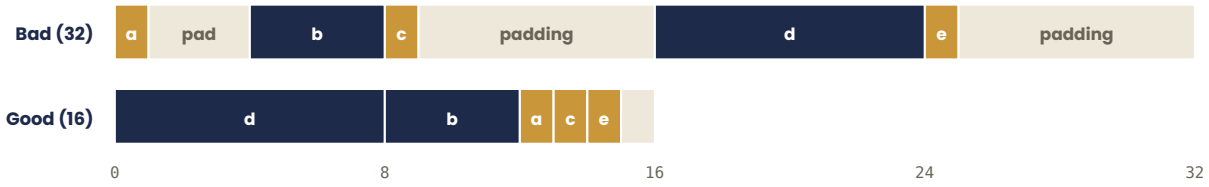
```c
#include <stdio.h>
#include <stddef.h>

struct Bad  { char a; int b; char c; double d; char e; };
struct Good { double d; int b; char a; char c; char e; };

int main(void) {
    printf("struct Bad : size %2zu\n", sizeof(struct Bad));
    printf("  a@%zu b@%zu c@%zu d@%zu e@%zu\n", offsetof(struct Bad, a), offsetof(struct Bad, b),
           offsetof(struct Bad, c), offsetof(struct Bad, d), offsetof(struct Bad, e));
    printf("struct Good: size %2zu\n", sizeof(struct Good));
    printf("  d@%zu b@%zu a@%zu c@%zu e@%zu\n", offsetof(struct Good, d), offsetof(struct Good, b),
           offsetof(struct Good, a), offsetof(struct Good, c), offsetof(struct Good, e));
    printf("alignof(double) = %zu, alignof(int) = %zu\n", _Alignof(double), _Alignof(int));
    return 0;
}
```

OUTPUT

```
struct Bad : size 32
  a@0 b@4 c@8 d@16 e@24
struct Good: size 16
  d@0 b@8 a@12 c@13 e@14
alignof(double) = 8, alignof(int) = 4
```



BYTE MAP: GREY BYTES ARE PADDING. REORDERING HALVES THE SIZE FROM 32 TO 16 BYTES

14.3.2 Reordering members to save space

Ordering members from **largest alignment to smallest** (doubles and pointers, then ints, then shorts, then chars) minimizes padding. In the example, the same five members take 16 bytes instead of 32: half the memory, a saving that matters when you store millions of records.

14.3.3 Packed structures (compiler-specific)

GCC and Clang accept `__attribute__((packed))` and MSVC uses `#pragma pack` to remove padding entirely. This is used to match an exact binary layout (file headers, network packets, hardware registers). Members may then be misaligned, which is slower and on some CPUs can fault, and taking their address is unsafe.

A PACKED HEADER (GCC/CLANG EXTENSION) main.c

```c
#include <stdio.h>
#include <stdint.h>

struct Normal { uint8_t type; uint32_t length; uint16_t flags; };
struct __attribute__((packed)) Packed { uint8_t type; uint32_t length; uint16_t flags; };

int main(void) {
    printf("Normal: %zu bytes\n", sizeof(struct Normal));
    printf("Packed: %zu bytes (1 + 4 + 2)\n", sizeof(struct Packed));
    struct Packed h = {1, 1500, 0x0003};
    printf("type %u, length %u, flags 0x%04x\n", h.type, (unsigned)h.length, h.flags);
    return 0;
}
```

OUTPUT

```
Normal: 12 bytes
Packed: 7 bytes (1 + 4 + 2)
type 1, length 1500, flags 0x0003
```

14.3.4 Flexible array members (C99)

The **last** member of a structure may be an array with no size: `int data[];`. It contributes nothing to sizeof; you allocate the structure together with as many elements as you need in a single malloc. This keeps a header and its variable-length data in one contiguous block.

A STRUCTURE WITH A FLEXIBLE ARRAY MEMBER main.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

struct Message {
    int id;
    size_t len;
    char text[];              /* flexible array member: must be last */
};

struct Message *msg_new(int id, const char *s) {
    size_t n = strlen(s) + 1;
    struct Message *m = malloc(sizeof *m + n);     /* header + text in one block */
    if (!m) return NULL;
    m->id = id;
    m->len = n - 1;
    memcpy(m->text, s, n);
    return m;
}
int main(void) {
    printf("sizeof(struct Message) = %zu (text adds nothing)\n", sizeof(struct Message));
    struct Message *m = msg_new(42, "flexible arrays save a malloc");
    printf("#%d [%zu] %s\n", m->id, m->len, m->text);
    free(m);                                       /* one free */
    return 0;
}
```

OUTPUT

```
sizeof(struct Message) = 16 (text adds nothing)
#42 [29] flexible arrays save a malloc
```

14.4 Bit fields

14.4.1 Declaring bit fields

A **bit field** is a structure member with an explicit width in bits: `unsigned int ready : 1;`. Several bit fields are packed into the same storage unit, so flags and small numbers take only the bits they need.

BIT FIELDS

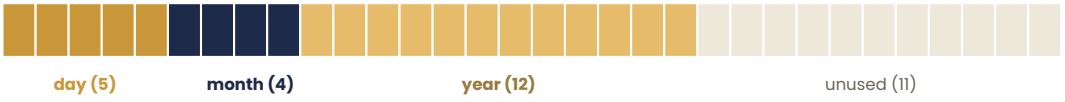
main.c

```c
#include <stdio.h>

struct Date {                   /* a full date in 4 bytes */
    unsigned int day   : 5;     /* 1..31   needs 5 bits  */
    unsigned int month : 4;     /* 1..12   needs 4 bits  */
    unsigned int year  : 12;    /* 0..4095 needs 12 bits */
};
struct Perms {
    unsigned int read  : 1;
    unsigned int write : 1;
    unsigned int exec  : 1;
};
int main(void) {
    struct Date d = {15, 8, 1947};
    printf("%02u/%02u/%u in %zu bytes\n", d.day, d.month, d.year, sizeof d);
    struct Perms p = {1, 0, 1};
    printf("r=%u w=%u x=%u\n", p.read, p.write, p.exec);
    d.day = 33;                 /* too big: only the low 5 bits are kept */
    printf("day after storing 33: %u\n", d.day);
    return 0;
}
```

OUTPUT

```
15/08/1947 in 4 bytes
r=1 w=0 x=1
day after storing 33: 1
```



TYPICAL GCC LAYOUT OF STRUCT DATE INSIDE ONE 32-BIT UNSIGNED INT

14.4.2 Uses and limitations

- **Uses:** compact flag sets, protocol headers, hardware register maps on embedded systems, large arrays of small records.
- **No addresses:** `&d.day` is an error, so you cannot `scanf` directly into a bit field.
- **Layout is implementation-defined:** bit order, whether fields straddle storage units, and the signedness of plain `int` fields vary between compilers. Do not use bit fields to match an external binary format portably; use shifts and masks (Module 17) instead.
- **Overflow wraps silently** as seen above, and access is slightly slower than a whole int.

14.5 Unions

14.5.1 Defining a union

A **union** is declared like a structure, but all its members share the **same memory**. Only one member holds a meaningful value at a time: the one most recently written.

SNIPPET

C

```c
union Value {
    int    i;
    float  f;
    char   s[8];
};
```

14.5.2 Shared memory between members

EVERY MEMBER STARTS AT THE SAME ADDRESS

main.c

```c
#include <stdio.h>
#include <string.h>

union Value { int i; float f; char s[8]; };

int main(void) {
    union Value v;
    v.i = 65;
    printf("after v.i = 65   : i = %d\n", v.i);
    v.f = 3.5f;                              /* overwrites the same bytes */
    printf("after v.f = 3.5  : f = %.1f, i now reads %d\n", v.f, v.i);
    strcpy(v.s, "hi");
    printf("after strcpy s   : s = %s\n", v.s);
    printf("&v.i = %p\n&v.f = %p\n&v.s = %p\n", (void *)&v.i, (void *)&v.f, (void *)&v.s);
    return 0;
}
```

OUTPUT (addresses differ per run)

```
after v.i = 65   : i = 65
after v.f = 3.5  : f = 3.5, i now reads 1080033280
after strcpy s   : s = hi
&v.i = 0x7ffd1d710630
&v.f = 0x7ffd1d710630
&v.s = 0x7ffd1d710630
```

Reading `v.i` right after writing `v.f` reinterprets the float's bits as an int: 1080033280 is 0x40600000, the IEEE 754 pattern of 3.5. C allows this **type punning** through unions, which is a common way to inspect the bits of a float.

14.5.3 Size of a union

A union is as big as its **largest member**, rounded up to the strictest alignment among its members.

14.5.4 Structure vs union

SIZEOF A STRUCTURE VS A UNION main.c

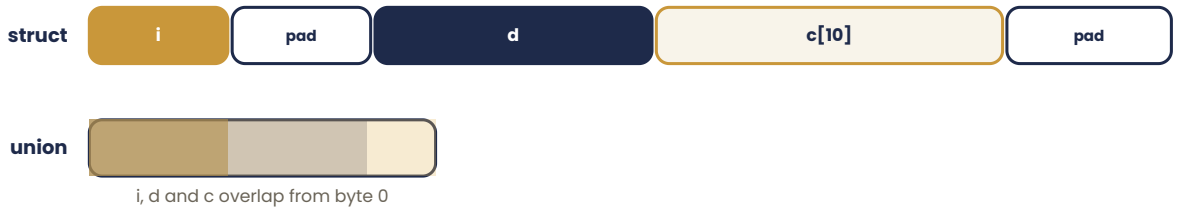
```c
#include <stdio.h>

struct S { int i; double d; char c[10]; };
union  U { int i; double d; char c[10]; };

int main(void) {
    printf("struct S: %zu bytes (all members side by side + padding)\n", sizeof(struct S));
    printf("union  U: %zu bytes (largest member 10, aligned to 8)\n", sizeof(union U));
    return 0;
}
```

OUTPUT

```
struct S: 32 bytes (all members side by side + padding)
union  U: 16 bytes (largest member 10, aligned to 8)
```



STRUCTURE MEMBERS SIT SIDE BY SIDE; UNION MEMBERS OVERLAP

	struct	union
Memory	separate space for each member	one shared space
Size	sum of members + padding	largest member (+ alignment)
Valid members at once	all	only the last one written
Typical use	records	one of several alternatives, saving memory, bit reinterpretation

14.5.5 Tagged unions

Since a union does not remember which member is active, pair it with an **enum tag** in a structure. The tag records the type currently stored, and code checks it before reading. This "tagged union" (also called a variant or sum type) is how interpreters store values of different types and how shapes, tokens and messages are modelled.

A TAGGED UNION OF SHAPES main.c

```c
#include <stdio.h>

enum ShapeKind { CIRCLE, RECT, TRIANGLE };

struct Shape {
    enum ShapeKind kind;              /* the tag */
    union {
        struct { double r; } circle;
        struct { double w, h; } rect;
        struct { double b, h; } tri;
    } u;
};

double area(const struct Shape *s) {
    switch (s->kind) {
    case CIRCLE:   return 3.14159265 * s->u.circle.r * s->u.circle.r;
    case RECT:     return s->u.rect.w * s->u.rect.h;
    case TRIANGLE: return 0.5 * s->u.tri.b * s->u.tri.h;
    }
    return 0;
}
int main(void) {
    struct Shape shapes[] = {
        {CIRCLE,   .u.circle = {2}},
        {RECT,     .u.rect   = {3, 4}},
        {TRIANGLE, .u.tri    = {6, 5}},
    };
    const char *names[] = {"circle", "rectangle", "triangle"};
    for (int i = 0; i < 3; i++)
        printf("%-9s area = %6.2f\n", names[shapes[i].kind], area(&shapes[i]));
    printf("sizeof(struct Shape) = %zu\n", sizeof(struct Shape));
    return 0;
}
```

OUTPUT

```
circle    area =  12.57
rectangle area =  12.00
triangle  area =  15.00
sizeof(struct Shape) = 24
```

14.6 Enumerations

14.6.1 Defining an enum

An **enumeration** defines a set of named integer constants. It makes code self-documenting: `if (day == SUNDAY)` instead of `if (day == 0)`.

SNIPPET C

```c
enum Day { SUNDAY, MONDAY, TUESDAY, WEDNESDAY, THURSDAY, FRIDAY, SATURDAY };
enum Day today = WEDNESDAY;
```

14.6.2 Default and custom values

By default the first constant is 0 and each next one is one more than the previous. You can assign any value explicitly; constants after it continue counting from there. Several constants may share a value. The constants have type `int`.

ENUM VALUES

main.c

```c
#include <stdio.h>

enum Day { SUNDAY, MONDAY, TUESDAY, WEDNESDAY, THURSDAY, FRIDAY, SATURDAY };
enum Http { OK = 200, CREATED, NOT_FOUND = 404, SERVER_ERROR = 500 };
enum Level { LOW = 1, MEDIUM = 5, HIGH = 10, MAX = HIGH };

int main(void) {
    printf("SUNDAY = %d, WEDNESDAY = %d, SATURDAY = %d\n", SUNDAY, WEDNESDAY, SATURDAY);
    printf("OK = %d, CREATED = %d, NOT_FOUND = %d\n", OK, CREATED, NOT_FOUND);
    printf("HIGH = %d, MAX = %d\n", HIGH, MAX);
    enum Day d = FRIDAY;
    d = (enum Day)((d + 2) % 7);          /* enums are ints: arithmetic works */
    printf("two days after FRIDAY is day %d (SUNDAY)\n", d);
    printf("sizeof(enum Day) = %zu\n", sizeof(enum Day));
    return 0;
}
```

OUTPUT

```
SUNDAY = 0, WEDNESDAY = 3, SATURDAY = 6
OK = 200, CREATED = 201, NOT_FOUND = 404
HIGH = 10, MAX = 10
two days after FRIDAY is day 0 (SUNDAY)
sizeof(enum Day) = 4
```

14.6.3 Enums in switch statements

Enums and switch belong together. With `-Wall`, GCC warns if a switch on an enum forgets one of its constants and has no default, which catches bugs when a new constant is added later. A parallel array of strings indexed by the enum gives readable output.

A TRAFFIC-LIGHT STATE MACHINE

main.c

```c
#include <stdio.h>

enum Light { RED, YELLOW, GREEN, LIGHT_COUNT };
static const char *light_name[LIGHT_COUNT] = {"RED", "YELLOW", "GREEN"};

enum Light next(enum Light l) {
    switch (l) {
    case RED:    return GREEN;
    case GREEN:  return YELLOW;
    case YELLOW: return RED;
    default:     return RED;
    }
}
int main(void) {
    enum Light l = RED;
    for (int step = 0; step < 6; step++) {
        printf("%s%s", light_name[l], step < 5 ? " -> " : "\n");
        l = next(l);
    }
    return 0;
}
```

OUTPUT

```
RED -> GREEN -> YELLOW -> RED -> GREEN -> YELLOW
```

TIP

Adding a final `..._COUNT` constant (here `LIGHT_COUNT`) gives you the number of real constants for free, ideal for sizing lookup arrays and loops.

14.7 typedef

14.7.1 Aliasing basic types

`typedef existing_type NewName;` creates a new **name** for an existing type (not a new type). It improves readability and portability: change the typedef in one place and every use follows.

SNIPPET

C

```c
typedef unsigned char  byte;
typedef unsigned long  ulong;
typedef double         Money;      /* intent is now clear in signatures */

Money price = 249.99;
byte  flags = 0x0F;
```

The standard library is full of typedefs: `size_t`, `ptrdiff_t`, `FILE`, `uint32_t` and `time_t` are all typedef names chosen by the platform.

14.7.2 typedef with struct

Writing `struct Student` everywhere is verbose. A typedef lets you write just `Student`. Keeping the struct tag as well (the same name is allowed) is needed for self-referential structures.

TYPEDEF WITH STRUCTURES

main.c

```c
#include <stdio.h>

typedef struct { double x, y; } Point;          /* anonymous struct + typedef */

typedef struct Node {                            /* tag needed for the self-pointer */
    int value;
    struct Node *next;
} Node;

int main(void) {
    Point p = {3, 4};
    Node b = {2, NULL}, a = {1, &b};
    printf("Point (%.0f, %.0f)\n", p.x, p.y);
    printf("list: %d -> %d\n", a.value, a.next->value);
    return 0;
}
```

OUTPUT

```
Point (3, 4)
list: 1 -> 2
```

14.7.3 typedef with function pointers

As seen in Module 12, a typedef turns an unreadable function-pointer declaration into a simple name. Here it is used inside a structure to build a small command table:

A COMMAND TABLE WITH A FUNCTION-POINTER TYPEDEF

main.c

```c
#include <stdio.h>
#include <string.h>

typedef int (*Handler)(int);

typedef struct {
    const char *name;
    Handler fn;
} Command;

int twice(int x)  { return 2 * x; }
int square(int x) { return x * x; }
int negate(int x) { return -x; }

int main(void) {
    Command cmds[] = {{"twice", twice}, {"square", square}, {"negate", negate}};
    const char *wanted[] = {"square", "negate", "cube"};
    for (int k = 0; k < 3; k++) {
        int found = 0;
        for (size_t i = 0; i < sizeof cmds / sizeof *cmds; i++)
            if (strcmp(cmds[i].name, wanted[k]) == 0) {
                printf("%s(7) = %d\n", wanted[k], cmds[i].fn(7));
                found = 1;
            }
        if (!found) printf("%s: unknown command\n", wanted[k]);
    }
    return 0;
}
```

OUTPUT

```
square(7) = 49
negate(7) = -7
cube: unknown command
```

14.8 Anonymous structures and unions (C11)

C11 allows a structure or union **without a name** to be nested directly inside another. Its members are accessed as if they belonged to the outer structure. This makes tagged unions and grouped fields much less verbose.

ANONYMOUS UNIONS AND STRUCTURES

main.c

```c
#include <stdio.h>
#include <stdint.h>

typedef struct {
    enum { INT, REAL, TEXT } type;
    union {                         /* anonymous union */
        long   i;
        double r;
        const char *t;
    };
} Value;

typedef union {
    uint32_t rgba;
    struct { uint8_t r, g, b, a; };   /* anonymous struct inside a union */
} Colour;

void show(Value v) {
    switch (v.type) {
    case INT:  printf("int  %ld\n", v.i); break;     /* v.i, not v.u.i */
    case REAL: printf("real %g\n", v.r); break;
    case TEXT: printf("text \"%s\"\n", v.t); break;
    }
}
int main(void) {
    show((Value){.type = INT, .i = 42});
    show((Value){.type = REAL, .r = 2.5});
    show((Value){.type = TEXT, .t = "anonymous"});
    Colour c = {.r = 0xC9, .g = 0x97, .b = 0x3A, .a = 0xFF};
    printf("gold = r %u g %u b %u, packed 0x%08X (little-endian)\n", c.r, c.g, c.b, (unsigned)c.rgba);
    return 0;
}
```

OUTPUT

```
int  42
real 2.5
text "anonymous"
gold = r 201 g 151 b 58, packed 0xFF3A97C9 (little-endian)
```

Structures group related data into one record; unions overlay alternatives in the same memory; enums give names to integer codes; typedef gives short names to any type. Together they let C model real-world data cleanly.

PRACTICE CHECKPOINT

Build a **Student Record system**: an array of structures with **nested address details**, **sorting by marks** (qsort), and a function that **prints a student through a pointer**.

SAMPLE SOLUTION

main.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef struct {
    char city[20];
    int pin;
```

```c
} Address;

typedef struct {
    int roll;
    char name[24];
    float marks;
    Address addr;                    /* nested structure */
} Student;

void print_student(const Student *s) {           /* through a pointer */
    printf("%3d  %-8s %6.1f  %-10s %d\n", s->roll, s->name, s->marks, s->addr.city, s->addr.pin);
}
int by_marks_desc(const void *a, const void *b) {
    const Student *x = a, *y = b;
    return (y->marks > x->marks) - (y->marks < x->marks);
}
int main(void) {
    int n;
    printf("How many students? ");
    if (scanf("%d", &n) != 1 || n <= 0 || n > 50) return 1;
    Student list[50];
    for (int i = 0; i < n; i++) {
        printf("roll name marks city pin: ");
        scanf("%d %23s %f %19s %d", &list[i].roll, list[i].name, &list[i].marks,
              list[i].addr.city, &list[i].addr.pin);
    }
    qsort(list, n, sizeof list[0], by_marks_desc);
    printf("\nRank Roll Name      Marks  City       PIN\n");
    float total = 0;
    for (int i = 0; i < n; i++) {
        printf("%-4d ", i + 1);
        print_student(&list[i]);
        total += list[i].marks;
    }
    printf("Class average: %.2f; topper: %s from %s\n", total / n, list[0].name, list[0].addr.city);
    return 0;
}
```

CONSOLE (user input in gold)

```
How many students? 4
roll name marks city pin: 1 Asha 78.5 Mysuru 570001
roll name marks city pin: 2 Ravi 91 Chennai 600001
roll name marks city pin: 3 Meera 85.5 Pune 411001
roll name marks city pin: 4 Kabir 66 Delhi 110001

Rank Roll Name      Marks  City       PIN
1      2  Ravi       91.0  Chennai    600001
2      3  Meera      85.5  Pune       411001
3      1  Asha       78.5  Mysuru     570001
4      4  Kabir      66.0  Delhi      110001
Class average: 80.25; topper: Ravi from Chennai
```

MODULE 15 · INTERMEDIATE

The preprocessor

Minimum duration **2 days** | Level **Intermediate**

Before the compiler sees a single line of your program, the preprocessor runs over the source and carries out every line that starts with #. It pastes in header files, replaces macros, and keeps or drops blocks of code depending on conditions. Understanding it explains what #include really does and how one source file builds differently for debug and release, Linux and Windows.

IN THIS MODULE

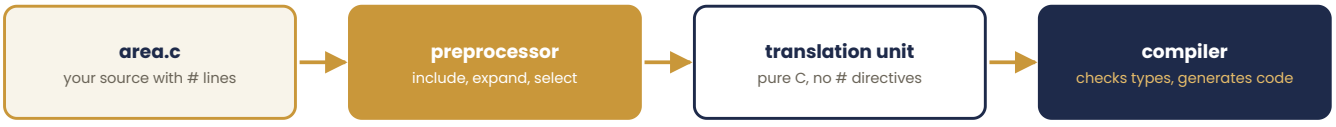
15.1 Introduction
15.2 File inclusion
15.3 Macros
15.4 Conditional compilation
15.5 Include guards
15.6 Other directives
15.7 Predefined macros
15.8 assert.h

15.1 Introduction

15.1.1 What the preprocessor does

The preprocessor is a **text transformer**. It knows nothing about types, variables or functions; it works on lines and tokens. Its jobs are:

- **File inclusion**: `#include` replaces the line with the entire contents of another file.
- **Macro expansion**: `#define` names are replaced by their definitions wherever they appear.
- **Conditional compilation**: `#if`, `#ifdef` and friends keep or delete blocks of code.
- **Removing comments** and joining lines that end with a backslash.
- **Other directives**: `#error`, `#pragma`, `#line` and, in C23, `#embed`.



THE PREPROCESSOR RUNS FIRST AND HANDS PLAIN C TO THE COMPILER

A directive starts with `#` as the first non-blank character of a line and ends at the end of that line. It is **not** a C statement, so it has no semicolon.

15.1.2 Viewing preprocessed output with gcc -E

`gcc -E file.c` stops after preprocessing and prints the result. It is the best way to debug a confusing macro. Take this small program:

AREA.C area.c

```c
#include <stdio.h>
#define PI 3.14159
#define SQUARE(x) ((x) * (x))

int main(void) {
    double r = 2.0;
    /* area of a circle */
    printf("%f\n", PI * SQUARE(r));
    return 0;
}
```

OUTPUT

```
12.566360
```

Terminal

```
$ gcc -E area.c | wc -l
823
$ gcc -E area.c | tail -8

# 5 "area.c"
int main(void) {
    double r = 2.0;

    printf("%f\n", 3.14159 * ((r) * (r)));
    return 0;
}
```

Two things stand out. The output is **823 lines** long: almost all of it is the text of `stdio.h` and the headers it includes. And in `main`, the comment has gone, `PI` became `3.14159` and `SQUARE(r)` became `((r) * (r))`. The `# 5 "area.c"` line is a **line marker** so that the compiler can still report errors with the original file name and line number.

15.2 File inclusion

15.2.1 #include <file> vs #include "file"

Form	Searched in	Used for
`#include <stdio.h>`	the system include directories (e.g. /usr/include) and any added with `-I`	standard and installed library headers
`#include "util.h"`	first the directory of the including file, then the same places as <>	your own project headers

`gcc -I dir` adds a directory to the search path, so a project can keep its headers in an `include/` folder. `gcc -H` prints every header actually opened, indented by nesting depth.

15.2.2 Writing your own header files

A header file (`.h`) holds the **declarations** that several source files share: function prototypes, type definitions, macros and `extern` declarations. The matching `.c` file holds the **definitions**. Any file that wants to use the functions includes the header.

GEOMETRY.H C

```c
#ifndef GEOMETRY_H
#define GEOMETRY_H

#define PI 3.14159265358979

typedef struct { double x, y; } Point;

double distance(Point a, Point b);
double circle_area(double r);

#endif
```

GEOMETRY.C C

```c
#include <math.h>
#include "geometry.h"

double distance(Point a, Point b) {
    return sqrt((a.x - b.x) * (a.x - b.x) + (a.y - b.y) * (a.y - b.y));
}
double circle_area(double r) {
    return PI * r * r;
}
```

MAIN.C (BUILT WITH: GCC MAIN.C GEOMETRY.C -LM) main.c

```c
#include <stdio.h>
#include "geometry.h"

int main(void) {
    Point a = {0, 0}, b = {3, 4};
    printf("distance = %.2f\n", distance(a, b));
    printf("area(r = 2) = %.4f\n", circle_area(2));
    return 0;
}
```

OUTPUT

```
distance = 5.00
area(r = 2) = 12.5664
```

Put in a header (.h)	Keep in the source (.c)
function prototypes	function bodies
struct / enum / typedef definitions	global variable definitions (int count = 0;)
macros and constants	static helper functions
`extern` variable declarations	anything private to that file

WATCH OUT

Never put function bodies or variable definitions in a header that is included by several .c files. Each file would get its own copy and the linker reports "multiple definition" errors.

15.3 Macros

15.3.1 Object-like macros #define PI 3.14159

`#define NAME replacement` tells the preprocessor to replace every later occurrence of the token NAME with the replacement text. By convention macro names are UPPERCASE. Macros are not variables: they have no type, no address and no scope; they are pure text substitution that lasts until the end of the file or an `#undef`.

OBJECT-LIKE MACROS

main.c

```c
#include <stdio.h>

#define PI         3.14159
#define MAX_USERS  100
#define GREETING   "Welcome to LBF"
#define NEWLINE    putchar('\n')

int main(void) {
    int ids[MAX_USERS];                    /* array size from a macro */
    printf("%s%c", GREETING, '!');
    NEWLINE;
    printf("capacity %d users, %zu bytes\n", MAX_USERS, sizeof ids);
    printf("circumference of r = 10: %.2f\n", 2 * PI * 10);
    printf("\"PI\" inside a string is not replaced\n");
    return 0;
}
```

OUTPUT

```
Welcome to LBF!
capacity 100 users, 400 bytes
circumference of r = 10: 62.83
"PI" inside a string is not replaced
```

TIP

For typed constants, `enum { MAX_USERS = 100 };` or `static const double PI = 3.14159;` are often better: they obey scope and appear in the debugger. Macros remain necessary for array sizes in C before C23 and for conditional compilation.

15.3.2 Function-like macros

A macro can take parameters: `#define SQUARE(x) ((x) * (x))`. There must be **no space** between the name and the opening parenthesis; otherwise it becomes an object-like macro whose text starts with "(x)". The arguments are substituted textually, so the macro works for any type.

FUNCTION-LIKE MACROS

main.c

```c
#include <stdio.h>

#define SQUARE(x)   ((x) * (x))
#define MAX(a, b)   ((a) > (b) ? (a) : (b))
#define ABS(x)      ((x) < 0 ? -(x) : (x))
#define ARRAY_LEN(a) (sizeof(a) / sizeof((a)[0]))

int main(void) {
    int marks[] = {72, 91, 65, 88};
    printf("SQUARE(7)     = %d\n", SQUARE(7));
    printf("SQUARE(1.5)   = %.2f\n", SQUARE(1.5));       /* works for double too */
    printf("MAX(3, 9)     = %d\n", MAX(3, 9));
    printf("ABS(-4.25)    = %.2f\n", ABS(-4.25));
    printf("ARRAY_LEN     = %zu\n", ARRAY_LEN(marks));
    return 0;
}
```

OUTPUT

```
SQUARE(7)     = 49
SQUARE(1.5)   = 2.25
MAX(3, 9)     = 9
ABS(-4.25)    = 4.25
ARRAY_LEN     = 4
```

15.3.3 Macro pitfalls: missing parentheses, double evaluation

Because macros are textual, they misbehave in ways functions never do.

Pitfall 1: missing parentheses. Without parentheses around each parameter and around the whole body, operator precedence rearranges the expression after substitution:

MISSING PARENTHESES CHANGE THE MEANING

main.c

```c
#include <stdio.h>

#define BAD_SQUARE(x)  x * x
#define BAD_DOUBLE(x)  (x) + (x)
#define SQUARE(x)      ((x) * (x))

int main(void) {
    printf("BAD_SQUARE(2 + 3) = %d   (2 + 3 * 2 + 3)\n", BAD_SQUARE(2 + 3));
    printf("10 / BAD_DOUBLE(5) = %d  (10 / (5) + (5))\n", 10 / BAD_DOUBLE(5));
    printf("SQUARE(2 + 3)     = %d\n", SQUARE(2 + 3));
    return 0;
}
```

OUTPUT

```
BAD_SQUARE(2 + 3) = 11   (2 + 3 * 2 + 3)
10 / BAD_DOUBLE(5) = 7  (10 / (5) + (5))
SQUARE(2 + 3)     = 25
```

Pitfall 2: double evaluation. A parameter that appears twice in the body is evaluated twice. If the argument has a side effect, it happens twice:

SIDE EFFECTS EVALUATED TWICE main.c

```c
#include <stdio.h>

#define SQUARE(x)  ((x) * (x))
#define MAX(a, b)  ((a) > (b) ? (a) : (b))

static int calls = 0;
int next_value(void) { return ++calls * 10; }

int main(void) {
    int i = 3;
    int big = MAX(i++, 2);            /* i++ runs twice! */
    printf("MAX(i++, 2) = %d, i is now %d (expected 4)\n", big, i);
    int s = SQUARE(next_value());     /* function called twice */
    printf("SQUARE(next_value()) = %d, calls = %d\n", s, calls);
    return 0;
}
```

OUTPUT

```
MAX(i++, 2) = 4, i is now 5 (expected 4)
SQUARE(next_value()) = 200, calls = 2
```

The fix is simple: never pass expressions with side effects to a macro, or use an `inline` function instead (15.3.7).

15.3.4 Multi-line macros and do { } while (0)

A macro definition ends at the end of the line. To continue it, end each line with a backslash `\`. When a macro contains several statements, wrap them in `do { ... } while (0)`. This makes the macro behave like a single statement that requires a semicolon, so it works correctly inside an `if` without braces.

THE DO { } WHILE (0) IDIOM main.c

```c
#include <stdio.h>

#define SWAP_BAD(a, b)  int t = a; a = b; b = t;

#define SWAP(type, a, b) do {   \
        type tmp_ = (a);        \
        (a) = (b);              \
        (b) = tmp_;             \
    } while (0)

int main(void) {
    int x = 1, y = 2;
    if (x < y)
        SWAP(int, x, y);          /* one statement: safe without braces */
    else
        printf("already ordered\n");
    printf("x = %d, y = %d\n", x, y);
    /* With SWAP_BAD, only "int t = a;" would belong to the if,
       and the "else" would no longer compile. */
    return 0;
}
```

OUTPUT

```
x = 2, y = 1
```

15.3.5 Stringizing # and token pasting

Two special operators work only inside macro definitions:

- `#param` (**stringizing**) turns the argument into a string literal: `#x` with argument `count` becomes `"count"`.
- `a ## b` (**token pasting**) glues two tokens into one: `get_ ## name` with argument `age` becomes `get_age`.

STRINGIZING AND TOKEN PASTING main.c

```c
#include <stdio.h>

#define PRINT_INT(var)   printf(#var " = %d\n", var)
#define SHOW(expr)       printf("%-14s -> %d\n", #expr, (expr))
#define MAKE_GETTER(field) int get_##field(void) { return field; }
#define STR(x)    #x
#define XSTR(x)   STR(x)            /* expand first, then stringize */
#define VERSION   3

int age = 21, score = 88;
MAKE_GETTER(age)                    /* defines int get_age(void)   */
MAKE_GETTER(score)                  /* defines int get_score(void) */

int main(void) {
    int total = 42;
    PRINT_INT(total);
    SHOW(total / 5);
    SHOW(7 % 3 + 1);
    printf("get_age() = %d, get_score() = %d\n", get_age(), get_score());
    printf("STR(VERSION)  = %s\n", STR(VERSION));
    printf("XSTR(VERSION) = %s\n", XSTR(VERSION));
    return 0;
}
```

OUTPUT

```
total = 42
total / 5      -> 8
7 % 3 + 1      -> 2
get_age() = 21, get_score() = 88
STR(VERSION)  = VERSION
XSTR(VERSION) = 3
```

NOTE

`STR(VERSION)` gives `"VERSION"` because `#` stringizes the argument before it is expanded. The two-level `XSTR` trick forces expansion first, giving `"3"`.

15.3.6 #undef

`#undef NAME` removes a macro definition. After it, NAME is an ordinary identifier again, and it may be redefined with a different value. Redefining a macro with a different body without #undef produces a warning.

#UNDEF AND REDEFINITION

main.c

```c
#include <stdio.h>

#define LIMIT 10
int small_limit(void) { return LIMIT; }
#undef LIMIT
#define LIMIT 500
int big_limit(void) { return LIMIT; }

int main(void) {
    printf("small_limit = %d, big_limit = %d\n", small_limit(), big_limit());
#undef LIMIT
#ifdef LIMIT
    printf("LIMIT still defined\n");
#else
    printf("LIMIT is no longer defined\n");
#endif
    return 0;
}
```

OUTPUT

```
small_limit = 10, big_limit = 500
LIMIT is no longer defined
```

15.3.7 Macros vs inline functions

C99 `static inline` functions give the speed of a macro (the compiler may paste the body at the call site) with the safety of a function: arguments are evaluated exactly once and type-checked.

SAME JOB, DIFFERENT BEHAVIOUR

main.c

```c
#include <stdio.h>

#define SQUARE_M(x) ((x) * (x))
static inline int square_f(int x) { return x * x; }

int main(void) {
    int a = 4, b = 4;
    int m = SQUARE_M(a++);          /* a incremented twice */
    int f = square_f(b++);          /* b incremented once  */
    printf("macro : result %d, a = %d\n", m, a);
    printf("inline: result %d, b = %d\n", f, b);
    return 0;
}
```

OUTPUT

```
macro : result 20, a = 6
inline: result 16, b = 5
```

	Macro	inline function
Type checking	none	full
Arguments evaluated	once per use in the body	exactly once
Works on any type	yes (generic)	no (one type, unless _Generic)
Visible in debugger	no	yes

	Macro	inline function
Can use # and ##	yes	no
Prefer for	constants in #if, code generation, stringizing	everything else

15.3.8 Variadic macros __VA_ARGS__

A macro whose parameter list ends with `...` accepts any number of arguments; `__VA_ARGS__` stands for them in the body. This is the standard way to build logging macros around printf. C23 adds `__VA_OPT__(,)` to drop the comma when there are no extra arguments; GCC also accepts `, ##__VA_ARGS__` for the same purpose.

A LOGGING MACRO WITH A VARIABLE NUMBER OF ARGUMENTS main.c

```c
#include <stdio.h>

#define LOG(level, fmt, ...) \
    fprintf(stdout, "[%s] " fmt "\n", level __VA_OPT__(,) __VA_ARGS__)
#define INFO(...)  LOG("INFO", __VA_ARGS__)
#define ERROR(...) LOG("ERROR", __VA_ARGS__)

int main(void) {
    int users = 42;
    INFO("server started");                    /* no extra arguments */
    INFO("%d users online", users);
    ERROR("disk %s is %d%% full", "/dev/sda1", 97);
    return 0;
}
```

OUTPUT

```
[INFO] server started
[INFO] 42 users online
[ERROR] disk /dev/sda1 is 97% full
```

15.4 Conditional compilation

Conditional directives decide which lines reach the compiler at all. Removed code is not compiled, so it may even contain code for another platform that would not compile here.

15.4.1 #if, #elif, #else, #endif

`#if` evaluates a **constant integer expression** made of literals, macros and operators (no variables, no sizeof). Non-zero keeps the block. Undefined names count as 0.

#IF CHAINS main.c

```c
#include <stdio.h>

#define LEVEL 2

int main(void) {
#if LEVEL == 1
    printf("basic edition\n");
#elif LEVEL == 2
    printf("standard edition\n");
#elif LEVEL >= 3
    printf("pro edition\n");
#else
    printf("unknown edition\n");
#endif

#if 0
    this text is never compiled, a handy way to disable a block
#endif
    return 0;
}
```

OUTPUT

```
standard edition
```

15.4.2 #ifdef and #ifndef

`#ifdef NAME` keeps the block if NAME is defined (with any value, even empty). `#ifndef NAME` keeps it if NAME is **not** defined. Macros can be defined on the compiler command line with `-DNAME` or `-DNAME=value`, which is how one source file produces different builds.

BUILT WITH: GCC -DVERBOSE '-DGREETING="NAMASTE"' MAIN.C main.c

```c
#include <stdio.h>

#ifndef GREETING
#define GREETING "Hello"           /* default unless set with -D */
#endif

int main(void) {
#ifdef VERBOSE
    printf("verbose mode on\n");
#endif
    printf("%s, world\n", GREETING);
    return 0;
}
```

OUTPUT

```
verbose mode on
Namaste, world
```

15.4.3 defined() operator

`defined(NAME)` is 1 if NAME is a macro, 0 otherwise. Unlike #ifdef it can be combined with `&&`, `||` and `!` in one #if.

SNIPPET C

```c
#if defined(DEBUG) && !defined(NDEBUG)
    /* debug build that still has assertions */
#endif

#if defined(_WIN32) || defined(__CYGWIN__)
    /* Windows-like environment */
#endif
```

15.4.4 Debug builds and platform-specific code

Two everyday uses: **debug logging** that disappears completely from release builds, and choosing **platform-specific code** using macros the compiler predefines for each system.

RELEASE BUILD: GCC MAIN.C main.c

```c
#include <stdio.h>

#ifdef DEBUG
#define DBG(fmt, ...) fprintf(stderr, "DEBUG %s:%d: " fmt "\n", __FILE__, __LINE__, __VA_ARGS__)
#else
#define DBG(fmt, ...) ((void)0)              /* compiles to nothing */
#endif

#if defined(_WIN32)
#define PLATFORM "Windows"
#elif defined(__APPLE__)
#define PLATFORM "macOS"
#elif defined(__linux__)
#define PLATFORM "Linux"
#else
#define PLATFORM "unknown"
#endif

int main(void) {
    int total = 0;
    for (int i = 1; i <= 3; i++) {
        total += i * i;
        DBG("i = %d, total = %d", i, total);
    }
    printf("running on %s, total = %d\n", PLATFORM, total);
    return 0;
}
```

OUTPUT

```
running on Linux, total = 14
```

DEBUG BUILD: GCC -DDEBUG MAIN.C (SAME SOURCE) main.c

```c
#include <stdio.h>

#ifdef DEBUG
#define DBG(fmt, ...) fprintf(stderr, "DEBUG %s:%d: " fmt "\n", __FILE__, __LINE__, __VA_ARGS__)
#else
#define DBG(fmt, ...) ((void)0)              /* compiles to nothing */
#endif

#if defined(_WIN32)
#define PLATFORM "Windows"
#elif defined(__APPLE__)
#define PLATFORM "macOS"
#elif defined(__linux__)
#define PLATFORM "Linux"
#else
#define PLATFORM "unknown"
#endif

int main(void) {
    int total = 0;
    for (int i = 1; i <= 3; i++) {
        total += i * i;
        DBG("i = %d, total = %d", i, total);
    }
    printf("running on %s, total = %d\n", PLATFORM, total);
    return 0;
}
```

OUTPUT

```
DEBUG main.c:23: i = 1, total = 1
DEBUG main.c:23: i = 2, total = 5
DEBUG main.c:23: i = 3, total = 14
running on Linux, total = 14
```

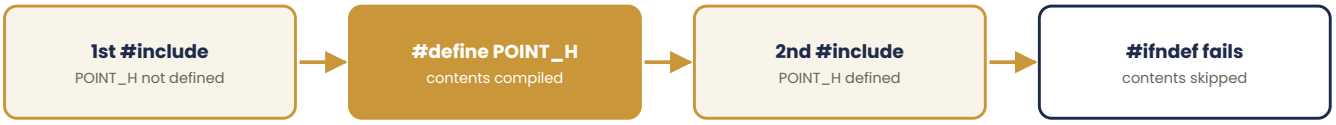
Macro	Defined on
`__linux__`	Linux
`_WIN32`	Windows, 32 and 64 bit
`_WIN64`	64-bit Windows
`__APPLE__`	macOS and iOS
`__unix__`	most Unix-like systems
`__GNUC__`	GCC (and Clang, which imitates it)
`_MSC_VER`	Microsoft Visual C++
`__x86_64__` / `__aarch64__`	64-bit Intel/AMD / 64-bit ARM

15.5 Include guards

15.5.1 #ifndef / #define / #endif pattern

If a header is included twice into the same file (often indirectly, through two other headers), its type definitions appear twice and the compiler reports "redefinition" errors. An **include guard** makes the

second inclusion empty:



HOW AN INCLUDE GUARD WORKS

WITHOUT A GUARD : POINT.H C

```c
typedef struct { int x, y; } Point;
```

WITHOUT A GUARD : SHAPE.H C

```c
#include "point.h"
typedef struct { Point corner; int w, h; } Rect;
```

DOUBLE INCLUSION WITHOUT A GUARD main.c

```c
#include <stdio.h>
#include "point.h"
#include "shape.h"          /* includes point.h a second time */

int main(void) {
    Rect r = {{1, 2}, 3, 4};
    printf("%d\n", r.w * r.h);
    return 0;
}
```

OUTPUT

```
In file included from shape.h:1,
                 from main.c:3:
point.h:1:30: error: conflicting types for ‘Point’; have ‘struct <anonymous>’
    1 | typedef struct { int x, y; } Point;
      |                              ^~~~~
In file included from main.c:2:
point.h:1:30: note: previous declaration of ‘Point’ with type ‘Point’
    1 | typedef struct { int x, y; } Point;
      |                              ^~~~~
```

WITH GUARDS : POINT.H C

```c
#ifndef POINT_H
#define POINT_H

typedef struct { int x, y; } Point;

#endif /* POINT_H */
```

WITH GUARDS : SHAPE.H C

```c
#ifndef SHAPE_H
#define SHAPE_H

#include "point.h"
typedef struct { Point corner; int w, h; } Rect;

#endif /* SHAPE_H */
```

THE SAME PROGRAM WITH GUARDED HEADERS

main.c

```c
#include <stdio.h>
#include "point.h"
#include "shape.h"

int main(void) {
    Rect r = {{1, 2}, 3, 4};
    printf("area = %d\n", r.w * r.h);
    return 0;
}
```

OUTPUT

```
area = 12
```

NOTE

C23 allows the same typedef to be repeated if it is identical, but struct definitions and many other things still clash. Guard every header, always.

15.5.2 #pragma once

`#pragma once` as the first line of a header tells the compiler to include the file only once. It is shorter and avoids name clashes between guard macros. It is not part of the C standard, but GCC, Clang and MSVC all support it. Many projects use classic guards for maximum portability; both are acceptable.

POINT.H WITH #PRAGMA ONCE

C

```c
#pragma once

typedef struct { int x, y; } Point;
```

15.6 Other directives

15.6.1 #error and #warning

`#error message` stops compilation with the message; it is used to reject unsupported configurations early. `#warning message` (standard since C23, long supported by GCC and Clang) prints a warning and continues.

W.C

C

```c
#include <stdio.h>
#if __STDC_VERSION__ < 201710L
#error "This program needs C17 or newer"
#endif
#warning "experimental feature enabled"
int main(void) { printf("built\n"); return 0; }
```

Terminal

```
$ gcc -std=c17 w.c -o w && ./w
w.c:5:2: warning: #warning "experimental feature enabled" [-Wcpp]
    5 | #warning "experimental feature enabled"
      |  ^~~~~~~
built
$ gcc -std=c99 w.c -o w
w.c:3:2: error: #error "This program needs C17 or newer"
    3 | #error "This program needs C17 or newer"
      |  ^~~~~
```

15.6.2 #pragma

`#pragma` passes implementation-specific instructions to the compiler. Unknown pragmas are ignored (with `-Wall`, warned about). Common ones:

Pragma	Effect
`#pragma once`	include this header only once
`#pragma pack(1)`	remove structure padding (MSVC, also GCC)
`#pragma GCC diagnostic ignored "-Wunused"`	silence one warning in a region
`#pragma omp parallel for`	parallelize a loop with OpenMP (-fopenmp)
`_Pragma("...")`	operator form, usable inside macros

TWO USEFUL PRAGMAS main.c

```c
#include <stdio.h>

#pragma GCC diagnostic push
#pragma GCC diagnostic ignored "-Wunused-variable"
void legacy(void) {
    int unused_counter;            /* no warning in this region */
}
#pragma GCC diagnostic pop

#pragma pack(push, 1)
struct Header { char tag; int length; };
#pragma pack(pop)

int main(void) {
    legacy();
    printf("packed Header is %zu bytes\n", sizeof(struct Header));
    return 0;
}
```

OUTPUT

```
packed Header is 5 bytes
```

15.6.3 #line

`#line number "file"` changes the line number and file name the compiler uses for messages and for `__LINE__` / `__FILE__`. Humans rarely write it; code generators (parser generators like yacc/bison, template engines) emit it so that errors point to the original input rather than to the generated C file.

#LINE CHANGES THE REPORTED LOCATION main.c

```c
#include <stdio.h>

int main(void) {
    printf("%s:%d\n", __FILE__, __LINE__);
#line 100 "grammar.y"
    printf("%s:%d\n", __FILE__, __LINE__);
    printf("%s:%d\n", __FILE__, __LINE__);
    return 0;
}
```

```
OUTPUT
main.c:4
grammar.y:100
grammar.y:101
```

15.6.4 #embed (C23)

C23 adds `#embed "file"`, which inserts the bytes of any file (an image, a font, a certificate) as a comma-separated list of integers, perfect for initializing an array. Before C23, programmers used external tools such as `xxd -i` to convert files to C arrays. Support arrived in GCC 15 and Clang 19; the GCC 13 compiler used for these notes does not support it yet, so this example is shown with its expected output.

EMBEDDING A FILE (C23) main.c

```c
#include <stdio.h>

int main(void) {
    static const unsigned char data[] = {
#embed "msg.txt"                /* msg.txt contains: Hi!\n */
    };
    printf("%zu bytes:", sizeof data);
    for (size_t i = 0; i < sizeof data; i++) printf(" %02X", data[i]);
    printf("\n");
    return 0;
}
```

```
OUTPUT (GCC 15+ / Clang 19+)
4 bytes: 48 69 21 0A
```

15.7 Predefined macros

15.7.1 __FILE__, __LINE__, __DATE__, __TIME__, __func__, __STDC_VERSION__

Name	Expands to
`__FILE__`	current source file name, as a string
`__LINE__`	current line number, as an int
`__DATE__`	compilation date, "Mmm dd yyyy"
`__TIME__`	compilation time, "hh:mm:ss"
`__func__`	name of the current function (a predefined identifier, not a macro)

Name	Expands to
`__STDC__`	1 on a conforming compiler
`__STDC_VERSION__`	199901L (C99), 201112L (C11), 201710L (C17), 202311L (C23)
`__STDC_HOSTED__`	1 with a full standard library, 0 on bare-metal

PREDEFINED MACROS main.c

```c
#include <stdio.h>

void report(void) {
    printf("in function %s at %s:%d\n", __func__, __FILE__, __LINE__);
}
int main(void) {
    printf("compiled on %s at %s\n", __DATE__, __TIME__);
    printf("C standard version: %ld\n", __STDC_VERSION__);
#if __STDC_VERSION__ >= 201112L
    printf("C11 or later features available\n");
#endif
    report();
    return 0;
}
```

OUTPUT (date and time are when it was compiled)

```
compiled on Oct  6 2026 at 09:58:50
C standard version: 201710
C11 or later features available
in function report at main.c:4
```

15.8 assert.h

15.8.1 assert() and NDEBUG

`assert(condition)` from `<assert.h>` checks an assumption at run time. If the condition is false, it prints the expression, file, line and function, then calls `abort()`. Assertions document what **must** be true for the code to be correct: they catch programming errors, not user errors.

A FAILING ASSERTION main.c

```c
#include <stdio.h>
#include <assert.h>

double average(const int *v, int n) {
    assert(v != NULL);
    assert(n > 0 && "average of an empty array");
    long sum = 0;
    for (int i = 0; i < n; i++) sum += v[i];
    return (double)sum / n;
}
int main(void) {
    int marks[] = {70, 80, 90};
    printf("average = %.1f\n", average(marks, 3));
    fflush(stdout);
    printf("average = %.1f\n", average(marks, 0));   /* bug: n is 0 */
    return 0;
}
```

OUTPUT (crash)

```
average = 80.0
prog: main.c:6: average: Assertion `n > 0 && "average of an empty array"' failed.
Aborted (core dumped)
```

Defining `NDEBUG` (usually with `-DNDEBUG` in release builds) turns every `assert` into nothing, so they cost nothing in production. Consequently, **never put necessary work inside an assert**: `assert(fclose(f) == 0);` would stop closing the file in release builds.

BUILT WITH: GCC -DNDEBUG MAIN.C main.c

```c
#include <stdio.h>
#include <assert.h>

int main(void) {
    int n = 0;
    assert(n > 0);
    printf("with -DNDEBUG the assert vanished; n = %d\n", n);
    return 0;
}
```

OUTPUT

```
with -DNDEBUG the assert vanished; n = 0
```

15.8.2 static_assert

`static_assert(condition, "message")` (C11 `_Static_assert`; a keyword without the header in C23) checks a **constant expression at compile time**. It costs nothing at run time and is ideal for verifying assumptions about sizes and layouts.

STATIC_ASSERT STOPS THE BUILD

main.c

```c
#include <assert.h>
#include <stdint.h>

struct Packet { uint8_t type; uint8_t flags; uint16_t length; uint32_t crc; };

static_assert(sizeof(int) >= 4, "int must be at least 32 bits");
static_assert(sizeof(struct Packet) == 8, "Packet layout changed");
static_assert(sizeof(void *) == 4, "this code assumes a 32-bit system");

int main(void) { return 0; }
```

OUTPUT

```
In file included from main.c:1:
main.c:8:1: error: static assertion failed: "this code assumes a 32-bit system"
    8 | static_assert(sizeof(void *) == 4, "this code assumes a 32-bit system");
      | ^~~~~~~~~~~~~
```

The preprocessor is a text tool that runs first: #include pastes files, #define substitutes text, and #if selects code. Parenthesize macro parameters, avoid side effects in macro arguments, guard every header, and prefer inline functions and constants when a macro is not truly needed.

PRACTICE CHECKPOINT

Write a **header file with include guards** and macros for **MAX, MIN and SQUARE**, a **DEBUG_LOG** macro that prints the file and line, and build the program **with and without -DDEBUG**.

SOLUTION : UTILS.H

C

```c
#ifndef UTILS_H
#define UTILS_H

#include <stdio.h>

#define MAX(a, b)    ((a) > (b) ? (a) : (b))
#define MIN(a, b)    ((a) < (b) ? (a) : (b))
#define SQUARE(x)    ((x) * (x))

#ifdef DEBUG
#define DEBUG_LOG(fmt, ...) \
    fprintf(stderr, "[DEBUG] %s:%d %s(): " fmt "\n", \
            __FILE__, __LINE__, __func__, __VA_ARGS__)
#else
#define DEBUG_LOG(fmt, ...) ((void)0)
#endif

#endif /* UTILS_H */
```

MAIN.C BUILT NORMALLY: GCC MAIN.C

main.c

```c
#include <stdio.h>
#include "utils.h"
#include "utils.h"          /* included twice on purpose: the guard makes it safe */

int main(void) {
    int data[] = {14, -3, 27, 8, 19};
    int n = sizeof data / sizeof data[0];
    int hi = data[0], lo = data[0];
    long sum_sq = 0;
    for (int i = 0; i < n; i++) {
        hi = MAX(hi, data[i]);
        lo = MIN(lo, data[i]);
        sum_sq += SQUARE(data[i]);
        DEBUG_LOG("i=%d value=%d hi=%d lo=%d", i, data[i], hi, lo);
    }
    printf("max = %d, min = %d, sum of squares = %ld\n", hi, lo, sum_sq);
    return 0;
}
```

OUTPUT

```
max = 27, min = -3, sum of squares = 1359
```

THE SAME MAIN.C BUILT WITH: GCC -DDEBUG MAIN.C

main.c

```c
#include <stdio.h>
#include "utils.h"
#include "utils.h"          /* included twice on purpose: the guard makes it safe */

int main(void) {
    int data[] = {14, -3, 27, 8, 19};
    int n = sizeof data / sizeof data[0];
    int hi = data[0], lo = data[0];
    long sum_sq = 0;
    for (int i = 0; i < n; i++) {
        hi = MAX(hi, data[i]);
        lo = MIN(lo, data[i]);
        sum_sq += SQUARE(data[i]);
        DEBUG_LOG("i=%d value=%d hi=%d lo=%d", i, data[i], hi, lo);
    }
    printf("max = %d, min = %d, sum of squares = %ld\n", hi, lo, sum_sq);
    return 0;
}
```

OUTPUT

```
[DEBUG] main.c:14 main(): i=0 value=14 hi=14 lo=14
[DEBUG] main.c:14 main(): i=1 value=-3 hi=14 lo=-3
[DEBUG] main.c:14 main(): i=2 value=27 hi=27 lo=-3
[DEBUG] main.c:14 main(): i=3 value=8 hi=27 lo=-3
[DEBUG] main.c:14 main(): i=4 value=19 hi=27 lo=-3
max = 27, min = -3, sum of squares = 1359
```

MODULE 16 · INTERMEDIATE

File handling

Minimum duration **3 days** | Level **Intermediate**

Variables disappear when a program ends. Files let a program store data permanently and read it back the next time it runs: settings, saved games, reports, logs, databases. C does all of this through the FILE stream functions in .

IN THIS MODULE

16.1 Introduction
16.2 Opening and closing files
16.3 Reading and writing text files
16.4 Binary files
16.5 Random access
16.6 Error handling
16.7 File operations
16.8 File programs

16.1 Introduction

16.1.1 Why files

- **Persistence**: data survives after the program ends and after the computer restarts.
- **Size**: files can hold far more data than fits comfortably in memory, and you can process them piece by piece.
- **Sharing**: other programs (a spreadsheet, a web server, another C program) can read what you write.
- **Input without typing**: test data, configuration and batch jobs come from files instead of the keyboard.

16.1.2 Text files vs binary files

A **text file** stores data as human-readable characters, organised in lines. The number 1500 is stored as the four characters `'1' '5' '0' '0'`. A **binary file** stores the raw bytes of values exactly as they are in memory: the int 1500 is stored as its 4 bytes `DC 05 00 00` (on a little-endian machine).



5 bytes, readable in any editor vs 4 bytes, exact, fast, not human-readable

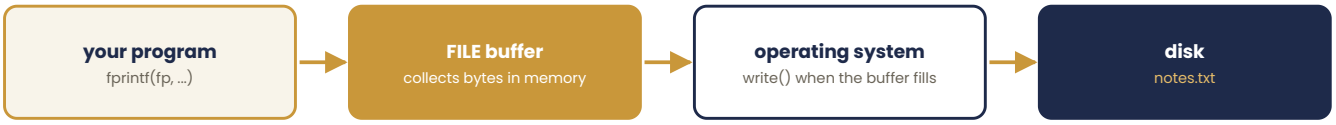
THE SAME NUMBER STORED AS TEXT AND AS BINARY

	Text file	Binary file
Contents	characters and newlines	raw bytes of variables
Readable in an editor	yes	no

	Text file	Binary file
Size	usually larger	compact, fixed size per record
Speed	slower (conversion with printf/scanf)	fast (bytes copied directly)
Portability	high	depends on sizes, endianness and padding
Functions	fprintf, fscanf, fgets, fputs, fgetc	fwrite, fread
Examples	.txt .csv .c .html .json	.jpg .mp3 .exe .db, saved records

16.1.3 The FILE pointer and streams

C treats every source or destination of data as a **stream**: a sequence of bytes you read from or write to in order. Opening a file gives you a pointer to a `FILE` structure (defined in stdio.h) that holds everything the library needs: the operating-system file descriptor, a **buffer**, the current **position**, and the **end-of-file** and **error** indicators. You never look inside the FILE structure; you pass the pointer to the stdio functions.



OUTPUT GOES THROUGH A BUFFER BEFORE IT REACHES THE DISK

16.1.4 Standard streams: stdin, stdout, stderr

Three streams are opened automatically before main starts. They are ordinary `FILE *` values, so every file function works with them too.

Stream	Default device	Buffering	Used by
`stdin`	keyboard	line buffered	scanf, getchar, fgets(..., stdin)
`stdout`	screen	line buffered on a terminal, fully buffered when redirected	printf, puts, putchar
`stderr`	screen	unbuffered	error messages: fprintf(stderr, ...)

THE STANDARD STREAMS ARE FILE POINTERS

main.c

```c
#include <stdio.h>

int main(void) {
    fprintf(stdout, "normal output goes to stdout\n");
    fprintf(stderr, "errors go to stderr\n");
    fputs("fputs works with stdout too\n", stdout);
    return 0;
}
```

OUTPUT

```
normal output goes to stdout
errors go to stderr
fputs works with stdout too
```

Keeping errors on stderr matters when output is redirected: `./prog > out.txt` sends stdout to the file, while error messages still appear on the screen.

16.2 Opening and closing files

16.2.1 fopen() and file modes

PROTOTYPE

C

```c
FILE *fopen(const char *filename, const char *mode);
```

Mode	Meaning	If the file exists	If it does not exist
`"r"`	read	read from the start	fails (NULL)
`"w"`	write	**contents erased**	created
`"a"`	append	writes go to the end	created
`"r+"`	read and write	kept, position at start	fails (NULL)
`"w+"`	read and write	**contents erased**	created
`"a+"`	read and append	read anywhere, writes at end	created
`"rb" "wb" "ab"` …	same, binary	no newline translation	
`"wx"` (C11)	exclusive create	fails (NULL)	created

The `b` modifier matters on Windows, where text mode converts `\n` to `\r\n` on output and back on input, and treats Ctrl-Z as end of file. On Linux and macOS text and binary modes behave the same, but writing `b` for binary data keeps your program portable.

> **WATCH OUT**
> Mode `"w"` silently destroys an existing file's contents the moment fopen succeeds. Use `"a"` to add to a file, or `"wx"` to refuse to overwrite.

16.2.2 Checking for NULL

fopen returns **NULL** if the file cannot be opened: it does not exist (for "r"), the directory does not exist, permission is denied, or too many files are open. Always check, and report the reason with `perror` (16.6).

16.2.3 fclose()

`fclose(fp)` flushes any buffered output to the disk, releases the operating-system file handle and frees the FILE structure. It returns 0 on success and EOF on error (for example, the disk filled up while flushing). Every successful fopen needs one fclose; a program that forgets may lose its last writes and eventually run out of file handles.

OPEN, WRITE, CLOSE, AND A FAILED OPEN main.c

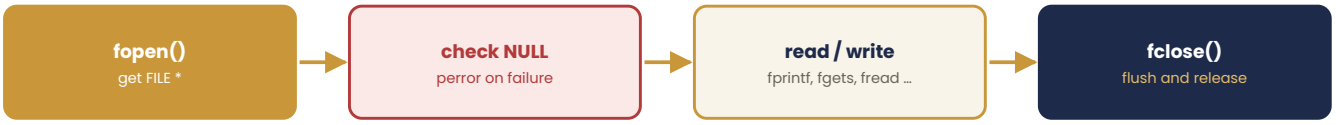
```c
#include <stdio.h>

int main(void) {
    FILE *fp = fopen("greeting.txt", "w");          /* create / truncate */
    if (fp == NULL) {
        perror("greeting.txt");
        return 1;
    }
    fprintf(fp, "Hello from C!\n");
    fprintf(fp, "Line %d of the file.\n", 2);
    if (fclose(fp) != 0) { perror("fclose"); return 1; }
    printf("file written\n");

    FILE *in = fopen("missing.txt", "r");           /* does not exist */
    if (in == NULL) perror("missing.txt");
    return 0;
}
```

OUTPUT

```
file written
missing.txt: No such file or directory
```



THE LIFE CYCLE OF EVERY FILE

16.3 Reading and writing text files

16.3.1 Character I/O: fgetc(), fputc()

`int fgetc(FILE *fp)` reads one character and returns it as an `unsigned char` converted to int, or **EOF** (-1) at the end of the file or on error. That is why the variable must be `int`, not char: EOF must be distinguishable from every valid byte. `fputc(c, fp)` writes one character.

COPYING A FILE CHARACTER BY CHARACTER, IN UPPER CASE

main.c

```c
#include <stdio.h>
#include <ctype.h>

int main(void) {
    FILE *in = fopen("poem.txt", "r");
    FILE *out = fopen("POEM.txt", "w");
    if (!in || !out) { perror("fopen"); return 1; }
    int c;                                  /* int, to hold EOF */
    long count = 0;
    while ((c = fgetc(in)) != EOF) {
        fputc(toupper(c), out);
        count++;
    }
    fclose(in);
    fclose(out);
    printf("%ld characters converted\n", count);

    FILE *check = fopen("POEM.txt", "r");
    while ((c = fgetc(check)) != EOF) putchar(c);
    fclose(check);
    return 0;
}
```

OUTPUT

```
56 characters converted
TWINKLE TWINKLE LITTLE STAR,
HOW I WONDER WHAT YOU ARE.
```

NOTE

Files given with this example (here `poem.txt`) are created in the program's folder before it runs. Its contents are:

```
Twinkle twinkle little star,
how I wonder what you are.
```

16.3.2 Line I/O: fgets(), fputs()

`char *fgets(char *buf, int size, FILE *fp)` reads **one line**, at most size - 1 characters, keeps the newline, and adds `'\0'`. It returns buf, or NULL at end of file. It never overflows the buffer, which makes it the safe way to read text. `fputs(s, fp)` writes a string without adding a newline.

READING A FILE LINE BY LINE

main.c

```c
#include <stdio.h>
#include <string.h>

int main(void) {
    FILE *fp = fopen("todo.txt", "r");
    if (!fp) { perror("todo.txt"); return 1; }
    char line[100];
    int n = 0;
    while (fgets(line, sizeof line, fp) != NULL) {
        line[strcspn(line, "\n")] = '\0';          /* strip the newline */
        if (line[0] == '\0') continue;              /* skip blank lines  */
        printf("%2d. %s\n", ++n, line);
    }
    fclose(fp);

    FILE *log = fopen("todo.txt", "a");               /* append a line */
    fputs("Revise file handling\n", log);
    fclose(log);
    printf("(added one task; the file now has %d tasks)\n", n + 1);
    return 0;
}
```

OUTPUT

```
 1. Buy notebooks
 2. Finish Module 15
 3. Practice pointers
(added one task; the file now has 4 tasks)
```

16.3.3 Formatted I/O: fprintf(), fscanf()

`fprintf` and `fscanf` work exactly like printf and scanf, with a FILE pointer as the first argument. fscanf returns the number of items successfully converted, so a loop reads records until it no longer gets a complete one.

WRITING AND READING FORMATTED RECORDS

main.c

```c
#include <stdio.h>

int main(void) {
    /* write a small table */
    FILE *fp = fopen("marks.txt", "w");
    if (!fp) return 1;
    const char *names[] = {"Asha", "Ravi", "Meera"};
    int marks[] = {88, 72, 95};
    for (int i = 0; i < 3; i++) fprintf(fp, "%s %d\n", names[i], marks[i]);
    fclose(fp);

    /* read it back */
    fp = fopen("marks.txt", "r");
    if (!fp) return 1;
    char name[30];
    int m, total = 0, count = 0;
    while (fscanf(fp, "%29s %d", name, &m) == 2) {
        printf("%-6s %3d\n", name, m);
        total += m;
        count++;
    }
    fclose(fp);
    printf("average %.2f\n", (double)total / count);
    return 0;
}
```

OUTPUT

```
Asha    88
Ravi    72
Meera   95
average 85.00
```

16.3.4 Detecting end of file: feof() and return values

`feof(fp)` returns true only **after** a read has already tried to go past the end. That is why the loop `while (!feof(fp))` is a classic bug: it runs one extra time and processes the last value twice. Always test the **return value** of the reading function instead, then use feof/ferror afterwards to find out why reading stopped.

THE WHILE (!FEOF) BUG

main.c

```c
#include <stdio.h>

int main(void) {
    FILE *fp = fopen("nums.txt", "r");
    if (!fp) return 1;
    int x, sum = 0;
    while (!feof(fp)) {                 /* WRONG */
        fscanf(fp, "%d", &x);
        sum += x;
    }
    printf("wrong loop: sum = %d\n", sum);

    rewind(fp);
    sum = 0;
    while (fscanf(fp, "%d", &x) == 1)    /* RIGHT: test the read itself */
        sum += x;
    printf("right loop: sum = %d\n", sum);
    if (feof(fp)) printf("stopped because: end of file\n");
    else if (ferror(fp)) printf("stopped because: read error\n");
    fclose(fp);
    return 0;
}
```

OUTPUT

```
wrong loop: sum = 90
right loop: sum = 60
stopped because: end of file
```

Function	Returns at end of file
`fgetc`	EOF
`fgets`	NULL
`fscanf`	EOF, or fewer items than requested
`fread`	fewer items than requested (possibly 0)

16.4 Binary files

16.4.1 fwrite() and fread()

PROTOTYPES

C

```c
size_t fwrite(const void *ptr, size_t size, size_t count, FILE *fp);
size_t fread (void *ptr,       size_t size, size_t count, FILE *fp);
```

Both copy `count` items of `size` bytes each between memory at ptr and the file, and return the number of **complete items** transferred. No conversion happens, so a whole array goes out or comes in with a single call.

WRITING AND READING AN INT ARRAY IN BINARY main.c

```c
#include <stdio.h>

int main(void) {
    int scores[5] = {1500, -2, 77, 65535, 42};
    FILE *fp = fopen("scores.bin", "wb");
    if (!fp) return 1;
    size_t w = fwrite(scores, sizeof scores[0], 5, fp);
    fclose(fp);
    printf("wrote %zu ints\n", w);

    int back[5] = {0};
    fp = fopen("scores.bin", "rb");
    if (!fp) return 1;
    size_t r = fread(back, sizeof back[0], 5, fp);
    fclose(fp);
    printf("read %zu ints:", r);
    for (int i = 0; i < 5; i++) printf(" %d", back[i]);
    printf("\n");

    /* look at the raw bytes of the file */
    fp = fopen("scores.bin", "rb");
    int c, k = 0;
    printf("file bytes:");
    while ((c = fgetc(fp)) != EOF) printf("%s%02X", k++ % 4 ? " " : "  ", c);
    printf("\n");
    fclose(fp);
    return 0;
}
```

OUTPUT

```
wrote 5 ints
read 5 ints: 1500 -2 77 65535 42
file bytes:  DC 05 00 00  FE FF FF FF  4D 00 00 00  FF FF 00 00  2A 00 00 00
```

The dump shows 20 bytes: 4 per int, least significant byte first (little-endian). 1500 is `DC 05 00 00` and -2 is `FE FF FF FF` in two's complement.

16.4.2 Saving and loading arrays of structures

Binary files shine with structures: a whole array of records is saved with one fwrite and loaded with one fread. A small header with a count lets the loader know how many records follow.

SAVING AND LOADING STRUCTURES

main.c

```c
#include <stdio.h>

typedef struct {
    int id;
    char name[20];
    double balance;
} Account;

int save(const char *path, const Account *a, int n) {
    FILE *fp = fopen(path, "wb");
    if (!fp) return 0;
    int ok = fwrite(&n, sizeof n, 1, fp) == 1 &&                /* header: count */
             fwrite(a, sizeof *a, (size_t)n, fp) == (size_t)n;   /* records */
    return fclose(fp) == 0 && ok;
}
int load(const char *path, Account *a, int max) {
    FILE *fp = fopen(path, "rb");
    if (!fp) return -1;
    int n = 0;
    if (fread(&n, sizeof n, 1, fp) != 1 || n > max) { fclose(fp); return -1; }
    int got = (int)fread(a, sizeof *a, (size_t)n, fp);
    fclose(fp);
    return got;
}
int main(void) {
    Account bank[] = {{1, "Asha", 2500.50}, {2, "Ravi", 120.00}, {3, "Meera", 98000.75}};
    if (!save("bank.dat", bank, 3)) { perror("save"); return 1; }
    printf("saved 3 accounts (%zu bytes each)\n", sizeof(Account));

    Account loaded[10];
    int n = load("bank.dat", loaded, 10);
    printf("loaded %d accounts\n", n);
    for (int i = 0; i < n; i++)
        printf("  #%d %-6s %10.2f\n", loaded[i].id, loaded[i].name, loaded[i].balance);
    return 0;
}
```

OUTPUT

```
saved 3 accounts (32 bytes each)
loaded 3 accounts
  #1 Asha      2500.50
  #2 Ravi       120.00
  #3 Meera    98000.75
```

WATCH OUT

Raw structure files are not portable between compilers or machines: int sizes, padding and byte order can differ. They are fine for your own program's data on one platform. For files exchanged between systems, write each field explicitly in a fixed format, or use text (CSV, JSON).

16.5 Random access

So far files were read from start to end. Every stream has a **file position indicator**, the offset where the next read or write happens. Random-access functions move it, so you can jump directly to record 500 without reading the first 499.

16.5.1 fseek() with SEEK_SET, SEEK_CUR, SEEK_END

SNIPPET C

```c
int fseek(FILE *fp, long offset, int origin);    /* 0 on success */
```

origin	offset is measured from	Example
`SEEK_SET`	the beginning of the file	`fseek(fp, 3 * sizeof(Rec), SEEK_SET)` : record 3
`SEEK_CUR`	the current position	`fseek(fp, -sizeof(Rec), SEEK_CUR)` : back one record
`SEEK_END`	the end of the file	`fseek(fp, 0, SEEK_END)` : go to the end



FIXED-SIZE RECORDS MAKE EVERY RECORD DIRECTLY ADDRESSABLE (40-BYTE RECORDS)

16.5.2 ftell() and rewind()

`long ftell(FILE *fp)` returns the current position in bytes. `rewind(fp)` goes back to the start and also clears the error indicator. A common trick: `fseek(fp, 0, SEEK_END)` followed by `ftell(fp)` gives the file size.

MOVING AROUND A FILE

main.c

```c
#include <stdio.h>

int main(void) {
    FILE *fp = fopen("alphabet.txt", "w+");
    if (!fp) return 1;
    fputs("ABCDEFGHIJKLMNOPQRSTUVWXYZ", fp);
    printf("after writing, position = %ld\n", ftell(fp));

    fseek(fp, 0, SEEK_END);
    printf("file size = %ld bytes\n", ftell(fp));

    fseek(fp, 7, SEEK_SET);
    printf("char at offset 7 = %c\n", fgetc(fp));       /* H, position now 8 */
    fseek(fp, 2, SEEK_CUR);
    printf("2 further on      = %c\n", fgetc(fp));       /* K */
    fseek(fp, -1, SEEK_END);
    printf("last character    = %c\n", fgetc(fp));       /* Z */
    rewind(fp);
    printf("after rewind, position = %ld\n", ftell(fp));
    printf("first character   = %c\n", fgetc(fp));
    fclose(fp);
    return 0;
}
```

OUTPUT

```
after writing, position = 26
file size = 26 bytes
char at offset 7 = H
2 further on      = K
last character    = Z
after rewind, position = 0
first character   = A
```

16.5.3 Updating a record in place

Open the file with `"r+b"` (read and write, keep contents), seek to the record, read it, change it, seek **back** to the same place and write it. The standard requires an fseek (or fflush) between a read and a following write on the same stream.

CHANGING ONE RECORD WITHOUT REWRITING THE FILE

main.c

```c
#include <stdio.h>

typedef struct { int id; char item[16]; int stock; } Product;

int main(void) {
    Product p[] = {{100, "pen", 50}, {101, "pencil", 80}, {102, "eraser", 30}};
    FILE *fp = fopen("stock.dat", "w+b");
    if (!fp) return 1;
    fwrite(p, sizeof p[0], 3, fp);

    int index = 1;                                  /* update record 1 */
    Product r;
    fseek(fp, index * (long)sizeof r, SEEK_SET);
    fread(&r, sizeof r, 1, fp);
    printf("before: %d %s %d\n", r.id, r.item, r.stock);
    r.stock -= 25;                                  /* sold 25 */
    fseek(fp, -(long)sizeof r, SEEK_CUR);           /* back to its start */
    fwrite(&r, sizeof r, 1, fp);

    rewind(fp);
    while (fread(&r, sizeof r, 1, fp) == 1)
        printf("  %d %-7s %3d\n", r.id, r.item, r.stock);
    fclose(fp);
    return 0;
}
```

OUTPUT

```
before: 101 pencil 80
  100 pen      50
  101 pencil   55
  102 eraser   30
```

16.6 Error handling

16.6.1 ferror() and clearerr()

Each stream has two flags. The **end-of-file indicator** (tested with feof) and the **error indicator** (tested with `ferror`), set when a read or write fails for a reason other than reaching the end, for example a disk error or writing to a stream opened for reading. Both stay set until `clearerr(fp)`, rewind or a successful fseek clears them.

THE ERROR INDICATOR

main.c

```c
#include <stdio.h>

int main(void) {
    FILE *fp = fopen("readonly.txt", "w");
    fputs("data\n", fp);
    fclose(fp);

    fp = fopen("readonly.txt", "r");                /* opened for reading */
    if (fputc('X', fp) == EOF)                       /* writing is an error */
        printf("fputc failed\n");
    printf("ferror = %d, feof = %d\n", ferror(fp) != 0, feof(fp) != 0);
    clearerr(fp);
    printf("after clearerr: ferror = %d\n", ferror(fp) != 0);
    fclose(fp);
    return 0;
}
```

OUTPUT

```
fputc failed
ferror = 1, feof = 0
after clearerr: ferror = 0
```

16.6.2 perror() and errno

When a library call fails, it stores an error code in the global variable `errno` (from `<errno.h>`). `perror("prefix")` prints the prefix, a colon and the text for the current errno to stderr. Read errno immediately after the failing call, because later successful calls may change it.

16.6.3 strerror()

`strerror(errnum)` from `<string.h>` returns the message text for an error code, so you can build your own messages.

REPORTING ERRORS main.c

```c
#include <stdio.h>
#include <errno.h>
#include <string.h>

int main(void) {
    FILE *fp = fopen("no/such/dir/file.txt", "r");
    if (fp == NULL) {
        int err = errno;                            /* save it at once */
        perror("fopen");
        printf("errno = %d (%s)\n", err, err == ENOENT ? "ENOENT" : "other");
        fprintf(stderr, "could not open config: %s\n", strerror(err));
    }
    FILE *dir = fopen("/", "w");                     /* a directory */
    if (dir == NULL) printf("open / for writing: %s\n", strerror(errno));
    for (int e = 1; e <= 3; e++) printf("strerror(%d) = %s\n", e, strerror(e));
    return 0;
}
```

OUTPUT

```
fopen: No such file or directory
errno = 2 (ENOENT)
could not open config: No such file or directory
open / for writing: Is a directory
strerror(1) = Operation not permitted
strerror(2) = No such file or directory
strerror(3) = No such process
```

errno	Name	Typical cause
2	ENOENT	file or directory does not exist
13	EACCES	permission denied
17	EEXIST	file exists (mode "wx")
21	EISDIR	is a directory
24	EMFILE	too many open files
28	ENOSPC	no space left on device

16.7 File operations

16.7.1 remove() and rename()

`remove(path)` deletes a file; `rename(old, new)` renames or moves it (within the same file system). Both return 0 on success. A common safe-save pattern writes to a temporary name and renames it over the real file only when writing succeeded, so a crash never leaves a half-written file.

SAFE SAVE: WRITE, THEN RENAME main.c

```c
#include <stdio.h>

int main(void) {
    FILE *fp = fopen("report.tmp", "w");
    if (!fp) return 1;
    fprintf(fp, "quarterly report\n");
    if (fclose(fp) != 0) { remove("report.tmp"); return 1; }

    if (rename("report.tmp", "report.txt") == 0) printf("renamed report.tmp -> report.txt\n");
    if (remove("report.tmp") != 0) perror("remove report.tmp");   /* already gone */
    if (remove("report.txt") == 0) printf("deleted report.txt\n");
    return 0;
}
```

OUTPUT

```
renamed report.tmp -> report.txt
remove report.tmp: No such file or directory
deleted report.txt
```

16.7.2 Temporary files with tmpfile()

`tmpfile()` creates and opens a unique temporary binary file in "w+b" mode. It is deleted automatically when closed or when the program ends, so it is ideal for scratch data too large for memory.

A SCRATCH FILE THAT CLEANS ITSELF UP main.c

```c
#include <stdio.h>

int main(void) {
    FILE *tmp = tmpfile();
    if (!tmp) { perror("tmpfile"); return 1; }
    for (int i = 1; i <= 5; i++) fprintf(tmp, "%d\n", i * i);
    rewind(tmp);
    int x, sum = 0;
    while (fscanf(tmp, "%d", &x) == 1) sum += x;
    printf("sum of squares from the temp file = %d\n", sum);
    fclose(tmp);                        /* file disappears here */
    return 0;
}
```

OUTPUT

```
sum of squares from the temp file = 55
```

16.7.3 Buffering and fflush()

Writes are collected in the stream's buffer and sent to the operating system in large chunks. `fflush(fp)` forces the buffer out now. You need it when another process (or a crash) must see the data immediately, before reading a stream you just wrote, and for prompts when stdout is not line buffered. `setvbuf` chooses the buffering mode.

Mode	Buffer is sent when	Typical stream
`_IOFBF` fully buffered	it is full, on fflush or fclose	files
`_IOLBF` line buffered	a newline is written, or it is full	stdout on a terminal
`_IONBF` unbuffered	immediately, every call	stderr

DATA SITS IN THE BUFFER UNTIL IT IS FLUSHED

main.c

```c
#include <stdio.h>

int main(void) {
    FILE *fp = fopen("log.txt", "w");
    if (!fp) return 1;
    fprintf(fp, "step 1 done\n");

    FILE *peek = fopen("log.txt", "r");          /* second, independent stream */
    char buf[64] = "";
    printf("before fflush, reader sees: \"%s\"\n", fgets(buf, sizeof buf, peek) ? buf : "");
    fflush(fp);                                   /* push the buffer to the file */
    clearerr(peek);
    if (fgets(buf, sizeof buf, peek)) printf("after fflush, reader sees : %s", buf);
    fclose(peek);
    fclose(fp);
    return 0;
}
```

OUTPUT

```
before fflush, reader sees: ""
after fflush, reader sees : step 1 done
```

16.8 File programs

16.8.1 Copy a file

A robust copy works on binary data in blocks, handles any file type and checks every step. Here source and destination come from the command line.

MYCP: RUN AS ./PROG NOTES.TXT BACKUP.TXT

main.c

```c
#include <stdio.h>

int main(int argc, char *argv[]) {
    if (argc != 3) { fprintf(stderr, "usage: %s SOURCE DEST\n", argv[0]); return 1; }
    FILE *in = fopen(argv[1], "rb");
    if (!in) { perror(argv[1]); return 1; }
    FILE *out = fopen(argv[2], "wb");
    if (!out) { perror(argv[2]); fclose(in); return 1; }

    char buf[4096];
    size_t n, total = 0;
    while ((n = fread(buf, 1, sizeof buf, in)) > 0) {
        if (fwrite(buf, 1, n, out) != n) { perror("write"); break; }
        total += n;
    }
    if (ferror(in)) perror("read");
    fclose(in);
    if (fclose(out) != 0) { perror("close"); return 1; }
    printf("copied %zu bytes from %s to %s\n", total, argv[1], argv[2]);
    return 0;
}
```

OUTPUT

```
copied 76 bytes from notes.txt to backup.txt
```

16.8.2 Count characters, words and lines

A word starts whenever a non-space character follows a space (or the start of the file). This is the logic of the Unix `wc` command.

A WORD COUNTER LIKE WC main.c

```c
#include <stdio.h>
#include <ctype.h>

int main(void) {
    FILE *fp = fopen("story.txt", "r");
    if (!fp) { perror("story.txt"); return 1; }
    long chars = 0, words = 0, lines = 0;
    int c, in_word = 0;
    while ((c = fgetc(fp)) != EOF) {
        chars++;
        if (c == '\n') lines++;
        if (isspace(c)) in_word = 0;
        else if (!in_word) { in_word = 1; words++; }
    }
    fclose(fp);
    printf("%ld lines, %ld words, %ld characters\n", lines, words, chars);
    return 0;
}
```

OUTPUT

```
4 lines, 12 words, 64 characters
```

16.8.3 Merge two files

Merging two files that are each sorted produces one sorted file by repeatedly taking the smaller of the two current values: the merge step of merge sort, applied to files.

MERGING TWO SORTED FILES main.c

```c
#include <stdio.h>

int main(void) {
    FILE *a = fopen("a.txt", "r"), *b = fopen("b.txt", "r"), *out = fopen("merged.txt", "w");
    if (!a || !b || !out) { perror("fopen"); return 1; }
    int x, y;
    int hx = fscanf(a, "%d", &x) == 1, hy = fscanf(b, "%d", &y) == 1;
    while (hx || hy) {
        if (hx && (!hy || x <= y)) { fprintf(out, "%d\n", x); hx = fscanf(a, "%d", &x) == 1; }
        else                       { fprintf(out, "%d\n", y); hy = fscanf(b, "%d", &y) == 1; }
    }
    fclose(a); fclose(b); fclose(out);

    out = fopen("merged.txt", "r");
    printf("merged:");
    while (fscanf(out, "%d", &x) == 1) printf(" %d", x);
    printf("\n");
    fclose(out);
    return 0;
}
```

OUTPUT

```
merged: 1 3 8 9 10 15 16 42 50 77
```

16.8.4 Read and write CSV data

CSV (comma-separated values) is the simplest way to exchange tables with spreadsheets. Each line is a record and commas separate the fields. Read each line with fgets and split it with `sscanf` using the scanset `%[^,]` ("everything up to a comma"), or with strtok.

PROCESSING A CSV FILE main.c

```c
#include <stdio.h>
#include <string.h>

int main(void) {
    FILE *in = fopen("sales.csv", "r");
    FILE *out = fopen("summary.csv", "w");
    if (!in || !out) { perror("fopen"); return 1; }
    char line[128], region[32], product[32];
    int units;
    double price, grand = 0;

    fgets(line, sizeof line, in);                       /* skip header */
    fprintf(out, "region,product,revenue\n");
    while (fgets(line, sizeof line, in)) {
        if (sscanf(line, "%31[^,],%31[^,],%d,%lf", region, product, &units, &price) != 4) {
            fprintf(stderr, "bad line: %s", line);
            continue;
        }
        double revenue = units * price;
        grand += revenue;
        fprintf(out, "%s,%s,%.2f\n", region, product, revenue);
    }
    fclose(in);
    fclose(out);

    out = fopen("summary.csv", "r");
    while (fgets(line, sizeof line, out)) fputs(line, stdout);
    fclose(out);
    printf("grand total: %.2f\n", grand);
    return 0;
}
```

OUTPUT

```
bad line: West,oops
region,product,revenue
North,Notebook,5460.00
South,Pen,3600.00
East,Bag,13485.00
North,Pen,960.00
grand total: 23505.00
```

NOTE

sales.csv contains a header line and five records, one of them deliberately broken (`West,oops`) to show the error check. Real CSV can contain quoted fields with commas inside; for that, use a proper CSV parser.

Open with fopen and check for NULL, choose text or binary mode on purpose, test the return value of every read instead of feof, use fseek/ftell for random access, report failures with perror or strerror, and always fclose.

PRACTICE CHECKPOINT

Extend the **Student Record system** from Module 14 to **save records to a binary file**, **load them on startup**, and **update one record using fseek**.

SAMPLE SOLUTION

main.c

```c
#include <stdio.h>
#include <string.h>

#define DB "students.dat"

typedef struct {
    int roll;
    char name[24];
    float marks;
    char city[20];
} Student;

int count_records(FILE *fp) {
    fseek(fp, 0, SEEK_END);
    long size = ftell(fp);
    rewind(fp);
    return (int)(size / (long)sizeof(Student));
}
void list_all(FILE *fp) {
    Student s;
    rewind(fp);
    printf("Roll  Name      Marks  City\n");
    while (fread(&s, sizeof s, 1, fp) == 1)
        printf("%-5d %-9s %5.1f  %s\n", s.roll, s.name, s.marks, s.city);
}
int update_marks(FILE *fp, int roll, float new_marks) {
    Student s;
    rewind(fp);
    for (long i = 0; fread(&s, sizeof s, 1, fp) == 1; i++) {
        if (s.roll == roll) {
            s.marks = new_marks;
            fseek(fp, i * (long)sizeof s, SEEK_SET);     /* jump to record i */
            fwrite(&s, sizeof s, 1, fp);
            fflush(fp);
            return 1;
        }
    }
    return 0;
}
int main(void) {
    /* 1. first run: create the file with some records */
    FILE *fp = fopen(DB, "wb");
    if (!fp) { perror(DB); return 1; }
    Student seed[] = {{1, "Asha", 78.5f, "Mysuru"}, {2, "Ravi", 91.0f, "Chennai"},
                      {3, "Meera", 85.5f, "Pune"}, {4, "Kabir", 66.0f, "Delhi"}};
    fwrite(seed, sizeof seed[0], 4, fp);
    fclose(fp);
    printf("saved 4 records to %s\n\n", DB);

    /* 2. next start-up: load and show */
    fp = fopen(DB, "r+b");
    if (!fp) { perror(DB); return 1; }
    printf("loaded %d records on startup\n", count_records(fp));
    list_all(fp);

    /* 3. update one record in place */
```

```
    int roll;
    float marks;
    printf("\nRoll number to update and new marks: ");
    if (scanf("%d %f", &roll, &marks) == 2) {
        if (update_marks(fp, roll, marks)) printf("record %d updated in place\n\n", roll);
        else printf("roll %d not found\n\n", roll);
    }
    list_all(fp);
    fclose(fp);
    return 0;
}
```

CONSOLE (user input in gold)

```
saved 4 records to students.dat

loaded 4 records on startup
Roll  Name      Marks  City
1     Asha       78.5  Mysuru
2     Ravi       91.0  Chennai
3     Meera      85.5  Pune
4     Kabir      66.0  Delhi

Roll number to update and new marks: 4 72.5
record 4 updated in place

Roll  Name      Marks  City
1     Asha       78.5  Mysuru
2     Ravi       91.0  Chennai
3     Meera      85.5  Pune
4     Kabir      72.5  Delhi
```

LEVEL 3

Advanced level

Outcome: bit manipulation, modern C features, data structures, build tools, debugging and Linux systems programming.

7 MODULES · 32 DAYS MINIMUM

Module 17 · Bit manipulation	**2 days**
Module 18 · Advanced C language features	**3 days**
Module 19 · Data structures in C	**10 days**
Module 20 · Multi-file programs, libraries and build tools	**3 days**
Module 21 · Debugging, testing and memory safety	**3 days**
Module 22 · Systems programming in C (Linux)	**6 days**
Module 23 · Projects, interview preparation and next steps	**5 days**

MODULE 17 · ADVANCED

Bit manipulation

Minimum duration **2 days** | Level **Advanced**

Every value in a computer is a pattern of bits. Working directly on those bits is essential in embedded systems (hardware registers), networking (protocol headers), graphics, compression and cryptography, and it is a favourite topic in programming interviews.

IN THIS MODULE

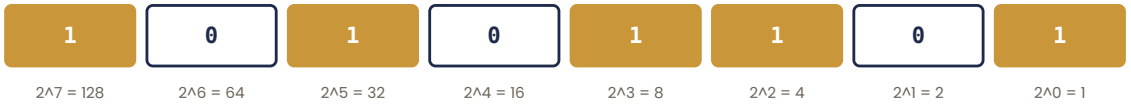
17.1 Number systems
17.2 Bitwise operators in depth
17.3 Bit masks
17.4 Bit tricks
17.5 Endianness

17.1 Number systems

17.1.1 Binary, octal, decimal and hexadecimal

A number system with **base b** uses digits 0 to b-1, and each position is worth b times the position to its right. Programmers use four bases:

System	Base	Digits	C literal	173 written in it
Binary	2	0 1	`0b10101101` (C23, GCC extension before)	10101101
Octal	8	0 to 7	`0255` (leading zero)	255
Decimal	10	0 to 9	`173`	173
Hexadecimal	16	0 to 9, A to F	`0xAD`	AD



`128 + 32 + 8 + 4 + 1 = 173`

PLACE VALUES OF THE BINARY NUMBER 10101101

Hexadecimal is popular because one hex digit is exactly **four bits** (a nibble), so a byte is always two hex digits: `1010 1101` = `A D`. Octal digits are three bits each and survive mainly in Unix file permissions (`chmod 0755`).

WATCH OUT

A leading zero makes a literal octal: `int x = 010;` is 8, not 10. Never pad decimal numbers with zeros in C source.

17.1.2 Conversions between them

Decimal to base b: divide by b repeatedly and collect the remainders; read them from last to first. **Base b to decimal**: multiply each digit by its place value and add. **Binary ↔ hex**: group bits in fours from the right and translate each group.

173 ÷ 2	quotient	remainder
173	86	1
86	43	0
43	21	1
21	10	1
10	5	0
5	2	1
2	1	0
1	0	1 ← read upwards: 10101101

CONVERTING BETWEEN BASES main.c

```c
#include <stdio.h>
#include <stdlib.h>

void print_base(unsigned n, unsigned base) {
    const char *digits = "0123456789ABCDEF";
    char buf[33];
    int i = 32;
    buf[i] = '\0';
    do {
        buf[--i] = digits[n % base];
        n /= base;
    } while (n > 0);
    printf("%s", &buf[i]);
}
void print_bits8(unsigned char b) {
    for (int i = 7; i >= 0; i--) {
        putchar((b >> i) & 1 ? '1' : '0');
        if (i == 4) putchar(' ');
    }
}
int main(void) {
    unsigned n = 173;
    printf("decimal %u = binary ", n);  print_base(n, 2);
    printf(" = octal %o = hex %X\n", n, n);
    printf("as nibbles: "); print_bits8((unsigned char)n); printf("\n");
    printf("base 5: "); print_base(n, 5); printf("\n");
    printf("strtol(\"10101101\", 2) = %ld\n", strtol("10101101", NULL, 2));
    printf("strtol(\"ad\", 16)       = %ld\n", strtol("ad", NULL, 16));
    printf("literals: %d %d %d\n", 0255, 0xAD, 0b10101101);
    return 0;
}
```

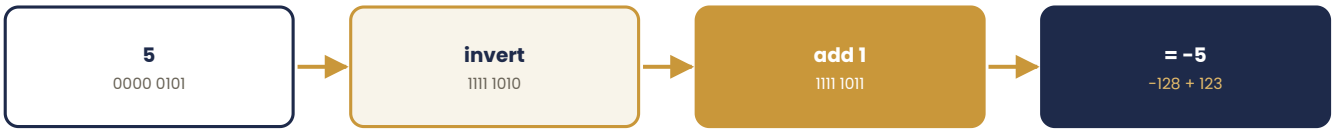
OUTPUT

```
decimal 173 = binary 10101101 = octal 255 = hex AD
as nibbles: 1010 1101
base 5: 1143
strtol("10101101", 2) = 173
strtol("ad", 16)       = 173
literals: 173 173 173
```

17.1.3 Signed numbers and two's complement

Almost every computer (and, since C23, the C standard itself) stores signed integers in **two's complement**. In an n-bit number the highest bit has the place value $\mathbf{-2^{n-1}}$ instead of $+2^{n-1}$. To negate a number: **invert all bits and add 1**.

8-bit pattern	unsigned	signed (two's complement)
0000 0000	0	0
0000 0101	5	5
0111 1111	127	127 (largest)
1000 0000	128	-128 (smallest)
1111 1011	251	-5
1111 1111	255	-1



NEGATING 5 IN TWO'S COMPLEMENT

TWO'S COMPLEMENT PATTERNS

main.c

```c
#include <stdio.h>
#include <stdint.h>

void bits(int8_t v) {
    uint8_t u = (uint8_t)v;
    for (int i = 7; i >= 0; i--) putchar((u >> i) & 1 ? '1' : '0');
}
int main(void) {
    int8_t vals[] = {5, -5, 127, -128, -1};
    for (int i = 0; i < 5; i++) {
        printf("%5d = ", vals[i]); bits(vals[i]);
        printf(" (as unsigned %u)\n", (uint8_t)vals[i]);
    }
    int8_t x = 5;
    printf("~x + 1 = %d\n", (int8_t)(~x + 1));
    return 0;
}
```

OUTPUT

```
    5 = 00000101 (as unsigned 5)
   -5 = 11111011 (as unsigned 251)
  127 = 01111111 (as unsigned 127)
 -128 = 10000000 (as unsigned 128)
   -1 = 11111111 (as unsigned 255)
~x + 1 = -5
```

The beauty of two's complement is that addition and subtraction use exactly the same circuit for signed and unsigned numbers, and there is only one zero.

17.2 Bitwise operators in depth

17.2.1 AND, OR, XOR, NOT

a	b	a & b	a | b	a ^ b	~a
0	0	0	0	0	1
0	1	0	1	1	1
1	0	0	1	1	0
1	1	1	1	0	0

- **AND &**: 1 only where both are 1. Used to **keep** selected bits and clear the rest (masking).
- **OR |**: 1 where either is 1. Used to **set** bits.
- **XOR ^**: 1 where they differ. Used to **toggle** bits; `x ^ x == 0` and `x ^ 0 == x`.
- **NOT ~**: flips every bit. `~0u` is all ones.

THE FOUR BITWISE OPERATORS main.c

```c
#include <stdio.h>

void show(const char *label, unsigned char v) {
    printf("%-8s ", label);
    for (int i = 7; i >= 0; i--) putchar((v >> i) & 1 ? '1' : '0');
    printf("  (%3u, 0x%02X)\n", v, v);
}
int main(void) {
    unsigned char a = 0xCA, b = 0x5F;        /* 1100 1010, 0101 1111 */
    show("a", a);
    show("b", b);
    show("a & b", a & b);
    show("a | b", a | b);
    show("a ^ b", a ^ b);
    show("~a", (unsigned char)~a);
    return 0;
}
```

OUTPUT

```
a        11001010  (202, 0xCA)
b        01011111  ( 95, 0x5F)
a & b    01001010  ( 74, 0x4A)
a | b    11011111  (223, 0xDF)
a ^ b    10010101  (149, 0x95)
~a       00110101  ( 53, 0x35)
```

NOTE

Do not confuse the bitwise operators with the logical ones: `6 & 1` is 0 but `6 && 1` is 1. Bitwise operators work on every bit and never short-circuit.

17.2.2 Left and right shifts

`x << n` moves every bit n places to the left, filling with zeros: it multiplies by 2^n. `x >> n` moves bits to the right, discarding the low bits: for unsigned values it divides by 2^n, rounding down.



SHIFTING LEFT MULTIPLIES, SHIFTING RIGHT DIVIDES

SHIFTS AS FAST MULTIPLICATION AND DIVISION main.c

```c
#include <stdio.h>

int main(void) {
    unsigned x = 13;
    for (int n = 0; n <= 3; n++)
        printf("13 << %d = %3u    13 >> %d = %u\n", n, x << n, n, x >> n);
    printf("1u << 10 = %u (2 to the power 10)\n", 1u << 10);
    printf("1000 / 8 via shift = %u\n", 1000u >> 3);
    return 0;
}
```

OUTPUT

```
13 << 0 =  13    13 >> 0 = 13
13 << 1 =  26    13 >> 1 = 6
13 << 2 =  52    13 >> 2 = 3
13 << 3 = 104    13 >> 3 = 1
1u << 10 = 1024 (2 to the power 10)
1000 / 8 via shift = 125
```

17.2.3 Arithmetic vs logical right shift

A **logical** right shift fills the vacated high bits with 0. An **arithmetic** right shift copies the sign bit, so negative numbers stay negative. In C, right-shifting an **unsigned** value is always logical. Right-shifting a **negative signed** value is implementation-defined; GCC, Clang and MSVC all do an arithmetic shift, which rounds towards minus infinity (unlike division, which rounds towards zero).

TWO KINDS OF RIGHT SHIFT main.c

```c
#include <stdio.h>
#include <stdint.h>

int main(void) {
    int32_t  s = -16;
    uint32_t u = (uint32_t)s;            /* same bits: 0xFFFFFFF0 */
    printf("signed   -16 >> 2 = %d (arithmetic: sign bit copied)\n", s >> 2);
    printf("unsigned 0x%08X >> 2 = 0x%08X (logical: zeros in)\n", u, u >> 2);
    printf("-7 >> 1 = %d but -7 / 2 = %d\n", -7 >> 1, -7 / 2);
    return 0;
}
```

OUTPUT

```
signed   -16 >> 2 = -4 (arithmetic: sign bit copied)
unsigned 0xFFFFFFF0 >> 2 = 0x3FFFFFFC (logical: zeros in)
-7 >> 1 = -4 but -7 / 2 = -3
```

TIP

Do bit manipulation on **unsigned** types (`unsigned` , `uint8_t` , `uint32_t`). Their behaviour is fully defined, while signed shifts and overflow are full of implementation-defined and undefined corners.

17.2.4 Undefined behaviour with shifts

The following shifts are **undefined behaviour**: the program may print anything, and optimisers assume they never happen.

- Shifting by a **negative** amount: `x << -1`.

- Shifting by **at least the width** of the (promoted) type: `1u << 32` for a 32-bit unsigned. On x86 the hardware uses only the low 5 bits of the count, so it often behaves like `1u << 0`.
- Left-shifting a **negative** signed value, or shifting a 1 into or past the sign bit of a signed value: `1 << 31` with 32-bit int (use `1u << 31`).

SHIFTING BY THE FULL WIDTH IS UNDEFINED main.c

```c
#include <stdio.h>

int main(void) {
    volatile unsigned n = 32;           /* volatile: stop the compiler folding it */
    unsigned one = 1;
    printf("1u << 31 = %u (fine)\n", one << 31);
    printf("1u << n (n = 32) = %u  <- undefined; x86 masks the count to 0\n", one << n);
    printf("safe: n < 32 ? 1u << n : 0 = %u\n", n < 32 ? one << n : 0);
    return 0;
}
```

OUTPUT (undefined behaviour; typical x86-64 result)

```
1u << 31 = 2147483648 (fine)
1u << n (n = 32) = 1  <- undefined; x86 masks the count to 0
safe: n < 32 ? 1u << n : 0 = 0
```

Compile with `-fsanitize=undefined` during development and such shifts are reported at run time with the file and line.

17.3 Bit masks

A **mask** is a value with 1s in the bit positions you want to work on. The mask for bit n is `1u << n`. The four basic operations each combine the value with a mask using one operator:

Operation	Expression	Why it works
Set bit n	`x |= 1u << n`	OR with 1 forces 1; OR with 0 keeps
Clear bit n	`x &= ~(1u << n)`	AND with 0 forces 0; AND with 1 keeps
Toggle bit n	`x ^= 1u << n`	XOR with 1 flips; XOR with 0 keeps
Check bit n	`(x >> n) & 1u` or `x & (1u << n)`	isolates the one bit

17.3.1 Set a bit

`x |= 1u << n` turns bit n on and leaves every other bit unchanged, whether bit n was 0 or already 1. Example: `0010 0101 | 1000 0000 = 1010 0101`.

17.3.2 Clear a bit

`x &= ~(1u << n)`: the inverted mask has a 0 only at position n, so AND forces that bit to 0 and keeps the rest. Example: `1010 0101 & 1111 1011 = 1010 0001`.

17.3.3 Toggle a bit

`x ^= 1u << n` flips bit n: 0 becomes 1 and 1 becomes 0. Toggling twice restores the original value.

17.3.4 Check a bit

`(x >> n) & 1u` moves bit n to position 0 and isolates it, giving exactly 0 or 1. `x & (1u << n)` gives 0 or the mask value, which is fine inside an `if`.

THE FOUR MASK OPERATIONS main.c

```c
#include <stdio.h>

void bits(unsigned char v) { for (int i = 7; i >= 0; i--) putchar((v >> i) & 1 ? '1' : '0'); }

unsigned char set_bit(unsigned char x, int n)    { return x | (1u << n); }
unsigned char clear_bit(unsigned char x, int n)  { return x & ~(1u << n); }
unsigned char toggle_bit(unsigned char x, int n) { return x ^ (1u << n); }
int           check_bit(unsigned char x, int n)  { return (x >> n) & 1u; }

int main(void) {
    unsigned char x = 0x25;                /* 0010 0101 */
    printf("start        : "); bits(x); printf("\n");
    x = set_bit(x, 7);    printf("set bit 7    : "); bits(x); printf("\n");
    x = clear_bit(x, 2);  printf("clear bit 2  : "); bits(x); printf("\n");
    x = toggle_bit(x, 0); printf("toggle bit 0 : "); bits(x); printf("\n");
    x = toggle_bit(x, 0); printf("toggle again : "); bits(x); printf("\n");
    printf("bit 5 is %d, bit 4 is %d\n", check_bit(x, 5), check_bit(x, 4));
    return 0;
}
```

OUTPUT

```
start        : 00100101
set bit 7    : 10100101
clear bit 2  : 10100001
toggle bit 0 : 10100000
toggle again : 10100001
bit 5 is 1, bit 4 is 0
```

Masks with several bits work the same way. For example `x & 0x0F` keeps the low nibble, `(x >> 4) & 0x0F` extracts the high nibble, and a field of width w at position p is read with `(x >> p) & ((1u << w) - 1)`.

EXTRACTING AND PACKING FIELDS OF AN RGB COLOUR main.c

```c
#include <stdio.h>
#include <stdint.h>

int main(void) {
    uint32_t colour = 0xC9973A;                /* LBF gold, 0xRRGGBB */
    unsigned r = (colour >> 16) & 0xFF;
    unsigned g = (colour >> 8)  & 0xFF;
    unsigned b =  colour        & 0xFF;
    printf("r = %u, g = %u, b = %u\n", r, g, b);
    uint32_t darker = ((r / 2) << 16) | ((g / 2) << 8) | (b / 2);   /* pack again */
    printf("half brightness = 0x%06X\n", darker);
    return 0;
}
```

OUTPUT

```
r = 201, g = 151, b = 58
half brightness = 0x644B1D
```

17.3.5 Flags packed in an integer

Many APIs pass a set of yes/no options as one integer, each option a power of two. Options are combined with |, tested with &, and removed with & ~. Unix file permissions and the `open()` flags in Module 22 work exactly this way.

PERMISSION FLAGS main.c

```c
#include <stdio.h>

enum {
    PERM_READ    = 1u << 0,      /* 0x01 */
    PERM_WRITE   = 1u << 1,      /* 0x02 */
    PERM_EXECUTE = 1u << 2,      /* 0x04 */
    PERM_ADMIN   = 1u << 3,      /* 0x08 */
};

void describe(const char *who, unsigned p) {
    printf("%-6s 0x%02X: %c%c%c%c\n", who, p,
           p & PERM_READ ? 'r' : '-', p & PERM_WRITE ? 'w' : '-',
           p & PERM_EXECUTE ? 'x' : '-', p & PERM_ADMIN ? 'A' : '-');
}
int main(void) {
    unsigned guest = PERM_READ;
    unsigned staff = PERM_READ | PERM_WRITE | PERM_EXECUTE;
    unsigned admin = staff | PERM_ADMIN;
    describe("guest", guest);
    describe("staff", staff);
    describe("admin", admin);
    staff &= ~PERM_EXECUTE;                     /* revoke execute */
    describe("staff", staff);
    if ((admin & (PERM_READ | PERM_WRITE)) == (PERM_READ | PERM_WRITE))
        printf("admin can read AND write\n");
    return 0;
}
```

OUTPUT

```
guest  0x01: r---
staff  0x07: rwx-
admin  0x0F: rwxA
staff  0x03: rw--
admin can read AND write
```

17.4 Bit tricks

17.4.1 Check even/odd

The lowest bit of an integer is 1 exactly when the number is odd: `n & 1`. (For negative two's complement numbers this still works.)

17.4.2 Check power of two

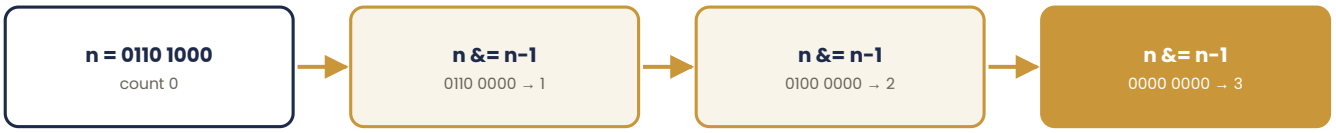
A power of two has exactly one bit set. Subtracting 1 turns that bit off and every bit below it on, so `n & (n - 1)` is zero only for powers of two (and for 0, which must be excluded).

n	binary	n - 1	n & (n - 1)
16	1 0000	0 1111	0 → power of two

n	binary	n - 1	n & (n - 1)
24	1 1000	1 0111	1 0000 → not

17.4.3 Count set bits (Brian Kernighan's algorithm)

`n &= n - 1` clears the **lowest** set bit. Repeating until n is 0 counts the 1 bits in as many steps as there are 1s, rather than one step per bit position.



EACH STEP REMOVES THE LOWEST 1 BIT

17.4.4 Swap without a temporary

Three XORs swap two variables: `a ^= b; b ^= a; a ^= b;`. It is a classic puzzle, but in real code use a temporary: it is clearer, just as fast, and the XOR version breaks if a and b are the same object (it zeroes it).

17.4.5 Find the unique element with XOR

Since `x ^ x == 0` and XOR is order-independent, XOR-ing all elements of an array in which every value appears twice except one leaves exactly that one value. O(n) time, O(1) memory.

17.4.6 Reverse bits

Take bits off the right end of n one by one and push them onto the right end of the result.

CLASSIC BIT TRICKS

main.c

```c
#include <stdio.h>
#include <stdint.h>

int is_odd(int n)              { return n & 1; }
int is_power_of_two(unsigned n){ return n != 0 && (n & (n - 1)) == 0; }
int count_bits(unsigned n) {
    int c = 0;
    while (n) { n &= n - 1; c++; }       /* Kernighan */
    return c;
}
int unique(const int *a, int n) {
    int x = 0;
    for (int i = 0; i < n; i++) x ^= a[i];
    return x;
}
uint8_t reverse8(uint8_t n) {
    uint8_t r = 0;
    for (int i = 0; i < 8; i++) { r = (uint8_t)((r << 1) | (n & 1)); n >>= 1; }
    return r;
}
int main(void) {
    printf("7 odd? %d   10 odd? %d\n", is_odd(7), is_odd(10));
    printf("powers of two up to 70:");
    for (unsigned n = 0; n <= 70; n++) if (is_power_of_two(n)) printf(" %u", n);
    printf("\nbits set in 0x68 = %d, in 255 = %d, in 0xFFFFFFFF = %d\n",
           count_bits(0x68), count_bits(255), count_bits(0xFFFFFFFFu));
    printf("builtin popcount(0x68) = %d\n", __builtin_popcount(0x68));

    int a = 25, b = 40;
    a ^= b; b ^= a; a ^= b;
    printf("after XOR swap: a = %d, b = %d\n", a, b);

    int v[] = {4, 9, 2, 7, 9, 4, 2};
    printf("unique element = %d\n", unique(v, 7));
    printf("reverse8(0b00010110) = 0x%02X (0b01101000)\n", reverse8(0x16));
    return 0;
}
```

OUTPUT

```
7 odd? 1   10 odd? 0
powers of two up to 70: 1 2 4 8 16 32 64
bits set in 0x68 = 3, in 255 = 8, in 0xFFFFFFFF = 32
builtin popcount(0x68) = 3
after XOR swap: a = 40, b = 25
unique element = 7
reverse8(0b00010110) = 0x68 (0b01101000)
```

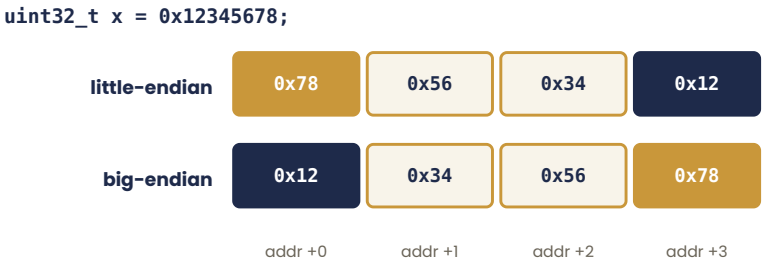
NOTE

GCC and Clang provide `__builtin_popcount`, `__builtin_clz` (count leading zeros) and `__builtin_ctz` (count trailing zeros), which compile to single CPU instructions. C23 standardizes them in `<stdbit.h>` as `stdc_count_ones`, `stdc_leading_zeros` and friends.

17.5 Endianness

17.5.1 Little-endian vs big-endian

A multi-byte value must be stored in several consecutive bytes. **Endianness** is the order. **Little-endian** machines (x86, x86-64, almost all ARM in practice) store the **least** significant byte at the lowest address. **Big-endian** ones (some older and network hardware) store the most significant byte first. Network protocols use big-endian, which is therefore also called **network byte order**.



THE SAME 32-BIT VALUE IN MEMORY ON THE TWO KINDS OF MACHINE

17.5.2 Detecting endianness in a program

INSPECTING BYTE ORDER

main.c

```c
#include <stdio.h>
#include <stdint.h>

int main(void) {
    uint32_t x = 0x12345678;
    unsigned char *p = (unsigned char *)&x;        /* view the bytes */
    printf("bytes in memory:");
    for (int i = 0; i < 4; i++) printf(" %02X", p[i]);
    printf("\n");
    printf("this machine is %s-endian\n", p[0] == 0x78 ? "little" : "big");

    union { uint16_t v; uint8_t b[2]; } probe = {.v = 1};   /* union method */
    printf("union probe says: %s-endian\n", probe.b[0] ? "little" : "big");
    return 0;
}
```

OUTPUT

```
bytes in memory: 78 56 34 12
this machine is little-endian
union probe says: little-endian
```

17.5.3 Byte swapping

When reading binary data written on a machine with the other byte order (or a network packet), the bytes must be reversed. Write it portably with shifts and masks, or use the optimized helpers: `htonl` / `ntohl` (host to network long and back) from `<arpa/inet.h>`, or GCC's `__builtin_bswap32`.

SWAPPING BYTE ORDER main.c

```c
#include <stdio.h>
#include <stdint.h>
#include <arpa/inet.h>

uint32_t swap32(uint32_t x) {
    return ((x & 0x000000FFu) << 24) |
           ((x & 0x0000FF00u) << 8)  |
           ((x & 0x00FF0000u) >> 8)  |
           ((x & 0xFF000000u) >> 24);
}
uint16_t swap16(uint16_t x) { return (uint16_t)((x << 8) | (x >> 8)); }

int main(void) {
    uint32_t v = 0x12345678;
    printf("swap32(0x%08X)  = 0x%08X\n", v, swap32(v));
    printf("bswap32 builtin = 0x%08X\n", __builtin_bswap32(v));
    printf("htonl           = 0x%08X (network order on this little-endian host)\n", htonl(v));
    printf("swap16(0x1F90)  = 0x%04X; port 8080 = 0x1F90 is sent as bytes 1F 90\n", swap16(0x1F90));
    printf("htons(8080) as stored: 0x%04X\n", htons(8080));
    return 0;
}
```

OUTPUT

```
swap32(0x12345678)  = 0x78563412
bswap32 builtin = 0x78563412
htonl           = 0x78563412 (network order on this little-endian host)
swap16(0x1F90)  = 0x901F; port 8080 = 0x1F90 is sent as bytes 1F 90
htons(8080) as stored: 0x901F
```

Use unsigned types for bit work. OR sets, AND with an inverted mask clears, XOR toggles, AND with a mask tests. n & (n - 1) clears the lowest set bit. Never shift by the type width or more, and remember that byte order matters whenever binary data leaves your machine.

PRACTICE CHECKPOINT

Store **8 on/off settings in one byte** with **set, clear, toggle and check** functions, and write a function that **counts the set bits**.

SAMPLE SOLUTION main.c

```c
#include <stdio.h>
#include <stdint.h>

enum Setting { WIFI, BLUETOOTH, GPS, DARK_MODE, SOUND, VIBRATE, AUTO_SYNC, AIRPLANE, SETTING_COUNT };
static const char *names[SETTING_COUNT] = {"WiFi", "Bluetooth", "GPS", "Dark mode",
                                           "Sound", "Vibrate", "Auto sync", "Airplane"};

void set_on(uint8_t *s, enum Setting k)     { *s |= (uint8_t)(1u << k); }
void set_off(uint8_t *s, enum Setting k)    { *s &= (uint8_t)~(1u << k); }
void toggle(uint8_t *s, enum Setting k)     { *s ^= (uint8_t)(1u << k); }
int  is_on(uint8_t s, enum Setting k)       { return (s >> k) & 1u; }
int  count_on(uint8_t s) {
    int c = 0;
    while (s) { s &= (uint8_t)(s - 1); c++; }
    return c;
}
void show(uint8_t s) {
```

```
    printf("settings byte = 0x%02X = ", s);
    for (int i = 7; i >= 0; i--) putchar(is_on(s, i) ? '1' : '0');
    printf("  (%d on)\n", count_on(s));
    for (int k = 0; k < SETTING_COUNT; k++)
        if (is_on(s, k)) printf("   [on] %s\n", names[k]);
}
int main(void) {
    uint8_t s = 0;
    set_on(&s, WIFI);
    set_on(&s, SOUND);
    set_on(&s, AUTO_SYNC);
    show(s);
    int cmd, k;
    printf("commands: 1 k = on, 2 k = off, 3 k = toggle, 0 = quit\n");
    while (printf("> "), scanf("%d", &cmd) == 1 && cmd != 0) {
        if (scanf("%d", &k) != 1 || k < 0 || k >= SETTING_COUNT) { printf("bad setting\n"); continue;
}
        if (cmd == 1) set_on(&s, k);
        else if (cmd == 2) set_off(&s, k);
        else if (cmd == 3) toggle(&s, k);
        printf("%s is now %s\n", names[k], is_on(s, k) ? "ON" : "OFF");
    }
    show(s);
    return 0;
}
```

CONSOLE (user input in gold)

```
settings byte = 0x51 = 01010001  (3 on)
   [on] WiFi
   [on] Sound
   [on] Auto sync
commands: 1 k = on, 2 k = off, 3 k = toggle, 0 = quit
> 3 7
Airplane is now ON
> 2 4
Sound is now OFF
> 1 3
Dark mode is now ON
> 3 0
WiFi is now OFF
> 0
settings byte = 0xC8 = 11001000  (3 on)
   [on] Dark mode
   [on] Auto sync
   [on] Airplane
```

MODULE 18 · ADVANCED

Advanced C language features

Minimum duration **3 days** Level **Advanced**

This module collects the qualifiers, techniques and rules that experienced C programmers rely on every day: what const, volatile, restrict and _Atomic really promise, functions with a variable number of arguments, generic code with void * and _Generic, the dark corners of undefined behaviour, error-handling patterns, the time and utility functions, and what each C standard added up to C23.

IN THIS MODULE

18.1 Type qualifiers
18.2 Variadic functions
18.3 Advanced function topics
18.4 Generic selection
18.5 Undefined, unspecified and implementation-defined behaviour
18.6 Error handling patterns in C
18.7 Time and utilities
18.8 Modern C standards

18.1 Type qualifiers

A **qualifier** adds a property to a type without changing its representation. C has four: `const`, `volatile`, `restrict` and `_Atomic`.

18.1.1 const in depth

`const` means "read-only through this name". The compiler rejects writes, which documents intent and catches mistakes. Key points beyond Module 12:

- A `const` object must be initialized, because it can never be assigned later.
- A `const int` in C is **not** a compile-time constant (unlike C++): before C23 it cannot size a file-scope array or be a `case` label. Use enum, #define or C23 `constexpr` for that.
- Converting `T *` to `const T *` is automatic; the reverse needs a cast and modifying a truly const object through it is undefined behaviour.
- String literals have type `char[]` in C for historical reasons but must be treated as const.
- Const-correct APIs: functions that only read through a pointer parameter should take `const T *`, so callers can pass both const and non-const data.

CONST-CORRECT CODE main.c

```c
#include <stdio.h>
#include <string.h>

size_t count_char(const char *s, char c) {     /* promises not to modify s */
    size_t n = 0;
    for (; *s; s++) if (*s == c) n++;
    return n;
}
int main(void) {
    const double TAX_RATE = 0.18;               /* must be initialized */
    char word[] = "bookkeeper";
    const char *lit = "committee";
    printf("e in %s: %zu\n", word, count_char(word, 'e'));   /* char * -> const char *: fine */
    printf("t in %s: %zu\n", lit, count_char(lit, 't'));
    printf("tax on 1000 = %.2f\n", 1000 * TAX_RATE);
    /* TAX_RATE = 0.2;         error: assignment of read-only variable */
    return 0;
}
```

OUTPUT

```
e in bookkeeper: 3
t in committee: 2
tax on 1000 = 180.00
```

18.1.2 volatile and hardware registers

`volatile` tells the compiler that an object can change (or that accessing it has effects) in ways the compiler cannot see: a memory-mapped hardware register, a variable changed by a signal handler, or memory shared with a device. Every read and write in the source then really happens, in order; the compiler may not cache the value in a register or remove "useless" accesses.

EMBEDDED EXAMPLE: POLLING A STATUS REGISTER C

```c
#define UART_STATUS (*(volatile uint32_t *)0x40011000u)   /* hardware register address */
#define TX_READY    (1u << 7)

while ((UART_STATUS & TX_READY) == 0)
    ;             /* without volatile the compiler could read once and loop forever */
```

VOLATILE WITH A SIGNAL HANDLER (LINUX)

main.c

```c
#include <stdio.h>
#include <signal.h>
#include <unistd.h>

static volatile sig_atomic_t stop = 0;      /* changed by the signal handler */

void on_alarm(int sig) { (void)sig; stop = 1; }

int main(void) {
    signal(SIGALRM, on_alarm);
    alarm(1);                               /* deliver SIGALRM after 1 second */
    unsigned long spins = 0;
    while (!stop) spins++;                  /* volatile forces a fresh read each time */
    printf("loop ended by the signal after %s spins\n", spins > 1000 ? "many" : "few");
    return 0;
}
```

OUTPUT

```
loop ended by the signal after many spins
```

WATCH OUT

volatile is **not** a tool for sharing data between threads: it gives no atomicity and no ordering between threads. Use `_Atomic` or mutexes (Module 22) for that.

18.1.3 restrict (C99)

`restrict` on a pointer parameter is a **promise** by the programmer: while this pointer is in use, the object it points to is accessed only through it (no other pointer **aliases** it). The compiler can then keep values in registers and vectorize loops. The standard library uses it, for example `void *memcpy(void *restrict dst, const void *restrict src, size_t n)`: overlapping buffers are not allowed (use memmove for those). Breaking the promise is undefined behaviour.

RESTRICT-QUALIFIED PARAMETERS

main.c

```c
#include <stdio.h>

/* Without restrict, the compiler must assume out may overlap a or b,
   and reload a[i] and b[i] after every store. */
void add_arrays(int n, int *restrict out, const int *restrict a, const int *restrict b) {
    for (int i = 0; i < n; i++) out[i] = a[i] + b[i];
}
int main(void) {
    int a[] = {1, 2, 3, 4}, b[] = {10, 20, 30, 40}, out[4];
    add_arrays(4, out, a, b);                /* three separate arrays: promise kept */
    for (int i = 0; i < 4; i++) printf("%d ", out[i]);
    printf("\n");
    return 0;
}
```

OUTPUT

```
11 22 33 44
```

18.1.4 _Atomic (C11)

An `_Atomic` object (or one of the typedefs in `<stdatomic.h>` such as `atomic_int`) can be read and modified by several threads at once without a data race: operations like `++` become indivisible read-

modify-write instructions. The example starts four threads that each add 100000 to two counters, one plain and one atomic.

ATOMIC VS PLAIN COUNTER WITH 4 THREADS main.c

```c
#include <stdio.h>
#include <stdatomic.h>
#include <pthread.h>

int plain = 0;
atomic_int safe = 0;

void *worker(void *arg) {
    (void)arg;
    for (int i = 0; i < 100000; i++) {
        plain++;                /* data race: increments get lost */
        safe++;                 /* atomic increment */
    }
    return NULL;
}
int main(void) {
    pthread_t t[4];
    for (int i = 0; i < 4; i++) pthread_create(&t[i], NULL, worker, NULL);
    for (int i = 0; i < 4; i++) pthread_join(t[i], NULL);
    printf("expected     : 400000\n");
    printf("atomic_int   : %d\n", atomic_load(&safe));
    printf("plain int    : %s\n", plain == 400000 ? "400000 (lucky this time)" : "less than 400000
(lost updates)");
    return 0;
}
```

OUTPUT

```
expected     : 400000
atomic_int   : 400000
plain int    : less than 400000 (lost updates)
```

18.2 Variadic functions

printf accepts any number of arguments. You can write such **variadic** functions yourself. The parameter list ends with `...` after at least one named parameter, and the macros of `<stdarg.h>` walk through the extra arguments.

18.2.1 stdarg.h: va_list, va_start, va_arg, va_end

Macro	Purpose
`va_list ap;`	a cursor over the variable arguments
`va_start(ap, last)`	start after the last named parameter (C23: `va_start(ap)` also allowed)
`va_arg(ap, type)`	fetch the next argument, which you must say the type of
`va_copy(dst, src)`	duplicate a cursor to walk the arguments twice
`va_end(ap)`	finish; required before returning

The function cannot know by itself how many arguments were passed or their types. It needs a convention: a **count** parameter, a **sentinel** value at the end, or a **format string** as in printf. Arguments

undergo default promotions: char and short arrive as int, float arrives as double.

TWO WAYS TO KNOW WHERE THE ARGUMENTS END

main.c

```c
#include <stdio.h>
#include <stdarg.h>

int sum(int count, ...) {                       /* convention 1: a count */
    va_list ap;
    va_start(ap, count);
    int total = 0;
    for (int i = 0; i < count; i++) total += va_arg(ap, int);
    va_end(ap);
    return total;
}
double max_of(double first, ...) {               /* convention 2: sentinel -1 */
    va_list ap;
    va_start(ap, first);
    double best = first, x;
    while ((x = va_arg(ap, double)) != -1)
        if (x > best) best = x;
    va_end(ap);
    return best;
}
int main(void) {
    printf("sum(3, 10, 20, 30) = %d\n", sum(3, 10, 20, 30));
    printf("sum(0)             = %d\n", sum(0));
    printf("max_of(...)        = %.2f\n", max_of(4.5, 9.25, 1.0, 7.75, -1.0));
    return 0;
}
```

OUTPUT

```
sum(3, 10, 20, 30) = 60
sum(0)             = 0
max_of(...)        = 9.25
```

WATCH OUT

Passing the wrong type to va_arg (for example reading an int with `va_arg(ap, double)`) or reading past the last argument is undefined behaviour, and the compiler cannot check it. That is why `max_of(4.5, 9, -1)` with int literals would break.

18.2.2 Writing your own printf-style function

The `v` versions of the printf family (`vprintf`, `vfprintf`, `vsnprintf`) take a `va_list` instead of `...`, so you can forward your arguments to them. Add the GCC attribute `format(printf, ...)` and the compiler checks your calls just like printf calls.

A LOGGING FUNCTION AND A MINI PRINTF main.c

```c
#include <stdio.h>
#include <stdarg.h>
#include <time.h>

__attribute__((format(printf, 2, 3)))
void log_msg(const char *level, const char *fmt, ...) {
    va_list ap;
    va_start(ap, fmt);
    fprintf(stdout, "[%-5s] ", level);
    vfprintf(stdout, fmt, ap);              /* forward the arguments */
    fputc('\n', stdout);
    va_end(ap);
}
/* a tiny printf that understands %d, %s and %c */
void mini_printf(const char *fmt, ...) {
    va_list ap;
    va_start(ap, fmt);
    for (const char *p = fmt; *p; p++) {
        if (*p != '%') { putchar(*p); continue; }
        switch (*++p) {
        case 'd': printf("%d", va_arg(ap, int)); break;
        case 's': fputs(va_arg(ap, const char *), stdout); break;
        case 'c': putchar(va_arg(ap, int)); break;          /* char promoted to int */
        case '%': putchar('%'); break;
        default:  putchar('?');
        }
    }
    va_end(ap);
}
int main(void) {
    log_msg("INFO", "server listening on port %d", 8080);
    log_msg("WARN", "%s is %d%% full", "disk", 91);
    mini_printf("%s scored %d%% grade %c\n", "Asha", 93, 'A');
    return 0;
}
```

OUTPUT

```
[INFO ] server listening on port 8080
[WARN ] disk is 91% full
Asha scored 93% grade A
```

18.3 Advanced function topics

18.3.1 Callbacks and generic programming with void *

C has no templates. Generic code is written with `void *` (a pointer to "anything"), an **element size** and **callback functions** that know the real type. qsort is the standard example. Inside generic code, bytes are moved with memcpy, and pointer arithmetic is done on `char *` because arithmetic on `void *` is not allowed in standard C.

18.3.2 Writing a generic sort or search

A GENERIC SORT AND SEARCH main.c

```c
#include <stdio.h>
#include <string.h>
```

```c
typedef int (*CmpFn)(const void *, const void *);

/* generic insertion sort: works for any element type */
void gsort(void *base, size_t n, size_t size, CmpFn cmp) {
    char *a = base;
    char tmp[64];                                   /* elements up to 64 bytes */
    for (size_t i = 1; i < n; i++) {
        memcpy(tmp, a + i * size, size);
        size_t j = i;
        while (j > 0 && cmp(a + (j - 1) * size, tmp) > 0) {
            memcpy(a + j * size, a + (j - 1) * size, size);
            j--;
        }
        memcpy(a + j * size, tmp, size);
    }
}
/* generic linear search: returns pointer to the element or NULL */
void *gfind(const void *key, void *base, size_t n, size_t size, CmpFn cmp) {
    char *a = base;
    for (size_t i = 0; i < n; i++)
        if (cmp(key, a + i * size) == 0) return a + i * size;
    return NULL;
}
typedef struct { char name[16]; double price; } Item;

int cmp_int(const void *x, const void *y) { int a = *(const int *)x, b = *(const int *)y; return (a >
b) - (a < b); }
int cmp_price(const void *x, const void *y) {
    double a = ((const Item *)x)->price, b = ((const Item *)y)->price;
    return (a > b) - (a < b);
}
int cmp_name(const void *x, const void *y) { return strcmp(((const Item *)x)->name, ((const Item *)y)-
>name); }

int main(void) {
    int v[] = {42, 7, 19, 3, 88};
    gsort(v, 5, sizeof *v, cmp_int);
    for (int i = 0; i < 5; i++) printf("%d ", v[i]);
    printf("\n");

    Item shop[] = {{"pen", 12.5}, {"bag", 899}, {"book", 245}, {"clip", 2}};
    gsort(shop, 4, sizeof *shop, cmp_price);
    for (int i = 0; i < 4; i++) printf("%s(%.1f) ", shop[i].name, shop[i].price);
    printf("\n");
    Item key = {"book", 0};
    Item *found = gfind(&key, shop, 4, sizeof *shop, cmp_name);
    if (found) printf("found %s at index %td\n", found->name, found - shop);
    return 0;
}
```

OUTPUT

```
3 7 19 42 88
clip(2.0) pen(12.5) book(245.0) bag(899.0)
found book at index 2
```

18.3.3 _Noreturn / [[noreturn]]

A function that never returns to its caller (it always calls exit, abort or longjmp, or loops forever) can be marked `_Noreturn` (C11, or `noreturn` from `<stdnoreturn.h>`) or with the C23 attribute `[[noreturn]]`. The compiler stops warning about missing return values after calls to it and can optimize better.

A FUNCTION THAT NEVER RETURNS (C23 ATTRIBUTE SYNTAX) main.c

```c
#include <stdio.h>
#include <stdlib.h>

[[noreturn]] void fatal(const char *msg) {
    fprintf(stderr, "fatal: %s\n", msg);
    exit(EXIT_FAILURE);
}
int parse_percent(int v) {
    if (v >= 0 && v <= 100) return v;
    fatal("percentage out of range");      /* no "missing return" warning */
}
int main(void) {
    printf("%d%%\n", parse_percent(75));
    printf("%d%%\n", parse_percent(130));
    return 0;
}
```

OUTPUT

```
75%
fatal: percentage out of range
```

18.3.4 Compound literals (C99)

`(type){ initializers }` creates an unnamed object of that type on the spot. It is an lvalue with automatic storage (inside a function), so you can take its address. Useful to pass a structure or array to a function without declaring a variable.

COMPOUND LITERALS main.c

```c
#include <stdio.h>

typedef struct { int x, y; } Point;

int sum(const int *a, int n) { int s = 0; for (int i = 0; i < n; i++) s += a[i]; return s; }
void show(Point p) { printf("(%d, %d)\n", p.x, p.y); }

int main(void) {
    show((Point){3, 4});                                /* struct literal */
    show((Point){.y = 9});                              /* designated, x = 0 */
    printf("sum = %d\n", sum((int[]){5, 10, 15, 20}, 4));  /* array literal */
    Point *p = &(Point){1, 1};                          /* address of a literal */
    p->x = 100;
    show(*p);
    return 0;
}
```

OUTPUT

```
(3, 4)
(0, 9)
sum = 50
(100, 1)
```

18.4 Generic selection

18.4.1 _Generic (C11) for type-based macros

`_Generic(expr, type1: result1, type2: result2, ..., default: result)` chooses one result at **compile time** based on the type of expr (expr is not evaluated). Wrapped in a macro, it gives C a simple form of function overloading. The `<tgmath.h>` header uses it so that `sqrt(x)` calls sqrtf, sqrt or sqrtl depending on x.

TYPE-BASED DISPATCH WITH _GENERIC main.c

```c
#include <stdio.h>
#include <math.h>

#define TYPE_NAME(x) _Generic((x),          \
    int: "int", unsigned: "unsigned",        \
    long: "long", float: "float",            \
    double: "double", char: "char",          \
    char *: "char *", const char *: "const char *", \
    default: "something else")

void print_int(int v)          { printf("int: %d\n", v); }
void print_double(double v)    { printf("double: %.3f\n", v); }
void print_str(const char *v)  { printf("string: \"%s\"\n", v); }

#define PRINT(x) _Generic((x),              \
    int: print_int,                         \
    double: print_double, float: print_double, \
    char *: print_str, const char *: print_str)(x)

#define ABS(x) _Generic((x), int: abs_i, double: fabs, default: fabs)(x)
static int abs_i(int v) { return v < 0 ? -v : v; }

int main(void) {
    printf("%s, %s, %s, %s\n", TYPE_NAME(42), TYPE_NAME(4.2), TYPE_NAME('A'), TYPE_NAME("hi"));
    PRINT(7);
    PRINT(2.5);
    PRINT("generic");
    printf("ABS(-3) = %d, ABS(-2.5) = %.1f\n", ABS(-3), ABS(-2.5));
    return 0;
}
```

OUTPUT

```
int, double, int, char *
int: 7
double: 2.500
string: "generic"
ABS(-3) = 3, ABS(-2.5) = 2.5
```

NOTE

`TYPE_NAME('A')` prints **int**: in C a character constant such as `'A'` has type int, not char (it is char in C++).

18.5 Undefined, unspecified and implementation-defined behaviour

18.5.1 What each term means

Term	Meaning	Example
Implementation-defined	the compiler chooses and **documents** the behaviour	sizeof(int); right shift of a negative number; whether char is signed
Unspecified	one of several allowed behaviours, not necessarily documented or consistent	order in which function arguments are evaluated: f(a(), b())
Undefined (UB)	the standard imposes **no requirements**: anything may happen	signed overflow, out-of-bounds access, NULL dereference, use after free
Locale-specific	depends on the current locale	whether isalpha('é') is true

Undefined behaviour is not "the program crashes". The compiler optimizes on the assumption that UB **never happens**, so a program with UB may work in debug builds, fail in release builds, or behave impossibly. Correct C programs must contain no UB.

18.5.2 Common undefined behaviour

SIGNED OVERFLOW: THE OPTIMIZER REMOVES THE CHECK (BUILT WITH -O2)

main.c

```c
#include <stdio.h>
#include <limits.h>

int will_overflow(int x) {
    return x + 1 < x;        /* "true only on overflow"... but overflow is UB */
}
int main(void) {
    printf("will_overflow(INT_MAX) = %d\n", will_overflow(INT_MAX));
    return 0;
}
```

OUTPUT

```
will_overflow(INT_MAX) = 0
```

With optimization the compiler reasons: "x + 1 < x can only be true if signed overflow happens, which is undefined, so it is always false", and the function becomes `return 0;`. The correct test compares **before** adding: `x == INT_MAX` or `x > INT_MAX - 1`.

UB	Example	Safe alternative
Signed integer overflow	`INT_MAX + 1`	check limits first; use unsigned or wider types; `__builtin_add_overflow` / C23 `ckd_add`
Out-of-bounds access	`a[10]` for `int a[10]`	track sizes; loop with `i < n`
NULL dereference	`*p` when p is NULL	check the pointer
Uninitialized read	`int x; printf("%d", x);`	initialize at declaration
Use after free	`free(p); *p = 1;`	set p = NULL after free

UB	Example	Safe alternative
Modifying a string literal	`char *s = "hi"; s[0] = 'H';`	`char s[] = "hi";`
Shift too far	`1u << 32`	keep the count below the width
Unsequenced modifications	`i = i++ + 1;`	one modification per statement
Division by zero	`x / 0`, `INT_MIN / -1`	check the divisor

UNDEFINEDBEHAVIORSANITIZER AND ADDRESSSANITIZER CATCH UB AT RUN TIME main.c

```c
#include <stdio.h>
#include <limits.h>

int main(void) {
    int arr[4] = {1, 2, 3, 4};
    int i = 4;
    int x = INT_MAX;
    x = x + i;                          /* signed overflow */
    printf("x = %d\n", x);
    printf("arr[i] = %d\n", arr[i]);  /* out of bounds */
    return 0;
}
```

OUTPUT (sanitizer report, trimmed)

```
$ gcc -g -fsanitize=undefined,address main.c -o prog && ./prog
main.c:8:7: runtime error: signed integer overflow: 2147483647 + 4 cannot be represented in type 'int'
x = -2147483645
main.c:10:32: runtime error: index 4 out of bounds for type 'int [4]'
=================================================================
==7800==ERROR: AddressSanitizer: stack-buffer-overflow on address 0x7f776bf00030
READ of size 4 at 0x7f776bf00030 thread T0
    #0 0x5608f34345aa in main main.c:10
  [32, 48) 'arr' (line 5) <== Memory access at offset 48 overflows this variable
SUMMARY: AddressSanitizer: stack-buffer-overflow main.c:10 in main
```

18.5.3 Strict aliasing rule

The **strict aliasing rule** says an object may only be accessed through an lvalue of its own (compatible) type, a signed/unsigned variant, or a character type. Reading a `float` through an `int *` breaks the rule, and optimizers assume it never happens: they may reorder or cache the accesses. To reinterpret bytes legally, use `memcpy` (compilers turn it into a single move) or a union.

REINTERPRETING BITS THE LEGAL WAY

main.c

```c
#include <stdio.h>
#include <string.h>
#include <stdint.h>

int main(void) {
    float f = 3.5f;
    /* uint32_t bad = *(uint32_t *)&f;     violates strict aliasing */
    uint32_t bits;
    memcpy(&bits, &f, sizeof bits);         /* correct and just as fast */
    printf("bits of 3.5f = 0x%08X\n", bits);
    printf("sign %u, exponent %u, mantissa 0x%06X\n", bits >> 31, (bits >> 23) & 0xFF, bits &
0x7FFFFF);

    unsigned char *bytes = (unsigned char *)&f;    /* char types may alias anything */
    printf("bytes:");
    for (size_t i = 0; i < sizeof f; i++) printf(" %02X", bytes[i]);
    printf("\n");
    return 0;
}
```

OUTPUT

```
bits of 3.5f = 0x40600000
sign 0, exponent 128, mantissa 0x600000
bytes: 00 00 60 40
```

18.6 Error handling patterns in C

C has no exceptions. Errors are reported through return values and handled explicitly at every call. Three patterns cover almost all real code.

18.6.1 Return codes and errno

Functions return a status (0 / -1, NULL, a special value, or an enum of error codes) and deliver results through pointer parameters. Library functions additionally set `errno`. Callers must check every status.

STATUS CODES PLUS OUT-PARAMETERS main.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <errno.h>
#include <limits.h>

typedef enum { PARSE_OK, PARSE_EMPTY, PARSE_JUNK, PARSE_RANGE } ParseStatus;
static const char *status_text[] = {"ok", "empty input", "not a number", "out of int range"};

ParseStatus parse_int(const char *s, int *out) {
    if (s == NULL || *s == '\0') return PARSE_EMPTY;
    char *end;
    errno = 0;
    long v = strtol(s, &end, 10);
    if (*end != '\0') return PARSE_JUNK;
    if (errno == ERANGE || v < INT_MIN || v > INT_MAX) return PARSE_RANGE;
    *out = (int)v;                      /* result through a pointer */
    return PARSE_OK;
}
int main(void) {
    const char *inputs[] = {"42", "", "12abc", "99999999999", "-7"};
    for (int i = 0; i < 5; i++) {
        int v = 0;
        ParseStatus st = parse_int(inputs[i], &v);
        if (st == PARSE_OK) printf("%-12s -> %d\n", inputs[i], v);
        else printf("%-12s -> error: %s\n", inputs[i], status_text[st]);
    }
    return 0;
}
```

OUTPUT

```
42           -> 42
             -> error: empty input
12abc        -> error: not a number
99999999999  -> error: out of int range
-7           -> -7
```

18.6.2 goto cleanup pattern

When a function acquires several resources (memory, files, locks), every error path must release exactly those acquired so far. Jumping forward to cleanup labels in reverse order of acquisition keeps this correct and readable. It is the one widely accepted use of goto, used throughout the Linux kernel.

RELEASING RESOURCES IN REVERSE ORDER

main.c

```c
#include <stdio.h>
#include <stdlib.h>

int process(const char *in_path, const char *out_path) {
    int rc = -1;
    FILE *in = NULL, *out = NULL;
    char *buf = NULL;

    in = fopen(in_path, "r");
    if (!in) { perror(in_path); goto done; }
    out = fopen(out_path, "w");
    if (!out) { perror(out_path); goto close_in; }
    buf = malloc(256);
    if (!buf) goto close_out;

    int lines = 0;
    while (fgets(buf, 256, in)) fprintf(out, "%3d| %s", ++lines, buf);
    printf("numbered %d lines into %s\n", lines, out_path);
    rc = 0;

    free(buf);
close_out:
    fclose(out);
close_in:
    fclose(in);
done:
    return rc;
}
int main(void) {
    process("input.txt", "numbered.txt");
    process("missing.txt", "x.txt");            /* fails early, still cleans up */
    FILE *f = fopen("numbered.txt", "r");
    int c;
    while ((c = fgetc(f)) != EOF) putchar(c);
    fclose(f);
    return 0;
}
```

OUTPUT

```
numbered 3 lines into numbered.txt
missing.txt: No such file or directory
  1| alpha
  2| beta
  3| gamma
```

18.6.3 setjmp() and longjmp()

`setjmp(env)` saves the current execution context and returns 0. A later `longjmp(env, value)`, even from deep inside nested calls, jumps back so that setjmp appears to return a second time, now with `value`. It is the C mechanism behind exception-like error handling in some libraries and interpreters. Use it sparingly: it skips every intermediate function's cleanup (memory is not freed, files not closed), and local variables modified after setjmp must be `volatile` to have defined values.

EXCEPTION-LIKE ERROR HANDLING WITH SETJMP/LONGJMP main.c

```c
#include <stdio.h>
#include <setjmp.h>

static jmp_buf on_error;

double divide(double a, double b) {
    if (b == 0) longjmp(on_error, 1);        /* "throw" error code 1 */
    return a / b;
}
double average_ratio(const double *a, const double *b, int n) {
    double s = 0;
    for (int i = 0; i < n; i++) s += divide(a[i], b[i]);
    return s / n;
}
int main(void) {
    double a[] = {10, 20, 30}, good[] = {2, 4, 5}, bad[] = {2, 0, 5};
    for (int t = 0; t < 2; t++) {
        int code = setjmp(on_error);          /* "try" */
        if (code == 0) {
            double r = average_ratio(a, t == 0 ? good : bad, 3);
            printf("test %d: average ratio = %.2f\n", t, r);
        } else {                              /* "catch" */
            printf("test %d: caught error %d (division by zero)\n", t, code);
        }
    }
    return 0;
}
```

OUTPUT

```
test 0: average ratio = 5.33
test 1: caught error 1 (division by zero)
```

18.7 Time and utilities

18.7.1 time.h: time(), clock(), difftime(), strftime()

Function / type	Purpose
`time_t time(time_t *t)`	current calendar time, seconds since 1970-01-01 UTC (the Unix epoch)
`struct tm *localtime(const time_t *)`	break time into fields: year, month, day, hour …
`size_t strftime(buf, size, fmt, tm)`	format a struct tm as text
`double difftime(time_t end, time_t start)`	difference in seconds
`time_t mktime(struct tm *)`	build a time_t from fields (and normalize them)
`clock_t clock(void)`	processor time used by the program; divide by CLOCKS_PER_SEC

CALENDAR TIME, DIFFERENCES AND CPU TIME main.c

```c
#include <stdio.h>
#include <time.h>

int main(void) {
    time_t now = time(NULL);
    struct tm *t = localtime(&now);
    char buf[64];
    strftime(buf, sizeof buf, "%A, %d %B %Y, %H:%M", t);
    printf("now: %s\n", buf);

    struct tm exam = {.tm_year = 2026 - 1900, .tm_mon = 11, .tm_mday = 25, .tm_hour = 9};
    time_t exam_t = mktime(&exam);
    printf("days until 25 Dec 2026, 09:00: %.1f\n", difftime(exam_t, now) / 86400);

    clock_t start = clock();
    volatile double x = 0;
    for (long i = 0; i < 30000000; i++) x += i * 0.5;
    double secs = (double)(clock() - start) / CLOCKS_PER_SEC;
    printf("loop used %s of CPU time\n", secs < 1 ? "under a second" : "over a second");
    return 0;
}
```

OUTPUT (depends on when it runs)

```
now: Tuesday, 06 October 2026, 09:58
days until 25 Dec 2026, 09:00: 80.0
loop used under a second of CPU time
```

strftime	Gives	strftime	Gives
%Y	2026	%H:%M:%S	14:05:09
%m / %d	10 / 06	%A / %a	Tuesday / Tue
%B / %b	October / Oct	%j	day of year 279
%F	2026-10-06 (ISO)	%s (GNU)	seconds since epoch

18.7.2 Random numbers: rand(), srand()

`rand()` returns a pseudo-random int between 0 and `RAND_MAX` (at least 32767). The sequence is fully determined by the **seed** set with `srand(seed)`: the same seed always gives the same numbers, which is useful for reproducible tests. Seed once, typically with `srand(time(NULL))`. To get a number in [lo, hi] use `lo + rand() % (hi - lo + 1)` (slightly biased, fine for games and exercises; never for security, where you need the OS random source such as `getrandom()`).

SEEDING AND USING RAND()

main.c

```c
#include <stdio.h>
#include <stdlib.h>

int roll(void) { return 1 + rand() % 6; }

int main(void) {
    srand(42);                                  /* fixed seed: reproducible */
    printf("seed 42: ");
    for (int i = 0; i < 8; i++) printf("%d ", roll());
    srand(42);
    printf("\nseed 42 again: ");
    for (int i = 0; i < 8; i++) printf("%d ", roll());

    srand(7);
    int count[7] = {0};
    for (int i = 0; i < 60000; i++) count[roll()]++;
    printf("\n60000 rolls:");
    for (int f = 1; f <= 6; f++) printf(" %d:%d", f, count[f]);
    printf("\nrandom double in [0,1): %.4f\n", rand() / (RAND_MAX + 1.0));
    return 0;
}
```

OUTPUT

```
seed 42: 1 1 6 4 5 5 2 1
seed 42 again: 1 1 6 4 5 5 2 1
60000 rolls: 1:9924 2:9918 3:9871 4:10124 5:10136 6:10027
random double in [0,1): 0.9956
```

18.7.3 stdlib.h utilities: qsort(), bsearch(), exit(), atexit(), system(), getenv()

SEARCHING, ENVIRONMENT, SHELL COMMANDS AND EXIT HANDLERS

main.c

```c
#include <stdio.h>
#include <stdlib.h>

int cmp(const void *a, const void *b) { int x = *(const int *)a, y = *(const int *)b; return (x > y) -
(x < y); }
void goodbye(void)  { printf("atexit: goodbye handler runs last\n"); }
void flush_log(void){ printf("atexit: flushing logs\n"); }

int main(void) {
    atexit(goodbye);                        /* registered handlers run in reverse order */
    atexit(flush_log);

    int v[] = {50, 10, 40, 20, 30};
    qsort(v, 5, sizeof *v, cmp);
    int key = 40;
    int *hit = bsearch(&key, v, 5, sizeof *v, cmp);   /* array must be sorted */
    printf("bsearch 40: %s at index %td\n", hit ? "found" : "missing", hit ? hit - v : -1);
    key = 35;
    printf("bsearch 35: %s\n", bsearch(&key, v, 5, sizeof *v, cmp) ? "found" : "missing");

    const char *home = getenv("HOME");
    printf("HOME is %s\n", home ? "set" : "not set");
    const char *missing = getenv("LBF_NOT_SET");
    printf("LBF_NOT_SET -> %s\n", missing ? missing : "(null)");

    fflush(stdout);
    int rc = system("echo hello from the shell");
    printf("system() returned %d\n", rc);
    exit(EXIT_SUCCESS);                     /* runs the atexit handlers */
}
```

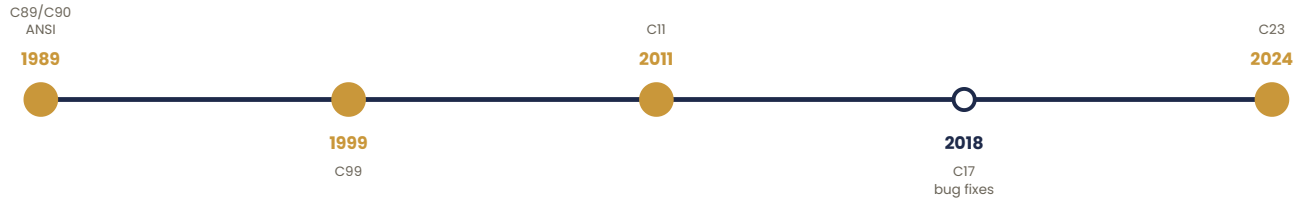
OUTPUT

```
bsearch 40: found at index 3
bsearch 35: missing
HOME is set
LBF_NOT_SET -> (null)
hello from the shell
system() returned 0
atexit: flushing logs
atexit: goodbye handler runs last
```

WATCH OUT

`system()` runs a full shell command, which is slow and dangerous if any part of the command comes from user input (command injection). Prefer specific library calls, or fork/exec (Module 22).

18.8 Modern C standards



THE MAJOR REVISIONS OF STANDARD C

18.8.1 C99 highlights

Feature	Example
`//` comments	`x++; // like C++`
`<stdbool.h>`	`bool done = true;`
`<stdint.h>` exact-width integers	`uint8_t`, `int64_t`
Declarations anywhere, in for	`for (int i = 0; ...)`
Variable-length arrays	`int a[n];`
Designated initializers	`{.x = 1, [5] = 9}`
Compound literals	`(Point){1, 2}`
`inline` functions	`static inline int sq(int x)`
`restrict`, `long long`, `%zu`, flexible array members, `snprintf`	

18.8.2 C11 highlights

Feature	Notes
`_Generic`	type-based selection (18.4)
`_Static_assert`	compile-time checks
Anonymous structs and unions	Module 14.8
`<threads.h>`	portable threads: thrd_create, mtx_lock (optional)
`_Atomic` and `<stdatomic.h>`	lock-free shared variables
`_Alignas`, `_Alignof`	control alignment
`_Noreturn`	functions that never return
Removal of `gets()`	it could not be used safely; use fgets
Exclusive open mode `"x"`, Unicode literals `u8"..."`	

18.8.3 C17: defect fixes

C17 (also called C18, published 2018) added **no new features**. It is a bug-fix release that clarified wording and resolved defect reports in C11. It is the default standard of GCC 8 to 14 and the one these notes use.

18.8.4 C23 highlights

C23 is the largest update since C99. Many features adopt ideas from C++ and long-standing compiler extensions. GCC 13 supports most of them with `-std=c2x` (GCC 14+: `-std=c23`).

A TOUR OF C23 FEATURES (GCC -STD=C2X)

main.c

```c
#include <stdio.h>

constexpr int MAX_USERS = 1'000;              /* constexpr + digit separators */
enum Color : unsigned char { RED, GREEN, BLUE }; /* enum with a fixed underlying type */

[[nodiscard]] int compute(void) { return 42; }
[[deprecated("use compute")]] int old_compute(void) { return 41; }

int main(void) {
    bool ok = true;                           /* bool, true, false are keywords */
    int *p = nullptr;                         /* nullptr constant */
    auto n = 0b1010'1010;                     /* auto inference, binary literal */
    typeof(n) copy = n;                       /* typeof */
    int users[MAX_USERS];                     /* constexpr usable as array size */
    printf("ok = %d, p is %s\n", ok, p == nullptr ? "nullptr" : "set");
    printf("n = %d (0b1010'1010), copy = %d\n", n, copy);
    printf("MAX_USERS = %d, sizeof users = %zu\n", MAX_USERS, sizeof users);
    printf("sizeof(enum Color) = %zu\n", sizeof(enum Color));
    printf("compute() = %d\n", compute());
    static_assert(sizeof(int) >= 4);          /* message now optional, no header */
    return 0;
}
```

OUTPUT

```
ok = 1, p is nullptr
n = 170 (0b1010'1010), copy = 170
MAX_USERS = 1000, sizeof users = 4000
sizeof(enum Color) = 1
compute() = 42
```

C23 feature	Before C23
`bool`, `true`, `false` keywords	`#include <stdbool.h>`
`nullptr`	`NULL` (may be plain 0)
`constexpr` objects	`#define` or enum
`typeof`, `typeof_unqual`	GCC extension `__typeof__`
`auto` type inference	none
`[[nodiscard]]`, `[[maybe_unused]]`, `[[deprecated]]`, `[[fallthrough]]`, `[[noreturn]]`	`__attribute__((...))`
Binary literals `0b1010`, digit separators `1'000'000`	hex only, no separators
`#embed`, `#elifdef`, `#warning`	external tools
`<stdckdint.h>` checked arithmetic, `<stdbit.h>`	compiler builtins
Two's complement required; K&R function definitions removed	

Qualifiers state intent (const), forbid caching (volatile), promise no aliasing (restrict) or provide thread safety (_Atomic). Generic C is written with void *, sizes and callbacks, or _Generic. Write code with no undefined behaviour, check every return value, release resources with goto cleanup, and know which standard your compiler targets.

PRACTICE CHECKPOINT

Write a **variadic function that averages any number of integers**, a **generic print macro using _Generic**, and a **resource-cleanup function using the goto cleanup pattern**.

SAMPLE SOLUTION

main.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <stdarg.h>

/* 1. variadic average: count first */
double average(int count, ...) {
    if (count <= 0) return 0.0;
    va_list ap;
    va_start(ap, count);
    long sum = 0;
    for (int i = 0; i < count; i++) sum += va_arg(ap, int);
    va_end(ap);
    return (double)sum / count;
}

/* 2. generic print */
void p_int(int x)            { printf("int    %d\n", x); }
void p_long(long x)          { printf("long   %ld\n", x); }
void p_double(double x)      { printf("double %.3f\n", x); }
void p_char(char x)          { printf("char   '%c'\n", x); }
void p_str(const char *x)    { printf("string \"%s\"\n", x); }
#define print(x) _Generic((x), int: p_int, long: p_long, double: p_double, float: p_double, \
                               char: p_char, char *: p_str, const char *: p_str)(x)

/* 3. goto cleanup: copy the first n numbers of a file into a new array, then save their squares */
int squares_report(const char *in_path, const char *out_path, int n) {
    int rc = -1;
    FILE *in = NULL, *out = NULL;
    int *data = NULL;

    if (!(in = fopen(in_path, "r")))   { perror(in_path); goto end; }
    if (!(data = malloc(n * sizeof *data))) goto close_in;
    if (!(out = fopen(out_path, "w"))) { perror(out_path); goto free_data; }

    int got = 0;
    while (got < n && fscanf(in, "%d", &data[got]) == 1) got++;
    for (int i = 0; i < got; i++) fprintf(out, "%d^2 = %d\n", data[i], data[i] * data[i]);
    printf("wrote %d squares to %s\n", got, out_path);
    rc = 0;

    fclose(out);
free_data:
    free(data);
close_in:
    fclose(in);
end:
    return rc;
}

int main(void) {
    printf("average(4, 10, 20, 30, 45) = %.2f\n", average(4, 10, 20, 30, 45));
    printf("average(1, 7)              = %.2f\n", average(1, 7));
    print(42);
    print(42L);
    print(3.14159);
    print((char)'Z');
```

```
    print("generic C");
    squares_report("nums.txt", "squares.txt", 4);
    squares_report("absent.txt", "squares.txt", 4);
    FILE *f = fopen("squares.txt", "r");
    char line[64];
    while (fgets(line, sizeof line, f)) fputs(line, stdout);
    fclose(f);
    return 0;
}
```

OUTPUT

```
average(4, 10, 20, 30, 45) = 26.25
average(1, 7)              = 7.00
int    42
long   42
double 3.142
char   'Z'
string "generic C"
wrote 4 squares to squares.txt
absent.txt: No such file or directory
3^2 = 9
5^2 = 25
12^2 = 144
7^2 = 49
```

MODULE 19 · ADVANCED

Data structures in C

Minimum duration **10 days** | Level **Advanced**

C has no built-in lists, stacks, maps or trees. Every one of them must be built from structures, pointers and dynamic memory, which makes C the best language for truly understanding how data structures work. This module builds the classic structures from scratch, with diagrams of every pointer change.

IN THIS MODULE

19.1 Fundamentals
19.2 Linked lists
19.3 Stack
19.4 Queue
19.5 Trees
19.6 Heaps
19.7 Hashing
19.8 Graphs
19.9 Sorting and searching algorithms

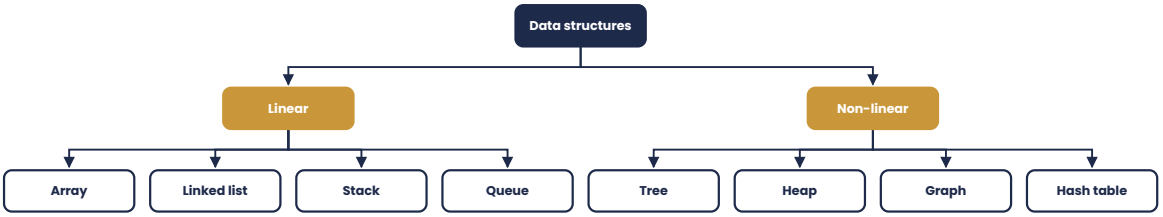
19.1 Fundamentals

19.1.1 What a data structure is

A **data structure** is a way of organising data in memory together with the operations allowed on it. The choice of structure decides how fast those operations are. A phone book sorted by name makes lookup by name fast and lookup by number slow; a different organisation reverses that. Choosing well is often the difference between a program that takes milliseconds and one that takes hours.

A data structure is usually described as an **abstract data type (ADT)**: what operations it offers (push, pop, insert, search) independent of how they are implemented. A stack, for example, can be implemented with an array or with a linked list, and the user of the stack need not know which.

19.1.2 Linear vs non-linear structures



CLASSIFICATION OF THE STRUCTURES IN THIS MODULE

	Linear	Non-linear
Arrangement	elements in a sequence, each has one next element	an element may connect to several others
Traversal	one pass from start to end	needs a strategy (depth-first, breadth-first)
Examples	array, linked list, stack, queue	tree, heap, graph, hash table (buckets)

	Linear	Non-linear
Models	lists, histories, waiting lines	hierarchies, networks, fast lookup

19.1.3 Time and space complexity, Big-O notation

Time complexity describes how the number of steps an algorithm takes grows with the input size n; **space complexity** describes how its extra memory grows. **Big-O** notation keeps only the dominant term and drops constants: an algorithm doing $3n^2 + 5n + 20$ steps is $O(n^2)$. It answers the question "what happens when n gets large?"

Big-O	Name	Example	n = 1,000	n = 1,000,000
$O(1)$	constant	array index, stack push	1	1
$O(\log n)$	logarithmic	binary search, balanced BST	10	20
$O(n)$	linear	linear search, list traversal	1,000	1,000,000
$O(n \log n)$	linearithmic	merge sort, heap sort	10,000	20,000,000
$O(n^2)$	quadratic	bubble sort, nested loops	1,000,000	10^{12}
$O(2^n)$	exponential	all subsets, naive Fibonacci	10^{301}	impossible



STEPS FOR N = 1,000 (BAR LENGTHS ARE ILLUSTRATIVE OF THE ORDER OF GROWTH)

We usually quote the **worst case**. Rules of thumb: a single loop over n items is $O(n)$; two nested loops over n are $O(n^2)$; halving the problem each step is $O(\log n)$; doing $O(\log n)$ work for each of n items is $O(n \log n)$.

O(N) VS O(LOG N) MEASURED main.c

```c
#include <stdio.h>

long steps;

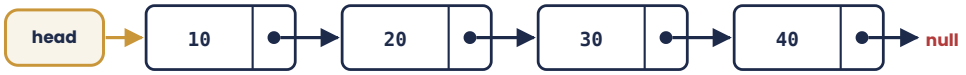
int linear_search(const int *a, int n, int key) {
    for (int i = 0; i < n; i++) { steps++; if (a[i] == key) return i; }
    return -1;
}
int binary_search(const int *a, int n, int key) {
    int lo = 0, hi = n - 1;
    while (lo <= hi) {
        steps++;
        int mid = lo + (hi - lo) / 2;
        if (a[mid] == key) return mid;
        if (a[mid] < key) lo = mid + 1; else hi = mid - 1;
    }
    return -1;
}
int main(void) {
    static int a[1000000];
    for (int i = 0; i < 1000000; i++) a[i] = 2 * i;          /* sorted */
    int sizes[] = {1000, 100000, 1000000};
    for (int k = 0; k < 3; k++) {
        int n = sizes[k], key = 2 * (n - 1);                   /* worst case: last element */
        steps = 0; linear_search(a, n, key); long lin = steps;
        steps = 0; binary_search(a, n, key); long bin = steps;
        printf("n = %7d: linear %7ld steps, binary %2ld steps\n", n, lin, bin);
    }
    return 0;
}
```

OUTPUT

```
n =    1000: linear    1000 steps, binary 10 steps
n =  100000: linear  100000 steps, binary 17 steps
n = 1000000: linear 1000000 steps, binary 20 steps
```

19.2 Linked lists

A **linked list** is a chain of **nodes**. Each node holds data and a pointer to the next node; the last node points to NULL. The list is reached through a **head** pointer. Unlike an array, the nodes are allocated one by one anywhere on the heap, so the list can grow and shrink freely and insertions do not move other elements.



A SINGLY LINKED LIST: HEAD → NODES → NULL

19.2.1 Singly linked list

All operations below are collected in one program at the end of this section. Each function that may change the **head** receives `Node **head` (Module 12.2.3), so it can update the caller's pointer.

THE NODE AND A CONSTRUCTOR

C

```c
typedef struct Node {
    int data;
    struct Node *next;
} Node;

Node *new_node(int data) {
    Node *n = malloc(sizeof *n);
    if (!n) { perror("malloc"); exit(1); }
    n->data = data;
    n->next = NULL;
    return n;
}
```

19.2.1.1 Create, traverse, count

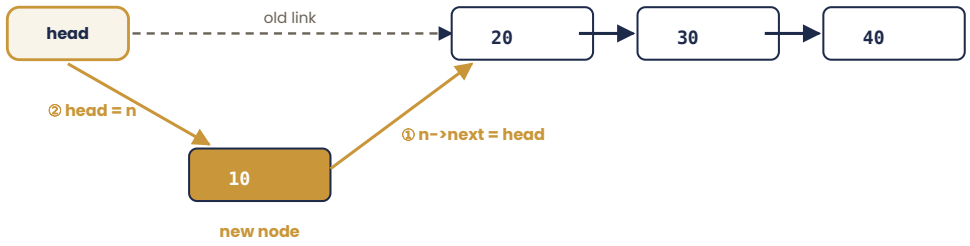
Traversal follows the next pointers from head until NULL. Counting is a traversal with a counter. Both are O(n).

SNIPPET

C

```c
void print_list(const Node *head) {
    for (const Node *p = head; p != NULL; p = p->next) printf("%d -> ", p->data);
    printf("NULL\n");
}
int count(const Node *head) {
    int c = 0;
    for (; head; head = head->next) c++;
    return c;
}
```

19.2.1.2 Insert at beginning, end and position



INSERT AT THE BEGINNING: LINK THE NEW NODE FIRST, THEN MOVE HEAD. O(1)

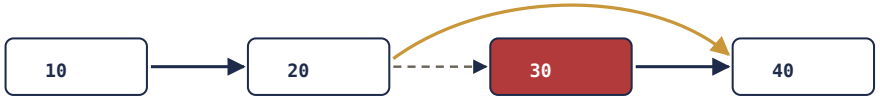
At the beginning: the new node points to the old first node, then head points to the new node. Order matters: moving head first would lose the rest of the list. **At the end**: walk to the last node (O(n)) and link it to the new node; if the list is empty, the new node becomes the head. **At position k**: walk to the node before position k and splice the new node in.

SNIPPET C

```c
void insert_front(Node **head, int data) {
    Node *n = new_node(data);
    n->next = *head;            /* 1: new node points to the old first */
    *head = n;                  /* 2: head points to the new node      */
}
void insert_end(Node **head, int data) {
    Node *n = new_node(data);
    if (*head == NULL) { *head = n; return; }
    Node *p = *head;
    while (p->next) p = p->next;      /* find the last node */
    p->next = n;
}
int insert_at(Node **head, int pos, int data) {      /* pos 0 = front */
    if (pos == 0) { insert_front(head, data); return 1; }
    Node *p = *head;
    for (int i = 0; p && i < pos - 1; i++) p = p->next;   /* node before pos */
    if (!p) return 0;                                     /* position too large */
    Node *n = new_node(data);
    n->next = p->next;
    p->next = n;
    return 1;
}
```

19.2.1.3 Delete by value and position

To delete a node, make its predecessor point past it, then free it. Deleting the first node changes head. Using a pointer-to-pointer that walks the **links** themselves removes the special case elegantly.



`prev->next = victim->next; free(victim);`

DELETING 30: BYPASS THE NODE, THEN FREE IT

SNIPPET C

```c
int delete_value(Node **head, int value) {
    for (Node **link = head; *link; link = &(*link)->next) {
        if ((*link)->data == value) {
            Node *victim = *link;
            *link = victim->next;      /* bypass (works for the head too) */
            free(victim);
            return 1;
        }
    }
    return 0;
}
int delete_at(Node **head, int pos) {
    Node **link = head;
    for (int i = 0; *link && i < pos; i++) link = &(*link)->next;
    if (*link == NULL) return 0;
    Node *victim = *link;
    *link = victim->next;
    free(victim);
    return 1;
}
```

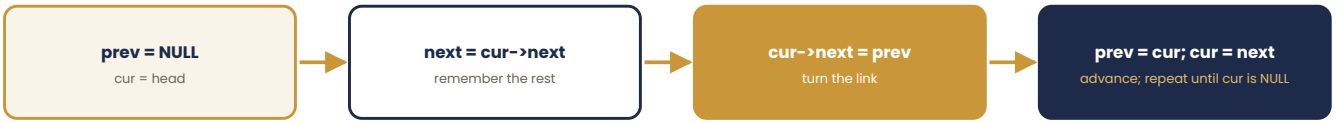
19.2.1.4 Search

SNIPPET C

```c
Node *search(Node *head, int key, int *index) {
    int i = 0;
    for (Node *p = head; p; p = p->next, i++)
        if (p->data == key) { if (index) *index = i; return p; }
    return NULL;
}
```

19.2.1.5 Reverse (iterative and recursive)

Iteratively, walk the list with three pointers (prev, cur, next) and turn each link around. **Recursively**, reverse the rest of the list first, then make the following node point back to the current one.



ITERATIVE REVERSAL: ONE PASS, O(N) TIME, O(1) SPACE

SNIPPET C

```c
void reverse(Node **head) {
    Node *prev = NULL, *cur = *head;
    while (cur) {
        Node *next = cur->next;
        cur->next = prev;
        prev = cur;
        cur = next;
    }
    *head = prev;
}
Node *reverse_rec(Node *head) {
    if (head == NULL || head->next == NULL) return head;   /* 0 or 1 node */
    Node *rest = reverse_rec(head->next);                  /* reverse the tail */
    head->next->next = head;                               /* next node points back */
    head->next = NULL;
    return rest;                                           /* new head */
}
```

19.2.1.6 Find the middle node, detect a loop (Floyd's algorithm)

Both problems use **two pointers moving at different speeds**. For the middle: **slow** moves one step and **fast** two; when fast reaches the end, slow is in the middle. For loop detection (Floyd's "tortoise and hare"): if the list has a cycle, fast eventually laps slow and they meet; if fast reaches NULL, there is no cycle. Both are O(n) time and O(1) space.

SNIPPET

C

```c
Node *middle(Node *head) {
    Node *slow = head, *fast = head;
    while (fast && fast->next) { slow = slow->next; fast = fast->next->next; }
    return slow;
}
int has_loop(const Node *head) {
    const Node *slow = head, *fast = head;
    while (fast && fast->next) {
        slow = slow->next;
        fast = fast->next->next;
        if (slow == fast) return 1;      /* they met inside a cycle */
    }
    return 0;
}
void free_list(Node **head) {
    while (*head) { Node *n = (*head)->next; free(*head); *head = n; }
}
```

The complete program combines all the functions above:

SINGLY LINKED LIST: ALL OPERATIONS

main.c

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct Node { int data; struct Node *next; } Node;

Node *new_node(int data) {
    Node *n = malloc(sizeof *n);
    if (!n) { perror("malloc"); exit(1); }
    n->data = data; n->next = NULL;
    return n;
}
void print_list(const Node *head) {
    for (const Node *p = head; p; p = p->next) printf("%d -> ", p->data);
    printf("NULL\n");
}
int count(const Node *head) { int c = 0; for (; head; head = head->next) c++; return c; }
void insert_front(Node **head, int data) { Node *n = new_node(data); n->next = *head; *head = n; }
void insert_end(Node **head, int data) {
    Node *n = new_node(data);
    if (!*head) { *head = n; return; }
    Node *p = *head;
    while (p->next) p = p->next;
    p->next = n;
}
int insert_at(Node **head, int pos, int data) {
    if (pos == 0) { insert_front(head, data); return 1; }
    Node *p = *head;
    for (int i = 0; p && i < pos - 1; i++) p = p->next;
    if (!p) return 0;
    Node *n = new_node(data); n->next = p->next; p->next = n;
    return 1;
}
int delete_value(Node **head, int value) {
    for (Node **link = head; *link; link = &(*link)->next)
        if ((*link)->data == value) { Node *v = *link; *link = v->next; free(v); return 1; }
    return 0;
}
int delete_at(Node **head, int pos) {
    Node **link = head;
```

```
    for (int i = 0; *link && i < pos; i++) link = &(*link)->next;
    if (!*link) return 0;
    Node *v = *link; *link = v->next; free(v);
    return 1;
}
Node *search(Node *head, int key, int *index) {
    int i = 0;
    for (Node *p = head; p; p = p->next, i++) if (p->data == key) { *index = i; return p; }
    return NULL;
}
void reverse(Node **head) {
    Node *prev = NULL, *cur = *head;
    while (cur) { Node *next = cur->next; cur->next = prev; prev = cur; cur = next; }
    *head = prev;
}
Node *reverse_rec(Node *head) {
    if (!head || !head->next) return head;
    Node *rest = reverse_rec(head->next);
    head->next->next = head; head->next = NULL;
    return rest;
}
Node *middle(Node *head) {
    Node *slow = head, *fast = head;
    while (fast && fast->next) { slow = slow->next; fast = fast->next->next; }
    return slow;
}
int has_loop(const Node *head) {
    const Node *slow = head, *fast = head;
    while (fast && fast->next) { slow = slow->next; fast = fast->next->next; if (slow == fast) return
1; }
    return 0;
}
void free_list(Node **head) { while (*head) { Node *n = (*head)->next; free(*head); *head = n; } }

int main(void) {
    Node *list = NULL;
    for (int v = 20; v <= 40; v += 10) insert_end(&list, v);
    insert_front(&list, 10);
    insert_at(&list, 2, 25);
    printf("list    : "); print_list(list);
    printf("count   : %d, middle: %d\n", count(list), middle(list)->data);
    int idx;
    if (search(list, 30, &idx)) printf("search 30: found at index %d\n", idx);
    delete_value(&list, 25);
    delete_at(&list, 0);
    printf("deleted : "); print_list(list);
    reverse(&list);
    printf("reversed: "); print_list(list);
    list = reverse_rec(list);
    printf("back    : "); print_list(list);

    printf("loop?   : %s\n", has_loop(list) ? "yes" : "no");
    Node *last = list; while (last->next) last = last->next;
    last->next = list->next;                        /* create a cycle on purpose */
    printf("loop?   : %s (after linking the last node back)\n", has_loop(list) ? "yes" : "no");
    last->next = NULL;                              /* break it again before freeing */
    free_list(&list);
    printf("freed, list is %s\n", list ? "not empty" : "NULL");
    return 0;
}
```

```
OUTPUT
list    : 10 -> 20 -> 25 -> 30 -> 40 -> NULL
count   : 5, middle: 25
search 30: found at index 3
deleted : 20 -> 30 -> 40 -> NULL
reversed: 40 -> 30 -> 20 -> NULL
back    : 20 -> 30 -> 40 -> NULL
loop?   : no
loop?   : yes (after linking the last node back)
freed, list is NULL
```

19.2.2 Doubly linked list

Each node of a **doubly linked list** has both `next` and `prev` pointers. You can walk in both directions, and a node can be deleted in O(1) when you have a pointer to it (no need to search for its predecessor). The cost is one extra pointer per node and two links to update on every change. Keeping a **tail** pointer makes insertion at the end O(1) too.



prev of first and next of last are NULL

DOUBLY LINKED LIST: NAVY ARROWS ARE NEXT, GOLD ARROWS ARE PREV

A DOUBLY LINKED LIST WITH HEAD AND TAIL main.c

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct DNode { int data; struct DNode *prev, *next; } DNode;
typedef struct { DNode *head, *tail; int size; } DList;

DNode *dnode(int v) { DNode *n = calloc(1, sizeof *n); n->data = v; return n; }

void push_back(DList *l, int v) {
    DNode *n = dnode(v);
    n->prev = l->tail;
    if (l->tail) l->tail->next = n; else l->head = n;
    l->tail = n; l->size++;
}
void push_front(DList *l, int v) {
    DNode *n = dnode(v);
    n->next = l->head;
    if (l->head) l->head->prev = n; else l->tail = n;
    l->head = n; l->size++;
}
void unlink_node(DList *l, DNode *n) {          /* O(1): no search needed */
    if (n->prev) n->prev->next = n->next; else l->head = n->next;
    if (n->next) n->next->prev = n->prev; else l->tail = n->prev;
    free(n); l->size--;
}
void show(const DList *l) {
    printf("forward : ");
    for (DNode *p = l->head; p; p = p->next) printf("%d ", p->data);
    printf("| backward: ");
    for (DNode *p = l->tail; p; p = p->prev) printf("%d ", p->data);
    printf("| size %d\n", l->size);
}
int main(void) {
```

```
    DList l = {0};
    push_back(&l, 20); push_back(&l, 30); push_front(&l, 10); push_back(&l, 40);
    show(&l);
    unlink_node(&l, l.head->next);          /* remove 20 */
    unlink_node(&l, l.tail);                 /* remove 40 */
    show(&l);
    while (l.head) unlink_node(&l, l.head);
    show(&l);
    return 0;
}
```

OUTPUT

```
forward : 10 20 30 40 | backward: 40 30 20 10 | size 4
forward : 10 30 | backward: 30 10 | size 2
forward : | backward: | size 0
```

19.2.3 Circular linked list

In a **circular linked list** the last node points back to the first instead of NULL. There is no natural end, so loops stop when they return to the start. Keeping a pointer to the **last** node gives O(1) access to both ends (the first is `last->next`). It suits round-robin scheduling, playlists on repeat and the Josephus problem.

A CIRCULAR LIST AND THE JOSEPHUS PROBLEM

main.c

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct CNode { int data; struct CNode *next; } CNode;

CNode *append(CNode *last, int v) {             /* returns the new last node */
    CNode *n = malloc(sizeof *n);
    n->data = v;
    if (!last) { n->next = n; return n; }        /* one node points to itself */
    n->next = last->next;                        /* new node -> first */
    last->next = n;
    return n;
}
void show(CNode *last) {
    if (!last) { printf("(empty)\n"); return; }
    CNode *p = last->next;
    do { printf("%d ", p->data); p = p->next; } while (p != last->next);
    printf("(back to %d)\n", last->next->data);
}
int main(void) {
    CNode *last = NULL;
    for (int i = 1; i <= 7; i++) last = append(last, i);
    show(last);
    /* Josephus: 7 people in a circle, every 3rd is removed */
    printf("removal order:");
    CNode *prev = last;
    while (prev->next != prev) {
        for (int k = 1; k < 3; k++) prev = prev->next;     /* move to the one before victim */
        CNode *victim = prev->next;
        printf(" %d", victim->data);
        prev->next = victim->next;
        free(victim);
    }
    printf("\nsurvivor: %d\n", prev->data);
    free(prev);
    return 0;
}
```

OUTPUT

```
1 2 3 4 5 6 7 (back to 1)
removal order: 3 6 2 7 5 1
survivor: 4
```

19.2.4 Array vs linked list

Operation / property	Array (dynamic)	Singly linked list
Access element i	O(1)	O(n)
Insert / delete at front	O(n) (shift)	O(1)
Insert / delete at end	O(1) amortized	O(n), O(1) with a tail pointer
Insert / delete in the middle (position known)	O(n)	O(1) after reaching it
Search (unsorted)	O(n)	O(n)
Binary search	O(log n) if sorted	not practical

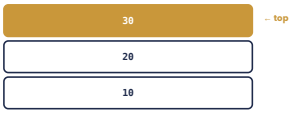
Operation / property	Array (dynamic)	Singly linked list
Memory per element	just the data	data + 8-byte pointer + allocator overhead
Cache friendliness	excellent (contiguous)	poor (scattered nodes)

TIP

On modern CPUs arrays are often faster than linked lists even for insertions in the middle, because contiguous memory is so cache-friendly. Use linked lists when you need stable node addresses, O(1) splicing, or very frequent insertion and deletion at known positions.

19.3 Stack

A **stack** is a LIFO (last in, first out) collection: the last element pushed is the first popped, like a pile of plates. Operations: **push** (add on top), **pop** (remove the top), **peek** (look at the top), **is_empty**. All are O(1).



PUSH 10, PUSH 20, PUSH 30: 30 IS POPPED FIRST

19.3.1 Array implementation

STACK WITH A FIXED ARRAY

main.c

```c
#include <stdio.h>
#include <stdbool.h>

#define CAP 5
typedef struct { int items[CAP]; int top; } Stack;      /* top = index of top item, -1 if empty */

void init(Stack *s)          { s->top = -1; }
bool is_empty(const Stack *s){ return s->top == -1; }
bool is_full(const Stack *s) { return s->top == CAP - 1; }
bool push(Stack *s, int v) {
    if (is_full(s)) { printf("overflow: cannot push %d\n", v); return false; }
    s->items[++s->top] = v;
    return true;
}
bool pop(Stack *s, int *out) {
    if (is_empty(s)) { printf("underflow: stack is empty\n"); return false; }
    *out = s->items[s->top--];
    return true;
}
int peek(const Stack *s) { return s->items[s->top]; }

int main(void) {
    Stack s; init(&s);
    for (int v = 10; v <= 60; v += 10) push(&s, v);       /* 60 overflows */
    printf("top is %d\n", peek(&s));
    int x;
    while (pop(&s, &x)) printf("popped %d\n", x);
    return 0;
}
```

OUTPUT

```
overflow: cannot push 60
top is 50
popped 50
popped 40
popped 30
popped 20
popped 10
underflow: stack is empty
```

19.3.2 Linked list implementation

Pushing and popping at the **head** of a linked list are O(1) and the stack never becomes full (until memory runs out).

STACK ON A LINKED LIST (REVERSING A WORD) main.c

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct SNode { int v; struct SNode *next; } SNode;
typedef struct { SNode *top; int size; } Stack;

void push(Stack *s, int v) {
    SNode *n = malloc(sizeof *n);
    n->v = v; n->next = s->top;
    s->top = n; s->size++;
}
int pop(Stack *s, int *out) {
    if (!s->top) return 0;
    SNode *n = s->top;
    *out = n->v; s->top = n->next; s->size--;
    free(n);
    return 1;
}
int main(void) {
    Stack s = {NULL, 0};
    const char *word = "STACK";
    for (const char *p = word; *p; p++) push(&s, *p);
    printf("%d items; popping reverses: ", s.size);
    int c;
    while (pop(&s, &c)) putchar(c);
    putchar('\n');
    return 0;
}
```

OUTPUT

```
5 items; popping reverses: KCATS
```

19.3.3 Applications: balanced parentheses, infix to postfix, postfix evaluation, undo

Balanced parentheses: push every opening bracket; on a closing bracket, the top of the stack must be the matching opener. At the end the stack must be empty.

CHECKING BRACKETS WITH A STACK

main.c

```c
#include <stdio.h>
#include <string.h>

int balanced(const char *s) {
    char st[100]; int top = -1;
    for (; *s; s++) {
        if (strchr("([{", *s)) st[++top] = *s;
        else if (strchr(")]}", *s)) {
            if (top < 0) return 0;
            char open = st[top--];
            if ((*s == ')' && open != '(') || (*s == ']' && open != '[') || (*s == '}' && open !=
'{'))
                return 0;
        }
    }
    return top == -1;
}
int main(void) {
    const char *tests[] = {"{a[i] = (b + c)}", "(a + b))", "([)]", "((x)", "f(g[h{1}])"};
    for (int i = 0; i < 5; i++) printf("%-18s %s\n", tests[i], balanced(tests[i]) ? "balanced" : "NOT
balanced");
    return 0;
}
```

OUTPUT

```
{a[i] = (b + c)}   balanced
(a + b))           NOT balanced
([)]               NOT balanced
((x)               NOT balanced
f(g[h{1}])         balanced
```

Infix to postfix (the shunting-yard algorithm by Dijkstra): humans write `a + b * c` (infix); computers evaluate `a b c * +` (postfix, Reverse Polish Notation) easily because it needs no parentheses or precedence. Scan the infix expression: operands go straight to the output; an operator first pops to the output every operator on the stack with higher or equal precedence (for left-associative operators), then is pushed; `(` is pushed; `)` pops until the matching `(`. At the end pop everything.

Read	Action	Stack	Output
a	operand → output		a
+	push	+	a
b	operand → output	+	a b
*	higher than + : push	+ *	a b
c	operand → output	+ *	a b c
end	pop all		a b c * +

Postfix evaluation: scan left to right; push numbers; on an operator, pop two numbers, apply it, push the result. The final value on the stack is the answer. Both algorithms are combined in the checkpoint at the end of this module.

Undo: editors push every action onto an undo stack; "undo" pops the most recent action and reverses it (and pushes it onto a redo stack).

UNDO WITH A STACK OF STATES main.c

```c
#include <stdio.h>
#include <string.h>

#define MAXH 10
char text[64] = "";
char history[MAXH][64]; int top = -1;          /* stack of previous states */

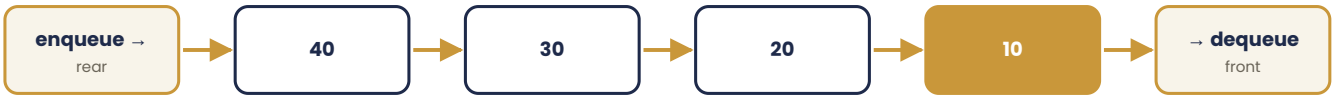
void type_text(const char *s) {
    if (top < MAXH - 1) strcpy(history[++top], text);   /* save state */
    strncat(text, s, sizeof text - strlen(text) - 1);
    printf("type %-8s -> \"%s\"\n", s, text);
}
void undo(void) {
    if (top < 0) { printf("nothing to undo\n"); return; }
    strcpy(text, history[top--]);
    printf("undo          -> \"%s\"\n", text);
}
int main(void) {
    type_text("Hello");
    type_text(", C");
    type_text("!!");
    undo();
    undo();
    type_text(" world");
    undo(); undo(); undo();
    return 0;
}
```

OUTPUT

```
type Hello    -> "Hello"
type , C      -> "Hello, C"
type !!       -> "Hello, C!!"
undo          -> "Hello, C"
undo          -> "Hello"
type  world   -> "Hello world"
undo          -> "Hello"
undo          -> ""
nothing to undo
```

19.4 Queue

A **queue** is FIFO (first in, first out), like a line at a ticket counter: elements are added at the **rear** (enqueue) and removed from the **front** (dequeue). Used for task scheduling, print spoolers, buffering between producer and consumer, and breadth-first search.



ELEMENTS LEAVE IN THE ORDER THEY ARRIVED

19.4.1 Simple queue (array and linked list)

In a simple array queue, front and rear indexes only move forward, so slots freed at the front are never reused: after enough operations the queue reports "full" while it is mostly empty. The linked-list version (head = front, tail = rear) has no such limit.

LINKED-LIST QUEUE

main.c

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct QNode { int v; struct QNode *next; } QNode;
typedef struct { QNode *front, *rear; } Queue;

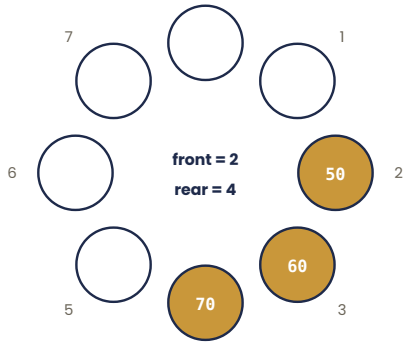
void enqueue(Queue *q, int v) {
    QNode *n = malloc(sizeof *n);
    n->v = v; n->next = NULL;
    if (q->rear) q->rear->next = n; else q->front = n;
    q->rear = n;
}
int dequeue(Queue *q, int *out) {
    if (!q->front) return 0;
    QNode *n = q->front;
    *out = n->v;
    q->front = n->next;
    if (!q->front) q->rear = NULL;          /* queue became empty */
    free(n);
    return 1;
}
int main(void) {
    Queue q = {NULL, NULL};
    const char *customers[] = {"Asha", "Ravi", "Meera", "Kabir"};
    for (int i = 0; i < 4; i++) { enqueue(&q, i); printf("arrives: %s\n", customers[i]); }
    int id;
    while (dequeue(&q, &id)) printf("served : %s\n", customers[id]);
    return 0;
}
```

OUTPUT

```
arrives: Asha
arrives: Ravi
arrives: Meera
arrives: Kabir
served : Asha
served : Ravi
served : Meera
served : Kabir
```

19.4.2 Circular queue

A **circular queue** (ring buffer) reuses the array by wrapping the indexes around with `% capacity`. Keeping a `count` distinguishes full from empty. Ring buffers are everywhere in systems code: keyboard buffers, audio streams, network drivers.



enqueue: rear = (rear + 1) % CAP

dequeue: front = (front + 1) % CAP

after slot 7 comes slot 0 again

A RING BUFFER OF 8 SLOTS HOLDING 50, 60, 70

CIRCULAR QUEUE IN A FIXED ARRAY main.c

```c
#include <stdio.h>
#include <stdbool.h>

#define CAP 4
typedef struct { int a[CAP]; int front, count; } CQueue;

bool enqueue(CQueue *q, int v) {
    if (q->count == CAP) return false;
    q->a[(q->front + q->count) % CAP] = v;     /* rear position wraps */
    q->count++;
    return true;
}
bool dequeue(CQueue *q, int *v) {
    if (q->count == 0) return false;
    *v = q->a[q->front];
    q->front = (q->front + 1) % CAP;
    q->count--;
    return true;
}
void show(const CQueue *q) {
    printf("[");
    for (int i = 0; i < q->count; i++) printf(" %d", q->a[(q->front + i) % CAP]);
    printf(" ] front index %d\n", q->front);
}
int main(void) {
    CQueue q = {{0}, 0, 0};
    for (int v = 1; v <= 5; v++) if (!enqueue(&q, v * 10)) printf("full, %d rejected\n", v * 10);
    show(&q);
    int x;
    dequeue(&q, &x); dequeue(&q, &x);
    printf("dequeued two, last was %d\n", x);
    enqueue(&q, 50); enqueue(&q, 60);          /* reuse the freed slots */
    show(&q);
    return 0;
}
```

OUTPUT

```
full, 50 rejected
[ 10 20 30 40 ] front index 0
dequeued two, last was 20
[ 30 40 50 60 ] front index 2
```

19.4.3 Deque

A **deque** (double-ended queue, pronounced "deck") allows insertion and removal at **both** ends in O(1). It can act as a stack or a queue. A circular array with a front index that can also move backwards implements it neatly.

A DEQUE ON A CIRCULAR ARRAY (BOUNDS CHECKS OMITTED FOR BREVITY)

main.c

```c
#include <stdio.h>

#define CAP 8
int dq[CAP], front = 0, count = 0;

void push_front(int v) { front = (front - 1 + CAP) % CAP; dq[front] = v; count++; }
void push_back(int v)  { dq[(front + count) % CAP] = v; count++; }
int pop_front(void)    { int v = dq[front]; front = (front + 1) % CAP; count--; return v; }
int pop_back(void)     { count--; return dq[(front + count) % CAP]; }
void show(void) { for (int i = 0; i < count; i++) printf("%d ", dq[(front + i) % CAP]); printf("\n");
}

int main(void) {
    push_back(3); push_back(4); push_front(2); push_front(1); push_back(5);
    show();
    int f = pop_front();
    int b = pop_back();
    printf("pop_front = %d, pop_back = %d\n", f, b);
    show();
    return 0;
}
```

OUTPUT

```
1 2 3 4 5
pop_front = 1, pop_back = 5
2 3 4
```

19.4.4 Priority queue

In a **priority queue** each element has a priority and the element with the highest (or lowest) priority leaves first, regardless of arrival order. Hospital triage and CPU schedulers work this way. A sorted array gives O(1) removal but O(n) insertion; the efficient implementation is the **binary heap** (19.6) with O(log n) for both.

A SIMPLE PRIORITY QUEUE (SORTED ARRAY) main.c

```c
#include <stdio.h>
#include <string.h>

typedef struct { char name[24]; int priority; } Patient;
Patient pq[10]; int n = 0;

void insert(const char *name, int pr) {          /* keep the array sorted by priority */
    int i = n++;
    while (i > 0 && pq[i - 1].priority < pr) { pq[i] = pq[i - 1]; i--; }
    strcpy(pq[i].name, name);
    pq[i].priority = pr;
}
Patient remove_top(void) {
    Patient top = pq[0];
    memmove(pq, pq + 1, (size_t)(--n) * sizeof *pq);
    return top;
}
int main(void) {
    insert("Asha (cold)", 2);
    insert("Ravi (fracture)", 7);
    insert("Meera (chest pain)", 10);
    insert("Kabir (cut)", 4);
    while (n > 0) { Patient p = remove_top(); printf("treat %-20s priority %d\n", p.name, p.priority);
}
    return 0;
}
```

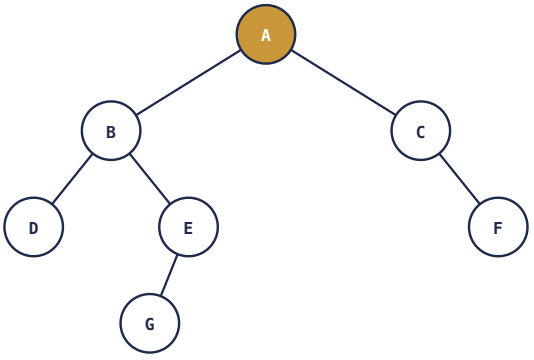
OUTPUT

```
treat Meera (chest pain)   priority 10
treat Ravi (fracture)      priority 7
treat Kabir (cut)          priority 4
treat Asha (cold)          priority 2
```

19.5 Trees

19.5.1 Tree terminology

A **tree** is a hierarchical structure of **nodes** connected by **edges**, with one special node at the top, the **root**. Every other node has exactly one **parent**. Trees model file systems, organisation charts, HTML documents, expression parsing and fast searching.



A BINARY TREE WITH ROOT A

Term	Meaning	In the figure
Root	the top node, no parent	A
Parent / child	a node directly above / below	B is the parent of D and E
Siblings	children of the same parent	D and E
Leaf	node with no children	D, G, F
Internal node	node with at least one child	A, B, C, E
Subtree	a node and all its descendants	B, D, E, G
Depth of a node	number of edges from the root	depth(G) = 3
Height of a tree	number of edges on the longest root-to-leaf path	3
Degree	number of children of a node	degree(C) = 1
Binary tree	every node has at most two children: left and right	this tree

19.5.2 Binary tree creation

A binary-tree node holds data and two child pointers. Trees are naturally **recursive**: a tree is either empty (NULL) or a node whose left and right children are themselves trees. Almost every tree algorithm is therefore a short recursive function.

SNIPPET C

```c
typedef struct TNode {
    int key;
    struct TNode *left, *right;
} TNode;

TNode *tnode(int key) {
    TNode *n = malloc(sizeof *n);
    n->key = key;
    n->left = n->right = NULL;
    return n;
}
```

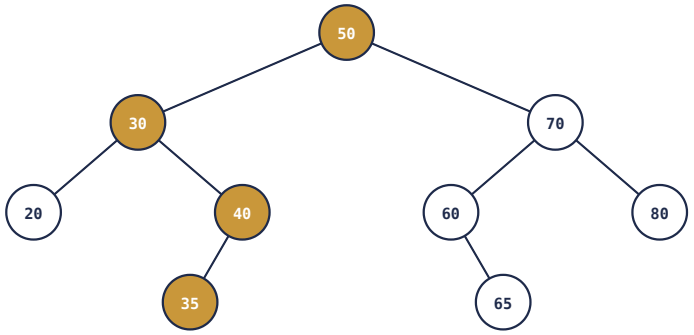
19.5.3 Traversals: inorder, preorder, postorder, level order

A **traversal** visits every node once. The three depth-first orders differ only in **when** the node itself is visited relative to its subtrees:

Order	Rule	Typical use
Preorder	node, left, right	copy a tree, print a directory listing, prefix expressions
Inorder	left, node, right	BST: visits keys in sorted order
Postorder	left, right, node	delete/free a tree, evaluate expression trees, folder sizes
Level order	level by level, left to right (uses a queue)	breadth-first processing, printing by level

19.5.4 Binary search tree: insert, search, delete, min, max

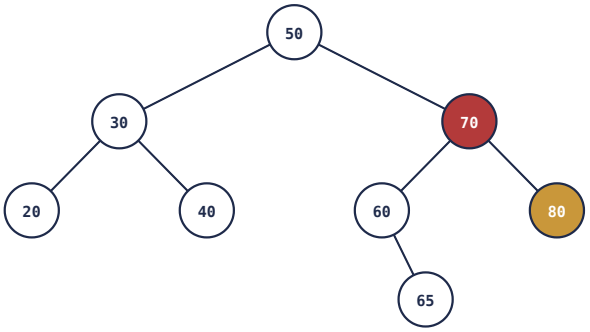
A **binary search tree (BST)** keeps the rule: every key in the **left** subtree is smaller than the node's key and every key in the **right** subtree is larger. Searching then follows a single path from the root, discarding half of the remaining tree at each step when the tree is balanced: O(log n). Inserting walks the same path and attaches a new leaf.



BST BUILT BY INSERTING 50, 30, 70, 20, 40, 60, 80, 35, 65. SEARCHING 35 VISITS 50 → 30 → 40 → 35

Minimum: follow left pointers to the end. **Maximum**: follow right pointers. **Delete** has three cases:

1. **Leaf**: simply remove it.
2. **One child**: replace the node by its child.
3. **Two children**: copy the **inorder successor** (the minimum of the right subtree) into the node, then delete that successor from the right subtree (it has at most one child).



DELETING 70 (TWO CHILDREN): ITS INORDER SUCCESSOR 80, THE SMALLEST KEY IN ITS RIGHT SUBTREE, IS COPIED INTO ITS PLACE AND THE OLD 80 LEAF IS REMOVED

19.5.5 Height, count of nodes and leaves

Each is one line of recursion: height = 1 + max(height(left), height(right)), with an empty tree having height -1 (counting edges); count = 1 + count(left) + count(right); a leaf counts 1 when both children are NULL.

A COMPLETE BINARY SEARCH TREE main.c

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct TNode { int key; struct TNode *left, *right; } TNode;

TNode *tnode(int k) { TNode *n = malloc(sizeof *n); n->key = k; n->left = n->right = NULL; return n; }

TNode *insert(TNode *root, int k) {
    if (!root) return tnode(k);
    if (k < root->key) root->left = insert(root->left, k);
    else if (k > root->key) root->right = insert(root->right, k);
```

```
    return root;                                    /* duplicates ignored */
}
TNode *search(TNode *root, int k, int *steps) {
    while (root) {
        (*steps)++;
        if (k == root->key) return root;
        root = k < root->key ? root->left : root->right;
    }
    return NULL;
}
TNode *min_node(TNode *r) { while (r && r->left) r = r->left; return r; }
TNode *max_node(TNode *r) { while (r && r->right) r = r->right; return r; }

TNode *delete(TNode *root, int k) {
    if (!root) return NULL;
    if (k < root->key) root->left = delete(root->left, k);
    else if (k > root->key) root->right = delete(root->right, k);
    else {
        if (!root->left)  { TNode *r = root->right; free(root); return r; }   /* 0 or 1 child */
        if (!root->right) { TNode *l = root->left;  free(root); return l; }
        TNode *succ = min_node(root->right);                                  /* 2 children */
        root->key = succ->key;
        root->right = delete(root->right, succ->key);
    }
    return root;
}
void preorder(const TNode *r)  { if (r) { printf("%d ", r->key); preorder(r->left); preorder(r-
>right); } }
void inorder(const TNode *r)   { if (r) { inorder(r->left); printf("%d ", r->key); inorder(r->right);
} }
void postorder(const TNode *r) { if (r) { postorder(r->left); postorder(r->right); printf("%d ", r-
>key); } }
void level_order(TNode *r) {
    TNode *q[100]; int head = 0, tail = 0;          /* simple array queue */
    if (r) q[tail++] = r;
    while (head < tail) {
        TNode *n = q[head++];
        printf("%d ", n->key);
        if (n->left) q[tail++] = n->left;
        if (n->right) q[tail++] = n->right;
    }
}
int height(const TNode *r) { if (!r) return -1; int a = height(r->left), b = height(r->right); return
1 + (a > b ? a : b); }
int count(const TNode *r)  { return r ? 1 + count(r->left) + count(r->right) : 0; }
int leaves(const TNode *r) { if (!r) return 0; if (!r->left && !r->right) return 1; return leaves(r-
>left) + leaves(r->right); }
void free_tree(TNode *r)   { if (r) { free_tree(r->left); free_tree(r->right); free(r); } }   /*
postorder */

int main(void) {
    int keys[] = {50, 30, 70, 20, 40, 60, 80, 35, 65};
    TNode *root = NULL;
    for (int i = 0; i < 9; i++) root = insert(root, keys[i]);
    printf("preorder   : "); preorder(root);    printf("\n");
    printf("inorder    : "); inorder(root);     printf("(sorted!)\n");
    printf("postorder  : "); postorder(root);   printf("\n");
    printf("level order: "); level_order(root); printf("\n");
    printf("min %d, max %d, height %d, nodes %d, leaves %d\n",
           min_node(root)->key, max_node(root)->key, height(root), count(root), leaves(root));
    int steps = 0;
    TNode *hit = search(root, 35, &steps);
    printf("search 35: %s in %d steps\n", hit ? "found" : "missing", steps);
    steps = 0;
```

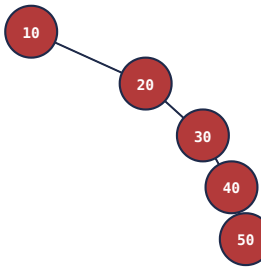
```
    hit = search(root, 55, &steps);
    printf("search 55: %s in %d steps\n", hit ? "found" : "missing", steps);
    root = delete(root, 20);   printf("delete 20 (leaf)        : "); inorder(root); printf("\n");
    root = delete(root, 40);   printf("delete 40 (one child)   : "); inorder(root); printf("\n");
    root = delete(root, 70);   printf("delete 70 (two children): "); inorder(root);
    printf("| root->right is now %d\n", root->right->key);
    free_tree(root);
    return 0;
}
```

OUTPUT

```
preorder   : 50 30 20 40 35 70 60 65 80
inorder    : 20 30 35 40 50 60 65 70 80 (sorted!)
postorder  : 20 35 40 30 65 60 80 70 50
level order: 50 30 70 20 40 60 80 35 65
min 20, max 80, height 3, nodes 9, leaves 4
search 35: found in 4 steps
search 55: missing in 3 steps
delete 20 (leaf)        : 30 35 40 50 60 65 70 80
delete 40 (one child)   : 30 35 50 60 65 70 80
delete 70 (two children): 30 35 50 60 65 80 | root->right is now 80
```

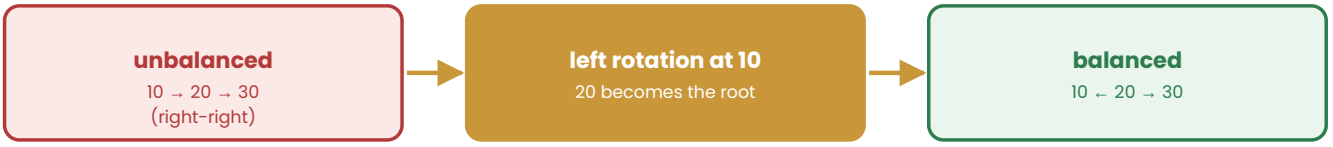
19.5.6 AVL trees (overview)

A BST is only fast if it is **balanced**. Inserting already sorted keys (10, 20, 30, 40 …) produces a tree that is just a linked list leaning right, and every operation becomes O(n).



INSERTING SORTED KEYS DEGENERATES A BST INTO A LIST: HEIGHT 4 FOR 5 NODES

An **AVL tree** (Adelson-Velsky and Landis, 1962) is a self-balancing BST. Every node stores its height, and the **balance factor** height(left) - height(right) must stay -1, 0 or +1. After each insertion or deletion, nodes on the path back to the root are checked; when one becomes unbalanced, a **rotation** (a constant-time rearrangement of three nodes) restores balance while preserving the BST order. This guarantees height O(log n), so search, insert and delete are always O(log n).



THE FOUR AVL CASES ARE LL, RR, LR AND RL; RR IS FIXED WITH ONE LEFT ROTATION

Case	Imbalance	Fix
LL	left child's left subtree too tall	one right rotation
RR	right child's right subtree too tall	one left rotation
LR	left child's right subtree too tall	left rotation on child, then right rotation

Case	Imbalance	Fix
RL	right child's left subtree too tall	right rotation on child, then left rotation

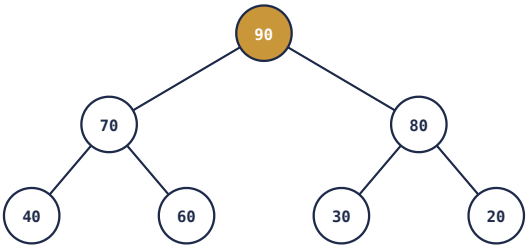
Red-black trees are a related self-balancing design with slightly looser balance and fewer rotations; they power C++ `std::map`, Java's `TreeMap` and the Linux kernel's scheduler.

19.6 Heaps

19.6.1 Min-heap and max-heap

A **binary heap** is a **complete** binary tree (all levels full except possibly the last, which fills from the left) with the **heap property**: in a **max-heap** every parent is ≥ its children, so the maximum is at the root; in a **min-heap** every parent is ≤ its children. Because it is complete, a heap is stored in a plain **array** with no pointers at all:

For the node at index i	Index
left child	2i + 1
right child	2i + 2
parent	(i - 1) / 2



MAX-HEAP STORED AS THE ARRAY [90, 70, 80, 40, 60, 30, 20]



THE SAME HEAP IN AN ARRAY: CHILDREN OF INDEX 1 (70) ARE AT 3 AND 4

Insert: place the new value at the end of the array, then **sift up**: swap with the parent while it is larger than the parent. **Extract max**: take the root, move the last element to the root, then **sift down**: swap with the larger child while it is smaller than that child. Both are O(log n), which makes heaps the standard priority queue.

19.6.2 Heapify and heap sort

Heapify turns an arbitrary array into a heap by sifting down every non-leaf node, from the last one up to the root. Surprisingly this takes only O(n). **Heap sort** then repeatedly swaps the root (the maximum) with the last element of the heap, shrinks the heap by one, and sifts the new root down. The result is sorted in place in O(n log n) time with O(1) extra memory.

HEAP OPERATIONS, A PRIORITY QUEUE AND HEAP SORT main.c

```c
#include <stdio.h>

static void swap(int *a, int *b) { int t = *a; *a = *b; *b = t; }
```

```c
void sift_down(int *a, int n, int i) {
    for (;;) {
        int largest = i, l = 2 * i + 1, r = 2 * i + 2;
        if (l < n && a[l] > a[largest]) largest = l;
        if (r < n && a[r] > a[largest]) largest = r;
        if (largest == i) return;
        swap(&a[i], &a[largest]);
        i = largest;
    }
}
void sift_up(int *a, int i) {
    while (i > 0 && a[(i - 1) / 2] < a[i]) { swap(&a[i], &a[(i - 1) / 2]); i = (i - 1) / 2; }
}
void heapify(int *a, int n) { for (int i = n / 2 - 1; i >= 0; i--) sift_down(a, n, i); }
void heap_sort(int *a, int n) {
    heapify(a, n);
    for (int end = n - 1; end > 0; end--) {
        swap(&a[0], &a[end]);          /* move the max to its final place */
        sift_down(a, end, 0);          /* restore the heap in a[0..end-1] */
    }
}
void show(const char *t, const int *a, int n) { printf("%-14s", t); for (int i = 0; i < n; i++)
printf(" %d", a[i]); printf("\n"); }

int main(void) {
    int a[] = {40, 20, 90, 60, 30, 80, 70};
    int n = 7;
    show("array", a, n);
    heapify(a, n);
    show("max-heap", a, n);

    int pq[16] = {0}, size = 0;          /* priority queue on a heap */
    int incoming[] = {15, 42, 7, 99, 23};
    for (int i = 0; i < 5; i++) { pq[size] = incoming[i]; sift_up(pq, size++); }
    printf("%-14s", "extract max");
    while (size > 0) { printf(" %d", pq[0]); pq[0] = pq[--size]; sift_down(pq, size, 0); }
    printf("\n");

    int b[] = {29, 3, 72, 15, 88, 41, 6, 57};
    heap_sort(b, 8);
    show("heap sorted", b, 8);
    return 0;
}
```

OUTPUT

```
array          40 20 90 60 30 80 70
max-heap       90 60 80 20 30 40 70
extract max    99 42 23 15 7
heap sorted    3 6 15 29 41 57 72 88
```

19.7 Hashing

A **hash table** stores key-value pairs and finds a key in **O(1) on average**, regardless of how many entries it holds. The idea: a **hash function** turns the key into an array index, so the entry is found by direct indexing instead of searching. Dictionaries in Python, objects in JavaScript, HashMap in Java and symbol tables in compilers are hash tables.

19.7.1 Hash functions

A good hash function is fast, deterministic and spreads keys **uniformly** over the table. For integers, `key % table_size` works (a prime size helps). For strings, combine all characters, for example the djb2 function by Dan Bernstein: start with 5381 and for each character compute `h = h * 33 + c`.

THE DJB2 STRING HASH main.c

```c
#include <stdio.h>

unsigned long djb2(const char *s) {
    unsigned long h = 5381;
    for (; *s; s++) h = h * 33 + (unsigned char)*s;
    return h;
}
int main(void) {
    const char *keys[] = {"apple", "banana", "cherry", "date", "elderberry", "fig"};
    for (int i = 0; i < 6; i++) {
        unsigned long h = djb2(keys[i]);
        printf("%-11s hash %20lu  -> bucket %lu of 7\n", keys[i], h, h % 7);
    }
    return 0;
}
```

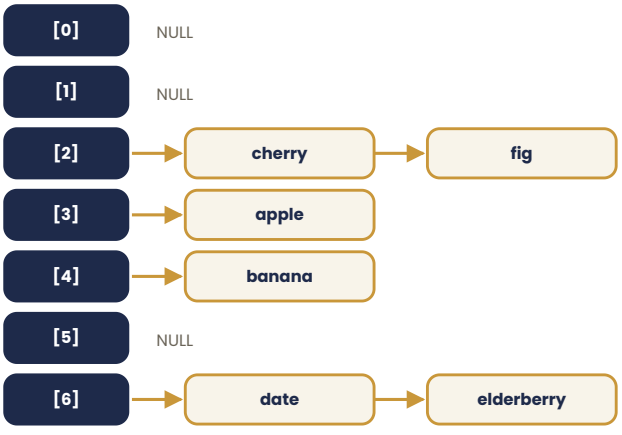
OUTPUT

```
apple       hash          210706734647  -> bucket 3 of 7
banana      hash         6953343506470  -> bucket 4 of 7
cherry      hash         6953390638546  -> bucket 2 of 7
date        hash            6385144163  -> bucket 6 of 7
elderberry  hash  8246270368627342645  -> bucket 6 of 7
fig         hash             193491643  -> bucket 2 of 7
```

19.7.2 Collision handling: chaining and open addressing

Different keys can land in the same bucket: a **collision**. Two strategies handle it:

- **Separate chaining**: each bucket holds a linked list of all entries that hash there. Simple, never "full"; performance degrades gracefully as lists grow.
- **Open addressing**: all entries live in the array itself. On a collision, **probe** other slots in a fixed sequence: linear probing tries h+1, h+2, ...; quadratic probing tries h+1, h+4, h+9, ...; double hashing uses a second hash for the step. Deleting needs "tombstone" markers so later searches do not stop early.



SEPARATE CHAINING WITH 7 BUCKETS: THE SIX FRUIT KEYS PLACED BY DJB2 % 7 AS COMPUTED ABOVE

The **load factor** α = entries / buckets measures fullness. Chaining keeps working above 1 but slows down; open addressing must stay below about 0.7. When α gets too high, the table is **resized** (typically doubled) and every entry is rehashed.

19.7.3 Building a simple hash table

A HASH MAP FROM STRINGS TO INTS WITH CHAINING

main.c

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define BUCKETS 7

typedef struct Entry {
    char *key;
    int value;
    struct Entry *next;
} Entry;

typedef struct { Entry *b[BUCKETS]; int count; } HashMap;

static unsigned long hash(const char *s) { unsigned long h = 5381; while (*s) h = h * 33 + (unsigned
char)*s++; return h; }

void put(HashMap *m, const char *key, int value) {
    unsigned long i = hash(key) % BUCKETS;
    for (Entry *e = m->b[i]; e; e = e->next)
        if (strcmp(e->key, key) == 0) { e->value = value; return; }      /* update */
    Entry *e = malloc(sizeof *e);
    e->key = malloc(strlen(key) + 1);
    strcpy(e->key, key);
    e->value = value;
    e->next = m->b[i];                    /* insert at the head of the chain */
    m->b[i] = e;
    m->count++;
}
int get(const HashMap *m, const char *key, int *out) {
    for (Entry *e = m->b[hash(key) % BUCKETS]; e; e = e->next)
        if (strcmp(e->key, key) == 0) { *out = e->value; return 1; }
    return 0;
}
int erase(HashMap *m, const char *key) {
    for (Entry **link = &m->b[hash(key) % BUCKETS]; *link; link = &(*link)->next)
        if (strcmp((*link)->key, key) == 0) {
            Entry *v = *link; *link = v->next;
            free(v->key); free(v); m->count--;
            return 1;
        }
    return 0;
}
void dump(const HashMap *m) {
    for (int i = 0; i < BUCKETS; i++) {
        printf("  [%d]", i);
        for (Entry *e = m->b[i]; e; e = e->next) printf(" -> %s:%d", e->key, e->value);
        printf("\n");
    }
}
void free_map(HashMap *m) {
    for (int i = 0; i < BUCKETS; i++)
        while (m->b[i]) { Entry *e = m->b[i]; m->b[i] = e->next; free(e->key); free(e); }
}
int main(void) {
```

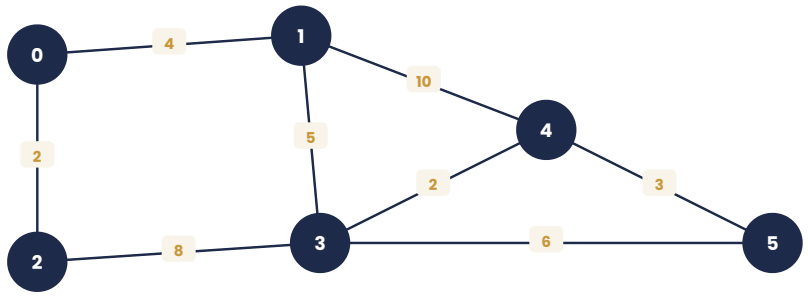
```
    HashMap stock = {{NULL}, 0};
    const char *fruit[] = {"apple", "banana", "cherry", "date", "elderberry", "fig", "grape", "kiwi"};
    for (int i = 0; i < 8; i++) put(&stock, fruit[i], (i + 1) * 10);
    put(&stock, "apple", 999);                      /* update existing key */
    dump(&stock);
    int v;
    if (get(&stock, "cherry", &v)) printf("get(cherry) = found %d\n", v);
    printf("get(mango)  = %s\n", get(&stock, "mango", &v) ? "found" : "missing");
    erase(&stock, "banana");
    printf("after erasing banana: %d entries, load factor %.2f\n", stock.count, (double)stock.count /
BUCKETS);
    free_map(&stock);
    return 0;
}
```

OUTPUT

```
  [0]
  [1] -> kiwi:80 -> grape:70
  [2] -> fig:60 -> cherry:30
  [3] -> apple:999
  [4] -> banana:20
  [5]
  [6] -> elderberry:50 -> date:40
get(cherry) = found 30
get(mango)  = missing
after erasing banana: 7 entries, load factor 1.00
```

19.8 Graphs

A **graph** G = (V, E) is a set of **vertices** (nodes) connected by **edges**. Edges may be **directed** (one-way, like follows on social media) or **undirected** (two-way, like roads), and may carry **weights** (distances, costs). Graphs model road maps, networks, dependencies between tasks and web links. A tree is a special graph: connected, with no cycles.



AN UNDIRECTED WEIGHTED GRAPH WITH 6 VERTICES AND 8 EDGES, USED IN ALL EXAMPLES BELOW

19.8.1 Representation: adjacency matrix and adjacency list

An **adjacency matrix** is a V × V array where `m[u][v]` holds the weight of edge u–v (0 for none). Checking an edge is O(1), but it uses O(V^2) memory even for sparse graphs. An **adjacency list** stores, for each vertex, a list of its neighbours: O(V + E) memory, ideal for sparse graphs (most real ones).

	[0]	[1]	[2]	[3]	[4]	[5]
[0]	0	4	2	0	0	0
[1]	4	0	0	5	10	0
[2]	2	0	0	8	0	0
[3]	0	5	8	0	2	6
[4]	0	10	0	2	0	3
[5]	0	0	0	6	3	0

ADJACENCY MATRIX (SYMMETRIC FOR AN UNDIRECTED GRAPH); EDGE 0–1 HAS WEIGHT 4

Vertex	Adjacency list (neighbour:weight)
0	1:4, 2:2
1	0:4, 3:5, 4:10
2	0:2, 3:8
3	1:5, 2:8, 4:2, 5:6
4	1:10, 3:2, 5:3
5	4:3, 3:6

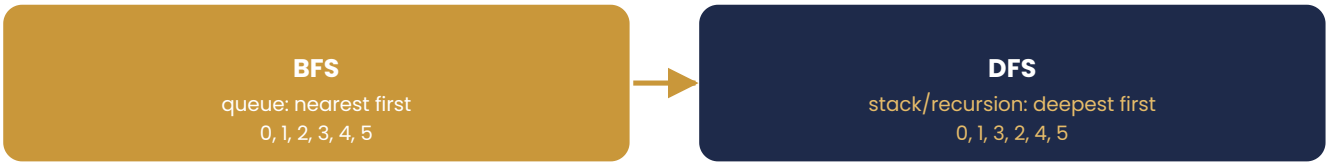
	Adjacency matrix	Adjacency list
Memory	$O(V^2)$	$O(V + E)$
Is u–v an edge?	$O(1)$	O(degree of u)
Visit all neighbours of u	$O(V)$	O(degree of u)
Best for	dense graphs, small V	sparse graphs, large V

19.8.2 Breadth-first search (BFS)

BFS explores the graph in layers: first the start vertex, then all its neighbours, then their unvisited neighbours, and so on. It uses a **queue** and a **visited** array. In an unweighted graph, BFS finds the path with the **fewest edges** to every vertex. Time $O(V + E)$.

19.8.3 Depth-first search (DFS)

DFS goes as deep as possible along one path before backtracking. It is naturally **recursive** (or uses an explicit stack). DFS is used for detecting cycles, topological sorting, finding connected components and solving mazes. Time $O(V + E)$.



VISITING ORDERS FROM VERTEX 0 (NEIGHBOURS TAKEN IN INCREASING ORDER)

19.8.4 Shortest path (Dijkstra, overview)

With weighted edges, fewest edges is not shortest distance. **Dijkstra's algorithm** (1956) finds the shortest distance from a source to every vertex when all weights are non-negative. It keeps a tentative distance for each vertex (0 for the source, infinity for the rest), repeatedly picks the unvisited vertex with the **smallest** tentative distance, marks it final, and **relaxes** its edges: if going through it gives a neighbour a shorter distance, update that distance. With a simple array it runs in $O(V^2)$; with a min-heap priority queue, $O((V + E) \log V)$. GPS navigation and network routing protocols are built on it.

BFS, DFS AND DIJKSTRA ON THE EXAMPLE GRAPH main.c

```
#include <stdio.h>
#include <limits.h>
#include <stdbool.h>

#define V 6
int g[V][V];                                  /* adjacency matrix with weights */

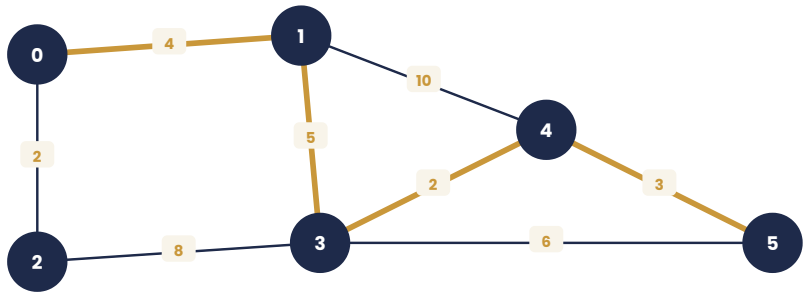
void add_edge(int u, int v, int w) { g[u][v] = g[v][u] = w; }

void bfs(int start) {
    bool seen[V] = {false};
    int queue[V], head = 0, tail = 0, dist[V];
    seen[start] = true; dist[start] = 0; queue[tail++] = start;
    printf("BFS from %d:", start);
    while (head < tail) {
        int u = queue[head++];
        printf(" %d(%d)", u, dist[u]);           /* vertex(edges from start) */
        for (int v = 0; v < V; v++)
            if (g[u][v] && !seen[v]) { seen[v] = true; dist[v] = dist[u] + 1; queue[tail++] = v; }
    }
    printf("\n");
}
void dfs_visit(int u, bool *seen) {
    seen[u] = true;
    printf(" %d", u);
    for (int v = 0; v < V; v++)
        if (g[u][v] && !seen[v]) dfs_visit(v, seen);
}
void dijkstra(int src) {
    int dist[V], prev[V];
    bool done[V] = {false};
    for (int i = 0; i < V; i++) { dist[i] = INT_MAX; prev[i] = -1; }
    dist[src] = 0;
    for (int k = 0; k < V; k++) {
        int u = -1;
        for (int i = 0; i < V; i++)                              /* closest unfinished vertex */
            if (!done[i] && dist[i] != INT_MAX && (u == -1 || dist[i] < dist[u])) u = i;
        if (u == -1) break;
        done[u] = true;
        for (int v = 0; v < V; v++)                              /* relax edges */
            if (g[u][v] && !done[v] && dist[u] + g[u][v] < dist[v]) { dist[v] = dist[u] + g[u][v];
prev[v] = u; }
    }
    printf("Dijkstra from %d:\n", src);
    for (int v = 0; v < V; v++) {
        int path[V], n = 0;
        for (int x = v; x != -1; x = prev[x]) path[n++] = x;
        printf("  to %d: distance %2d, path", v, dist[v]);
        while (n) printf(" %d", path[--n]);
        printf("\n");
    }
}
```

```c
int main(void) {
    add_edge(0, 1, 4); add_edge(0, 2, 2); add_edge(1, 3, 5); add_edge(2, 3, 8);
    add_edge(1, 4, 10); add_edge(3, 4, 2); add_edge(4, 5, 3); add_edge(3, 5, 6);
    bfs(0);
    bool seen[V] = {false};
    printf("DFS from 0:");
    dfs_visit(0, seen);
    printf("\n");
    dijkstra(0);
    return 0;
}
```

OUTPUT

```
BFS from 0: 0(0) 1(1) 2(1) 3(2) 4(2) 5(3)
DFS from 0: 0 1 3 2 4 5
Dijkstra from 0:
  to 0: distance  0, path 0
  to 1: distance  4, path 0 1
  to 2: distance  2, path 0 2
  to 3: distance  9, path 0 1 3
  to 4: distance 11, path 0 1 3 4
  to 5: distance 14, path 0 1 3 4 5
```



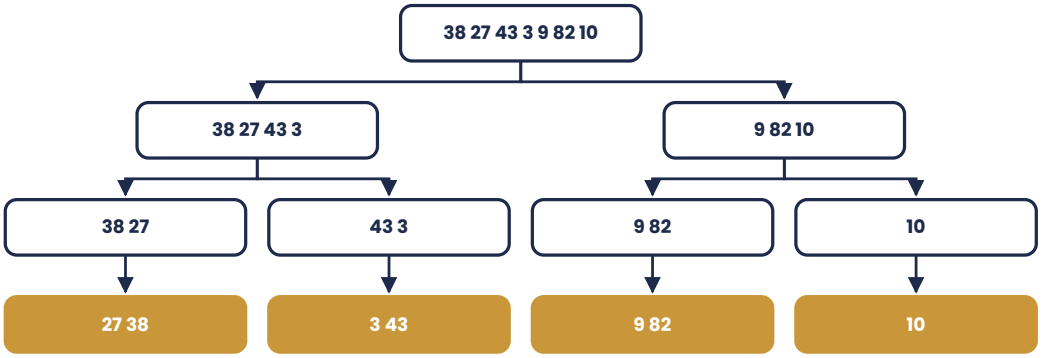
SHORTEST PATH FROM 0 TO 5: 0 → 1 → 3 → 4 → 5 (4 + 5 + 2 + 3 = 14), CHEAPER THAN 0 → 1 → 3 → 5 (15)

19.9 Sorting and searching algorithms

Module 7 introduced the $O(n^2)$ sorts (bubble, selection, insertion) and binary search. This section adds the fast divide-and-conquer sorts and a linear-time sort for small integer ranges.

19.9.1 Merge sort

Merge sort splits the array in half, sorts each half recursively, then **merges** the two sorted halves by repeatedly taking the smaller front element. There are log n levels of splitting and each level does O(n) merging work: O(n log n) always. It is **stable** (equal keys keep their order) but needs O(n) extra memory.



SPLITTING DOWN TO SORTED PAIRS; MERGING BACK UP GIVES 3 9 10 27 38 43 82

19.9.2 Quick sort

Quick sort picks a **pivot**, **partitions** the array so that smaller elements come before it and larger after it (the pivot is then in its final place), and recursively sorts the two parts. Average O(n log n) and in-place, usually the fastest in practice; worst case $O(n^2)$ when pivots are consistently bad (for example, the last element of an already sorted array), which random or median-of-three pivots avoid. Not stable. The C library's qsort is typically a tuned quicksort or introsort variant.

19.9.3 Counting sort

When keys are small integers in a known range [0, k], **counting sort** counts how many times each value occurs, turns the counts into starting positions (prefix sums), and places each element directly. It does no comparisons at all: O(n + k) time, O(n + k) memory, and it is stable. Ideal for marks out of 100, ages or bytes.

MERGE SORT, QUICK SORT AND COUNTING SORT main.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

static void merge(int *a, int lo, int mid, int hi, int *tmp) {
    int i = lo, j = mid, k = lo;
    while (i < mid && j < hi) tmp[k++] = a[i] <= a[j] ? a[i++] : a[j++];   /* <= keeps it stable */
    while (i < mid) tmp[k++] = a[i++];
    while (j < hi)  tmp[k++] = a[j++];
    memcpy(a + lo, tmp + lo, (size_t)(hi - lo) * sizeof *a);
}
static void msort(int *a, int lo, int hi, int *tmp) {        /* sorts a[lo..hi) */
    if (hi - lo < 2) return;
    int mid = lo + (hi - lo) / 2;
    msort(a, lo, mid, tmp);
    msort(a, mid, hi, tmp);
    merge(a, lo, mid, hi, tmp);
}
void merge_sort(int *a, int n) { int *tmp = malloc(n * sizeof *tmp); msort(a, 0, n, tmp); free(tmp); }

static void swap(int *x, int *y) { int t = *x; *x = *y; *y = t; }
static int partition(int *a, int lo, int hi) {               /* Lomuto, pivot = last */
    int pivot = a[hi], i = lo;
    for (int j = lo; j < hi; j++) if (a[j] < pivot) swap(&a[i++], &a[j]);
    swap(&a[i], &a[hi]);
    return i;
}
void quick_sort(int *a, int lo, int hi) {                    /* sorts a[lo..hi] */
    if (lo >= hi) return;
    int mid = lo + (hi - lo) / 2;
```

```
    swap(&a[mid], &a[hi]);                                  /* middle element as pivot */
    int p = partition(a, lo, hi);
    quick_sort(a, lo, p - 1);
    quick_sort(a, p + 1, hi);
}
void counting_sort(int *a, int n, int maxv) {
    int *count = calloc(maxv + 1, sizeof *count), *out = malloc(n * sizeof *out);
    for (int i = 0; i < n; i++) count[a[i]]++;
    for (int v = 1; v <= maxv; v++) count[v] += count[v - 1];   /* prefix sums = end positions */
    for (int i = n - 1; i >= 0; i--) out[--count[a[i]]] = a[i];  /* backwards keeps it stable */
    memcpy(a, out, n * sizeof *a);
    free(count); free(out);
}
void show(const char *t, const int *a, int n) { printf("%-14s", t); for (int i = 0; i < n; i++)
printf(" %d", a[i]); printf("\n"); }

int main(void) {
    int a[] = {38, 27, 43, 3, 9, 82, 10};
    int b[] = {38, 27, 43, 3, 9, 82, 10};
    int marks[] = {72, 45, 91, 45, 60, 100, 0, 72, 88};
    merge_sort(a, 7);              show("merge sort", a, 7);
    quick_sort(b, 0, 6);           show("quick sort", b, 7);
    counting_sort(marks, 9, 100);  show("counting sort", marks, 9);
    return 0;
}
```

OUTPUT

```
merge sort     3 9 10 27 38 43 82
quick sort     3 9 10 27 38 43 82
counting sort  0 45 45 60 72 72 88 91 100
```

19.9.4 Comparison of sorting algorithms by time, space and stability

Algorithm	Best	Average	Worst	Extra space	Stable	Notes
Bubble sort	O(n)	O(n²)	O(n²)	O(1)	yes	teaching only
Selection sort	O(n²)	O(n²)	O(n²)	O(1)	no	fewest swaps
Insertion sort	O(n)	O(n²)	O(n²)	O(1)	yes	great for small or nearly sorted data
Merge sort	O(n log n)	O(n log n)	O(n log n)	O(n)	yes	predictable; linked lists; external sorting
Quick sort	O(n log n)	O(n log n)	O(n²)	O(log n)	no	fastest in practice
Heap sort	O(n log n)	O(n log n)	O(n log n)	O(1)	no	guaranteed, in place
Counting sort	O(n + k)	O(n + k)	O(n + k)	O(n + k)	yes	small integer keys only

MEASURING QUADRATIC VS N LOG N main.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>

static void insertion(int *a, int n) {
    for (int i = 1; i < n; i++) { int k = a[i], j = i - 1; while (j >= 0 && a[j] > k) { a[j + 1] =
a[j]; j--; } a[j + 1] = k; }
}
static int cmp(const void *x, const void *y) { int a = *(const int *)x, b = *(const int *)y; return (a
> b) - (a < b); }

int main(void) {
    int sizes[] = {5000, 20000, 40000};
    printf("%8s %16s %16s\n", "n", "insertion O(n^2)", "qsort O(n log n)");
    for (int s = 0; s < 3; s++) {
        int n = sizes[s];
        int *a = malloc(n * sizeof *a), *b = malloc(n * sizeof *b);
        srand(1);
        for (int i = 0; i < n; i++) a[i] = b[i] = rand();
        clock_t t0 = clock(); insertion(a, n); double ti = (double)(clock() - t0) / CLOCKS_PER_SEC;
        t0 = clock(); qsort(b, n, sizeof *b, cmp); double tq = (double)(clock() - t0) /
CLOCKS_PER_SEC;
        printf("%8d %14.3f s %14.4f s\n", n, ti, tq);
        free(a); free(b);
    }
    return 0;
}
```

OUTPUT (times vary by machine)

```
       n insertion O(n^2) qsort O(n log n)
    5000          0.005 s         0.0006 s
   20000          0.050 s         0.0019 s
   40000          0.130 s         0.0041 s
```

Doubling n roughly **quadruples** the insertion-sort time but only slightly more than doubles the qsort time: the growth rates of the table, seen on a real machine.

Linked structures (lists, trees, graphs) are nodes joined by pointers; array-based ones (stacks, queues, heaps, hash tables) compute positions. Pick the structure by the operations you need most and their Big-O cost, draw the pointers before you code them, and free every node you allocate.

PRACTICE CHECKPOINT

Build three programs: a **menu-driven linked list** with all operations, a **stack-based infix-to-postfix converter** (with evaluation), and a **binary search tree with insert, delete and all traversals**.

Part 1: menu-driven linked list. It reuses the functions from 19.2.1 inside a menu loop.

PART 1: LINKED LIST MENU main.c

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct Node { int data; struct Node *next; } Node;
```

```
Node *new_node(int data) {
    Node *n = malloc(sizeof *n);
    if (!n) { perror("malloc"); exit(1); }
    n->data = data; n->next = NULL;
    return n;
}
void print_list(const Node *head) {
    for (const Node *p = head; p; p = p->next) printf("%d -> ", p->data);
    printf("NULL\n");
}
int count(const Node *head) { int c = 0; for (; head; head = head->next) c++; return c; }
void insert_front(Node **head, int data) { Node *n = new_node(data); n->next = *head; *head = n; }
void insert_end(Node **head, int data) {
    Node *n = new_node(data);
    if (!*head) { *head = n; return; }
    Node *p = *head;
    while (p->next) p = p->next;
    p->next = n;
}
int insert_at(Node **head, int pos, int data) {
    if (pos == 0) { insert_front(head, data); return 1; }
    Node *p = *head;
    for (int i = 0; p && i < pos - 1; i++) p = p->next;
    if (!p) return 0;
    Node *n = new_node(data); n->next = p->next; p->next = n;
    return 1;
}
int delete_value(Node **head, int value) {
    for (Node **link = head; *link; link = &(*link)->next)
        if ((*link)->data == value) { Node *v = *link; *link = v->next; free(v); return 1; }
    return 0;
}
int delete_at(Node **head, int pos) {
    Node **link = head;
    for (int i = 0; *link && i < pos; i++) link = &(*link)->next;
    if (!*link) return 0;
    Node *v = *link; *link = v->next; free(v);
    return 1;
}
Node *search(Node *head, int key, int *index) {
    int i = 0;
    for (Node *p = head; p; p = p->next, i++) if (p->data == key) { *index = i; return p; }
    return NULL;
}
void reverse(Node **head) {
    Node *prev = NULL, *cur = *head;
    while (cur) { Node *next = cur->next; cur->next = prev; prev = cur; cur = next; }
    *head = prev;
}
Node *reverse_rec(Node *head) {
    if (!head || !head->next) return head;
    Node *rest = reverse_rec(head->next);
    head->next->next = head; head->next = NULL;
    return rest;
}
Node *middle(Node *head) {
    Node *slow = head, *fast = head;
    while (fast && fast->next) { slow = slow->next; fast = fast->next->next; }
    return slow;
}
int has_loop(const Node *head) {
    const Node *slow = head, *fast = head;
    while (fast && fast->next) { slow = slow->next; fast = fast->next->next; if (slow == fast) return
1; }
```

```
    return 0;
}
void free_list(Node **head) { while (*head) { Node *n = (*head)->next; free(*head); *head = n; } }

int main(void) {
    Node *list = NULL;
    int choice, v, pos, idx;
    do {
        printf("\n1 front 2 end 3 at-pos 4 del-value 5 del-pos 6 search 7 reverse 8 show 0 quit: ");
        if (scanf("%d", &choice) != 1) break;
        switch (choice) {
        case 1: scanf("%d", &v); insert_front(&list, v); break;
        case 2: scanf("%d", &v); insert_end(&list, v); break;
        case 3: scanf("%d %d", &pos, &v); if (!insert_at(&list, pos, v)) printf("bad position");
break;
        case 4: scanf("%d", &v); printf(delete_value(&list, v) ? "deleted %d" : "%d not found", v);
break;
        case 5: scanf("%d", &pos); printf(delete_at(&list, pos) ? "deleted" : "bad position"); break;
        case 6: scanf("%d", &v); if (search(list, v, &idx)) printf("%d at index %d", v, idx); else
printf("not found"); break;
        case 7: reverse(&list); printf("reversed"); break;
        case 8: print_list(list); printf("count %d, middle %d", count(list), list ? middle(list)->data
: 0); break;
        }
    } while (choice != 0);
    free_list(&list);
    printf("bye\n");
    return 0;
}
```

CONSOLE (user input in gold)

```
1 front 2 end 3 at-pos 4 del-value 5 del-pos 6 search 7 reverse 8 show 0 quit: 2 20

1 front 2 end 3 at-pos 4 del-value 5 del-pos 6 search 7 reverse 8 show 0 quit: 2 40

1 front 2 end 3 at-pos 4 del-value 5 del-pos 6 search 7 reverse 8 show 0 quit: 1 10

1 front 2 end 3 at-pos 4 del-value 5 del-pos 6 search 7 reverse 8 show 0 quit: 3 2 30

1 front 2 end 3 at-pos 4 del-value 5 del-pos 6 search 7 reverse 8 show 0 quit: 8
10 -> 20 -> 30 -> 40 -> NULL
count 4, middle 30
1 front 2 end 3 at-pos 4 del-value 5 del-pos 6 search 7 reverse 8 show 0 quit: 4 20
deleted 20
1 front 2 end 3 at-pos 4 del-value 5 del-pos 6 search 7 reverse 8 show 0 quit: 6 40
40 at index 2
1 front 2 end 3 at-pos 4 del-value 5 del-pos 6 search 7 reverse 8 show 0 quit: 7
reversed
1 front 2 end 3 at-pos 4 del-value 5 del-pos 6 search 7 reverse 8 show 0 quit: 8
40 -> 30 -> 10 -> NULL
count 3, middle 30
1 front 2 end 3 at-pos 4 del-value 5 del-pos 6 search 7 reverse 8 show 0 quit: 0
bye
```

Part 2: infix to postfix with evaluation. Single-digit and multi-digit numbers, the operators + - * / ^ (^ is right-associative) and parentheses are supported.

PART 2: INFIX TO POSTFIX CONVERTER AND EVALUATOR — main.c

```
#include <stdio.h>
#include <stdlib.h>
#include <ctype.h>
```

```c
#include <string.h>
#include <math.h>

static int prec(char op) { return op == '^' ? 3 : (op == '*' || op == '/') ? 2 : (op == '+' || op ==
'-') ? 1 : 0; }

/* shunting-yard: tokens in the output are separated by spaces */
int to_postfix(const char *in, char *out) {
    char st[64]; int top = -1;
    *out = '\0';
    for (const char *p = in; *p; p++) {
        if (isspace((unsigned char)*p)) continue;
        if (isdigit((unsigned char)*p)) {
            while (isdigit((unsigned char)*p)) strncat(out, p++, 1);
            strcat(out, " ");
            p--;
        } else if (*p == '(') st[++top] = '(';
        else if (*p == ')') {
            while (top >= 0 && st[top] != '(') { strncat(out, &st[top--], 1); strcat(out, " "); }
            if (top < 0) return 0;                    /* unmatched ) */
            top--;                                    /* discard ( */
        } else if (prec(*p)) {
            while (top >= 0 && st[top] != '(' &&
                   (prec(st[top]) > prec(*p) || (prec(st[top]) == prec(*p) && *p != '^'))) {
                strncat(out, &st[top--], 1); strcat(out, " ");
            }
            st[++top] = *p;
        } else return 0;                              /* bad character */
    }
    while (top >= 0) { if (st[top] == '(') return 0; strncat(out, &st[top--], 1); strcat(out, " "); }
    return 1;
}
double eval_postfix(const char *pf) {
    double st[64]; int top = -1;
    char buf[128];
    strcpy(buf, pf);
    for (char *tok = strtok(buf, " "); tok; tok = strtok(NULL, " ")) {
        if (isdigit((unsigned char)tok[0])) { st[++top] = atof(tok); continue; }
        double b = st[top--], a = st[top--];
        switch (tok[0]) {
        case '+': st[++top] = a + b; break;
        case '-': st[++top] = a - b; break;
        case '*': st[++top] = a * b; break;
        case '/': st[++top] = a / b; break;
        case '^': st[++top] = pow(a, b); break;
        }
    }
    return st[top];
}
int main(void) {
    char line[128], pf[256];
    while (fgets(line, sizeof line, stdin)) {
        line[strcspn(line, "\n")] = '\0';
        if (!to_postfix(line, pf)) { printf("%-24s -> invalid expression\n", line); continue; }
        printf("%-24s -> %-26s = %g\n", line, pf, eval_postfix(pf));
    }
    return 0;
}
```

```
CONSOLE (user input in gold)
2 + 3 * 4
2 + 3 * 4               -> 2 3 4 * +              = 14
(2 + 3) * 4
(2 + 3) * 4             -> 2 3 + 4 *              = 20
10 - 4 - 3
10 - 4 - 3              -> 10 4 - 3 -             = 3
2 ^ 3 ^ 2
2 ^ 3 ^ 2               -> 2 3 2 ^ ^              = 512
(8 + 2) / (3 - 1) * 7
(8 + 2) / (3 - 1) * 7   -> 8 2 + 3 1 - / 7 *      = 35
((1 + 2)
((1 + 2)                -> invalid expression
```

Part 3: binary search tree. The BST functions from 19.5.4 driven by commands.

PART 3: BST DRIVEN BY COMMANDS (I = INSERT, D = DELETE, P = PRINT, Q = QUIT) — main.c

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct TNode { int key; struct TNode *left, *right; } TNode;

TNode *tnode(int k) { TNode *n = malloc(sizeof *n); n->key = k; n->left = n->right = NULL; return n; }

TNode *insert(TNode *root, int k) {
    if (!root) return tnode(k);
    if (k < root->key) root->left = insert(root->left, k);
    else if (k > root->key) root->right = insert(root->right, k);
    return root;                                    /* duplicates ignored */
}
TNode *search(TNode *root, int k, int *steps) {
    while (root) {
        (*steps)++;
        if (k == root->key) return root;
        root = k < root->key ? root->left : root->right;
    }
    return NULL;
}
TNode *min_node(TNode *r) { while (r && r->left) r = r->left; return r; }
TNode *max_node(TNode *r) { while (r && r->right) r = r->right; return r; }

TNode *delete(TNode *root, int k) {
    if (!root) return NULL;
    if (k < root->key) root->left = delete(root->left, k);
    else if (k > root->key) root->right = delete(root->right, k);
    else {
        if (!root->left)  { TNode *r = root->right; free(root); return r; }   /* 0 or 1 child */
        if (!root->right) { TNode *l = root->left;  free(root); return l; }
        TNode *succ = min_node(root->right);                                  /* 2 children */
        root->key = succ->key;
        root->right = delete(root->right, succ->key);
    }
    return root;
}
void preorder(const TNode *r)  { if (r) { printf("%d ", r->key); preorder(r->left); preorder(r->right); } }
void inorder(const TNode *r)   { if (r) { inorder(r->left); printf("%d ", r->key); inorder(r->right); } }
void postorder(const TNode *r) { if (r) { postorder(r->left); postorder(r->right); printf("%d ", r->key); } }
void level_order(TNode *r) {
    TNode *q[100]; int head = 0, tail = 0;          /* simple array queue */
```

```
    if (r) q[tail++] = r;
    while (head < tail) {
        TNode *n = q[head++];
        printf("%d ", n->key);
        if (n->left) q[tail++] = n->left;
        if (n->right) q[tail++] = n->right;
    }
}
int height(const TNode *r) { if (!r) return -1; int a = height(r->left), b = height(r->right); return
1 + (a > b ? a : b); }
int count(const TNode *r)  { return r ? 1 + count(r->left) + count(r->right) : 0; }
int leaves(const TNode *r) { if (!r) return 0; if (!r->left && !r->right) return 1; return leaves(r-
>left) + leaves(r->right); }
void free_tree(TNode *r)   { if (r) { free_tree(r->left); free_tree(r->right); free(r); } }   /*
postorder */

int main(void) {
    TNode *root = NULL;
    char cmd;
    int k;
    while (scanf(" %c", &cmd) == 1 && cmd != 'q') {
        if (cmd == 'i') { scanf("%d", &k); root = insert(root, k); }
        else if (cmd == 'd') { scanf("%d", &k); root = delete(root, k); printf("deleted %d\n", k); }
        else if (cmd == 'p') {
            printf("  inorder    : "); inorder(root);     printf("\n");
            printf("  preorder   : "); preorder(root);    printf("\n");
            printf("  postorder  : "); postorder(root);   printf("\n");
            printf("  level order: "); level_order(root); printf("\n");
            printf("  height %d, %d nodes, %d leaves\n", height(root), count(root), leaves(root));
        }
    }
    free_tree(root);
    return 0;
}
```

CONSOLE (user input in gold)

```
i 50 i 30 i 70 i 20 i 40 i 60 i 80
p
  inorder    : 20 30 40 50 60 70 80
  preorder   : 50 30 20 40 70 60 80
  postorder  : 20 40 30 60 80 70 50
  level order: 50 30 70 20 40 60 80
  height 2, 7 nodes, 4 leaves
d 30
deleted 30
d 50
deleted 50
p
  inorder    : 20 40 60 70 80
  preorder   : 60 40 20 70 80
  postorder  : 20 40 80 70 60
  level order: 60 40 70 20 80
  height 2, 5 nodes, 2 leaves
q
```

MODULE 20 · ADVANCED

Multi-file programs, libraries and build tools

Minimum duration **3 days** | Level **Advanced**

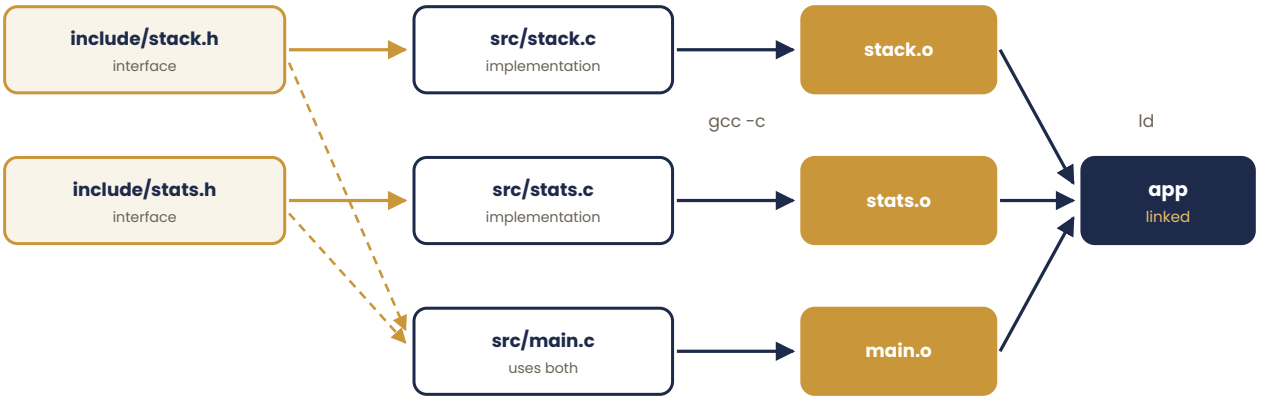
Real C projects contain dozens or thousands of source files. This module shows how to split a program into modules with clean interfaces, package reusable code as static and shared libraries, automate builds with Make and CMake, and keep the project organised and under version control. All terminal output in this module comes from building the same small project for real.

IN THIS MODULE

20.1 Splitting a program into files
20.2 Libraries
20.3 Make and Makefiles
20.4 CMake basics
20.5 Code organisation and style
20.6 Version control

20.1 Splitting a program into files

A single 5,000-line file is hard to read, slow to recompile (every change recompiles everything) and impossible for a team to edit at once. C's answer is **separate compilation**: each `.c` file is compiled on its own into an object file, and the linker joins them. Each module is a pair of files: a header that says **what** it offers and a source file that says **how**.



THE EXAMPLE PROJECT: HEADERS ARE #INCLUDED (DASHED), SOURCES ARE COMPILED SEPARATELY, OBJECTS ARE LINKED

Project layout

```
project/
├── include/
│   ├── stack.h        declarations: what the stack module offers
│   └── stats.h
├── src/
│   ├── stack.c        definitions: how it works
│   ├── stats.c
│   └── main.c         the program that uses both modules
├── Makefile
├── CMakeLists.txt
└── .gitignore
```

20.1.1 Header files (.h) for declarations

A header contains everything another file needs to **use** the module and nothing more: function prototypes, type names, constants and `extern` declarations of shared variables. It must be safe to include from any number of files, so it never defines functions or variables (except `static inline` functions).

INCLUDE/STACK.H C

```c
#ifndef STACK_H
#define STACK_H

#include <stdbool.h>
#include <stddef.h>

typedef struct Stack Stack;          /* opaque: fields are hidden in stack.c */

Stack *stack_create(void);
void   stack_destroy(Stack *s);
bool   stack_push(Stack *s, int value);
bool   stack_pop(Stack *s, int *out);
size_t stack_size(const Stack *s);

#endif /* STACK_H */
```

INCLUDE/STATS.H C

```c
#ifndef STATS_H
#define STATS_H

extern int stats_calls;              /* defined once in stats.c */

double mean(const int *a, int n);
int    maximum(const int *a, int n);

#endif /* STATS_H */
```

20.1.2 Source files (.c) for definitions

The source file includes **its own header first** (so the compiler checks that the definitions match the declarations), then defines every function and variable that the header promises.

SRC/STACK.C

C

```c
#include <stdlib.h>
#include "stack.h"

struct Stack {                      /* the full definition lives only here */
    int *items;
    size_t size, cap;
};

static bool grow(Stack *s) {        /* static: private to this file */
    size_t ncap = s->cap ? s->cap * 2 : 4;
    int *tmp = realloc(s->items, ncap * sizeof *tmp);
    if (!tmp) return false;
    s->items = tmp;
    s->cap = ncap;
    return true;
}
Stack *stack_create(void) { return calloc(1, sizeof(Stack)); }
void stack_destroy(Stack *s) { if (s) { free(s->items); free(s); } }
bool stack_push(Stack *s, int v) {
    if (s->size == s->cap && !grow(s)) return false;
    s->items[s->size++] = v;
    return true;
}
bool stack_pop(Stack *s, int *out) {
    if (s->size == 0) return false;
    *out = s->items[--s->size];
    return true;
}
size_t stack_size(const Stack *s) { return s->size; }
```

SRC/STATS.C

C

```c
#include "stats.h"

int stats_calls = 0;                /* the one definition */
static int last_n = 0;              /* private to this file */

double mean(const int *a, int n) {
    stats_calls++; last_n = n;
    long s = 0;
    for (int i = 0; i < n; i++) s += a[i];
    return n ? (double)s / n : 0.0;
}
int maximum(const int *a, int n) {
    stats_calls++; last_n = n;
    int m = a[0];
    for (int i = 1; i < n; i++) if (a[i] > m) m = a[i];
    return m;
}
```

20.1.3 Include guards

Every header above starts with `#ifndef X_H / #define X_H` and ends with `#endif` (Module 15.5). In a multi-file project this is essential: `main.c` may include `stats.h` directly and again through another header, and without the guard the compiler would see the declarations twice. Name the guard after the file, in upper case, to keep guards unique.

20.1.4 Compiling and linking multiple files

The simplest way passes all sources to gcc in one command. `-Iinclude` adds the include directory to the header search path.

SRC/MAIN.C (BUILT WITH: GCC -IINCLUDE SRC/MAIN.C SRC/STACK.C SRC/STATS.C -O APP) main.c

```c
#include <stdio.h>
#include "stack.h"
#include "stats.h"

int main(void) {
    int marks[] = {72, 91, 65, 88, 79};
    printf("mean = %.1f, max = %d\n", mean(marks, 5), maximum(marks, 5));
    printf("stats functions called %d times\n", stats_calls);

    Stack *s = stack_create();
    for (int i = 0; i < 5; i++) stack_push(s, marks[i]);
    printf("stack holds %zu items; popping:", stack_size(s));
    int v;
    while (stack_pop(s, &v)) printf(" %d", v);
    printf("\n");
    stack_destroy(s);
    return 0;
}
```

```
OUTPUT
mean = 79.0, max = 91
stats functions called 2 times
stack holds 5 items; popping: 79 88 65 91 72
```

Real projects compile each file separately with `-c` (compile only, producing a `.o` object file) and link the objects in a final step. After a change, only the changed file is recompiled.

Separate compilation

```
$ gcc -std=c17 -Wall -Iinclude -c src/stack.c -o stack.o
$ gcc -std=c17 -Wall -Iinclude -c src/stats.c -o stats.o
$ gcc -std=c17 -Wall -Iinclude -c src/main.c  -o main.o
$ ls *.o
main.o
stack.o
stats.o
$ gcc main.o stack.o stats.o -o app
$ ./app
mean = 79.0, max = 91
stats functions called 2 times
stack holds 5 items; popping: 79 88 65 91 72
```

`nm` lists the symbols in an object file. **T** means a function defined in the text (code) section, **B** a global variable in BSS, and a lower-case letter a symbol that is **local** to the file, such as a static variable. Forgetting an object file at link time produces the familiar "undefined reference" errors:

Terminal

```
$ nm stats.o
0000000000000004 b last_n
0000000000000085 T maximum
0000000000000000 T mean
0000000000000000 B stats_calls
$ gcc main.o stack.o -o app2              # stats.o forgotten
/usr/bin/ld: main.o: in function `main':
main.c:(.text+0x4c): undefined reference to `maximum'
/usr/bin/ld: main.c:(.text+0x5f): undefined reference to `mean'
/usr/bin/ld: main.c:(.text+0x85): undefined reference to `stats_calls'
collect2: error: ld returned 1 exit status
```

20.1.5 Using static and extern correctly across files

Want	Write	Where
A function only this file uses	`static void helper(void) {...}`	.c file only
A function other files call	prototype in .h, definition in .c	both
A global shared by several files	`extern int count;` in .h, `int count = 0;` in exactly one .c	both
A global private to one file	`static int count;`	.c file only
A constant shared by several files	`#define` or `enum` in .h, or `extern const`	.h

In the project, `grow()` and `last_n` are **static**: invisible outside their files (lower-case `b` in nm), so another module may use the same names without conflict. `stats_calls` is shared: declared `extern` in the header, defined once in stats.c. Default to static and expose only what other files need; global variables should be rare.

> **WATCH OUT**
> Defining the same non-static global in two .c files (for example writing `int stats_calls = 0;` in a header included twice) gives a "multiple definition of 'stats_calls'" link error. Since GCC 10 the old "common symbol" leniency is off by default (-fno-common).

20.2 Libraries

A **library** packages compiled object files so that many programs can reuse them without recompiling the sources. Users get the header (the interface) and the library file (the compiled implementation).

20.2.1 Static libraries (.a / .lib) with ar

A static library is simply an **archive** of object files made with `ar` (`.a` on Linux/macOS, `.lib` on Windows). At link time the linker copies the needed object files **into the executable**.

Creating and using a static library

```
$ ar rcs libds.a stack.o stats.o      # r: insert, c: create, s: write an index
$ ar t libds.a                          # list the members
stack.o
stats.o
$ gcc main.o -L. -lds -o app_static
$ ./app_static
mean = 79.0, max = 91
stats functions called 2 times
stack holds 5 items; popping: 79 88 65 91 72
```

20.2.2 Shared libraries (.so / .dll)

A shared library (`.so` on Linux, `.dylib` on macOS, `.dll` on Windows) is **not** copied into the program. The program records its name, and the **dynamic loader** finds and maps it into memory each time the program starts. Code must be compiled as position-independent (`-fPIC`) and linked with `-shared`.

Building and running with a shared library

```
$ gcc -fPIC -Iinclude -c src/stack.c -o stack.pic.o
$ gcc -fPIC -Iinclude -c src/stats.c -o stats.pic.o
$ gcc -shared stack.pic.o stats.pic.o -o libds.so
$ gcc main.o -L. -lds -o app_shared
$ ./app_shared
./app_shared: error while loading shared libraries: libds.so: cannot open shared object file: No such
file or directory
$ LD_LIBRARY_PATH=. ./app_shared
mean = 79.0, max = 91
stats functions called 2 times
stack holds 5 items; popping: 79 88 65 91 72
$ ldd app_shared
        linux-vdso.so.1 (0x00007f84597ce000)
        libds.so => not found
        libc.so.6 => /lib/x86_64-linux-gnu/libc.so.6 (0x00007f8459400000)
```

The first run fails because the loader searches only standard directories (/lib, /usr/lib, those listed in /etc/ld.so.conf). Fixes: install the library there and run `ldconfig`, set `LD_LIBRARY_PATH` for testing, or embed a search path in the program with `-Wl,-rpath,'$ORIGIN'`. `ldd` shows which shared libraries a program needs and where they were found.

20.2.3 Linking with -l and -L

`-lNAME` asks the linker for `libNAME.so` or `libNAME.a`; `-L dir` adds a directory to search. So `-L. -lds` finds `./libds.a` or `./libds.so` (the shared one is preferred if both exist). The math library is the classic example: `-lm` links `libm`.

> **WATCH OUT**
> Order matters with static libraries: the linker processes arguments left to right and only pulls from a library the symbols that are **already** needed. Put libraries **after** the files that use them: `gcc main.o -lds` works, `gcc -lds main.o` fails with undefined references.

20.2.4 Static vs dynamic linking

	Static (.a)	Shared (.so)
Library code is	copied into each executable	loaded at run time, shared in memory by all programs

	Static (.a)	Shared (.so)
Executable size	larger	smaller
Deployment	one self-contained file	the .so must be present on the target machine
Updating the library	relink every program	replace the .so; programs pick it up (security fixes!)
Start-up	slightly faster	small loading cost
Version problems	none at run time	possible ("DLL hell"), handled with sonames and versions

Sizes of the three builds

```
$ ls -l app_static app_shared | awk '{print $5, $9}'
16336 app_shared
16584 app_static
$ gcc -static main.o -L. -l:libds.a -o app_full     # everything static, libc included
$ ls -l app_full | awk '{print $5}'
785824
$ ldd app_full
        not a dynamic executable
```

20.3 Make and Makefiles

20.3.1 Why build automation

Typing compile commands by hand does not scale: you must remember every flag, compile files in the right order, and know which files changed. **make** reads a **Makefile** describing how each file is built from others and rebuilds **only what is out of date**, comparing file modification times.

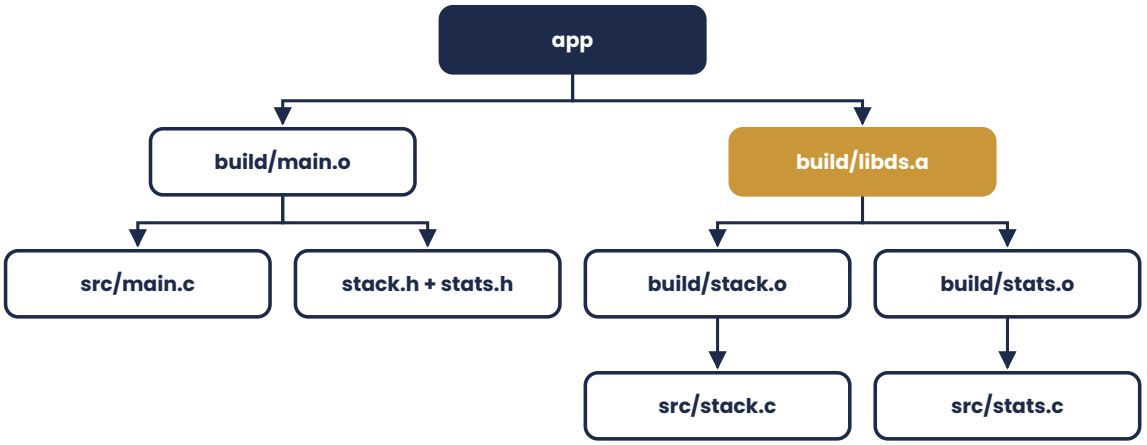
20.3.2 Targets, dependencies and recipes

A RULE

C

```
target: dependencies
<TAB>recipe (shell commands that build the target)
```

A **rule** says: to make **target**, first make sure every **dependency** is up to date; if any dependency is newer than the target (or the target does not exist), run the **recipe**. Recipe lines must start with a real **tab** character, not spaces; this is the most common Makefile error ("missing separator").



THE DEPENDENCY GRAPH MAKE WALKS FOR THE PROJECT

20.3.3 Variables and pattern rules

Variables keep settings in one place (`CC`, `CFLAGS`). **Automatic variables** refer to parts of the current rule: `$@` the target, `$<` the first dependency, `$^` all dependencies. A **pattern rule** with `%` describes how to build any file of one kind from another, so one rule compiles every .c file.

MAKEFILE

shell

```
# ---- variables ----
CC      := gcc
CFLAGS  := -std=c17 -Wall -Wextra -g -Iinclude
LDFLAGS :=
SRCDIR  := src
OBJDIR  := build
LIB     := $(OBJDIR)/libds.a
TARGET  := app

LIB_SRCS := $(SRCDIR)/stack.c $(SRCDIR)/stats.c
LIB_OBJS := $(LIB_SRCS:$(SRCDIR)/%.c=$(OBJDIR)/%.o)

# ---- targets ----
.PHONY: all clean run

all: $(TARGET)

$(TARGET): $(OBJDIR)/main.o $(LIB)
        $(CC) $(LDFLAGS) $(OBJDIR)/main.o -L$(OBJDIR) -lds -o $@

$(LIB): $(LIB_OBJS)
        ar rcs $@ $^

# pattern rule: how to make any build/X.o from src/X.c
$(OBJDIR)/%.o: $(SRCDIR)/%.c | $(OBJDIR)
        $(CC) $(CFLAGS) -c $< -o $@

$(OBJDIR):
        mkdir -p $@

# headers: rebuild objects when a header changes
$(OBJDIR)/main.o: include/stack.h include/stats.h
$(OBJDIR)/stack.o: include/stack.h
$(OBJDIR)/stats.o: include/stats.h

run: $(TARGET)
        ./$(TARGET)

clean:
        rm -rf $(OBJDIR) $(TARGET)
```

make rebuilds only what changed

```
$ make
mkdir -p build
gcc -std=c17 -Wall -Wextra -g -Iinclude -c src/main.c -o build/main.o
gcc -std=c17 -Wall -Wextra -g -Iinclude -c src/stack.c -o build/stack.o
gcc -std=c17 -Wall -Wextra -g -Iinclude -c src/stats.c -o build/stats.o
ar rcs build/libds.a build/stack.o build/stats.o
gcc  build/main.o -Lbuild -lds -o app
$ make
make: Nothing to be done for 'all'.
$ touch include/stats.h          # pretend we edited the header
$ make
gcc -std=c17 -Wall -Wextra -g -Iinclude -c src/main.c -o build/main.o
gcc -std=c17 -Wall -Wextra -g -Iinclude -c src/stats.c -o build/stats.o
ar rcs build/libds.a build/stack.o build/stats.o
gcc  build/main.o -Lbuild -lds -o app
```

After touching `stats.h`, make recompiled exactly the two files that include it and relinked; `stack.o` was untouched. In larger projects the header dependencies are generated automatically with `gcc -MMD -MP` instead of being listed by hand.

Automatic variable	Means	In `$(OBJDIR)/%.o: $(SRCDIR)/%.c`
`$@`	the target	build/stack.o
`$<`	first dependency	src/stack.c
`$^`	all dependencies	(all of them, used in the ar rule)
`$*`	the stem matched by %	stack

20.3.4 make clean and phony targets

Some targets are not files but **actions**: `all`, `clean`, `run`, `test`, `install`. Declaring them `.PHONY` tells make they are always out of date and never to be confused with a file of the same name (if a file called `clean` existed, make clean would otherwise do nothing).

Terminal

```
$ make run
./app
mean = 79.0, max = 91
stats functions called 2 times
stack holds 5 items; popping: 79 88 65 91 72
$ make clean
rm -rf build app
$ ls
Makefile  include  src
```

TIP

Useful flags: `make -j8` builds 8 files in parallel; `make -n` prints the commands without running them; `make CFLAGS=-O2` overrides a variable from the command line.

20.4 CMake basics

Makefiles are Unix-specific and get complicated for large, cross-platform projects. **CMake** is a **build-system generator**: you describe the project once in `CMakeLists.txt`, and CMake generates Makefiles, Ninja files, or Visual Studio and Xcode projects. Most modern open-source C and C++ projects use it, and IDEs such as CLion and VS Code open CMake projects directly.

20.4.1 CMakeLists.txt

CMAKELISTS.TXT

shell

```
cmake_minimum_required(VERSION 3.16)
project(DataStructures VERSION 1.0 LANGUAGES C)

set(CMAKE_C_STANDARD 17)
set(CMAKE_C_STANDARD_REQUIRED ON)
add_compile_options(-Wall -Wextra)

# a static library from the reusable sources
add_library(ds STATIC src/stack.c src/stats.c)
target_include_directories(ds PUBLIC include)

# the program, linked against the library
add_executable(app src/main.c)
target_link_libraries(app PRIVATE ds)
```

Command	Purpose
`project()`	name, version and languages
`set(CMAKE_C_STANDARD 17)`	adds -std=c17 (or the right flag for each compiler)
`add_library(name STATIC|SHARED files)`	builds a library
`add_executable(name files)`	builds a program
`target_include_directories()`	adds -I paths; PUBLIC passes them on to users of the target
`target_link_libraries()`	links a target against libraries

20.4.2 Building out of source

CMake builds in a **separate directory**, so generated files never mix with the sources and a clean build is just deleting that directory. The two-step workflow is **configure** then **build**:

Terminal

```
$ cmake -S . -B build             # configure: source dir . , build dir build
-- The C compiler identification is GNU 13.3.0
-- Detecting C compiler ABI info - done
-- Check for working C compiler: /usr/bin/cc - skipped
-- Detecting C compile features - done
-- Configuring done (0.2s)
-- Generating done (0.0s)
-- Build files have been written to: /home/student/project/build
$ cmake --build build             # build (runs make or ninja for you)
[ 20%] Building C object CMakeFiles/ds.dir/src/stack.c.o
[ 40%] Building C object CMakeFiles/ds.dir/src/stats.c.o
[ 60%] Linking C static library libds.a
[ 60%] Built target ds
[ 80%] Building C object CMakeFiles/app.dir/src/main.c.o
[100%] Linking C executable app
[100%] Built target app
$ ./build/app
mean = 79.0, max = 91
stats functions called 2 times
stack holds 5 items; popping: 79 88 65 91 72
```

Pass `-DCMAKE_BUILD_TYPE=Debug` or `Release` when configuring to choose -g or -O3 style settings, and `-G Ninja` to use the faster Ninja build tool.

20.5 Code organisation and style

20.5.1 Modular design and interfaces

- **One module, one responsibility**: stack.c manages stacks; it does not print menus or read files.
- **Small interfaces**: expose the minimum in the header; everything else is static.
- **Consistent prefixes**: C has one global namespace, so module functions share a prefix (`stack_push`, `stack_pop`) to avoid clashes.
- **Create/destroy pairs** and documented ownership (Module 13.5.2).
- **Dependencies flow one way**: low-level modules never include high-level headers.

20.5.2 Opaque pointers for information hiding

`stack.h` declares `typedef struct Stack Stack;` without its fields: an **incomplete type**. Users can hold a `Stack *` and pass it to the stack functions, but cannot see or touch the fields, cannot allocate a Stack themselves, and cannot depend on its layout. The full `struct Stack` appears only in stack.c. This is C's version of private members: you can change the implementation (say, from an array to a linked list) without recompiling or breaking any user. The standard `FILE` type works the same way from the programmer's point of view.

USERS CANNOT REACH INSIDE AN OPAQUE TYPE main.c

```c
#include <stdio.h>
#include "stack.h"

int main(void) {
    Stack *s = stack_create();
    stack_push(s, 1);
    printf("%zu\n", s->size);        /* not allowed: fields are hidden */
    Stack local;                      /* not allowed: size unknown here */
    return 0;
}
```

OUTPUT

```
main.c: In function ‘main’:
main.c:7:22: error: invalid use of incomplete typedef ‘Stack’
    7 |     printf("%zu\n", s->size);        /* not allowed: fields are hidden */
      |                      ^~
main.c:8:11: error: storage size of ‘local’ isn’t known
    8 |     Stack local;                      /* not allowed: size unknown here */
      |           ^~~~~
main.c:8:11: warning: unused variable ‘local’ [-Wunused-variable]
```

20.5.3 Coding standards (consistent naming, formatting with clang-format)

Teams agree on a **coding standard** so that all code looks as if one person wrote it. Typical C conventions:

Item	Common convention
Functions and variables	`snake_case`: `stack_push`, `item_count`

Item	Common convention
Types	`PascalCase` or `snake_case_t` (the `_t` suffix is reserved by POSIX, so many avoid it)
Macros and constants	`UPPER_CASE`: `MAX_USERS`
Files	lower case, one module per .h/.c pair
Indentation	4 spaces (or the kernel's 8-wide tabs), braces style chosen once
Well-known standards	Linux kernel style, GNU style, MISRA C (safety-critical), CERT C (security)

Formatting should be automatic, not argued about. **clang-format** rewrites source files according to a small `.clang-format` configuration; editors can run it on every save.

clang-format in action

```
$ cat .clang-format
BasedOnStyle: LLVM
IndentWidth: 4
ColumnLimit: 100
$ cat messy.c
#include <stdio.h>
int   sum(int*a,int n){int s=0;for(int i=0;i<n;i++)s+=a[i];return s;}
int main(void){ int v[]={1,2,3};
    printf("%d\n",sum(v,3)); return 0;}
$ clang-format messy.c
#include <stdio.h>
int sum(int *a, int n) {
    int s = 0;
    for (int i = 0; i < n; i++)
        s += a[i];
    return s;
}
int main(void) {
    int v[] = {1, 2, 3};
    printf("%d\n", sum(v, 3));
    return 0;
}
```

20.6 Version control

20.6.1 Git basics for C projects

Git records the history of a project as a series of **commits** (snapshots), lets you go back to any earlier version, work on **branches**, and collaborate through hosting services such as GitHub or GitLab. Every C project, even a student exercise, should live in a Git repository.

Command	Purpose
`git init`	create a repository in the current folder
`git status`	show changed and untracked files
`git add FILES`	stage changes for the next commit
`git commit -m "msg"`	record a snapshot
`git log --oneline`	list commits

Command	Purpose
`git diff`	show unstaged changes
`git switch -c feature`	create and switch to a branch
`git push / git pull`	exchange commits with a remote such as GitHub

A first commit

```
$ git init -b main
$ make -s                       # build products now exist too
$ git status --short            # build/ and app are hidden by .gitignore
?? .gitignore
?? CMakeLists.txt
?? Makefile
?? include/
?? src/
$ git add .
$ git commit -m "Stack and stats library with Makefile"
$ git log --oneline
c06be07 Stack and stats library with Makefile
$ echo '// tweak' >> src/main.c
$ git diff --stat
 src/main.c | 1 +
 1 file changed, 1 insertion(+)
```

20.6.2 .gitignore for build outputs

Only **sources** belong in version control: code, headers, build scripts, documentation, test data. Generated files (objects, libraries, executables, build directories) are recreated by the build, differ between machines and bloat the history. List them in `.gitignore`:

.GITIGNORE shell

```
build/
app
*.o
*.a
*.so
```

Split programs into header/source pairs with include guards, keep internals static, and hide structures behind opaque pointers. Package reusable code as static (.a) or shared (.so) libraries, build with Make or CMake so only changed files recompile, format automatically, and commit only sources to Git.

PRACTICE CHECKPOINT

Split the data structures project into separate **.h and .c files**, build it into a **static library**, and compile everything with a **Makefile**.

This module's project is exactly that solution, with the stack and statistics modules as the library. To extend it with the Module 19 structures, add one header/source pair per structure (for example `include/list.h` + `src/list.c`, `include/bst.h` + `src/bst.c`) and append them to `LIB_SRCS`; the pattern rule compiles them automatically. A typical session:

Checkpoint build

```
$ make clean && make -j4
rm -rf build app
mkdir -p build
gcc -std=c17 -Wall -Wextra -g -Iinclude -c src/main.c -o build/main.o
gcc -std=c17 -Wall -Wextra -g -Iinclude -c src/stack.c -o build/stack.o
gcc -std=c17 -Wall -Wextra -g -Iinclude -c src/stats.c -o build/stats.o
ar rcs build/libds.a build/stack.o build/stats.o
gcc  build/main.o -Lbuild -lds -o app
$ ar t build/libds.a
stack.o
stats.o
$ ./app
mean = 79.0, max = 91
stats functions called 2 times
stack holds 5 items; popping: 79 88 65 91 72
```

MODULE 21 · ADVANCED

Debugging, testing and memory safety

Minimum duration **3 days** Level **Advanced**

C gives you no safety net: an out-of-bounds write or a forgotten free compiles cleanly and may only misbehave weeks later. Professional C programmers therefore rely on tools: a debugger to watch the program run, memory checkers to catch invalid accesses and leaks, compiler warnings and static analysers to find bugs before running, tests to prove the code works, and secure-coding habits. Every session shown in this module was captured from the real tools.

IN THIS MODULE

21.1 Debugging with GDB
21.2 Memory error detection
21.3 Static analysis
21.4 Testing C code
21.5 Secure coding
21.6 Performance

21.1 Debugging with GDB

GDB, the GNU debugger, runs your program under its control. You can stop it at any line, execute it one line at a time, inspect and change variables, and examine the call stack after a crash. It replaces guesswork and scattered printf calls with direct observation.

21.1.1 Compiling with -g

`-g` stores **debugging information** in the executable: the mapping from machine instructions back to source lines, and the names and types of variables. Without it GDB shows only addresses and assembly. Also use `-O0` (no optimisation) while debugging; with optimisation, variables may be "optimized out" and lines executed out of order. Here is the program we will debug. The total is right but the average is absurd:

AVG.C: A BUG TO FIND avg.c

```c
#include <stdio.h>

double average(const int *a, int n) {
    int sum = 0;
    for (int i = 0; i <= n; i++)      /* BUG: <= reads one past the end */
        sum += a[i];
    return (double)sum / n;
}

int main(void) {
    int marks[5] = {70, 80, 90, 60, 50};
    int total = 0;
    for (int i = 0; i < 5; i++)
        total += marks[i];
    printf("total = %d\n", total);
    printf("average = %.2f\n", average(marks, 5));
    return 0;
}
```

OUTPUT (wrong average; the garbage value differs per run)

```
total = 350
average = 6623.00
```

21.1.2 Starting gdb, run, break, next, step, continue

Command	Short	Effect
`gdb ./prog`		load the program (add `--args ./prog a b` for arguments)
`break avg.c:5` / `break average`	b	stop at a line or at entry to a function
`run`	r	start the program
`next`	n	execute the current line, stepping **over** function calls
`step`	s	execute the current line, stepping **into** function calls
`continue`	c	run until the next breakpoint
`finish`		run until the current function returns, show its return value
`info breakpoints` / `delete 1`		list / remove breakpoints
`quit`	q	leave gdb

21.1.3 Inspecting variables: print, display, info locals

`print expr` evaluates any C expression with the program's current values; `print *a@5` shows 5 elements starting at a pointer. `display expr` re-prints the expression after every step. `info locals` and `info args` list all local variables and parameters.

A GDB session (lines typed by the user follow the (gdb) prompt)

```
$ gcc -g -O0 avg.c -o avg
$ gdb -q ./avg
(gdb) break average
Breakpoint 1 at 0x1178: file avg.c, line 4.
(gdb) run
Breakpoint 1, average (a=0x7fffffff9150, n=5) at avg.c:4
4           int sum = 0;
(gdb) info args
a = 0x7fffffff9150
n = 5
(gdb) next
5           for (int i = 0; i <= n; i++)      /* BUG: <= reads one past the end */
(gdb) next
6               sum += a[i];
(gdb) next
5           for (int i = 0; i <= n; i++)      /* BUG: <= reads one past the end */
(gdb) print sum
$1 = 70
(gdb) print a[0]
$2 = 70
(gdb) print *a@5
$3 = {70, 80, 90, 60, 50}
(gdb) display i
(gdb) display sum
(gdb) next
6               sum += a[i];
1: i = 1
2: sum = 70
(gdb) next
5           for (int i = 0; i <= n; i++)      /* BUG: <= reads one past the end */
1: i = 1
2: sum = 150
(gdb) info locals
i = 2
sum = 240
(gdb) finish
Run till exit from #0  average (a=0x7fffffff9150, n=5) at avg.c:5
0x0000555555555253 in main () at avg.c:16
16          printf("average = %.2f\n", average(marks, 5));
Value returned is $4 = 6623.3999999999996
```

Stepping through the loop shows sum growing correctly for the five marks; letting it run to the end with `finish` returns 6623.4, so the extra iteration with `i == 5` added a garbage value read from beyond the array. The fix is `i < n`.

21.1.4 Backtrace with bt after a crash

When a program crashes under GDB, it stops at the faulting instruction. `bt` (backtrace) lists the chain of function calls that led there, innermost first. `frame N` selects a frame so that you can print the variables of that function.

CRASH.C

crash.c

```c
#include <stdio.h>
#include <string.h>

typedef struct { char name[20]; int age; } Person;

int name_length(const Person *p) {
    return (int)strlen(p->name);          /* crashes if p is NULL */
}
Person *find(Person *list, int n, const char *name) {
    for (int i = 0; i < n; i++)
        if (strcmp(list[i].name, name) == 0) return &list[i];
    return NULL;
}
int main(void) {
    Person people[] = {{"Asha", 21}, {"Ravi", 23}};
    Person *p = find(people, 2, "Meera");          /* not found: NULL */
    printf("length = %d\n", name_length(p));
    return 0;
}
```

OUTPUT (crash)

```
Segmentation fault (core dumped)
```

Finding the cause of a segmentation fault

```
$ gcc -g crash.c -o crash
$ gdb -q ./crash
(gdb) run
Program received signal SIGSEGV, Segmentation fault.
__strlen_evex () at ../sysdeps/x86_64/multiarch/strlen-evex-base.S:81
(gdb) bt
#0  __strlen_evex () at ../sysdeps/x86_64/multiarch/strlen-evex-base.S:81
#1  0x00005555555551c5 in name_length (p=0x0) at crash.c:7
#2  0x00005555555552cb in main () at crash.c:17
(gdb) frame 2
#2  0x00005555555552cb in main () at crash.c:17
17          printf("length = %d\n", name_length(p));
(gdb) print p
$1 = (Person *) 0x0
(gdb) info locals
people = {{name = "Asha", '\000' <repeats 15 times>, age = 21}, {name = "Ravi", '\000' <repeats 15
times>, age = 23}}
p = 0x0
```

Read the backtrace from the bottom up: main called name_length with `p=0x0`, which passed a NULL-based address to strlen inside the C library (frame #0 is library code without source, which is normal). The real bug is in main: it did not check find's NULL result.

TIP

Without GDB, enable core dumps (`ulimit -c unlimited`) and load the dump later: `gdb ./crash core` then `bt` . Or simply compile with `-fsanitize=address` , which prints a similar backtrace automatically when the crash happens.

21.1.5 Watchpoints

A **watchpoint** stops the program whenever a variable's value **changes**, wherever in the code that happens, and shows the old and new values. It answers "who modified this?" without guessing where to

put breakpoints. `rwatch` stops on reads and `awatch` on any access.

WATCH.C watch.c

```c
#include <stdio.h>

int balance = 1000;

void deposit(int amt)  { balance += amt; }
void withdraw(int amt) { balance -= amt; }
void apply_fee(void)   { balance -= balance / 10; }   /* who changed the balance? */

int main(void) {
    deposit(500);
    withdraw(200);
    apply_fee();
    deposit(50);
    printf("balance = %d\n", balance);
    return 0;
}
```

OUTPUT

```
balance = 1220
```

Catching every change to balance

```
(gdb) break main
(gdb) run
Breakpoint 1, main () at watch.c:10
10          deposit(500);
(gdb) watch balance
Hardware watchpoint 2: balance
(gdb) continue
Hardware watchpoint 2: balance
Old value = 1000
New value = 1500
deposit (amt=500) at watch.c:5
(gdb) continue
Old value = 1500
New value = 1300
withdraw (amt=200) at watch.c:6
(gdb) continue
Old value = 1300
New value = 1170
apply_fee () at watch.c:7
7       void apply_fee(void)   { balance -= balance / 10; }   /* who changed the balance? */
```

21.1.6 Debugging in an IDE

IDEs put a graphical front end on the same debuggers: click in the margin to set a breakpoint, hover over variables to see values, and use toolbar buttons for step over (F10), step into (F11) and continue (F5).

IDE / editor	Debugger used	Notes
VS Code + C/C++ extension	gdb or lldb	configure in launch.json; works with CMake Tools
CLion	gdb or lldb (bundled)	opens CMake projects directly
Visual Studio (Windows)	MSVC debugger	excellent; also shows memory windows
Code::Blocks, Dev-C++	gdb	common in colleges

IDE / editor	Debugger used	Notes
Xcode (macOS)	lldb	lldb commands are close to gdb's

21.2 Memory error detection

Memory bugs often produce no visible symptom: the wrong value, a crash much later, or nothing at all until a different machine. Dynamic checkers watch every memory access while the program runs and report the exact line. The following program has **three planted bugs**: a heap overflow (missing +1 for the terminator), a use after free, and a leak. Run normally, it seems to work:

LEAKY.C: THREE HIDDEN BUGS

leaky.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

char *make_greeting(const char *name) {
    char *g = malloc(strlen("Hello, ") + strlen(name));   /* BUG 1: no room for '\0' */
    strcpy(g, "Hello, ");
    strcat(g, name);
    return g;
}
int main(void) {
    int *scores = malloc(5 * sizeof *scores);
    for (int i = 0; i < 5; i++) scores[i] = i * 10;
    free(scores);
    printf("first score: %d\n", scores[0]);             /* BUG 2: use after free */

    for (int i = 0; i < 3; i++) {
        char *g = make_greeting("Asha");
        printf("%s\n", g);                               /* BUG 3: never freed */
    }
    return 0;
}
```

OUTPUT (looks almost fine; the first value is garbage)

```
first score: 1506834206
Hello, Asha
Hello, Asha
Hello, Asha
```

21.2.1 Valgrind Memcheck: leaks, invalid reads and writes

Valgrind runs the unmodified program on a simulated CPU and checks every memory access. No recompilation is needed (but `-g` gives line numbers). Programs run 10 to 50 times slower. It is available on Linux; macOS and Windows users use the sanitizers instead.

Valgrind finds all three bugs (output trimmed)

```
$ gcc -g leaky.c -o leaky
$ valgrind --leak-check=full ./leaky
==10038== Invalid read of size 4
==10038==    at 0x10929F: main (leaky.c:15)
==10038==  Address 0x4a7d040 is 0 bytes inside a block of size 20 free'd
==10038==    at 0x484988F: free (in /usr/libexec/valgrind/vgpreload_memcheck-amd64-linux.so)
==10038==    by 0x10929A: main (leaky.c:14)
==10038==  Block was alloc'd at
==10038==    at 0x4846828: malloc (in /usr/libexec/valgrind/vgpreload_memcheck-amd64-linux.so)
==10038==    by 0x109254: main (leaky.c:12)
==10038==
==10038== Invalid write of size 1
==10038==    at 0x484EF6E: strcat (in /usr/libexec/valgrind/vgpreload_memcheck-amd64-linux.so)
==10038==    by 0x109238: make_greeting (leaky.c:8)
==10038==    by 0x1092CE: main (leaky.c:18)
==10038==  Address 0x4a7e0eb is 0 bytes after a block of size 11 alloc'd
==10038==    at 0x4846828: malloc (in /usr/libexec/valgrind/vgpreload_memcheck-amd64-linux.so)
==10038==    by 0x109210: make_greeting (leaky.c:6)
first score: 0
Hello, Asha
Hello, Asha
Hello, Asha
==10038== HEAP SUMMARY:
==10038==     in use at exit: 33 bytes in 3 blocks
==10038==   total heap usage: 5 allocs, 2 frees, 4,149 bytes allocated
==10038==
==10038== 33 bytes in 3 blocks are definitely lost in loss record 1 of 1
==10038==    at 0x4846828: malloc (in /usr/libexec/valgrind/vgpreload_memcheck-amd64-linux.so)
==10038==    by 0x109210: make_greeting (leaky.c:6)
==10038==    by 0x1092CE: main (leaky.c:18)
==10038==
==10038== ERROR SUMMARY: 8 errors from 4 contexts (suppressed: 0 from 0)
```

Valgrind message	Meaning
Invalid read / write of size N	access outside any valid block: overflow, underflow or freed memory
… inside a block … free'd	use after free; it also tells you where it was freed and allocated
… 0 bytes after a block of size 11	one past the end: classic off-by-one
Conditional jump depends on uninitialised value(s)	a decision made with garbage
definitely lost	leak: no pointer to the block remains
still reachable	not freed but still pointed to at exit (often acceptable)
Invalid free() / double free	freeing something not allocated, or twice

21.2.2 AddressSanitizer (-fsanitize=address)

AddressSanitizer (ASan) is built into GCC and Clang. The compiler inserts checks around every memory access and surrounds every block with poisoned "red zones". It is much faster than Valgrind (about 2× slowdown), finds stack and global overflows that Valgrind misses, works on Linux, macOS and Windows (MSVC), and stops at the **first** error with a clear report. It includes LeakSanitizer, which reports leaks at exit.

ASan stops at the first bug (shadow-memory dump trimmed)

```
$ gcc -g -fsanitize=address -fno-omit-frame-pointer leaky.c -o leaky_asan
$ ./leaky_asan
=================================================================
==10048==ERROR: AddressSanitizer: heap-use-after-free on address 0x503000000040
READ of size 4 at 0x503000000040 thread T0
    #0 0x5640283003f5 in main leaky.c:15

0x503000000040 is located 0 bytes inside of 20-byte region [0x503000000040,0x503000000054)
freed by thread T0 here:
    #0 0x7f738e0fc4d8 in free ../../../../src/libsanitizer/asan/asan_malloc_linux.cpp:52
    #1 0x5640283003be in main leaky.c:14

previously allocated by thread T0 here:
    #0 0x7f738e0fd9c7 in malloc ../../../../src/libsanitizer/asan/asan_malloc_linux.cpp:69
    #1 0x56402830033e in main leaky.c:12

SUMMARY: AddressSanitizer: heap-use-after-free leaky.c:15 in main
```

Fix that bug, rebuild, and ASan reports the next one (the heap-buffer-overflow in make_greeting at line 8), and after that, LeakSanitizer reports the 33 leaked bytes. Make `-fsanitize=address,undefined` the default for all debug builds.

21.2.3 UndefinedBehaviorSanitizer (-fsanitize=undefined)

UBSan detects undefined behaviour that is not about memory: signed overflow, shifts out of range, division by zero, misaligned pointers, invalid enum or bool values, out-of-bounds indexes on arrays of known size. Each is reported as a `runtime error` with the line, and by default the program continues.

BUILT WITH -FSANITIZE=UNDEFINED

main.c

```c
#include <stdio.h>
#include <limits.h>

int main(void) {
    int big = INT_MAX;
    int shift = 33;
    int zero = 0;
    int arr[3] = {1, 2, 3};
    int idx = 3;
    printf("%d\n", big + 1);          /* signed overflow   */
    printf("%d\n", 1 << shift);       /* shift too far     */
    printf("%d\n", arr[idx]);         /* index out of range */
    printf("%d\n", 10 / zero);        /* division by zero  */
    return 0;
}
```

OUTPUT (UBSan reports; values after each report are undefined)

```
main.c:10:5: runtime error: signed integer overflow: 2147483647 + 1 cannot be represented in type
'int'
-2147483648
main.c:11:22: runtime error: shift exponent 33 is too large for 32-bit type 'int'
2
main.c:12:23: runtime error: index 3 out of bounds for type 'int [3]'
main.c:12:5: runtime error: load of address 0x7ffd0c33cd98 with insufficient space for an object of
type 'int'
0x7ffd0c33cd98: note: pointer points here
 03 00 00 00  00 31 f3 46 ce 99 9c 3a  00 00 00 00 00 00 00 00  d8 ce 33 0c fd 7f 00 00  50 ce 33 0c
              ^
1190342912
main.c:13:23: runtime error: division by zero
Floating point exception (core dumped)
```

21.3 Static analysis

Static analysis examines the **source code without running it**. It finds bugs on paths your tests never execute, at the cost of occasional false alarms.

21.3.1 Compiler warnings (-Wall -Wextra -Werror)

The cheapest static analyser is the compiler itself. By default GCC warns about very little. `-Wall` enables the common warnings, `-Wextra` adds more, and `-Werror` turns every warning into an error so they cannot be ignored. Every warning deserves to be fixed.

FIVE BUGS, BUILT WITH -WALL -WEXTRA -WERROR

warn.c

```c
#include <stdio.h>

int grade(int marks) {
    if (marks >= 90) return 'A';
    else if (marks >= 75) return 'B';
}
int main(void) {
    int count;
    unsigned int limit = 10;
    for (int i = 0; i < limit; i++)
        count++;
    if (count = 10) printf("ten\n");
    printf("%d %s\n", grade(80));
    return 0;
}
```

OUTPUT

```
warn.c: In function ‘main’:
warn.c:10:23: error: comparison of integer expressions of different signedness: ‘int’ and ‘unsigned
int’ [-Werror=sign-compare]
   10 |     for (int i = 0; i < limit; i++)
      |                       ^
warn.c:12:9: error: suggest parentheses around assignment used as truth value [-Werror=parentheses]
   12 |     if (count = 10) printf("ten\n");
      |         ^~~~~
warn.c:13:17: error: format ‘%s’ expects a matching ‘char *’ argument [-Werror=format=]
   13 |     printf("%d %s\n", grade(80));
      |                ~^
      |                 |
      |                 char *
warn.c: In function ‘grade’:
warn.c:6:1: error: control reaches end of non-void function [-Werror=return-type]
    6 | }
      | ^
cc1: all warnings being treated as errors
```

Without any flags, GCC reported only the format mismatch. With `-Wall -Wextra` it also catches the missing return, the assignment used as a condition and the signed/unsigned comparison. The uninitialised `count` is reported only when optimisation is on (`-O2 -Wall`), because that analysis runs inside the optimiser, which is a good reason to compile with both debug and optimised settings in CI. Other useful flags: `-Wshadow`, `-Wconversion`, `-Wpedantic`, `-Wformat=2`, `-Wnull-dereference`.

21.3.2 Tools: cppcheck, clang-tidy

Dedicated analysers follow values along every path through a function. **clang-tidy** (part of LLVM) combines the Clang Static Analyzer with hundreds of style and bug-prone checks. **cppcheck** is a lightweight, free analyser that is easy to run on whole directories (`cppcheck --enable=all src/`). Commercial tools (Coverity, PVS-Studio, Polyspace) go further.

clang-tidy on warn.c

```
$ clang-tidy warn.c --checks='-*,clang-analyzer-*,bugprone-*' -- -std=c17
warn.c:11:9: warning: The expression is an uninitialized value. The computed value will also be
garbage [clang-analyzer-core.uninitialized.Assign]
warn.c:12:9: warning: Although the value stored to 'count' is used in the enclosing expression, the
value is never actually read from 'count' [clang-analyzer-deadcode.DeadStores]
warn.c:12:15: warning: an assignment within an 'if' condition is bug-prone [bugprone-assignment-in-if-
condition]
```

TIDY2.C

C

```c
int copy_name(const char *src) {
    char *buf = malloc(8);
    strcpy(buf, src);
    if (strlen(buf) > 5) return 1;      /* leaks buf on this path */
    free(buf);
    return 0;
}
```

A leak found without running the code

```
$ clang-tidy tidy2.c --checks='-*,clang-analyzer-*' -- -std=c17
tidy2.c:6:5: warning: Call to function 'strcpy' is insecure as it does not provide bounding of the
memory buffer. Replace unbounded copy functions with analogous functions that support length arguments
such as 'strlcpy'. CWE-119 [clang-analyzer-security.insecureAPI.strcpy]
tidy2.c:7:33: warning: Potential leak of memory pointed to by 'buf' [clang-analyzer-unix.Malloc]
```

Technique	Finds	Cost
Compiler warnings	type mismatches, suspicious constructs	free, every build
Static analysers	path-dependent bugs: leaks, NULL use, dead code	seconds to minutes; some false positives
Sanitizers (ASan, UBSan)	real memory and UB errors on executed paths	2× slower test runs
Valgrind	memory errors and leaks in any binary	10 to 50× slower
Tests	wrong results	writing the tests

21.4 Testing C code

21.4.1 Writing test functions with assert

A **unit test** calls a function with known inputs and checks the outputs. Without any framework, a test is a function full of `assert`s, and a test program runs them all. Test the normal cases, the **edge cases** (empty, one element, maximum values, negative numbers) and the error cases.

TESTS WRITTEN WITH ASSERT

main.c

```c
#include <stdio.h>
#include <assert.h>
#include <string.h>
#include <ctype.h>

/* ---- code under test ---- */
int is_palindrome(const char *s) {
    size_t i = 0, j = strlen(s);
    while (i < j) {
        if (!isalnum((unsigned char)s[i])) { i++; continue; }
        if (!isalnum((unsigned char)s[j - 1])) { j--; continue; }
        if (tolower((unsigned char)s[i]) != tolower((unsigned char)s[j - 1])) return 0;
        i++; j--;
    }
    return 1;
}
int clamp(int v, int lo, int hi) { return v < lo ? lo : v > hi ? hi : v; }
```

```c
/* ---- tests ---- */
static int passed = 0;
#define CHECK(expr) do { assert(expr); passed++; } while (0)

void test_palindrome(void) {
    CHECK(is_palindrome("madam"));
    CHECK(is_palindrome("A man, a plan, a canal: Panama"));
    CHECK(is_palindrome(""));                 /* edge: empty */
    CHECK(is_palindrome("x"));                /* edge: one char */
    CHECK(!is_palindrome("hello"));
}
void test_clamp(void) {
    CHECK(clamp(5, 0, 10) == 5);
    CHECK(clamp(-3, 0, 10) == 0);             /* below range */
    CHECK(clamp(99, 0, 10) == 10);            /* above range */
    CHECK(clamp(10, 0, 10) == 10);            /* boundary */
}
int main(void) {
    test_palindrome();
    test_clamp();
    printf("all %d checks passed\n", passed);
    return 0;
}
```

OUTPUT

```
all 9 checks passed
```

If any check fails, assert prints the failing expression with its file and line and aborts, so the problem is obvious. The drawback is that the first failure stops everything; frameworks run all tests and report a summary.

21.4.2 Unit testing frameworks (Unity, CMocka, Check)

Framework	Style	Notes
Unity	single .c/.h pair, `TEST_ASSERT_EQUAL_INT(5, f())`	tiny, popular in embedded systems; pairs with CMock for mocks
CMocka	`assert_int_equal()`, built-in mocking	used by Samba, libssh; each test isolated
Check	runs each test in a separate process	a crashing test does not stop the suite
Criterion	automatic test registration, parallel runs	modern, little boilerplate
CTest (CMake)	runs any test programs	add_test() in CMakeLists.txt; make test

WHAT A UNITY TEST FILE LOOKS LIKE

C

```c
#include "unity.h"
#include "stats.h"

void setUp(void) {}       /* runs before each test */
void tearDown(void) {}    /* runs after each test  */

void test_mean_of_three(void) {
    int a[] = {1, 2, 3};
    TEST_ASSERT_EQUAL_DOUBLE(2.0, mean(a, 3));
}
void test_maximum_negative(void) {
    int a[] = {-5, -2, -9};
    TEST_ASSERT_EQUAL_INT(-2, maximum(a, 3));
}
int main(void) {
    UNITY_BEGIN();
    RUN_TEST(test_mean_of_three);
    RUN_TEST(test_maximum_negative);
    return UNITY_END();          /* prints "2 Tests 0 Failures 0 Ignored" */
}
```

21.5 Secure coding

Memory bugs in C are not just crashes: attackers exploit them to run their own code. A large share of serious security vulnerabilities in C and C++ software are memory-safety bugs. Secure coding means treating **all input as hostile** and never trusting a size you did not check.

21.5.1 Buffer overflow and how to prevent it

A **buffer overflow** writes past the end of an array. On the stack it can overwrite the saved return address, letting an attacker redirect execution. Modern defences (stack canaries, non-executable stacks, ASLR) make exploitation harder, but the only real fix is never to overflow.

A STACK BUFFER OVERFLOW CAUGHT BY THE STACK PROTECTOR

main.c

```c
#include <stdio.h>
#include <string.h>

void greet_unsafe(const char *name) {
    char buf[8];
    strcpy(buf, name);                      /* no size check */
    printf("hi %s\n", buf);
}
int main(void) {
    greet_unsafe("Asha");
    greet_unsafe("Bartholomew-the-third");  /* 21 chars into 8 bytes */
    return 0;
}
```

OUTPUT (crash)

```
hi Asha
hi Bartholomew-the-third
*** stack smashing detected ***: terminated
Aborted (core dumped)
```

Unsafe	Safe replacement
`gets(buf)` (removed in C11)	`fgets(buf, sizeof buf, stdin)`
`strcpy(dst, src)`	`snprintf(dst, sizeof dst, "%s", src)`, or check `strlen(src) < sizeof dst` first
`strcat(dst, src)`	track remaining space; `snprintf` at an offset
`sprintf(buf, ...)`	`snprintf(buf, sizeof buf, ...)` and check the return value
`scanf("%s", buf)`	`scanf("%19s", buf)` with a width one less than the size
`strncpy`	does not always add \0; terminate manually or use snprintf

21.5.2 Format string vulnerabilities

Passing user input as the **format** argument (`printf(input)`) lets the user insert conversion specifiers. `%p` or `%x` read values from the stack that were never passed (leaking secrets such as addresses that defeat ASLR), and `%n` even writes to memory. Always use a constant format: `printf("%s", input)`.

USER INPUT USED AS A FORMAT STRING — main.c

```c
#include <stdio.h>
#include <string.h>

int main(void) {
    char input[64];
    if (!fgets(input, sizeof input, stdin)) return 1;
    input[strcspn(input, "\n")] = '\0';
    printf("safe  : ");
    printf("%s", input);          /* input is data */
    printf("\nunsafe: ");
    printf(input);                /* input is treated as a format! */
    printf("\n");
    return 0;
}
```

CONSOLE (user input in gold)

```
%p %p %p %p
safe  : %p %p %p %p
unsafe: 0x55972e0ec6b0 (nil) (nil) 0x73
```

GCC warns about this with `-Wformat -Wformat-security` (enabled by default on many Linux distributions).

21.5.3 Integer overflow

Integer overflow becomes a security bug when the result is used as a **size**. If `count * size` wraps around, malloc allocates a tiny block and the following loop writes far beyond it. Check before multiplying, or use the overflow-checking builtins (C23 standardises them as `ckd_add`, `ckd_mul` in `<stdckdint.h>`).

OVERFLOW IN A SIZE CALCULATION

main.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <stdint.h>

void *alloc_array(size_t count, size_t size) {
    size_t bytes;
    if (__builtin_mul_overflow(count, size, &bytes)) {      /* C23: ckd_mul(&bytes, count, size) */
        fprintf(stderr, "refused: %zu x %zu overflows\n", count, size);
        return NULL;
    }
    return malloc(bytes);
}
int main(void) {
    size_t count = SIZE_MAX / 4 + 2;              /* attacker-controlled count */
    printf("naive count * 4 = %zu bytes (wrapped around!)\n", count * 4);
    void *p = alloc_array(count, 4);
    printf("checked allocation: %s\n", p ? "allocated" : "NULL");
    void *q = alloc_array(1000, 4);
    printf("1000 x 4: %s\n", q ? "allocated" : "NULL");
    free(q);
    return 0;
}
```

OUTPUT

```
naive count * 4 = 4 bytes (wrapped around!)
refused: 4611686018427387905 x 4 overflows
checked allocation: NULL
1000 x 4: allocated
```

21.5.4 Safe functions and input validation

- **Validate every input**: range-check numbers, length-check strings, reject unexpected characters. Prefer an allow-list ("digits only") to a deny-list.
- Read numbers with `strtol` / `strtod` and check `errno` and the end pointer, never `atoi`.
- Check the return value of **every** library call: malloc, fopen, scanf, snprintf, fclose.
- Use sizes from `sizeof`, never hard-coded twice.
- Never build shell commands from input (`system()`); use exec with an argument array.
- Compile with protections: `-D_FORTIFY_SOURCE=2 -O2 -fstack-protector-strong -fPIE -pie`, and test with sanitizers.
- Follow a standard such as **SEI CERT C** or **MISRA C** in safety-critical work.

STRICT VALIDATION OF A NUMBER main.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <errno.h>

/* returns 1 and stores the age if the line is a whole number 0..150 */
int read_age(const char *line, int *age) {
    char *end;
    errno = 0;
    long v = strtol(line, &end, 10);
    if (end == line) return 0;                          /* no digits */
    while (*end == ' ' || *end == '\n') end++;
    if (*end != '\0') return 0;                         /* trailing junk */
    if (errno == ERANGE || v < 0 || v > 150) return 0; /* out of range */
    *age = (int)v;
    return 1;
}
int main(void) {
    char line[32];
    while (fgets(line, sizeof line, stdin)) {
        int age;
        line[strcspn(line, "\n")] = '\0';
        if (read_age(line, &age)) printf("%-22s -> accepted age %d\n", line, age);
        else printf("%-22s -> rejected\n", line);
    }
    return 0;
}
```

```
CONSOLE (user input in gold)
42
42                     -> accepted age 42
-5
-5                     -> rejected
20 years
20 years               -> rejected

                       -> rejected
99999999999999999999
99999999999999999999   -> rejected
150
150                    -> accepted age 150
```

21.6 Performance

21.6.1 Optimisation levels -O0 to -O3

Level	What the compiler does	Use for
`-O0`	nothing: every variable lives in memory (default)	debugging
`-O1`	basic optimisations, quick to compile	debug builds that should run reasonably
`-O2`	almost all safe optimisations: inlining, scheduling, loop optimisations	release builds (the usual choice)

Level	What the compiler does	Use for
`-O3`	-O2 plus aggressive inlining and vectorisation	number crunching; measure, it is not always faster
`-Os` / `-Oz`	optimise for size	embedded systems
`-Og`	optimisations that do not hurt debugging	development
`-march=native`	use every instruction of this CPU	programs that run only on this machine

A one-line function shows the difference. At -O0 the parameter is stored to the stack and loaded back; at -O2 it stays in registers:

Generated assembly at -O0 and -O2

```
$ cat square.c
int square(int x) { return x * x; }
$ gcc -S -O0 square.c -o -          # (directives removed)
        pushq   %rbp
        movq    %rsp, %rbp
        movl    %edi, -4(%rbp)
        movl    -4(%rbp), %eax
        imull   %eax, %eax
        popq    %rbp
        ret
$ gcc -S -O2 square.c -o -
        imull   %edi, %edi
        movl    %edi, %eax
        ret
```

A loop summing 200 million ints: -O2 is 3.6 times faster than -O0 here

```
$ gcc -O0 bench.c -o bench && ./bench
total = 7450000000, time 0.513 s
$ gcc -O1 bench.c -o bench && ./bench
total = 7450000000, time 0.226 s
$ gcc -O2 bench.c -o bench && ./bench
total = 7450000000, time 0.143 s
$ gcc -O3 bench.c -o bench && ./bench
total = 7450000000, time 0.145 s
```

21.6.2 Profiling with gprof or perf

Never guess where a program spends its time; **measure**. A profiler shows which functions consume the time, so you optimise the 5% of the code that matters. **gprof** needs the program compiled with `-pg`; running it writes `gmon.out`. **perf** (Linux) samples any program without recompiling: `perf record ./prog` then `perf report`.

PROF.C (EXCERPT) C

```c
int is_prime(int n) {
    if (n < 2) return 0;
    for (int d = 2; d < n; d++)          /* slow: tries every divisor */
        if (n % d == 0) return 0;
    return 1;
}
long sum_digits(long n) { long s = 0; while (n) { s += n % 10; n /= 10; } return s; }
/* main calls both for every i below 60000 */
```

gprof points straight at the hot function

```
$ gcc -O0 -pg prof.c -o prof
$ ./prof
6057 primes below 60000, digit sum 1230000
$ gprof -b -p prof gmon.out
Flat profile:

Each sample counts as 0.01 seconds.
  %   cumulative   self              self     total
 time   seconds   seconds    calls  us/call  us/call  name
100.00      0.41     0.41    60000     6.83     6.83  is_prime
  0.00      0.41     0.00    60000     0.00     0.00  sum_digits
```

All the time is in `is_prime`. Testing divisors only up to √n (`d * d <= n`) turns its O(n) loop into O(√n), a far bigger win than any compiler flag: algorithmic improvements beat micro-optimisations.

21.6.3 Cache-friendly code (overview)

Main memory is about 100 times slower than the CPU. The CPU hides this with **caches** that load memory in **64-byte lines**. Code that accesses memory **sequentially** uses every byte of each line it loads; code that jumps around wastes most of each line. C stores 2D arrays row by row, so iterating row-wise is sequential and column-wise is strided:

SAME WORK, DIFFERENT MEMORY ORDER (BUILT WITH -O2) main.c

```c
#include <stdio.h>
#include <time.h>

#define N 4096
static int m[N][N];

int main(void) {
    for (int i = 0; i < N; i++) for (int j = 0; j < N; j++) m[i][j] = 1;
    long s = 0;
    clock_t t = clock();
    for (int i = 0; i < N; i++)              /* row by row: sequential */
        for (int j = 0; j < N; j++) s += m[i][j];
    double row = (double)(clock() - t) / CLOCKS_PER_SEC;
    t = clock();
    for (int j = 0; j < N; j++)              /* column by column: jumps 16 KB each step */
        for (int i = 0; i < N; i++) s += m[i][j];
    double col = (double)(clock() - t) / CLOCKS_PER_SEC;
    printf("sum %ld\nrow-major    %.3f s\ncolumn-major %.3f s (%.1fx slower)\n", s, row, col, col /
row);
    return 0;
}
```

OUTPUT (times vary by machine)

```
sum 33554432
row-major    0.010 s
column-major 0.171 s (17.9x slower)
```

- Traverse arrays in memory order; keep data that is used together close together.
- Prefer arrays of structures that are processed sequentially to pointer-heavy linked structures for hot loops.
- Reorder structure members to reduce padding (Module 14.3) so more records fit in each cache line.
- Move work out of inner loops, and let the profiler confirm every change.

Debug with GDB (-g -O0): breakpoints, stepping, print, backtrace and watchpoints. Catch memory errors with Valgrind and AddressSanitizer, undefined behaviour with UBSan, and suspicious code with -Wall -Wextra -Werror and clang-tidy. Test every function, validate every input, use bounded functions, and measure with a profiler before optimising.

PRACTICE CHECKPOINT

Find and fix a planted **memory leak**, a **buffer overflow** and a **use-after-free** using gdb, Valgrind and AddressSanitizer.

Use `leaky.c` from 21.2. The tools located all three bugs: Valgrind reported the invalid read at line 15 (use after free, freed at line 14), the invalid write in strcat at line 8 into an 11-byte block allocated at line 6 (overflow: "Hello, Asha" needs 12 bytes), and 33 bytes definitely lost from line 6 (leak). In GDB, `break make_greeting`, `finish` and `print strlen(g)` confirm the 11-character string in an 11-byte block. The corrected program:

FIXED.C, BUILT WITH -FSANITIZE=ADDRESS: NO REPORTS fixed.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

char *make_greeting(const char *name) {
    char *g = malloc(strlen("Hello, ") + strlen(name) + 1);  /* FIX 1: +1 for '\0' */
    if (!g) return NULL;
    strcpy(g, "Hello, ");
    strcat(g, name);
    return g;
}
int main(void) {
    int *scores = malloc(5 * sizeof *scores);
    if (!scores) return 1;
    for (int i = 0; i < 5; i++) scores[i] = i * 10;
    printf("first score: %d\n", scores[0]);
    free(scores);                                          /* FIX 2: free after the last use */
    scores = NULL;

    for (int i = 0; i < 3; i++) {
        char *g = make_greeting("Asha");
        if (!g) return 1;
        printf("%s\n", g);
        free(g);                                           /* FIX 3: free every greeting */
    }
    return 0;
}
```

OUTPUT

```
first score: 0
Hello, Asha
Hello, Asha
Hello, Asha
```

Valgrind confirms the fix

```
$ gcc -g fixed.c -o fixed
$ valgrind --leak-check=full ./fixed
first score: 0
Hello, Asha
Hello, Asha
Hello, Asha
==10058== HEAP SUMMARY:
==10058==     in use at exit: 0 bytes in 0 blocks
==10058==   total heap usage: 5 allocs, 5 frees, 4,152 bytes allocated
==10058==
==10058== All heap blocks were freed -- no leaks are possible
==10058==
==10058== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
```

MODULE 22 · ADVANCED

Systems programming in C (Linux)

Minimum duration **6 days** | Level **Advanced**

C is the language of operating systems: the Linux kernel, the C library and most system tools are written in it. This module uses C to talk to the operating system directly: raw file descriptors, creating and controlling processes, pipes between processes, signals, POSIX threads with proper synchronisation, and TCP network sockets. All programs run on Linux (most also on macOS); process IDs and addresses in the outputs differ on every run.

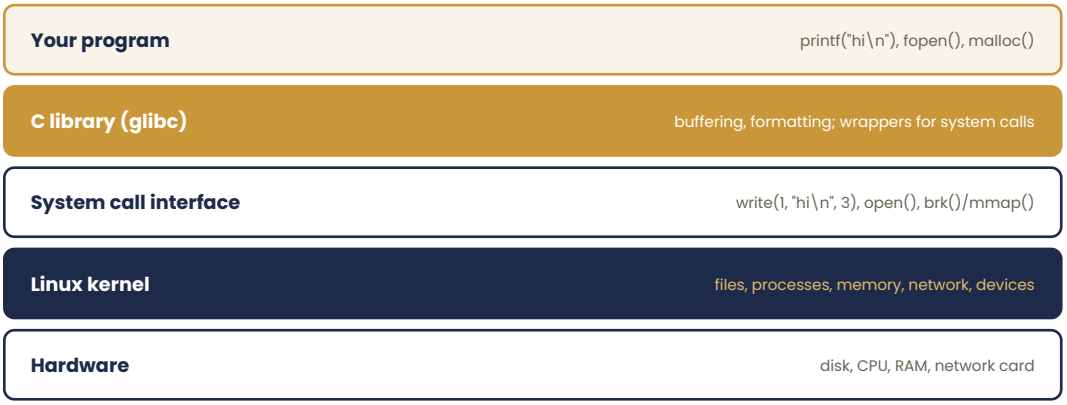
IN THIS MODULE

22.1 Introduction
22.2 Low-level file I/O
22.3 Processes
22.4 Inter-process communication
22.5 Signals
22.6 Threads (POSIX threads)
22.7 Network programming

22.1 Introduction

22.1.1 System calls vs library functions

A user program cannot touch hardware or other processes directly. It asks the **kernel** through **system calls** such as `read`, `write`, `open`, `fork` and `socket`. Each system call switches the CPU from user mode to kernel mode and back, which is relatively expensive. **Library functions** such as `printf`, `fopen` and `malloc` run in user space and call system calls when needed: printf formats text into a buffer, then calls `write` once when the buffer is flushed.



LIBRARY FUNCTIONS SIT ON TOP OF SYSTEM CALLS

strace shows the system calls a program makes: two printf calls, one write

```
$ cat hi.c
#include <stdio.h>
int main(void) { printf("hello\n"); printf("world\n"); return 0; }
$ gcc hi.c -o hi && strace -e trace=write ./hi > /dev/null
write(1, "hello\nworld\n", 12)          = 12
+++ exited with 0 +++
```

22.1.2 POSIX standard

POSIX (Portable Operating System Interface, IEEE 1003) standardises the system interface of Unix-like systems: functions such as `open`, `fork`, `pthread_create` and `socket`, the shell and the basic utilities. Code written to POSIX runs on Linux, macOS, the BSDs and other Unix systems with little or no change. Windows is not POSIX; it has its own Win32 API (CreateProcess, CreateThread, Winsock), though WSL runs Linux programs unchanged. The headers live in `<unistd.h>`, `<fcntl.h>`, `<sys/types.h>`, `<sys/wait.h>` and friends.

> NOTE
> Compiling with a strict standard such as `-std=c17` hides everything that is not ISO C. To use POSIX functions you must request them with a **feature-test macro** defined before the first #include: `#define _POSIX_C_SOURCE 200809L` for POSIX 2008, or `#define _DEFAULT_SOURCE` (glibc) for POSIX plus common BSD/Linux extensions such as usleep. Every program in this module starts with that line. With `-std=gnu17` (GCC's default) they are visible automatically.

22.1.3 Man pages (man 2, man 3)

The manual is the primary reference. It is divided into sections; the same name may exist in several, so give the section number:

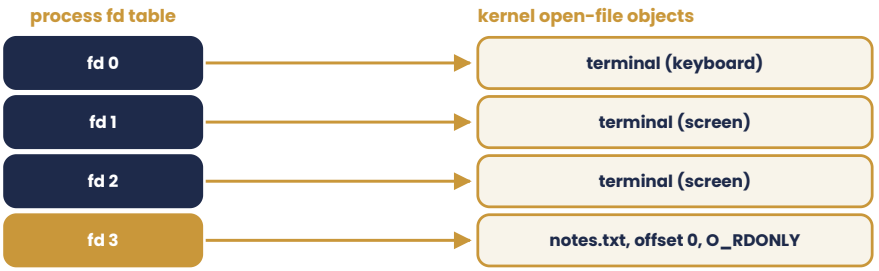
Section	Contents	Example
1	user commands	`man 1 printf` (the shell command)
2	system calls	`man 2 open`, `man 2 fork`
3	library functions	`man 3 printf`, `man 3 pthread_create`
7	overviews	`man 7 signal`, `man 7 socket`

Each page lists the headers to include, the prototype, the return value, the **ERRORS** (errno values) and examples. `man -k keyword` searches all pages.

22.2 Low-level file I/O

22.2.1 File descriptors

The kernel refers to every open file, pipe, socket or terminal of a process by a small non-negative integer, a **file descriptor** (fd): an index into the process's table of open files. Three are open at start: **0** = standard input, **1** = standard output, **2** = standard error. `open` returns the lowest unused number, so the first file you open is usually 3. A `FILE *` from stdio is a buffered wrapper around a file descriptor (`fileno(fp)` returns it).



FILE DESCRIPTORS INDEX THE KERNEL'S OPEN FILES

22.2.2 open(), read(), write(), close(), lseek()

Call	Prototype	Returns
open	`int open(const char *path, int flags, mode_t mode)`	new fd, or -1
read	`ssize_t read(int fd, void *buf, size_t n)`	bytes read (may be fewer than n), 0 at end of file, -1 on error
write	`ssize_t write(int fd, const void *buf, size_t n)`	bytes written (may be fewer), -1 on error
lseek	`off_t lseek(int fd, off_t offset, int whence)`	new offset, or -1
close	`int close(int fd)`	0, or -1

`flags` combine an access mode (`O_RDONLY`, `O_WRONLY`, `O_RDWR`) with options (`O_CREAT`, `O_TRUNC`, `O_APPEND`, `O_EXCL`) using | (bit flags, Module 17.3.5). There is no buffering: each call is a system call, so read and write in large blocks.

RAW FILE I/O WITH FILE DESCRIPTORS

main.c

```c
#define _DEFAULT_SOURCE     /* expose POSIX/Linux functions under -std=c17 */
#include <stdio.h>
#include <string.h>
#include <fcntl.h>
#include <unistd.h>

int main(void) {
    int fd = open("data.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if (fd == -1) { perror("open"); return 1; }
    printf("opened data.txt as fd %d\n", fd);
    const char *msg = "Systems programming in C\n";
    ssize_t w = write(fd, msg, strlen(msg));
    printf("wrote %zd bytes\n", w);
    close(fd);

    fd = open("data.txt", O_RDONLY);
    char buf[64];
    ssize_t n = read(fd, buf, sizeof buf - 1);
    buf[n] = '\0';
    printf("read %zd bytes: %s", n, buf);

    off_t pos = lseek(fd, 8, SEEK_SET);            /* jump to offset 8 */
    n = read(fd, buf, 11);
    buf[n] = '\0';
    printf("from offset %ld: \"%s\"\n", (long)pos, buf);
    printf("file size via lseek: %ld\n", (long)lseek(fd, 0, SEEK_END));
    close(fd);

    write(STDOUT_FILENO, "written directly to fd 1\n", 25);
    return 0;
}
```

OUTPUT

```
opened data.txt as fd 3
wrote 25 bytes
read 25 bytes: Systems programming in C
from offset 8: "programming"
file size via lseek: 25
written directly to fd 1
```

A robust copy loop must handle **short reads and writes**: read may return fewer bytes than asked, and write may write fewer than given (common with pipes and sockets).

CP WITH SYSTEM CALLS: ./PROG NOTES.TXT COPY.TXT

main.c

```c
#define _DEFAULT_SOURCE     /* expose POSIX/Linux functions under -std=c17 */
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
#include <errno.h>

ssize_t write_all(int fd, const char *buf, size_t n) {
    size_t done = 0;
    while (done < n) {
        ssize_t w = write(fd, buf + done, n - done);
        if (w == -1) { if (errno == EINTR) continue; return -1; }   /* retry if interrupted */
        done += (size_t)w;
    }
    return (ssize_t)done;
}
int main(int argc, char *argv[]) {
    if (argc != 3) { fprintf(stderr, "usage: %s SRC DST\n", argv[0]); return 1; }
    int in = open(argv[1], O_RDONLY);
    if (in == -1) { perror(argv[1]); return 1; }
    int out = open(argv[2], O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if (out == -1) { perror(argv[2]); close(in); return 1; }
    char buf[8192];
    ssize_t n;
    long total = 0;
    while ((n = read(in, buf, sizeof buf)) > 0) {
        if (write_all(out, buf, (size_t)n) == -1) { perror("write"); break; }
        total += n;
    }
    if (n == -1) perror("read");
    close(in);
    close(out);
    printf("copied %ld bytes\n", total);
    return 0;
}
```

OUTPUT

```
copied 77 bytes
```

22.2.3 File permissions

Every file has an owner, a group, and nine permission bits: read (r = 4), write (w = 2) and execute (x = 1) for the **user** (owner), the **group** and **others**. They are written in octal: `0644` = `rw-r--r--`, `0755` = `rwxr-xr-x`. The `mode` argument of open sets them for a new file, reduced by the process's **umask** (typically 022). `chmod()` changes them and `stat()` reads them.

Octal	Symbolic	Typical for
0644	rw-r--r--	ordinary files
0600	rw-------	private files: keys, passwords
0755	rwxr-xr-x	programs and directories
0700	rwx------	private directories

SETTING AND READING PERMISSIONS

main.c

```c
#define _DEFAULT_SOURCE     /* expose POSIX/Linux functions under -std=c17 */
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/stat.h>

void show_mode(const char *path) {
    struct stat st;
    if (stat(path, &st) == -1) { perror(path); return; }
    char s[10];
    const char *rwx = "rwxrwxrwx";
    for (int i = 0; i < 9; i++) s[i] = (st.st_mode & (0400 >> i)) ? rwx[i] : '-';
    s[9] = '\0';
    printf("%-12s %04o %s  %lld bytes\n", path, st.st_mode & 0777, s, (long long)st.st_size);
}
int main(void) {
    umask(022);
    int fd = open("secret.txt", O_WRONLY | O_CREAT | O_TRUNC, 0600);
    write(fd, "pin=1234\n", 9);
    close(fd);
    fd = open("public.txt", O_WRONLY | O_CREAT | O_TRUNC, 0666);   /* umask removes w for group/others
*/
    close(fd);
    show_mode("secret.txt");
    show_mode("public.txt");
    chmod("public.txt", 0755);
    show_mode("public.txt");
    return 0;
}
```

OUTPUT

```
secret.txt   0600 rw-------  9 bytes
public.txt   0644 rw-r--r--  0 bytes
public.txt   0755 rwxr-xr-x  0 bytes
```

22.3 Processes

A **process** is a running program: its code, its memory (stack, heap, data), its open file descriptors and other state, identified by a **process ID** (PID). Processes form a tree: every process except the first (PID 1, init/systemd) is created by a parent.

22.3.1 Process ID: getpid(), getppid()

WHO AM I? main.c

```c
#define _DEFAULT_SOURCE     /* expose POSIX/Linux functions under -std=c17 */
#include <stdio.h>
#include <unistd.h>

int main(void) {
    printf("my PID      : %d\n", (int)getpid());
    printf("parent PID  : %d (the shell or program that started me)\n", (int)getppid());
    printf("user ID     : %d\n", (int)getuid());
    return 0;
}
```

OUTPUT (IDs differ per run)

```
my PID      : 14143
parent PID  : 11975 (the shell or program that started me)
user ID     : 0
```

22.3.2 fork() and parent/child processes

`pid_t fork(void)` creates a new process that is an almost exact **copy** of the caller: same code, a copy of its memory, copies of its file descriptors, and it continues from the same point, returning from fork. fork is called once but **returns twice**: in the **parent** it returns the child's PID; in the **child** it returns 0; on failure it returns -1. After fork both processes run concurrently, so the order of their output is not guaranteed.



ONE CALL, TWO RETURNS: THE RETURN VALUE TELLS EACH PROCESS WHO IT IS

FORK: SEPARATE COPIES OF MEMORY main.c

```c
#define _DEFAULT_SOURCE     /* expose POSIX/Linux functions under -std=c17 */
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main(void) {
    int x = 10;
    printf("before fork: PID %d\n", (int)getpid());
    fflush(stdout);                         /* avoid duplicating buffered output */
    pid_t pid = fork();
    if (pid == -1) { perror("fork"); return 1; }
    if (pid == 0) {                         /* child */
        x += 5;
        printf("child : PID %d, parent %d, x = %d\n", (int)getpid(), (int)getppid(), x);
        return 0;
    }
    wait(NULL);                             /* parent waits so the order is fixed */
    x -= 5;
    printf("parent: PID %d, child was %d, x = %d\n", (int)getpid(), (int)pid, x);
    return 0;
}
```

OUTPUT (PIDs differ per run)

```
before fork: PID 14149
child : PID 14150, parent 14149, x = 15
parent: PID 14149, child was 14150, x = 5
```

Each process changed **its own copy** of x. Copying is cheap because Linux uses **copy-on-write**: pages are shared until one of the processes writes to them.

WATCH OUT

Call `fflush(stdout)` before fork. Otherwise text still sitting in the stdio buffer is copied into the child and printed twice, a classic puzzle when output is redirected to a file.

22.3.3 exec() family

`exec` **replaces** the current program with a new one: same PID, same open file descriptors, entirely new code and memory. If it succeeds it never returns. fork + exec is how every Unix shell runs commands: fork a child, and the child execs the command.

Function	Arguments given as	Searches PATH	Environment
`execl`	list: `execl("/bin/ls", "ls", "-l", NULL)`	no	inherited
`execlp`	list	yes	inherited
`execv`	array: `char *argv[]`	no	inherited
`execvp`	array	yes	inherited
`execle` / `execve`	list / array	no	given explicitly (execve is the real system call)

22.3.4 wait() and waitpid()

A parent collects a finished child's **exit status** with `wait(&status)` (any child) or `waitpid(pid, &status, options)` (a specific child; `WNOHANG` makes it non-blocking). Macros decode the status: `WIFEXITED` / `WEXITSTATUS` for a normal exit, `WIFSIGNALED` / `WTERMSIG` if a signal killed it.

FORK, EXEC AND WAITPID TOGETHER main.c

```c
#define _DEFAULT_SOURCE     /* expose POSIX/Linux functions under -std=c17 */
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

void run(char *const argv[]) {
    pid_t pid = fork();
    if (pid == 0) {
        execvp(argv[0], argv);          /* only returns on error */
        perror(argv[0]);
        _exit(127);                     /* conventional "command not found" code */
    }
    int status;
    waitpid(pid, &status, 0);
    if (WIFEXITED(status))
        printf("[%s exited with status %d]\n", argv[0], WEXITSTATUS(status));
    else if (WIFSIGNALED(status))
        printf("[%s killed by signal %d]\n", argv[0], WTERMSIG(status));
}
int main(void) {
    char *cmd1[] = {"echo", "hello from echo", NULL};
    char *cmd2[] = {"ls", "/no/such/dir", NULL};
    char *cmd3[] = {"no_such_program", NULL};
    char *cmd4[] = {"sh", "-c", "kill -TERM $$", NULL};
    run(cmd1);
    run(cmd2);
    run(cmd3);
    run(cmd4);
    return 0;
}
```

OUTPUT

```
hello from echo
[echo exited with status 0]
ls: cannot access '/no/such/dir': No such file or directory
[ls exited with status 2]
no_such_program: No such file or directory
[no_such_program exited with status 127]
[sh killed by signal 15]
```

22.3.5 Zombie and orphan processes

When a child exits, the kernel keeps a small entry with its exit status until the parent collects it with wait. In between, the child is a **zombie** (state `Z`, shown as <defunct> in ps). Zombies use no memory but occupy a process-table slot; a long-running parent that never waits accumulates them. If the **parent** exits first, the child becomes an **orphan** and is adopted by PID 1 (or a "subreaper" process), which waits for it automatically.

WATCHING A ZOMBIE APPEAR AND DISAPPEAR main.c

```c
#define _DEFAULT_SOURCE     /* expose POSIX/Linux functions under -std=c17 */
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main(void) {
    pid_t pid = fork();
    if (pid == 0) _exit(0);                     /* child exits immediately */
    sleep(1);                                   /* parent has not waited yet */
    char cmd[64];
    snprintf(cmd, sizeof cmd, "ps -o pid=,stat=,comm= -p %d", (int)pid);
    printf("before wait: ");
    fflush(stdout);
    system(cmd);                                /* shows state Z: zombie */
    waitpid(pid, NULL, 0);                      /* reap it */
    printf("after wait : ");
    fflush(stdout);
    if (system(cmd) != 0) printf("(gone: the zombie was reaped)\n");
    return 0;
}
```

OUTPUT (PID differs per run)

```
before wait: 14169 Z+   prog
after wait : (gone: the zombie was reaped)
```

TIP

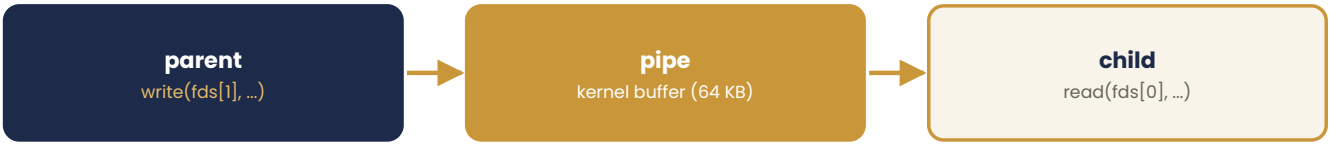
A parent that does not care about its children's status can avoid zombies by ignoring SIGCHLD (`signal(SIGCHLD, SIG_IGN)`), or by reaping with `waitpid(-1, NULL, WNOHANG)` in a loop.

22.4 Inter-process communication

Processes have separate memory, so they need operating-system mechanisms to exchange data: **inter-process communication (IPC)**.

22.4.1 Pipes

`pipe(fds)` creates a one-way channel: `fds[1]` is the write end and `fds[0]` the read end. Created before fork, it is shared by parent and child. Each process closes the end it does not use. A read returns 0 (end of file) when every write end has been closed, which is why closing unused ends matters.



A PIPE CARRIES BYTES FROM ONE PROCESS TO ANOTHER

TWO PIPES: A REQUEST AND A REPLY

main.c

```c
#define _DEFAULT_SOURCE     /* expose POSIX/Linux functions under -std=c17 */
#include <stdio.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>
#include <ctype.h>

int main(void) {
    int to_child[2], to_parent[2];
    pipe(to_child);
    pipe(to_parent);
    if (fork() == 0) {                              /* child: upper-cases text */
        close(to_child[1]); close(to_parent[0]);
        char buf[128];
        ssize_t n;
        while ((n = read(to_child[0], buf, sizeof buf)) > 0) {
            for (ssize_t i = 0; i < n; i++) buf[i] = (char)toupper((unsigned char)buf[i]);
            write(to_parent[1], buf, (size_t)n);
        }
        close(to_child[0]); close(to_parent[1]);
        _exit(0);
    }
    close(to_child[0]); close(to_parent[1]);          /* parent */
    const char *msg = "pipes connect processes";
    write(to_child[1], msg, strlen(msg));
    close(to_child[1]);                               /* child sees end of file */
    char reply[128];
    ssize_t n = read(to_parent[0], reply, sizeof reply - 1);
    reply[n > 0 ? n : 0] = '\0';
    printf("parent sent    : %s\nchild returned : %s\n", msg, reply);
    close(to_parent[0]);
    wait(NULL);
    return 0;
}
```

OUTPUT

```
parent sent    : pipes connect processes
child returned : PIPES CONNECT PROCESSES
```

Shell pipelines work exactly like this. For `ls | wc -l` the shell creates a pipe, forks twice, and uses `dup2` to make the pipe ends become fd 1 of ls and fd 0 of wc before exec:

BUILDING A SHELL PIPELINE WITH DUP2

main.c

```c
#define _DEFAULT_SOURCE     /* expose POSIX/Linux functions under -std=c17 */
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main(void) {                        /* runs: printf "b\na\nc\n" | sort */
    int fds[2];
    pipe(fds);
    if (fork() == 0) {                  /* left side writes into the pipe */
        dup2(fds[1], STDOUT_FILENO);
        close(fds[0]); close(fds[1]);
        execlp("printf", "printf", "banana\napple\ncherry\n", (char *)NULL);
        _exit(127);
    }
    if (fork() == 0) {                  /* right side reads from the pipe */
        dup2(fds[0], STDIN_FILENO);
        close(fds[0]); close(fds[1]);
        execlp("sort", "sort", (char *)NULL);
        _exit(127);
    }
    close(fds[0]); close(fds[1]);       /* parent keeps no ends open */
    wait(NULL); wait(NULL);
    printf("pipeline finished\n");
    return 0;
}
```

OUTPUT

```
apple
banana
cherry
pipeline finished
```

22.4.2 Named pipes (FIFOs)

An ordinary pipe exists only between related processes. A **named pipe** (FIFO) has a name in the file system, created with `mkfifo()` or the `mkfifo` command, so **unrelated** processes can open it like a file. Opening a FIFO for reading blocks until a writer opens it, and vice versa.

A FIFO BETWEEN TWO PROCESSES

main.c

```c
#define _DEFAULT_SOURCE     /* expose POSIX/Linux functions under -std=c17 */
#include <stdio.h>
#include <string.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/stat.h>
#include <sys/wait.h>

int main(void) {
    const char *path = "chat.fifo";
    unlink(path);
    if (mkfifo(path, 0600) == -1) { perror("mkfifo"); return 1; }
    if (fork() == 0) {                          /* writer: could be a separate program */
        int fd = open(path, O_WRONLY);          /* blocks until a reader opens */
        const char *lines[] = {"hello through a FIFO\n", "second message\n"};
        for (int i = 0; i < 2; i++) write(fd, lines[i], strlen(lines[i]));
        close(fd);
        _exit(0);
    }
    int fd = open(path, O_RDONLY);
    char buf[128];
    ssize_t n;
    while ((n = read(fd, buf, sizeof buf - 1)) > 0) { buf[n] = '\0'; printf("reader got: %s", buf); }
    close(fd);
    wait(NULL);
    unlink(path);                               /* remove the FIFO file */
    return 0;
}
```

OUTPUT

```
reader got: hello through a FIFO
second message
```

22.4.3 Shared memory and message queues (overview)

Mechanism	API (POSIX)	Characteristics
Pipe / FIFO	`pipe`, `mkfifo`	byte stream, one direction, simple
Shared memory	`shm_open` + `mmap`	fastest: processes read and write the same RAM; needs synchronisation (semaphores, mutexes)
Message queue	`mq_open`, `mq_send`, `mq_receive`	discrete messages with priorities, kernel-managed
Semaphore	`sem_open`, `sem_wait`, `sem_post`	counting lock between processes
Unix domain socket	`socket(AF_UNIX, ...)`	like a network socket but local; two-way; can pass file descriptors
Signals	`kill`	tiny notifications, no data

SHARED MEMORY BETWEEN PARENT AND CHILD WITH MMAP main.c

```c
#define _DEFAULT_SOURCE     /* expose POSIX/Linux functions under -std=c17 */
#include <stdio.h>
#include <sys/mman.h>
#include <sys/wait.h>
#include <unistd.h>

int main(void) {
    /* anonymous shared mapping: visible to parent and child after fork */
    int *shared = mmap(NULL, sizeof(int) * 4, PROT_READ | PROT_WRITE,
                       MAP_SHARED | MAP_ANONYMOUS, -1, 0);
    if (shared == MAP_FAILED) { perror("mmap"); return 1; }
    int not_shared = 0;
    if (fork() == 0) {
        for (int i = 0; i < 4; i++) shared[i] = (i + 1) * 100;
        not_shared = 999;
        _exit(0);
    }
    wait(NULL);
    printf("shared memory  : %d %d %d %d (written by the child)\n", shared[0], shared[1], shared[2],
shared[3]);
    printf("ordinary memory: %d (child changed only its own copy)\n", not_shared);
    munmap(shared, sizeof(int) * 4);
    return 0;
}
```

OUTPUT

```
shared memory  : 100 200 300 400 (written by the child)
ordinary memory: 0 (child changed only its own copy)
```

22.5 Signals

22.5.1 Common signals: SIGINT, SIGTERM, SIGKILL, SIGSEGV

A **signal** is an asynchronous notification sent to a process by the kernel, by another process (`kill`), or by itself. Each signal has a default action: terminate, terminate with core dump, ignore, or stop.

Signal	No.	Sent when	Default	Catchable
SIGINT	2	Ctrl-C in the terminal	terminate	yes
SIGTERM	15	`kill PID` : polite request to exit	terminate	yes
SIGKILL	9	`kill -9 PID`	terminate	**no**
SIGSEGV	11	invalid memory access	core dump	yes (rarely useful)
SIGFPE	8	integer division by zero	core dump	yes
SIGABRT	6	`abort()` , failed assert	core dump	yes
SIGCHLD	17	a child stopped or exited	ignore	yes
SIGALRM	14	timer from `alarm()`	terminate	yes
SIGPIPE	13	writing to a pipe or socket with no reader	terminate	yes
SIGSTOP / SIGCONT	19 / 18	Ctrl-Z / fg	stop / continue	SIGSTOP no

22.5.2 signal() and sigaction()

`signal(sig, handler)` is the simple ANSI C interface; its exact behaviour differs between systems. `sigaction()` is the POSIX interface with precise control: which signals are blocked while the handler runs, and flags such as `SA_RESTART` (automatically restart interrupted system calls). Use sigaction in real programs. The handler can be a function, `SIG_IGN` (ignore) or `SIG_DFL` (default action).

22.5.3 Writing a signal handler

A handler can run at **any** moment, between any two instructions of your program. It must therefore only do **async-signal-safe** things: set a `volatile sig_atomic_t` flag, or call the few safe functions such as `write` and `_exit`. printf and malloc are **not** safe in a handler (they may be interrupted halfway through their own work). The standard pattern: the handler sets a flag, the main loop checks it.

GRACEFUL SHUTDOWN WITH SIGACTION main.c

```c
#define _DEFAULT_SOURCE     /* expose POSIX/Linux functions under -std=c17 */
#include <stdio.h>
#include <string.h>
#include <signal.h>
#include <unistd.h>

static volatile sig_atomic_t got_term = 0, got_usr1 = 0;

static void on_signal(int sig) {
    if (sig == SIGTERM || sig == SIGINT) got_term = 1;
    else if (sig == SIGUSR1) got_usr1++;
    const char msg[] = "  (handler ran)\n";
    write(STDOUT_FILENO, msg, sizeof msg - 1);        /* async-signal-safe */
}
int main(void) {
    struct sigaction sa;
    memset(&sa, 0, sizeof sa);
    sa.sa_handler = on_signal;
    sigemptyset(&sa.sa_mask);
    sa.sa_flags = SA_RESTART;
    sigaction(SIGTERM, &sa, NULL);
    sigaction(SIGINT, &sa, NULL);
    sigaction(SIGUSR1, &sa, NULL);
    signal(SIGPIPE, SIG_IGN);                          /* ignore entirely */

    printf("worker %d running; sending myself signals\n", (int)getpid());
    fflush(stdout);
    for (int tick = 1; !got_term; tick++) {
        printf("tick %d, SIGUSR1 seen %d time(s)\n", tick, (int)got_usr1);
        fflush(stdout);
        if (tick == 2) kill(getpid(), SIGUSR1);
        if (tick == 3) raise(SIGUSR1);
        if (tick == 4) kill(getpid(), SIGTERM);        /* like "kill PID" from a shell */
        usleep(100000);
    }
    printf("SIGTERM received: saving work and exiting cleanly\n");
    return 0;
}
```

OUTPUT (PID differs per run)

```
worker 14199 running; sending myself signals
tick 1, SIGUSR1 seen 0 time(s)
tick 2, SIGUSR1 seen 0 time(s)
  (handler ran)
tick 3, SIGUSR1 seen 1 time(s)
  (handler ran)
tick 4, SIGUSR1 seen 2 time(s)
  (handler ran)
SIGTERM received: saving work and exiting cleanly
```

22.6 Threads (POSIX threads)

A **thread** is an independent flow of execution **inside** a process. All threads of a process share its memory (globals, heap, open files) but each has its own stack and registers. Threads are cheaper to create than processes and communicate simply through shared variables, which is exactly what makes them dangerous. Compile and link with `-pthread`.



THREADS SHARE THE PROCESS'S MEMORY BUT HAVE PRIVATE STACKS

22.6.1 pthread_create() and pthread_join()

PROTOTYPES IN C

```c
int pthread_create(pthread_t *thread, const pthread_attr_t *attr,
                   void *(*start)(void *), void *arg);
int pthread_join(pthread_t thread, void **result);
```

`pthread_create` starts a new thread running `start(arg)`. The single `void *` argument is usually a pointer to a structure with everything the thread needs. `pthread_join` waits for a thread to finish and collects its return value, like waitpid for processes.

SPLITTING A SUM ACROSS 4 THREADS main.c

```c
#define _DEFAULT_SOURCE     /* expose POSIX/Linux functions under -std=c17 */
#include <stdio.h>
#include <pthread.h>

typedef struct { int id; long from, to; long result; } Job;

void *sum_range(void *arg) {
    Job *job = arg;
    long s = 0;
    for (long i = job->from; i <= job->to; i++) s += i;
    job->result = s;                    /* each thread writes only its own Job */
    return NULL;
}
int main(void) {
    enum { T = 4 };
    pthread_t threads[T];
    Job jobs[T];
    long n = 100000000, chunk = n / T;
    for (int i = 0; i < T; i++) {
        jobs[i] = (Job){i, i * chunk + 1, (i + 1) * chunk, 0};
        pthread_create(&threads[i], NULL, sum_range, &jobs[i]);
    }
    long total = 0;
    for (int i = 0; i < T; i++) {
        pthread_join(threads[i], NULL);
        printf("thread %d summed %ld..%ld = %ld\n", jobs[i].id, jobs[i].from, jobs[i].to,
jobs[i].result);
        total += jobs[i].result;
    }
    printf("total = %ld (expected %ld)\n", total, n * (n + 1) / 2);
    return 0;
}
```

OUTPUT

```
thread 0 summed 1..25000000 = 312500012500000
thread 1 summed 25000001..50000000 = 937500012500000
thread 2 summed 50000001..75000000 = 1562500012500000
thread 3 summed 75000001..100000000 = 2187500012500000
total = 5000000050000000 (expected 5000000050000000)
```

22.6.2 Race conditions

A **race condition** occurs when the result depends on the unpredictable timing of threads. `counter++` looks like one step but is three: load the value, add 1, store it back. If two threads load the same value before either stores, one increment is lost.

Time	Thread A	Thread B	counter in memory
1	load 41		41
2		load 41	41
3	add → 42		41
4		add → 42	41
5	store 42		42
6		store 42	42 (one increment lost)

A DATA RACE

main.c

```c
#define _DEFAULT_SOURCE     /* expose POSIX/Linux functions under -std=c17 */
#include <stdio.h>
#include <pthread.h>

#define THREADS 4
#define ITERS   1000000

long counter = 0;

void *work(void *arg) {
    (void)arg;
    for (int i = 0; i < ITERS; i++)
        counter++;                      /* not atomic: load, add 1, store */
    return NULL;
}
int main(void) {
    pthread_t t[THREADS];
    for (int i = 0; i < THREADS; i++) pthread_create(&t[i], NULL, work, NULL);
    for (int i = 0; i < THREADS; i++) pthread_join(t[i], NULL);
    printf("counter = %ld, expected %d, lost %ld updates\n", counter, THREADS * ITERS, THREADS * ITERS
- counter);
    return 0;
}
```

OUTPUT (the number lost changes every run)

```
counter = 3080381, expected 4000000, lost 919619 updates
```

22.6.3 Mutexes

A **mutex** (mutual exclusion lock) protects a **critical section**: only one thread can hold it at a time; others calling `pthread_mutex_lock` wait until it is unlocked. Keep critical sections short, and always unlock on every path.

THE SAME PROGRAM WITH A MUTEX main.c

```c
#define _DEFAULT_SOURCE     /* expose POSIX/Linux functions under -std=c17 */
#include <stdio.h>
#include <pthread.h>

#define THREADS 4
#define ITERS   1000000

long counter = 0;
pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;

void *work(void *arg) {
    (void)arg;
    for (int i = 0; i < ITERS; i++)
    {
        pthread_mutex_lock(&lock);      /* enter the critical section */
        counter++;
        pthread_mutex_unlock(&lock);    /* leave it */
    }
    return NULL;
}
int main(void) {
    pthread_t t[THREADS];
    for (int i = 0; i < THREADS; i++) pthread_create(&t[i], NULL, work, NULL);
    for (int i = 0; i < THREADS; i++) pthread_join(t[i], NULL);
    printf("counter = %ld, expected %d, lost %ld updates\n", counter, THREADS * ITERS, THREADS * ITERS
- counter);
    return 0;
}
```

OUTPUT

```
counter = 4000000, expected 4000000, lost 0 updates
```

Locking a million times per thread is slow. Better designs reduce sharing: each thread counts in a local variable and adds it to the total **once** under the lock, or uses an atomic counter (Module 18.1.4).

22.6.4 Condition variables

A **condition variable** lets a thread **sleep until a condition becomes true** instead of spinning in a loop. It is always used with a mutex: the waiting thread calls `pthread_cond_wait(&cv, &m)`, which atomically unlocks the mutex and sleeps; when another thread changes the shared state and calls `pthread_cond_signal` (or `_broadcast`), the waiter wakes up holding the mutex again. Always wait in a `while` loop that re-checks the condition, because wake-ups can be spurious.

A BOUNDED PRODUCER/CONSUMER QUEUE main.c

```c
#define _DEFAULT_SOURCE     /* expose POSIX/Linux functions under -std=c17 */
#include <stdio.h>
#include <pthread.h>

#define CAP 3
int buf[CAP], count = 0, in = 0, out = 0;
pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t not_full = PTHREAD_COND_INITIALIZER, not_empty = PTHREAD_COND_INITIALIZER;

void *producer(void *arg) {
    (void)arg;
    for (int item = 1; item <= 6; item++) {
        pthread_mutex_lock(&m);
        while (count == CAP) pthread_cond_wait(&not_full, &m);     /* sleep while full */
```

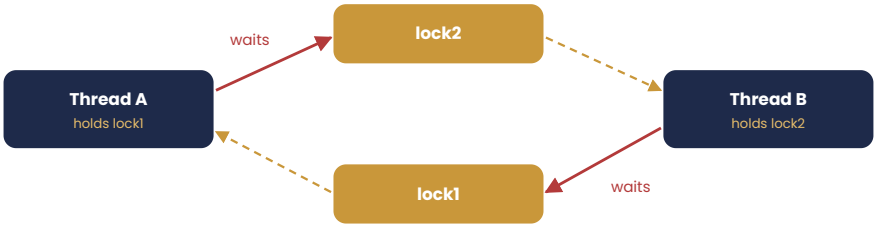
```
        buf[in] = item; in = (in + 1) % CAP; count++;
        printf("produced %d (buffer has %d)\n", item, count);
        pthread_cond_signal(&not_empty);
        pthread_mutex_unlock(&m);
    }
    return NULL;
}
void *consumer(void *arg) {
    (void)arg;
    long sum = 0;
    for (int k = 0; k < 6; k++) {
        pthread_mutex_lock(&m);
        while (count == 0) pthread_cond_wait(&not_empty, &m);      /* sleep while empty */
        int item = buf[out]; out = (out + 1) % CAP; count--;
        pthread_cond_signal(&not_full);
        pthread_mutex_unlock(&m);
        sum += item;
    }
    printf("consumer received all items, sum = %ld\n", sum);
    return NULL;
}
int main(void) {
    pthread_t p, c;
    pthread_create(&p, NULL, producer, NULL);
    pthread_create(&c, NULL, consumer, NULL);
    pthread_join(p, NULL);
    pthread_join(c, NULL);
    return 0;
}
```

OUTPUT (interleaving can vary)

```
produced 1 (buffer has 1)
produced 2 (buffer has 1)
produced 3 (buffer has 1)
produced 4 (buffer has 1)
produced 5 (buffer has 1)
produced 6 (buffer has 1)
consumer received all items, sum = 21
```

22.6.5 Deadlock

A **deadlock** happens when threads wait for each other forever. Classic case: thread A holds lock 1 and waits for lock 2, while thread B holds lock 2 and waits for lock 1. Four conditions must all hold (mutual exclusion, hold and wait, no preemption, circular wait); breaking any one prevents deadlock. The usual rule: **always acquire locks in the same global order**.



CIRCULAR WAIT: EACH THREAD HOLDS WHAT THE OTHER NEEDS

PROVOKING A DEADLOCK (TIMED LOCKS LET THE DEMO FINISH) main.c

```c
#define _DEFAULT_SOURCE     /* expose POSIX/Linux functions under -std=c17 */
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>
#include <errno.h>
#include <time.h>

pthread_mutex_t lock1 = PTHREAD_MUTEX_INITIALIZER, lock2 = PTHREAD_MUTEX_INITIALIZER;

void *thread_a(void *arg) {
    (void)arg;
    pthread_mutex_lock(&lock1);
    usleep(100000);                                  /* make the bad timing certain */
    struct timespec ts; clock_gettime(CLOCK_REALTIME, &ts); ts.tv_sec += 1;
    if (pthread_mutex_timedlock(&lock2, &ts) == ETIMEDOUT)
        printf("A: holding lock1, gave up waiting for lock2 -> deadlock\n");
    else pthread_mutex_unlock(&lock2);
    pthread_mutex_unlock(&lock1);
    return NULL;
}
void *thread_b(void *arg) {
    (void)arg;
    pthread_mutex_lock(&lock2);                      /* opposite order: the bug */
    usleep(100000);
    struct timespec ts; clock_gettime(CLOCK_REALTIME, &ts); ts.tv_sec += 1;
    if (pthread_mutex_timedlock(&lock1, &ts) == ETIMEDOUT)
        printf("B: holding lock2, gave up waiting for lock1 -> deadlock\n");
    else pthread_mutex_unlock(&lock1);
    pthread_mutex_unlock(&lock2);
    return NULL;
}
int main(void) {
    pthread_t a, b;
    pthread_create(&a, NULL, thread_a, NULL);
    pthread_create(&b, NULL, thread_b, NULL);
    pthread_join(a, NULL);
    pthread_join(b, NULL);
    printf("fix: make both threads lock lock1 before lock2\n");
    return 0;
}
```

OUTPUT (whichever thread times out first reports it; then the other can proceed)

```
B: holding lock2, gave up waiting for lock1 -> deadlock
A: holding lock1, gave up waiting for lock2 -> deadlock
fix: make both threads lock lock1 before lock2
```

22.6.6 C11 threads (threads.h) overview

C11 added an optional portable threads library in `<threads.h>` modelled on pthreads, so the same code can compile on any conforming platform (glibc supports it since 2.28; MSVC since 2022). Most Linux code still uses pthreads directly, which offer more features.

C11 threads.h	POSIX equivalent
`thrd_create(&t, fn, arg)`	`pthread_create` (fn returns int)
`thrd_join(t, &res)`	`pthread_join`
`mtx_init / mtx_lock / mtx_unlock`	`pthread_mutex_*`

C11 threads.h	POSIX equivalent
`cnd_wait / cnd_signal`	`pthread_cond_*`
`thread_local` (`_Thread_local`)	`__thread` / pthread keys

THE SAME IDEAS WITH C11 THREADS

main.c

```c
#define _DEFAULT_SOURCE     /* expose POSIX/Linux functions under -std=c17 */
#include <stdio.h>
#include <threads.h>

mtx_t lock;
long total = 0;

int worker(void *arg) {
    int n = *(int *)arg;
    long local = 0;
    for (int i = 1; i <= n; i++) local += i;   /* work without the lock */
    mtx_lock(&lock);
    total += local;                            /* one short critical section */
    mtx_unlock(&lock);
    return 0;
}
int main(void) {
    thrd_t t[3];
    int n[3] = {100, 1000, 10000};
    mtx_init(&lock, mtx_plain);
    for (int i = 0; i < 3; i++) thrd_create(&t[i], worker, &n[i]);
    for (int i = 0; i < 3; i++) thrd_join(t[i], NULL);
    mtx_destroy(&lock);
    printf("total = %ld\n", total);
    return 0;
}
```

OUTPUT

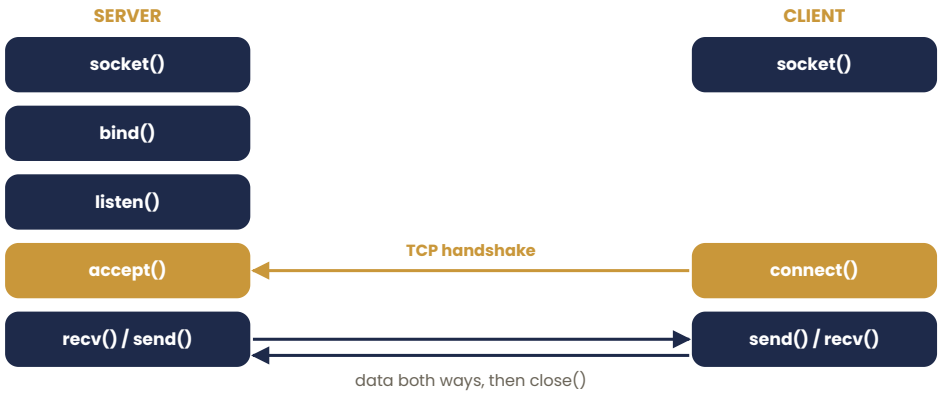
```
total = 50510550
```

22.7 Network programming

22.7.1 Sockets overview (TCP vs UDP)

A **socket** is an endpoint for communication, used through a file descriptor like a file or pipe. An Internet socket address is an **IP address** (which machine) plus a **port** (which program on it, 0 to 65535). The two main transport protocols:

	TCP (SOCK_STREAM)	UDP (SOCK_DGRAM)
Connection	connection-oriented: handshake first	connectionless: just send
Reliability	guaranteed, in order, no duplicates	packets may be lost, duplicated or reordered
Data	a byte stream (no message boundaries)	separate datagrams
Speed / overhead	more overhead	minimal
Used by	HTTP, HTTPS, SSH, email, databases	DNS, video calls, games, streaming



THE SEQUENCE OF CALLS FOR A TCP SERVER AND CLIENT

22.7.2 Server: socket(), bind(), listen(), accept()

1. `socket(AF_INET, SOCK_STREAM, 0)` creates a TCP socket.
2. `bind()` attaches it to a local address and port. Port numbers must be converted to network byte order with `htons` (Module 17.5).
3. `listen(fd, backlog)` marks it as a passive socket that accepts connections.
4. `accept()` waits for a client and returns a **new** socket for that one connection; the listening socket keeps accepting others.

22.7.3 Client: connect(), send(), recv()

The client creates a socket and calls `connect()` with the server's address. Then both sides use `send()` / `recv()` (or plain write/read). `recv` returns 0 when the other side has closed the connection. TCP is a stream, so one send may arrive as several recvs; real protocols mark message boundaries, for example with newlines or length prefixes.

22.7.4 A simple echo server and client

To show both sides in one runnable example, the program forks: the child is the client and the parent is the server, communicating over the loopback address 127.0.0.1. Normally they are two separate programs, possibly on different machines.

AN ECHO SERVER AND CLIENT OVER TCP — main.c

```c
#define _DEFAULT_SOURCE     /* expose POSIX/Linux functions under -std=c17 */
#include <stdio.h>
#include <string.h>
#include <ctype.h>
#include <unistd.h>
#include <arpa/inet.h>
#include <sys/socket.h>
#include <sys/wait.h>

#define PORT 5599

static void run_client(void) {
    int s = socket(AF_INET, SOCK_STREAM, 0);
    struct sockaddr_in srv = {.sin_family = AF_INET, .sin_port = htons(PORT)};
    inet_pton(AF_INET, "127.0.0.1", &srv.sin_addr);
    if (connect(s, (struct sockaddr *)&srv, sizeof srv) == -1) { perror("connect"); _exit(1); }
    const char *msgs[] = {"hello server", "sockets are file descriptors", "bye"};
    char buf[256];
    for (int i = 0; i < 3; i++) {
        send(s, msgs[i], strlen(msgs[i]), 0);
```

```
        ssize_t n = recv(s, buf, sizeof buf - 1, 0);
        if (n <= 0) break;
        buf[n] = '\0';
        printf("client got : %s\n", buf);
        fflush(stdout);
    }
    close(s);
}
int main(void) {
    int lfd = socket(AF_INET, SOCK_STREAM, 0);
    int yes = 1;
    setsockopt(lfd, SOL_SOCKET, SO_REUSEADDR, &yes, sizeof yes);   /* allow quick restarts */
    struct sockaddr_in addr = {.sin_family = AF_INET, .sin_port = htons(PORT),
                               .sin_addr.s_addr = htonl(INADDR_LOOPBACK)};
    if (bind(lfd, (struct sockaddr *)&addr, sizeof addr) == -1) { perror("bind"); return 1; }
    listen(lfd, 5);
    printf("server listening on 127.0.0.1:%d\n", PORT);
    fflush(stdout);

    if (fork() == 0) { close(lfd); run_client(); _exit(0); }       /* child = client */

    struct sockaddr_in peer;
    socklen_t plen = sizeof peer;
    int cfd = accept(lfd, (struct sockaddr *)&peer, &plen);
    printf("server: client connected from %s\n", inet_ntoa(peer.sin_addr));
    fflush(stdout);
    char buf[256];
    ssize_t n;
    while ((n = recv(cfd, buf, sizeof buf, 0)) > 0) {              /* echo in upper case */
        for (ssize_t i = 0; i < n; i++) buf[i] = (char)toupper((unsigned char)buf[i]);
        send(cfd, buf, (size_t)n, 0);
    }
    printf("server: client closed the connection\n");
    close(cfd);
    close(lfd);
    wait(NULL);
    return 0;
}
```

OUTPUT

```
server listening on 127.0.0.1:5599
server: client connected from 127.0.0.1
client got : HELLO SERVER
client got : SOCKETS ARE FILE DESCRIPTORS
client got : BYE
server: client closed the connection
```

Production servers must handle many clients at once: fork a process per client, start a thread per client, or use one thread with I/O multiplexing (`poll`, `epoll`), which is how high-performance servers such as nginx work. Use `getaddrinfo()` to resolve host names and support IPv6.

System calls are the doorway to the kernel. Files, pipes and sockets are all file descriptors used with read and write. fork copies a process, exec replaces its program, wait reaps it. Signals are asynchronous: handlers only set flags. Threads share memory, so protect shared data with mutexes, wait with condition variables, and take locks in a fixed order.

PRACTICE CHECKPOINT

Build a **mini shell** that reads commands and runs them with **fork and exec**, and a **multithreaded counter fixed with a mutex**.

Part 1: mini shell. It prints a prompt, splits the line into words, handles the built-ins `cd` and `exit` itself (they must change the shell process, so they cannot run in a child), and runs everything else with fork + execvp + waitpid.

PART 1: MYSH main.c

```c
#define _DEFAULT_SOURCE     /* expose POSIX/Linux functions under -std=c17 */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>

#define MAXARGS 32

int parse(char *line, char **argv) {
    int argc = 0;
    for (char *tok = strtok(line, " \t\n"); tok && argc < MAXARGS - 1; tok = strtok(NULL, " \t\n"))
        argv[argc++] = tok;
    argv[argc] = NULL;
    return argc;
}
int main(void) {
    char line[256], *argv[MAXARGS];
    int last_status = 0;
    for (;;) {
        printf("mysh[%d]$ ", last_status);
        fflush(stdout);
        if (!fgets(line, sizeof line, stdin)) { printf("\n"); break; }      /* Ctrl-D */
        if (parse(line, argv) == 0) continue;                                /* empty line */
        if (strcmp(argv[0], "exit") == 0) break;
        if (strcmp(argv[0], "cd") == 0) {                                    /* built-in */
            if (chdir(argv[1] ? argv[1] : getenv("HOME")) == -1) perror("cd");
            continue;
        }
        pid_t pid = fork();
        if (pid == -1) { perror("fork"); continue; }
        if (pid == 0) {
            execvp(argv[0], argv);
            fprintf(stderr, "mysh: %s: command not found\n", argv[0]);
            _exit(127);
        }
        int status;
        waitpid(pid, &status, 0);
        last_status = WIFEXITED(status) ? WEXITSTATUS(status) : 128 + WTERMSIG(status);
    }
    printf("goodbye\n");
    return 0;
}
```

```
CONSOLE (user input in gold)
mysh[0]$ echo hello from my shell
hello from my shell
mysh[0]$ mkdir -p demo_dir
mysh[0]$ cd demo_dir
mysh[0]$ pwd
/tmp/lbfleke431s/demo_dir
mysh[0]$ ls /no/such/path
ls: cannot access '/no/such/path': No such file or directory
mysh[2]$ whatisthis
mysh: whatisthis: command not found
mysh[127]$ expr 6 + 7
13
mysh[0]$ exit
goodbye
```

Part 2: multithreaded counter. Run with and without the mutex to see the race and its fix side by side.

PART 2: RACE AND FIX

main.c

```c
#define _DEFAULT_SOURCE     /* expose POSIX/Linux functions under -std=c17 */
#include <stdio.h>
#include <pthread.h>

#define THREADS 8
#define ITERS   500000

long unsafe_count = 0, safe_count = 0;
pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;

void *unsafe_worker(void *arg) {
    (void)arg;
    for (int i = 0; i < ITERS; i++) unsafe_count++;          /* data race */
    return NULL;
}
void *safe_worker(void *arg) {
    (void)arg;
    for (int i = 0; i < ITERS; i++) {
        pthread_mutex_lock(&lock);
        safe_count++;
        pthread_mutex_unlock(&lock);
    }
    return NULL;
}
void run(void *(*fn)(void *)) {
    pthread_t t[THREADS];
    for (int i = 0; i < THREADS; i++) pthread_create(&t[i], NULL, fn, NULL);
    for (int i = 0; i < THREADS; i++) pthread_join(t[i], NULL);
}
int main(void) {
    run(unsafe_worker);
    run(safe_worker);
    printf("expected      : %d\n", THREADS * ITERS);
    printf("without mutex : %ld  (%s)\n", unsafe_count, unsafe_count == THREADS * ITERS ? "lucky" :
"updates lost");
    printf("with mutex    : %ld  (%s)\n", safe_count, safe_count == THREADS * ITERS ? "correct" :
"wrong");
    return 0;
}
```

OUTPUT (the unsafe value changes every run)

```
expected      : 4000000
without mutex : 2471178  (updates lost)
with mutex    : 4000000  (correct)
```

MODULE 23 · ADVANCED

Projects, interview preparation and next steps

Minimum duration **5 days** | Level **Advanced**

The final module brings everything together. Six capstone projects combine structures, files, dynamic memory, linked lists and algorithms into complete programs. Then comes focused interview preparation: a topic checklist, output-prediction and pointer puzzles with explained answers, classic coding problems, and how to present your project. The module ends with a map of where to go after C.

IN THIS MODULE

23.1 Capstone projects
23.2 Interview preparation
23.3 Next steps after C

23.1 Capstone projects

Each project below is a complete, working program, kept compact so that it fits in these notes. Treat each one as a starting point: split it into modules with a Makefile (Module 20), add the extensions listed, test it with the sanitizers (Module 21), and keep it in Git. A project you have extended yourself is worth far more in an interview than one you copied.

Project	Main skills	Difficulty
Student Management System	structures, text files, menus	★★
Bank Management System	binary files, fseek, validation	★★★
Library Management System	linked lists, file persistence	★★★
Contact Book	dynamic arrays, realloc, qsort, search	★★★
Tic-Tac-Toe	2D arrays, game loop, win detection, simple AI	★★
File compression (RLE)	binary I/O, bytes, command-line tools	★★★★

23.1.1 Student Management System (structures + file handling)

Records are kept in an array of structures in memory and saved to a **text file** (one student per line, fields separated by |) so the file can be inspected and edited by hand. The menu supports add, list, search, update, delete and a report.

STUDENT MANAGEMENT SYSTEM

main.c

```c
#include <stdio.h>
#include <string.h>

#define MAX 100
#define DB "students.txt"
```

```
typedef struct { int roll; char name[32]; char course[16]; float marks; } Student;

Student db[MAX];
int count = 0;

void load(void) {
    FILE *f = fopen(DB, "r");
    if (!f) return;                                  /* first run: no file yet */
    while (count < MAX && fscanf(f, "%d|%31[^|]|%15[^|]|%f\n", &db[count].roll,
                                 db[count].name, db[count].course, &db[count].marks) == 4)
        count++;
    fclose(f);
}
void save(void) {
    FILE *f = fopen(DB, "w");
    if (!f) { perror(DB); return; }
    for (int i = 0; i < count; i++)
        fprintf(f, "%d|%s|%s|%.1f\n", db[i].roll, db[i].name, db[i].course, db[i].marks);
    fclose(f);
}
int find(int roll) { for (int i = 0; i < count; i++) if (db[i].roll == roll) return i; return -1; }
char grade(float m) { return m >= 90 ? 'A' : m >= 75 ? 'B' : m >= 60 ? 'C' : m >= 40 ? 'D' : 'F'; }
void show(const Student *s) { printf("%4d  %-12s %-8s %6.1f  %c\n", s->roll, s->name, s->course, s-
>marks, grade(s->marks)); }

int main(void) {
    load();
    printf("%d record(s) loaded from %s\n", count, DB);
    int choice;
    while (printf("\n1 Add  2 List  3 Search  4 Update  5 Delete  6 Report  0 Save & exit\nChoice: "),
           scanf("%d", &choice) == 1 && choice != 0) {
        int roll, i;
        switch (choice) {
        case 1:
            if (count == MAX) { printf("database full\n"); break; }
            printf("roll name course marks: ");
            scanf("%d %31s %15s %f", &db[count].roll, db[count].name, db[count].course,
&db[count].marks);
            if (find(db[count].roll) != -1) { printf("roll %d already exists\n", db[count].roll);
break; }
            count++;
            printf("added\n");
            break;
        case 2:
            printf("Roll  Name         Course    Marks  Grade\n");
            for (i = 0; i < count; i++) show(&db[i]);
            break;
        case 3:
            printf("roll: "); scanf("%d", &roll);
            if ((i = find(roll)) >= 0) show(&db[i]); else printf("not found\n");
            break;
        case 4:
            printf("roll and new marks: "); scanf("%d", &roll);
            if ((i = find(roll)) >= 0) { scanf("%f", &db[i].marks); printf("updated: "); show(&db[i]);
}
            else { float skip; scanf("%f", &skip); printf("not found\n"); }
            break;
        case 5:
            printf("roll: "); scanf("%d", &roll);
            if ((i = find(roll)) >= 0) { db[i] = db[--count]; printf("deleted\n"); } else printf("not
found\n");
            break;
        case 6: {
```

```
            float sum = 0; int top = 0;
            for (i = 0; i < count; i++) { sum += db[i].marks; if (db[i].marks > db[top].marks) top =
i; }
            if (count) printf("%d students, average %.2f, topper %s (%.1f)\n", count, sum / count,
db[top].name, db[top].marks);
            break;
        }
        default: printf("invalid choice\n");
        }
    }
    save();
    printf("saved %d record(s)\n", count);
    return 0;
}
```

CONSOLE (user input in gold)

```
0 record(s) loaded from students.txt

1 Add  2 List  3 Search  4 Update  5 Delete  6 Report  0 Save & exit
Choice: 1
roll name course marks: 101 Asha BCA 88.5
added

1 Add  2 List  3 Search  4 Update  5 Delete  6 Report  0 Save & exit
Choice: 1
roll name course marks: 102 Ravi BSc 71
added

1 Add  2 List  3 Search  4 Update  5 Delete  6 Report  0 Save & exit
Choice: 1
roll name course marks: 103 Meera BCA 93
added

1 Add  2 List  3 Search  4 Update  5 Delete  6 Report  0 Save & exit
Choice: 2
Roll  Name         Course    Marks  Grade
 101  Asha         BCA        88.5  B
 102  Ravi         BSc        71.0  C
 103  Meera        BCA        93.0  A

1 Add  2 List  3 Search  4 Update  5 Delete  6 Report  0 Save & exit
Choice: 4
roll and new marks: 102 76.5
updated:  102  Ravi         BSc        76.5  B

1 Add  2 List  3 Search  4 Update  5 Delete  6 Report  0 Save & exit
Choice: 5
roll: 101
deleted

1 Add  2 List  3 Search  4 Update  5 Delete  6 Report  0 Save & exit
Choice: 6
2 students, average 84.75, topper Meera (93.0)

1 Add  2 List  3 Search  4 Update  5 Delete  6 Report  0 Save & exit
Choice: 0
saved 2 record(s)
```

Extensions: sort the list by marks or name with qsort; store records in a dynamic array instead of a fixed one; validate input (marks 0..100, unique roll); export a CSV report; add a password for the update and delete options.

23.1.2 Bank Management System (structures + binary files + menus)

Accounts are fixed-size records in a **binary file**, so any account can be read or updated directly with `fseek` without loading the whole file (Module 16.5). The account number doubles as the record index. Every transaction is validated.

BANK MANAGEMENT SYSTEM main.c

```
#include <stdio.h>
#include <string.h>

#define DB "bank.dat"
typedef struct { int number; char holder[32]; double balance; int active; } Account;

FILE *db;

int read_acc(int no, Account *a) {
    if (no < 1) return 0;
    fseek(db, (long)(no - 1) * (long)sizeof *a, SEEK_SET);
    return fread(a, sizeof *a, 1, db) == 1 && a->active;
}
void write_acc(const Account *a) {
    fseek(db, (long)(a->number - 1) * (long)sizeof *a, SEEK_SET);
    fwrite(a, sizeof *a, 1, db);
    fflush(db);
}
int next_number(void) {
    fseek(db, 0, SEEK_END);
    return (int)(ftell(db) / (long)sizeof(Account)) + 1;
}
int main(void) {
    db = fopen(DB, "r+b");
    if (!db) db = fopen(DB, "w+b");                    /* create on first run */
    if (!db) { perror(DB); return 1; }
    int choice, no;
    double amt;
    Account a, b;
    while (printf("\n1 Open 2 Deposit 3 Withdraw 4 Transfer 5 Balance 6 Close 0 Exit: "),
           scanf("%d", &choice) == 1 && choice != 0) {
        switch (choice) {
        case 1:
            memset(&a, 0, sizeof a);
            printf("holder name and opening deposit: ");
            scanf("%31s %lf", a.holder, &a.balance);
            if (a.balance < 500) { printf("minimum opening deposit is 500\n"); break; }
            a.number = next_number(); a.active = 1;
            write_acc(&a);
            printf("account %d opened for %s\n", a.number, a.holder);
            break;
        case 2: case 3:
            printf("account and amount: "); scanf("%d %lf", &no, &amt);
            if (!read_acc(no, &a)) { printf("no such account\n"); break; }
            if (amt <= 0) { printf("amount must be positive\n"); break; }
            if (choice == 3 && amt > a.balance) { printf("insufficient funds (balance %.2f)\n",
a.balance); break; }
            a.balance += choice == 2 ? amt : -amt;
            write_acc(&a);
            printf("done, new balance %.2f\n", a.balance);
            break;
        case 4:
            printf("from, to, amount: "); scanf("%d", &no);
            int to; scanf("%d %lf", &to, &amt);
            if (!read_acc(no, &a) || !read_acc(to, &b) || no == to) { printf("invalid accounts\n");
break; }
```

```
            if (amt <= 0 || amt > a.balance) { printf("invalid amount\n"); break; }
            a.balance -= amt; b.balance += amt;
            write_acc(&a); write_acc(&b);
            printf("transferred %.2f from %d to %d\n", amt, no, to);
            break;
        case 5:
            printf("account: "); scanf("%d", &no);
            if (read_acc(no, &a)) printf("%d %-10s %12.2f\n", a.number, a.holder, a.balance);
            else printf("no such account\n");
            break;
        case 6:
            printf("account: "); scanf("%d", &no);
            if (read_acc(no, &a)) { a.active = 0; write_acc(&a); printf("account %d closed, paid out
%.2f\n", no, a.balance); }
            else printf("no such account\n");
            break;
        }
    }
    fclose(db);
    return 0;
}
```

CONSOLE (user input in gold)

```
1 Open 2 Deposit 3 Withdraw 4 Transfer 5 Balance 6 Close 0 Exit: 1
holder name and opening deposit: Asha 5000
account 1 opened for Asha

1 Open 2 Deposit 3 Withdraw 4 Transfer 5 Balance 6 Close 0 Exit: 1
holder name and opening deposit: Ravi 300
minimum opening deposit is 500

1 Open 2 Deposit 3 Withdraw 4 Transfer 5 Balance 6 Close 0 Exit: 1
holder name and opening deposit: Ravi 1200
account 2 opened for Ravi

1 Open 2 Deposit 3 Withdraw 4 Transfer 5 Balance 6 Close 0 Exit: 2
account and amount: 1 2500
done, new balance 7500.00

1 Open 2 Deposit 3 Withdraw 4 Transfer 5 Balance 6 Close 0 Exit: 3
account and amount: 2 5000
insufficient funds (balance 1200.00)

1 Open 2 Deposit 3 Withdraw 4 Transfer 5 Balance 6 Close 0 Exit: 4
from, to, amount: 1 2 1000
transferred 1000.00 from 1 to 2

1 Open 2 Deposit 3 Withdraw 4 Transfer 5 Balance 6 Close 0 Exit: 5
account: 1
1 Asha            6500.00

1 Open 2 Deposit 3 Withdraw 4 Transfer 5 Balance 6 Close 0 Exit: 5
account: 2
2 Ravi            2200.00

1 Open 2 Deposit 3 Withdraw 4 Transfer 5 Balance 6 Close 0 Exit: 6
account: 2
account 2 closed, paid out 2200.00

1 Open 2 Deposit 3 Withdraw 4 Transfer 5 Balance 6 Close 0 Exit: 5
account: 2
no such account

1 Open 2 Deposit 3 Withdraw 4 Transfer 5 Balance 6 Close 0 Exit: 0
```

Extensions: a transaction log file with date and time (Module 18.7.1); PIN protection; interest calculation; a monthly statement; storing money as an integer number of paise instead of double to avoid rounding errors.

23.1.3 Library Management System (linked lists + files)

Books are held in a **sorted singly linked list** that grows and shrinks with no fixed limit, and the list is saved to and loaded from a file. Issue and return operations update each book's status.

LIBRARY MANAGEMENT SYSTEM (DRIVEN BY A SCRIPTED DEMO) — main.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef struct Book {
    int id;
    char title[48];
```

```
    char issued_to[24];          /* empty string = available */
    struct Book *next;
} Book;

Book *insert_sorted(Book *head, Book *b) {          /* keep ids in order */
    if (!head || b->id < head->id) { b->next = head; return b; }
    Book *p = head;
    while (p->next && p->next->id < b->id) p = p->next;
    b->next = p->next; p->next = b;
    return head;
}
Book *find(Book *head, int id) { for (; head; head = head->next) if (head->id == id) return head;
return NULL; }
Book *add(Book *head, int id, const char *title) {
    Book *b = calloc(1, sizeof *b);
    b->id = id;
    snprintf(b->title, sizeof b->title, "%s", title);
    return insert_sorted(head, b);
}
Book *remove_book(Book *head, int id) {
    for (Book **link = &head; *link; link = &(*link)->next)
        if ((*link)->id == id) { Book *v = *link; *link = v->next; free(v); break; }
    return head;
}
void list(const Book *b) {
    for (; b; b = b->next)
        printf("  %3d  %-28s %s\n", b->id, b->title, b->issued_to[0] ? b->issued_to : "(available)");
}
void save(const Book *b, const char *path) {
    FILE *f = fopen(path, "w");
    for (; b; b = b->next) fprintf(f, "%d|%s|%s\n", b->id, b->title, b->issued_to);
    fclose(f);
}
Book *load(const char *path) {
    Book *head = NULL;
    FILE *f = fopen(path, "r");
    if (!f) return NULL;
    char line[128];
    while (fgets(line, sizeof line, f)) {
        Book *b = calloc(1, sizeof *b);
        if (sscanf(line, "%d|%47[^|]|%23[^\n]", &b->id, b->title, b->issued_to) >= 2) head =
insert_sorted(head, b);
        else free(b);
    }
    fclose(f);
    return head;
}
void free_all(Book *b) { while (b) { Book *n = b->next; free(b); b = n; } }

int main(void) {
    Book *lib = NULL;
    lib = add(lib, 30, "The C Programming Language");
    lib = add(lib, 10, "Let Us C");
    lib = add(lib, 20, "Data Structures in C");
    lib = add(lib, 40, "Operating System Concepts");
    Book *b = find(lib, 20);
    if (b && !b->issued_to[0]) { strcpy(b->issued_to, "issued to Meera"); printf("book 20 issued\n");
}
    b = find(lib, 30);
    if (b) strcpy(b->issued_to, "issued to Kabir");
    lib = remove_book(lib, 40);
    save(lib, "library.txt");
    free_all(lib);
```

```
    Book *reloaded = load("library.txt");              /* as if the program restarted */
    printf("catalogue after restart:\n");
    list(reloaded);
    b = find(reloaded, 30);
    if (b) { b->issued_to[0] = '\0'; printf("book 30 returned\n"); }
    list(reloaded);
    free_all(reloaded);
    return 0;
}
```

OUTPUT

```
book 20 issued
catalogue after restart:
   10  Let Us C                     (available)
   20  Data Structures in C         issued to Meera
   30  The C Programming Language   issued to Kabir
book 30 returned
   10  Let Us C                     (available)
   20  Data Structures in C         issued to Meera
   30  The C Programming Language   (available)
```

Extensions: a member list (a second linked list) with a limit on books per member; due dates and fines; search by partial title with `strstr`; a menu like project 23.1.1.

23.1.4 Contact Book with search and sort (dynamic memory + qsort)

Contacts live in a **dynamic array** that doubles with realloc, are sorted with **qsort** by different keys, found by name with **bsearch** on the sorted array, and filtered by partial match.

CONTACT BOOK

main.c

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <strings.h>

typedef struct { char name[32]; char phone[16]; char city[20]; } Contact;
typedef struct { Contact *items; size_t size, cap; } Book;

int add(Book *b, const char *name, const char *phone, const char *city) {
    if (b->size == b->cap) {
        size_t ncap = b->cap ? b->cap * 2 : 2;
        Contact *tmp = realloc(b->items, ncap * sizeof *tmp);
        if (!tmp) return 0;
        b->items = tmp; b->cap = ncap;
    }
    Contact *c = &b->items[b->size++];
    snprintf(c->name, sizeof c->name, "%s", name);
    snprintf(c->phone, sizeof c->phone, "%s", phone);
    snprintf(c->city, sizeof c->city, "%s", city);
    return 1;
}
int by_name(const void *a, const void *b) { return strcasecmp(((const Contact *)a)->name, ((const
Contact *)b)->name); }
int by_city(const void *a, const void *b) {
    int r = strcmp(((const Contact *)a)->city, ((const Contact *)b)->city);
    return r ? r : by_name(a, b);
}
void print(const Book *b, const char *title) {
    printf("%s\n", title);
    for (size_t i = 0; i < b->size; i++)
        printf("  %-10s %-12s %s\n", b->items[i].name, b->items[i].phone, b->items[i].city);
```

```
}
int main(void) {
    Book book = {NULL, 0, 0};
    add(&book, "Ravi", "98450-11111", "Chennai");
    add(&book, "asha", "99000-22222", "Mysuru");
    add(&book, "Meera", "97400-33333", "Pune");
    add(&book, "Kabir", "96320-44444", "Chennai");
    add(&book, "Zoya", "95910-55555", "Mysuru");
    printf("%zu contacts, capacity %zu\n", book.size, book.cap);

    qsort(book.items, book.size, sizeof *book.items, by_city);
    print(&book, "sorted by city, then name:");
    qsort(book.items, book.size, sizeof *book.items, by_name);
    print(&book, "sorted by name:");

    Contact key = {"meera", "", ""};
    Contact *hit = bsearch(&key, book.items, book.size, sizeof *book.items, by_name);
    printf("bsearch \"meera\": %s\n", hit ? hit->phone : "not found");

    printf("names containing \"a\":");
    for (size_t i = 0; i < book.size; i++)
        if (strchr(book.items[i].name, 'a') || strchr(book.items[i].name, 'A')) printf(" %s",
book.items[i].name);
    printf("\n");
    free(book.items);
    return 0;
}
```

OUTPUT

```
5 contacts, capacity 8
sorted by city, then name:
  Kabir      96320-44444  Chennai
  Ravi       98450-11111  Chennai
  asha       99000-22222  Mysuru
  Zoya       95910-55555  Mysuru
  Meera      97400-33333  Pune
sorted by name:
  asha       99000-22222  Mysuru
  Kabir      96320-44444  Chennai
  Meera      97400-33333  Pune
  Ravi       98450-11111  Chennai
  Zoya       95910-55555  Mysuru
bsearch "meera": 97400-33333
names containing "a": asha Kabir Meera Ravi Zoya
```

Extensions: save and load as CSV; edit and delete; detect duplicate phone numbers with a hash table (Module 19.7); case-insensitive partial search with a custom function.

23.1.5 Tic-Tac-Toe or Snake game in the console

A two-player (human vs computer) Tic-Tac-Toe on a 3 × 3 array. The computer uses simple priorities: win if it can, block if it must, take the centre, then a corner, then any cell. Win detection checks the 8 possible lines from a table.

TIC-TAC-TOE AGAINST THE COMPUTER — main.c

```
#include <stdio.h>

char board[9] = {'1', '2', '3', '4', '5', '6', '7', '8', '9'};
const int lines[8][3] = {{0,1,2},{3,4,5},{6,7,8},{0,3,6},{1,4,7},{2,5,8},{0,4,8},{2,4,6}};

void draw(void) {
```

```
    for (int r = 0; r < 3; r++) {
        printf(" %c | %c | %c\n", board[r * 3], board[r * 3 + 1], board[r * 3 + 2]);
        if (r < 2) printf("---+---+---\n");
    }
}
char winner(void) {
    for (int i = 0; i < 8; i++) {
        char a = board[lines[i][0]];
        if (a == board[lines[i][1]] && a == board[lines[i][2]]) return a;
    }
    return 0;
}
int free_cell(int i) { return board[i] != 'X' && board[i] != 'O'; }
int completing_move(char who) {                 /* a cell that completes a line for who */
    for (int i = 0; i < 9; i++) {
        if (!free_cell(i)) continue;
        char save = board[i];
        board[i] = who;
        int wins = winner() == who;
        board[i] = save;
        if (wins) return i;
    }
    return -1;
}
int computer_move(void) {
    int m;
    if ((m = completing_move('O')) >= 0) return m;      /* win */
    if ((m = completing_move('X')) >= 0) return m;      /* block */
    if (free_cell(4)) return 4;                         /* centre */
    int pref[] = {0, 2, 6, 8, 1, 3, 5, 7};
    for (int i = 0; i < 8; i++) if (free_cell(pref[i])) return pref[i];
    return -1;
}
int main(void) {
    draw();
    for (int turn = 0; turn < 9 && !winner(); turn++) {
        if (turn % 2 == 0) {
            int cell;
            printf("Your move (1-9): ");
            if (scanf("%d", &cell) != 1) return 1;
            if (cell < 1 || cell > 9 || !free_cell(cell - 1)) { printf("invalid, try again\n"); turn-
-; continue; }
            board[cell - 1] = 'X';
        } else {
            int m = computer_move();
            board[m] = 'O';
            printf("Computer plays %d\n", m + 1);
            draw();
        }
    }
    char w = winner();
    if (!w) draw();
    printf(w == 'X' ? "You win!\n" : w == 'O' ? "Computer wins!\n" : "Draw.\n");
    return 0;
}
```

CONSOLE (user input in gold)

```
 1 | 2 | 3
---+---+---
 4 | 5 | 6
---+---+---
 7 | 8 | 9
Your move (1-9): 1
Computer plays 5
 X | 2 | 3
---+---+---
 4 | O | 6
---+---+---
 7 | 8 | 9
Your move (1-9): 9
Computer plays 3
 X | 2 | O
---+---+---
 4 | O | 6
---+---+---
 7 | 8 | X
Your move (1-9): 7
Computer plays 4
 X | 2 | O
---+---+---
 O | O | 6
---+---+---
 X | 8 | X
Your move (1-9): 6
Computer plays 8
 X | 2 | O
---+---+---
 O | O | X
---+---+---
 X | O | X
Your move (1-9): 2
 X | X | O
---+---+---
 O | O | X
---+---+---
 X | O | X
Draw.
```

Extensions: an unbeatable computer using the minimax algorithm (recursion over all future moves); a score board across games; a Snake game using a circular queue for the snake's body (Module 19.4.2) and non-blocking keyboard input (termios on Linux, conio.h on Windows).

23.1.6 Mini text editor or file compression tool (advanced)

Run-length encoding (RLE) replaces runs of identical bytes with a count and the byte: `AAAABBB` becomes `4A3B`. It is the simplest real compression scheme, used inside image formats such as BMP and TIFF. This tool compresses and decompresses any file as binary data, choosing the mode from the command line like a real utility, and verifies the round trip.

RLE: RUN AS ./PROG -C PIXELS.TXT PIXELS.RLE

main.c

```c
#include <stdio.h>
#include <string.h>

/* format: pairs of bytes (count 1..255, value) */
long compress(FILE *in, FILE *out) {
    int c, prev = fgetc(in);
```

```
    long pairs = 0;
    if (prev == EOF) return 0;
    int run = 1;
    while ((c = fgetc(in)) != EOF) {
        if (c == prev && run < 255) { run++; continue; }
        fputc(run, out); fputc(prev, out); pairs++;
        prev = c; run = 1;
    }
    fputc(run, out); fputc(prev, out);
    return pairs + 1;
}
long expand(FILE *in, FILE *out) {
    int count, value;
    long bytes = 0;
    while ((count = fgetc(in)) != EOF && (value = fgetc(in)) != EOF)
        for (int i = 0; i < count; i++, bytes++) fputc(value, out);
    return bytes;
}
long file_size(const char *p) { FILE *f = fopen(p, "rb"); fseek(f, 0, SEEK_END); long n = ftell(f);
fclose(f); return n; }

int main(int argc, char *argv[]) {
    if (argc != 4 || (strcmp(argv[1], "-c") && strcmp(argv[1], "-d"))) {
        fprintf(stderr, "usage: %s -c|-d INPUT OUTPUT\n", argv[0]);
        return 1;
    }
    FILE *in = fopen(argv[2], "rb"), *out = fopen(argv[3], "wb");
    if (!in || !out) { perror("fopen"); return 1; }
    if (argv[1][1] == 'c') compress(in, out); else expand(in, out);
    fclose(in); fclose(out);
    long a = file_size(argv[2]), b = file_size(argv[3]);
    printf("%s: %ld -> %ld bytes (%.1f%%)\n", argv[1][1] == 'c' ? "compressed" : "expanded", a, b,
100.0 * b / a);

    /* demo: expand again and compare with the original */
    in = fopen(argv[3], "rb"); out = fopen("check.bin", "wb");
    expand(in, out);
    fclose(in); fclose(out);
    FILE *x = fopen(argv[2], "rb"), *y = fopen("check.bin", "rb");
    int c1, c2, same = 1;
    do { c1 = fgetc(x); c2 = fgetc(y); if (c1 != c2) same = 0; } while (c1 != EOF && same);
    fclose(x); fclose(y);
    printf("round trip %s\n", same ? "OK: identical to the original" : "FAILED");
    return 0;
}
```

OUTPUT

```
compressed: 310 -> 64 bytes (20.6%)
round trip OK: identical to the original
```

RLE shines on data with long runs (this 310-byte "image" of dots and hashes) and makes text with few repeats **larger**. Real compressors (gzip's DEFLATE) combine LZ77 back-references with Huffman coding; implementing **Huffman coding** with a priority queue and a binary tree (Module 19) is an excellent advanced extension. A **mini text editor** project instead keeps the file as a linked list or array of lines and implements commands such as `p` (print), `a` (append), `d 3` (delete line 3), `s/old/new/` and `w` (write), in the style of the classic Unix editor `ed`.

23.2 Interview preparation

23.2.1 Core C interview topics checklist

Make sure you can explain each item in your own words and write a short example from memory.

Area	You should be able to explain
Pointers	pointer arithmetic; `*p++` vs `(*p)++`; pointer vs array; `int *p[5]` vs `int (*p)[5]`; double pointers; function pointers; void pointers; NULL, wild and dangling pointers; const with pointers
Memory	stack vs heap; malloc/calloc/realloc/free; leaks; double free; use after free; memory layout (text, data, BSS, heap, stack); sizeof vs strlen
Storage classes	auto, register, static (local and global), extern; scope vs lifetime; default values; internal vs external linkage
Structures	padding and alignment; struct vs union; bit fields; self-referential structures; passing by value vs pointer; flexible array members
Preprocessor	macro pitfalls; #define vs const vs enum; include guards; conditional compilation; # and ##; predefined macros
Language rules	integer promotion and conversions; signed vs unsigned comparison; operator precedence; sequence points and undefined behaviour; volatile
Strings	char array vs string literal; writing strlen, strcpy, strcmp, strrev; why gets is unsafe
Compilation	preprocess, compile, assemble, link; static vs dynamic linking; what -g, -O2, -Wall do

23.2.2 Output-prediction questions

Predict the output **before** reading the answer. Every program here was compiled and run with the same GCC used throughout these notes.

EIGHT CLASSIC OUTPUT QUESTIONS

main.c

```c
#include <stdio.h>

int main(void) {
    int a = 5;
    int b = a++ + ++a;          /* Q1: unsequenced modification */
    printf("Q1: %d\n", b);

    unsigned int u = 1;
    int n = -1;
    printf("Q2: %s\n", n < u ? "-1 < 1" : "-1 > 1  (n converted to unsigned)");

    char c = 'A' + 2;
    printf("Q3: %c %d\n", c, c);

    int x = 10;
    printf("Q4: %d %d\n", x == 10, x = 20);

    printf("Q5: %zu %zu\n", sizeof('a'), sizeof(char));

    int i = 0;
    if (i = 0) printf("Q6: true\n"); else printf("Q6: false\n");

    printf("Q7: %d\n", 7 / 2 * 2 + 7 % 2);
    printf("Q8: %.2f\n", 7 / 2 * 2.0);
    return 0;
}
```

OUTPUT

```
Q1: 12
Q2: -1 > 1  (n converted to unsigned)
Q3: C 67
Q4: 0 20
Q5: 4 1
Q6: false
Q7: 7
Q8: 6.00
```

Q	Explanation
1	**Undefined behaviour**: a is modified twice without a sequence point. GCC happened to print 12 here, but another compiler or flag may differ. The right interview answer is "undefined".
2	In `n < u`, n is converted to unsigned: -1 becomes 4294967295, which is greater than 1.
3	Characters are small integers: 'A' is 65, plus 2 is 67, which is 'C'.
4	The order in which function arguments are evaluated is **unspecified**, and here x is read and modified in the same call, which is undefined. GCC evaluated the assignment first.
5	In C a character constant has type **int**, so sizeof('a') is 4, while sizeof(char) is always 1. (In C++ both are 1.)
6	`i = 0` is an assignment with value 0, which is false.
7	Integer division: 7 / 2 = 3, 3 * 2 = 6, 7 % 2 = 1, total 7.
8	7 / 2 is integer division (3) before multiplying by 2.0, so 6.00, not 7.00.

SEVEN MORE: RECURSION, STATIC, POINTERS, SIZEOF, SWITCH FALL-THROUGH

main.c

```c
#include <stdio.h>

void f(int n) { if (n > 0) { f(n - 1); printf("%d ", n); } }
int counter(void) { static int c = 0; return ++c; }

int main(void) {
    printf("Q9 : "); f(4); printf("\n");                        /* recursion order */
    counter(); counter();
    printf("Q10: %d\n", counter());                             /* static local */
    int arr[] = {10, 20, 30, 40};
    int *p = arr;
    printf("Q11: %d ", *p++);
    printf("%d ", *++p);
    printf("%d\n", ++*p);
    printf("Q12: %zu %zu\n", sizeof arr / sizeof arr[0], sizeof p);
    char s[] = "hello";
    printf("Q13: %zu %zu\n", sizeof s, sizeof "hello" - 1);
    for (int i = 0; i < 3; i++) { if (i == 1) continue; printf("Q14: %d\n", i); }
    int k = 3;
    switch (k) { case 3: printf("Q15: three "); case 4: printf("four "); break; default:
printf("other"); }
    printf("\n");
    return 0;
}
```

OUTPUT

```
Q9 : 1 2 3 4
Q10: 3
Q11: 10 30 31
Q12: 4 8
Q13: 6 5
Q14: 0
Q14: 2
Q15: three four
```

Q	Explanation
9	Each call prints **after** its recursive call returns, so the deepest call (1) prints first: 1 2 3 4.
10	A static local keeps its value between calls: the third call returns 3.
11	`*p++` reads 10 then moves p to 20; `*++p` moves to 30 and reads it; `++*p` increments the 30 to 31.
12	4 elements; p is a pointer, 8 bytes on 64-bit systems.
13	The array holds 6 bytes including '\0'; strlen-style length is 5.
14	continue skips i == 1, printing 0 and 2.
15	No break after case 3, so execution falls through into case 4.

23.2.3 Pointer and memory puzzle questions

POINTER PUZZLES

main.c

```c
#include <stdio.h>
#include <string.h>

int main(void) {
    int a[3][4] = {{1, 2, 3, 4}, {5, 6, 7, 8}, {9, 10, 11, 12}};
    printf("P1: %d %d %d\n", **a, *(*(a + 1) + 2), *(a[2] + 3));
    printf("P2: %td %td\n", (char *)(a + 1) - (char *)a, (char *)(&a + 1) - (char *)a);

    char *names[] = {"red", "green", "blue"};
    char **pp = names;
    printf("P3: %s %c %s\n", *(pp + 1), *names[2], *pp + 1);

    int x = 5, *p = &x, **q = &p;
    **q = 50;
    printf("P4: %d\n", x);

    char buf[] = "pointer";
    char *s = buf + strlen(buf) - 1;
    printf("P5: ");
    while (s >= buf) putchar(*s--);
    printf("\n");

    int v[] = {1, 2, 3, 4, 5};
    int *e = &v[4];
    printf("P6: %td %d\n", e - v, *(v + (e - v) / 2));

    struct { char c; int i; char d; } st;
    printf("P7: %zu\n", sizeof st);
    union { int i; char c[4]; } u = {.i = 0x01020304};
    printf("P8: %d (first byte on this machine)\n", u.c[0]);
    return 0;
}
```

OUTPUT

```
P1: 1 7 12
P2: 16 48
P3: green b ed
P4: 50
P5: retniop
P6: 4 3
P7: 12
P8: 4 (first byte on this machine)
```

P	Explanation
1	`**a` is a[0][0] = 1; `*(*(a+1)+2)` is a[1][2] = 7; `*(a[2]+3)` is a[2][3] = 12.
2	`a + 1` moves one row (4 ints = 16 bytes); `&a + 1` moves past the whole array (48 bytes).
3	`*(pp+1)` is "green"; `*names[2]` is the first char of "blue"; `*pp + 1` is "red" without its first char: "ed".
4	q points to p, p points to x, so `**q = 50` changes x.
5	Walking a pointer backwards from the last character prints the string reversed.
6	Pointer difference counts elements (4), and v + 2 points to the middle value 3.
7	char, padding to 4, int, char, padding to 4: 12 bytes (Module 14.3).

P	Explanation
8	On a little-endian machine the least significant byte (4) is stored first.

23.2.4 Coding problems on arrays, strings, linked lists and recursion

These are among the most frequently asked in campus placements. Practise writing them on paper, stating the time and space complexity, and testing edge cases (empty input, one element, duplicates, negatives).

SEVEN FREQUENTLY ASKED PROBLEMS main.c

```c
#include <stdio.h>
#include <string.h>
#include <ctype.h>

/* 1. second largest element, O(n) */
int second_largest(const int *a, int n) {
    int first = a[0], second = -2147483647 - 1;
    for (int i = 1; i < n; i++) {
        if (a[i] > first) { second = first; first = a[i]; }
        else if (a[i] > second && a[i] != first) second = a[i];
    }
    return second;
}
/* 2. move zeros to the end keeping order, O(n), in place */
void move_zeros(int *a, int n) {
    int k = 0;
    for (int i = 0; i < n; i++) if (a[i]) a[k++] = a[i];
    while (k < n) a[k++] = 0;
}
/* 3. maximum subarray sum (Kadane), O(n) */
int max_subarray(const int *a, int n) {
    int best = a[0], cur = a[0];
    for (int i = 1; i < n; i++) {
        cur = a[i] > cur + a[i] ? a[i] : cur + a[i];
        if (cur > best) best = cur;
    }
    return best;
}
/* 4. are two strings anagrams? O(n) with counting */
int anagram(const char *s, const char *t) {
    int cnt[26] = {0};
    for (; *s; s++) if (isalpha((unsigned char)*s)) cnt[tolower((unsigned char)*s) - 'a']++;
    for (; *t; t++) if (isalpha((unsigned char)*t)) cnt[tolower((unsigned char)*t) - 'a']--;
    for (int i = 0; i < 26; i++) if (cnt[i]) return 0;
    return 1;
}
/* 5. reverse the words of a sentence in place */
void reverse_range(char *l, char *r) { while (l < r) { char t = *l; *l++ = *r; *r-- = t; } }
void reverse_words(char *s) {
    reverse_range(s, s + strlen(s) - 1);                /* whole string */
    for (char *w = s; *w; ) {
        while (*w == ' ') w++;
        char *e = w;
        while (*e && *e != ' ') e++;
        reverse_range(w, e - 1);                         /* each word back */
        w = e;
    }
}
/* 6. recursion: power set of {a, b, c} */
void subsets(const char *set, int i, char *cur, int len) {
    if (set[i] == '\0') { cur[len] = '\0'; printf("{%s} ", cur); return; }
```

```
    subsets(set, i + 1, cur, len);                      /* exclude set[i] */
    cur[len] = set[i];
    subsets(set, i + 1, cur, len + 1);                  /* include set[i] */
}
/* 7. recursion: Tower of Hanoi moves */
int hanoi(int n, char from, char to, char via) {
    if (n == 0) return 0;
    int moves = hanoi(n - 1, from, via, to);
    if (n == 3) printf("  move disk 3 %c -> %c\n", from, to);
    return moves + 1 + hanoi(n - 1, via, to, from);
}
int main(void) {
    int a[] = {12, 35, 1, 10, 34, 35};
    printf("1. second largest: %d\n", second_largest(a, 6));
    int z[] = {0, 1, 0, 3, 12, 0, 5};
    move_zeros(z, 7);
    printf("2. zeros moved: ");
    for (int i = 0; i < 7; i++) printf("%d ", z[i]);
    int k[] = {-2, 1, -3, 4, -1, 2, 1, -5, 4};
    printf("\n3. max subarray sum: %d\n", max_subarray(k, 9));
    printf("4. anagram(\"Listen\", \"Silent\") = %d, (\"apple\", \"paper\") = %d\n", anagram("Listen",
"Silent"), anagram("apple", "paper"));
    char sent[] = "learn being forward";
    reverse_words(sent);
    printf("5. %s\n", sent);
    char cur[4];
    printf("6. ");
    subsets("abc", 0, cur, 0);
    printf("\n7. Hanoi with 3 disks:\n");
    printf("   total moves: %d\n", hanoi(3, 'A', 'C', 'B'));
    return 0;
}
```

OUTPUT

```
1. second largest: 34
2. zeros moved: 1 3 12 5 0 0 0
3. max subarray sum: 6
4. anagram("Listen", "Silent") = 1, ("apple", "paper") = 0
5. forward being learn
6. {} {c} {b} {bc} {a} {ac} {ab} {abc}
7. Hanoi with 3 disks:
  move disk 3 A -> C
   total moves: 7
```

Linked-list favourites (reverse, middle, loop detection, merge two sorted lists, remove the n-th node from the end) and tree problems (height, traversals, BST validation, lowest common ancestor) were covered in Module 19; revisit them and time yourself.

23.2.5 Explaining your project clearly

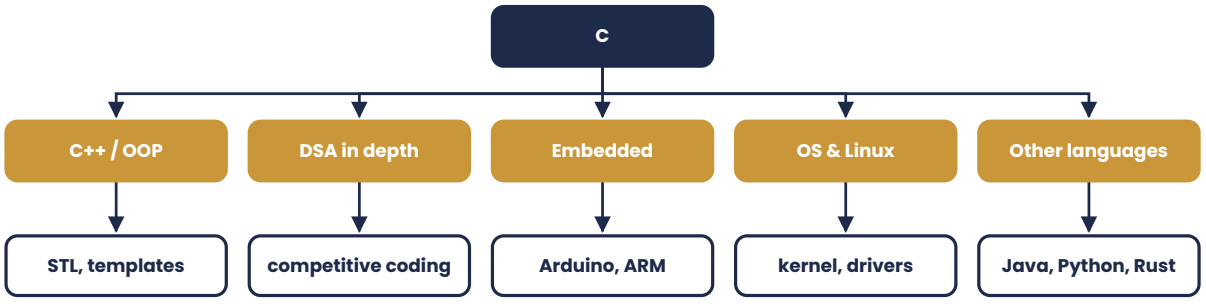
Interviewers judge how well you understand and communicate your work as much as the work itself. Prepare a two-minute explanation using this structure:

1. **Problem**: what it does and for whom, in one sentence. "A bank account manager that stores accounts in a binary file so any account can be updated instantly."
2. **Design**: the main data structures and why you chose them. "Fixed-size records so the account number gives the file offset with fseek, O(1) access instead of reading the whole file."
3. **Your challenges**: one real bug and how you found it. "Withdrawals sometimes corrupted the next record; Valgrind showed I was writing sizeof a pointer instead of sizeof the struct."

4. **Quality**: Makefile, multiple modules, warnings enabled, tested with AddressSanitizer, input validation, Git history.
5. **What you would improve**: shows you think beyond the assignment. "Store money as integers, add a transaction log, use SQLite for concurrent users."

> **TIP**
> Be ready to open the code and walk through any function, to draw your data structure on a whiteboard, and to answer "what happens if the file is missing / the input is negative / memory runs out?". Never claim code you cannot explain.

23.3 Next steps after C



WHERE C LEADS

23.3.1 C++ and object-oriented programming

C++ began as "C with classes" and keeps almost all of C, adding **classes** (data and functions together, with access control), **constructors and destructors** (automatic resource management, RAII: no more forgotten free), **templates** (generic code checked by the compiler, unlike void *), the **Standard Template Library** (vector, map, unordered_map, sort, algorithms ready to use), references, exceptions and lambdas. Everything you learned about pointers, memory and the build process carries over directly. LBF's C++ course is the natural next step.

23.3.2 Data structures and algorithms in depth

Module 19 covered the essentials. Going further means: advanced trees (segment trees, tries, B-trees), graph algorithms (topological sort, minimum spanning trees with Kruskal and Prim, Bellman-Ford, Floyd-Warshall), dynamic programming, greedy algorithms, backtracking, and complexity analysis. Practise regularly on platforms such as LeetCode, HackerRank, CodeChef and Codeforces; product companies interview heavily on these topics.

23.3.3 Embedded systems and microcontrollers

C is the dominant language of embedded systems: washing machines, car controllers, medical devices, IoT sensors. You will use bit manipulation on hardware registers (Module 17), `volatile` (Module 18), fixed-width integer types and very limited memory. Start with an Arduino (programmed in C/C++), then move to ARM Cortex-M boards (STM32), learn about interrupts, timers, UART/SPI/I2C communication, and real-time operating systems such as FreeRTOS. Coding standards such as MISRA C matter here.

23.3.4 Operating systems and Linux internals

Module 22 opened the door to the kernel. Next: study processes and scheduling, virtual memory and paging, file systems and concurrency in an operating systems course (the free textbook "Operating

Systems: Three Easy Pieces" is excellent), read "The Linux Programming Interface", write a simple shell with pipes and redirection, then a loadable kernel module or device driver. Contributing to open-source C projects (the Linux kernel, Git, Redis, SQLite, curl) is outstanding experience.

23.3.5 Other languages: Java, Python, Rust

Language	What C knowledge gives you	Good for
Java	same syntax for expressions, loops and operators; you will appreciate garbage collection and know what a reference really is	enterprise back ends, Android
Python	you understand what the interpreter (itself written in C) does for you; you can write C extensions for speed	scripting, data science, AI, automation
JavaScript / TypeScript	C-style syntax; event loops make sense after Module 22	web front end and back end
C#	C-family syntax, value vs reference types, unsafe pointers	Windows, games (Unity)
Rust	you know exactly which bugs (use after free, data races) its ownership and borrow checker prevent at compile time	systems programming with memory safety
Go	simple C-like syntax with garbage collection and goroutines	cloud services, networking tools

Projects turn knowledge into skill: build them in modules with a Makefile, test them with sanitizers, keep them in Git, and be able to explain every line. For interviews, master pointers, memory, storage classes, structures and the preprocessor, and practise predicting output and coding classic problems. C is the foundation; every path after it builds on what you now know.

PRACTICE CHECKPOINT

Build **one capstone project** with a **Makefile**, **multiple source files**, **file storage** and a **Git repository**, and **present it as you would in an interview**.

A suggested layout for the Bank Management System (23.1.2), organised exactly as in Module 20:

Project layout

```
bank/
├── include/
│   ├── account.h       Account type, account_read/account_write (fseek-based)
│   ├── bank.h          open, deposit, withdraw, transfer, close: validation rules
│   └── ui.h            menus and input helpers
├── src/
│   ├── account.c       file storage: the only module that touches bank.dat
│   ├── bank.c          business rules, no printf/scanf
│   ├── ui.c            all input/output, safe number reading with strtol
│   └── main.c          opens the database, runs the menu loop
├── tests/
│   └── test_bank.c     assert-based tests of bank.c on a temporary file
├── Makefile            all, run, test, clean; CFLAGS with -Wall -Wextra -g -fsanitize=address
├── .gitignore          build/, *.o, bank.dat
└── README.md           what it does, how to build, design decisions
```

1. `git init`, commit the skeleton, then commit each feature separately with a clear message.

2. Write `tests/test_bank.c` first for the rules in bank.c (no negative deposits, no overdraft, transfer is all-or-nothing); `make test` must pass before each commit.
3. Run the program under Valgrind and AddressSanitizer with a scripted input file (`./bank < demo_input.txt`) and fix every report.
4. Prepare the two-minute explanation from 23.2.5, a diagram of the record layout in the file, and one bug story.
5. Present it to a friend or mentor, and answer their "what if" questions by showing the code.